A Vendor-Neutral Multi-Layer Graph Model for Network Protocol State

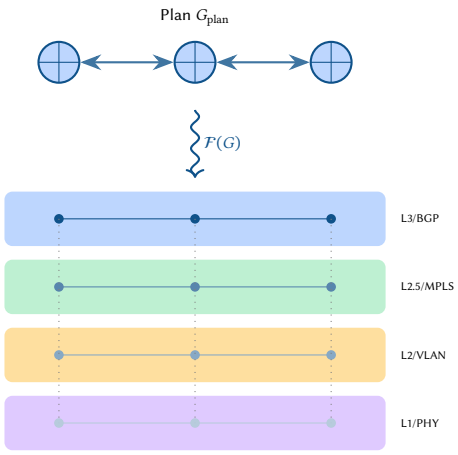
From Plan Topologies to Multi-Layer Overlays via Functorial Mappings

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. Network infrastructure state spans multiple protocol layers, from physical media and Ethernet switching through MPLS label paths, IP routing, and overlay encapsulations. Despite decades of standardisation in RFCs and IEEE specifications, no single vendor-neutral graph model captures the full stack of configurable and operational state across all layers. This report conducts an exhaustive survey of every major protocol, standard, and data model relevant to network state representation. We introduce a *multi-layer property hypergraph* formalism grounded in the supra-adjacency tensor of multilayer network theory, and define a *plan topology* G_{plan} that captures all protocol state as flat attributes on nodes and endpoints. A functorial overlay mapping $\mathcal{F} : G_{\text{plan}} \to G_{\text{overlay}}$ transforms the plan into protocol-specific overlay graphs using algebraic graph rewriting rules. The model is designed for implementation in the Network Topology Engine (NTE), providing correctness-by-construction guarantees through pre-operation validation and columnar SIMD queries.

Contents

1 Introduction and Motivation

Network infrastructure state is distributed across protocol layers, device configurations, and runtime data structures. Engineers reason about physical connectivity, VLAN membership, MPLS label paths, IP routing tables, BGP peering state, VRF isolation, and overlay encapsulations as logically distinct but tightly coupled concerns. Yet the data models used to represent this state are fragmented: YANG/OpenConfig models [1, 2] describe per-device configuration trees; NetBox [3] captures inventory and IPAM as relational tables; Batfish [4] builds a vendor-neutral forwarding model but does not expose a general-purpose graph API; and RFC 8345 [5] defines a YANG-based topology model that lacks protocol state depth.

Insight: *No single existing model captures the full stack of network state in a vendor-neutral, graph-native form.*

The Operational Cost of Fragmentation. This fragmentation is not merely an academic concern; it is the root cause of brittle automation and widespread network outages. When intent is scattered across disjoint databases and hierarchical vendor templates, there is no single source of truth capable of mathematically guaranteeing that an OSPF area boundary aligns with a firewall security zone, or that an EVPN route target matches its underlying VRF. To eliminate these classes of errors before deployment, the network must be modeled not as a collection of text files, but as a rigorous mathematical structure.

This report addresses three questions:

1. **What state exists?** An exhaustive survey of every protocol layer, from physical PHY training to BGP path attributes, identifying all configurable and operational state elements.
2. **How should it be modelled?** A multi-layer property hypergraph formalism where state is stored as flat attributes on nodes and endpoints, not as nested hierarchies or edge properties.
3. **How do layers relate?** A functorial mapping $\mathcal{F} : G_{\text{plan}} \to G_{\text{overlay}}$ that transforms a single plan topology into protocol-specific overlay graphs via algebraic graph rewriting.

Design Rule 1: Flat Attributes Principle

All protocol state shall be represented as flat key-value attributes on **nodes** (devices) and **endpoints** (interfaces/ports), never as nested structures and never as edge properties. Edges carry only structural semantics (connectivity, ownership, hierarchy).

Design Rule 2: Vendor Neutrality

The model shall not reference any vendor-specific CLI syntax, proprietary protocol extensions, or platform-dependent state. All attributes map to standards-track RFCs, IEEE specifications, or OpenConfig YANG models.

Design Rule 3: Configuration State Scope

The plan topology captures **configuration state**: parameters that an operator or automation system *pushes* to a device (interface speed, OSPF cost, BGP peer address, ACL rules). It does **not** model **operational state**: values that are *derived* at runtime by protocol operation or hardware measurement (OSPF neighbour FSM position, BGP session FSM state, STP port forwarding/blocking status, interface error counters, DOM telemetry readings, forwarding table entries). Operational attributes are noted in the survey tables for completeness—marked with "(operational)"—but they are not part of the plan topology's authoritative attribute set. The overlay functors $\mathcal{F}_\ell$ *compute* derived operational state from configuration inputs; they do not read it from the plan.

From Formalism to Practice

The mathematical concepts introduced throughout this report—flat attribute maps, functorial mappings, graph rewriting rules—are not abstract for the sake of abstraction. Each formal construction maps directly to a concrete operation in the NTE implementation. The purpose of the formalism is to provide precise, composable semantics for operations that network engineers already perform informally: filtering devices by protocol attributes, computing overlay topologies from a plan, and applying configuration transformations.

Design Decision 1: Mathematics to Implementation

Formal Concept	Implementation	Concrete Example
Flat attributes	DataFrame columns	`df.filter(pl.col("ospf_area") == "0.0.0.0")`
Functor evaluation	Function call	`fn build_ospf_overlay(plan: &PlanGraph) -> OspfOverlay`
Graph query	Traversal with predicate	`plan.neighbors(device_id).filter(|e| e.has_attr("ospf_area"))`
DPO rule application	Match + transform	Find all endpoint pairs sharing an `ospf_area` value; insert an adjacency edge between them

OSPF Overlay from Three Routers

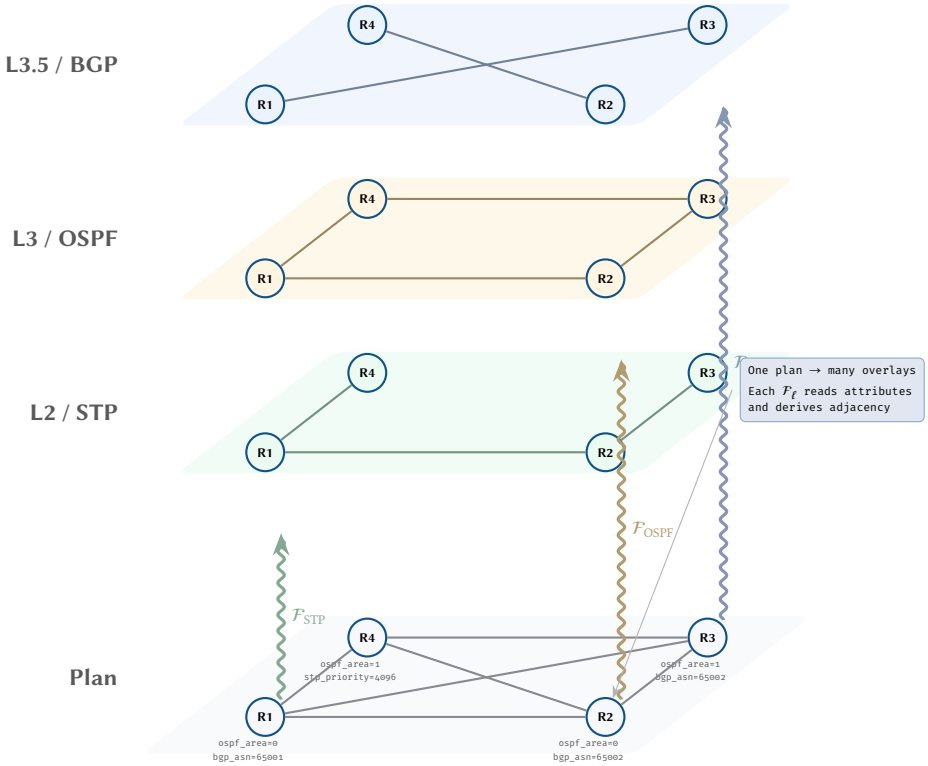
Consider three routers with the following plan-topology attributes:

Node	Endpoint	Key Attributes
R1	eth0	`ospf_area = 0`, `ipv4_address = 10.0.12.1/30`
R2	eth0	`ospf_area = 0`, `ipv4_address = 10.0.12.2/30`
R2	eth1	`ospf_area = 0`, `ipv4_address = 10.0.23.1/30`
R3	eth0	`ospf_area = 0`, `ipv4_address = 10.0.23.2/30`

The OSPF functor $\mathcal{F}_{\text{OSPF}}$ scans the plan topology for endpoints that share (a) a common `ospf_area` value and (b) a physical connectivity edge between them. It produces an overlay graph with two OSPF adjacency edges:

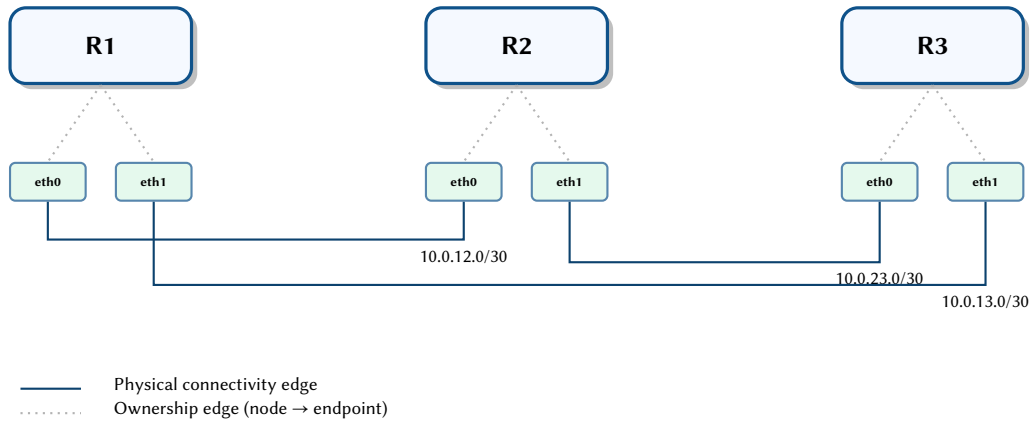
1. R1 $\overset{\text{area 0}}{\longleftrightarrow}$ R2 (via R1:eth0 ↔ R2:eth0, subnet 10.0.12.0/30)
2. R2 $\overset{\text{area 0}}{\longleftrightarrow}$ R3 (via R2:eth1 ↔ R3:eth0, subnet 10.0.23.0/30)

In code, this is a filter-and-join on the endpoint DataFrame followed by graph edge insertion—exactly the columnar operations that the flat attribute principle is designed to enable.



Insight: *Every mathematical operation in this report corresponds to a query, filter, or graph mutation that can be expressed in a few lines of code. The formalism exists to guarantee that these operations compose correctly—not to obscure the engineering.*

Figure 1. The plan topology as a single source of truth. Overlay functors $\mathcal{F}_\ell$ derive protocol-specific views from the same underlying attributed graph.



Element	Type	Key Attributes
R1	Node	`hostname=R1, ospf_router_id=1.1.1.1, bgp_asn=65001`
R1:eth0	Endpoint	`ipv4_address=10.0.12.1/30, ospf_area=0, ospf_cost=10`
R1:eth1	Endpoint	`ipv4_address=10.0.13.1/30, ospf_area=0, ospf_cost=20`
R2	Node	`hostname=R2, ospf_router_id=2.2.2.2, bgp_asn=65001`
R2:eth0	Endpoint	`ipv4_address=10.0.12.2/30, ospf_area=0, ospf_cost=10`
R2:eth1	Endpoint	`ipv4_address=10.0.23.1/30, ospf_area=0, ospf_cost=10`
R3	Node	`hostname=R3, ospf_router_id=3.3.3.3, bgp_asn=65001`
R3:eth0	Endpoint	`ipv4_address=10.0.23.2/30, ospf_area=0, ospf_cost=10`
R3:eth1	Endpoint	`ipv4_address=10.0.13.2/30, ospf_area=0, ospf_cost=20`

Figure 2. Structural view of a three-node plan topology with flat attribute annotations. Solid lines represent physical connectivity edges; dashed lines represent ownership edges from nodes to their endpoints. Each element's flat attributes map directly to DataFrame columns in the NTE storage layer.

Flattening Conventions

The Flat Attributes Principle requires that complex data types—structs, sets, lists, and maps—be decomposed into scalar key-value pairs before storage. Table 1 defines the canonical flattening rules.

Design Decision 2: Flattening Rules

Logical Type	Flat Encoding	Example
struct	Prefixed scalar keys	`error_counters_crc`, `error_counters_runts`
set	Per-element booleans	`vlan_100 = true`, `vlan_200 = true`
list	Indexed keys	`ntp_server_0 = 10.0.0.1`, `ntp_server_1 = 10.0.0.2`
map	Composite keys	`evpn_vni_100 = 10100`, `evpn_vni_200 = 10200`
range	Min/max pair	`mpls_label_range_min = 16`, `mpls_label_range_max = 1048575`
list⟨struct⟩	Index + prefix	`static_route_0_prefix = 0.0.0.0/0`, `static_route_0_nexthop = 10.0.0.1`

Attribute Naming Conventions

Attribute keys follow a structured naming convention: each key begins with a protocol prefix (`ospf_`, `bgp_`, `isis_`, etc.) followed by a field name. Indexed keys append a numeric suffix for ordered sequences (e.g., ACL rules). The formal EBNF grammar appears in Appendix A.7.

Insight: *Flattening enables SIMD-friendly columnar storage: each flat key becomes a column in a Polars DataFrame, supporting vectorised predicate evaluation across all nodes or endpoints simultaneously.*

Indexed attributes. When a protocol feature contains an ordered list of entries, each entry is assigned a zero-based numeric index that becomes part of the key. For example, ACL rule sequences flatten as `acl_MYACL_seq_10_action`, `acl_MYACL_seq_10_protocol`, and so on, where `10` is the sequence number carried from the original ACL definition. Similarly, static routes flatten as `static_route_0_prefix`, `static_route_0_nexthop`, `static_route_1_prefix`, etc. Each index represents an ordered position within the enclosing list, preserving the original sequence semantics from the source configuration.

Per-AFI and per-VLAN attributes. Address-family-qualified attributes embed the AFI/SAFI into the key to avoid collisions between protocol families. BGP peers, for instance, carry per-AFI activation flags of the form `bgp_peer_afi_safi` (e.g. `ipv4_unicast`, `l2vpn_evpn`), with each AFI value stored as a separate boolean attribute. Per-VLAN attributes follow the same pattern: STP multi-instance mappings flatten as `stp_msti_vlan_100 = 1`, `stp_msti_vlan_200 = 2`, where the VLAN ID is absorbed into the key and the MST instance number is the scalar value.

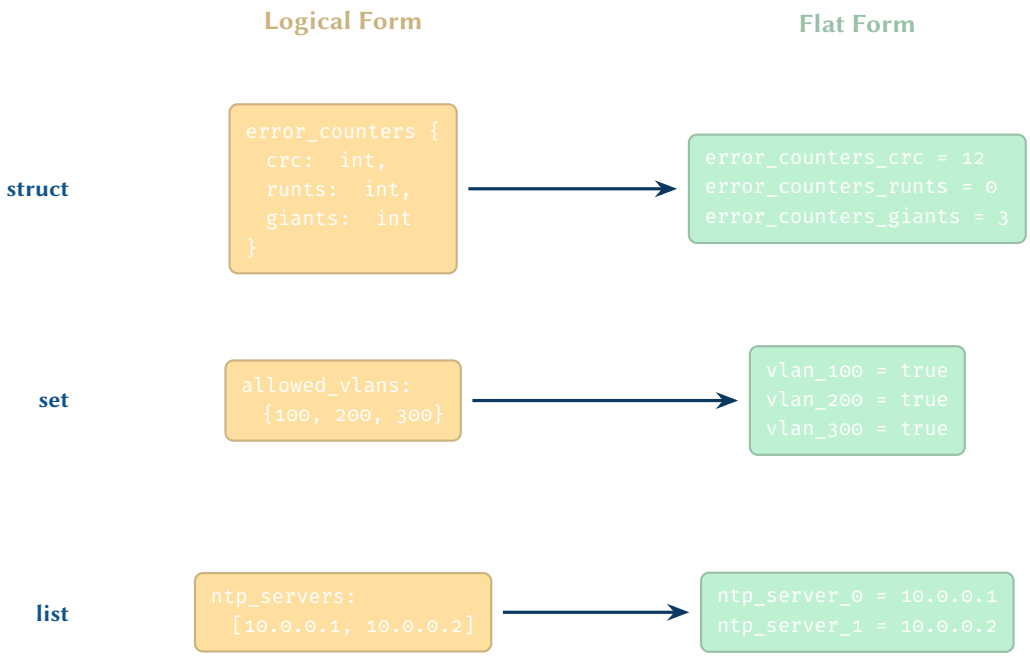
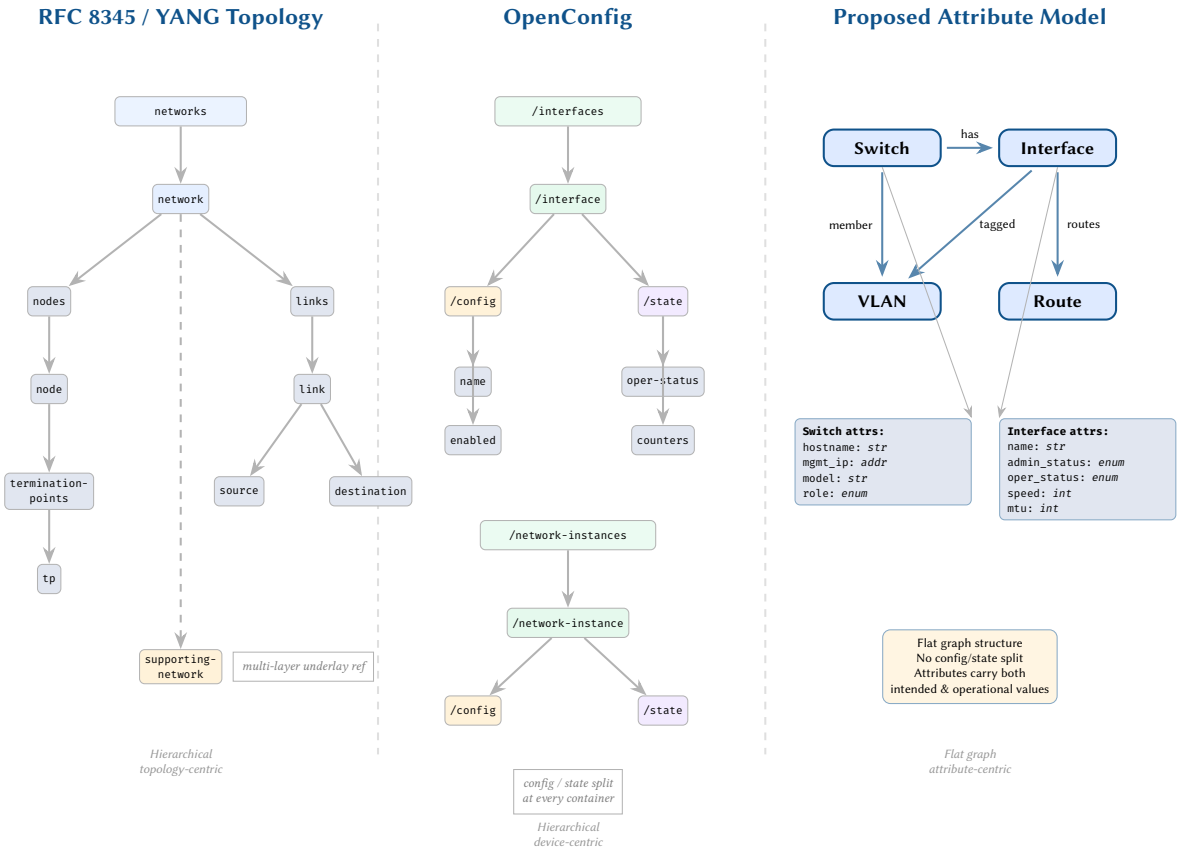


Figure 3. Flattening conventions: struct, set, and list types mapped to flat key-value attributes.

These naming rules work in conjunction with the flattening conventions above (Section 1): the flattening rules define how complex types decompose into scalar values, while the naming conventions define the key structure under which those values are stored.



Insight: *The attribute naming convention ensures that functor prefix-matching (filtering all `ospf_*` attributes) is both complete and unambiguous: no attribute can match two protocol prefixes.*

Figure 4. Comparison of network data models. RFC 8345 provides a hierarchical topology view; OpenConfig separates intended configuration from operational state at each container; our proposed model uses a flat attributed graph that unifies both state types.

1.1 Related Work

The problem of formally representing, reasoning about, and safely modifying network state sits at the intersection of several distinct fields: IETF standardisation, applied network theory, formal verification, and intent-based configuration synthesis. Historically, these domains have evolved in isolation—standards bodies

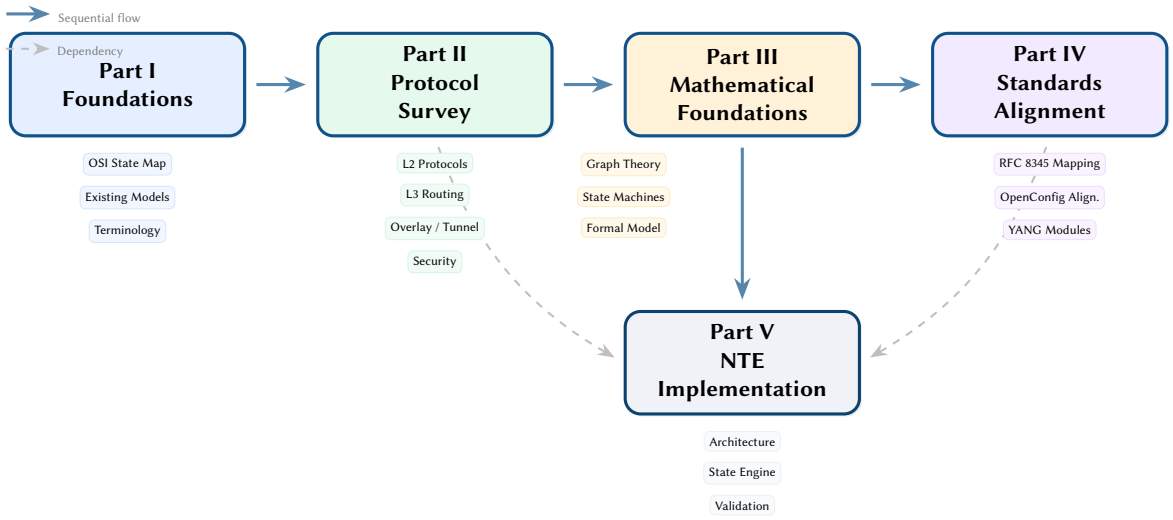


Figure 5. Report roadmap showing the five parts and their dependencies. Parts I–IV proceed sequentially, while Part V draws on Parts II–IV for the reference implementation.

focus on data models, academics on formal proofs, and practitioners on templating automation. This framework attempts to bridge these gaps, creating an engine that is simultaneously standards-aligned, mathematically rigorous, and operationally viable. This section explores how our work relates to and departs from existing literature.

1.1.1 Standards-Track Topology Models

The networking industry has invested heavily in YANG as a ubiquitous data modelling language. At the structural level, RFC 8345 [5] and its augmentations—including RFC 8346 [6] (Layer 3), RFC 8944 [7] (Layer 2), and RFC 8795 [8] (Traffic Engineering)—define comprehensive, vendor-neutral, multi-layer topology models. These models standardise the concepts of nodes, links, and termination points. Further specifications such as RFC 8343 [9] and RFC 8344 [10] model device-level interface and IP management, while RFC 8299 [11] and RFC 8466 [12] model VPN service delivery.

Synthesis & Departure: Our model strictly adopts the structural primitives of RFC 8345; we use the exact concepts of nodes, endpoints (termination points), and edges. However, we fundamentally reject the deep, hierarchical nesting characteristic of YANG trees. Hierarchical document structures (like XML or JSON representations of YANG) are notoriously difficult to traverse recursively or query globally across a large graph database. By flattening these deep hierarchies into columnar, prefix-matched attribute maps, we trade the strict rigid schema validation of a YANG tree for high-performance, SIMD-accelerated graph traversals and relational algebraic processing. This decoupling of the protocol schema from its storage representation enables $O(1)$ relational queries while maintaining semantic alignment with the IETF standards.

1.1.2 Multi-Layer Network Theory

In the realm of pure mathematics and physics, De Domenico et al. [13] introduced the supra-adjacency tensor formalism to model complex interacting systems. This was subsequently extended by Kivelä et al. [14] and Boccaletti et al. [15] into a comprehensive multilayer network framework. Porter [16] provides an accessible survey of these concepts, and Buldyrev et al. [17] famously demonstrated how cascading failures propagate across interdependent infrastructure networks. Shchurov [18] specifically applied these conceptual models to computer networks.

Synthesis & Departure: The vast majority of multi-layer network theory focuses on *observational* analysis—studying failure rates or diffusion in existing social networks, transport grids, or biological systems. These theoretical models typically treat layers as structurally similar graphs of pure connectivity. In contrast, practical network engineering requires modelling highly specific, idiosyncratic protocol semantics (e.g., BGP route reflectors, OSPF area boundaries, EVPN route targets). Our contribution adapts this purely analytical mathematical framework into a *generative* software engineering tool. By encoding concrete network protocols as algebraic functor rules within the tensor, we transition from merely observing fate-sharing to proactively and deterministically computing it during the configuration compilation phase.

1.1.3 Category-Theoretic Frameworks for Composition

Applied category theory has increasingly been used to model compositional systems. Baez and Fong [19] developed frameworks for open networks using decorated cospans, which allows systems with explicit boundaries to be glued together rigorously. Fong's thesis [20] formalises the algebra of open interconnected systems, and Spivak [21] explores the operad of wiring diagrams. More recently, Baez and Courser [22] extend

these methods with structured cospans. Concurrently, Robinson [23] and Hansen [24] have demonstrated how sheaf theory can be used to analyse local-to-global consistency in networks.

Synthesis & Departure: Our approach is heavily inspired by these categorical semantics, specifically in how we define the network "plan" as a compositional structure and use Functors and Double Pushout (DPO) rewriting to map the configuration space into protocol-specific overlay graphs. However, while categorical models offer profound theoretical guarantees, they are often too abstract for practical software implementation by network engineers. We deliberately restrict our implementation to concrete graph transformations and relational DataFrames rather than full categorical abstraction. This ensures the model remains computationally tractable and can be implemented efficiently in systems languages like Rust.

1.1.4 Network Verification and Analysis

The last decade has seen a paradigm shift towards rigorous formal verification of network state. Early seminal work like Header Space Analysis (HSA) [25] enabled static checking of packet reachability, while systems like VeriFlow [26] pushed this into real-time verification of data-plane invariants. NetKAT [27] introduced a powerful Kleene algebra with tests to reason formally about SDN policies. At the control-plane level, Minesweeper [28] leverages SMT solvers to verify routing configurations, and Batfish [4, 29] has become the industry standard for parsing vendor-specific configurations to construct offline routing and forwarding information bases for pre-deployment analysis.

Synthesis & Departure: Industry-standard tools like Batfish are exceptionally powerful, but they operate primarily as *post-hoc verifiers*—they require the final vendor configuration text as input. Our algebraic graph model serves a complementary, upstream purpose. By capturing the complete intended state in a vendor-neutral Plan Topology (G_{plan}), our model is *constructive* (Correctness-by-Construction). It provides the mathematically rigorous graph substrate from which device configurations are generated, allowing the system to catch structural violations (like overlapping VRFs, disconnected routing loops, or missing MACsec ciphers) via algebraic compilation *before* a traditional verification tool would even have a vendor configuration to parse.

1.1.5 Intent-Based Configuration and Sources of Truth

In the operational domain, Intent-Based Networking (IBN) seeks to close the gap between high-level business logic and low-level device configuration. Research systems like Propane [30] compile abstract routing policies directly into BGP configurations, while Merlin [31] provisions bandwidth and path resources from declarative specifications. Lumi [32] explores natural-language interactions for managing intent. Commercially, tools like NetBox [3] and Nautobot [33] have become ubiquitous as relational Sources of Truth (SoT) for IP Address Management (IPAM) and asset inventory, and NAPALM [34] provides a unified API layer for multi-vendor interactions.

Synthesis & Departure: The critical bottleneck identified in IBN literature is the "translation phase"—reliably converting abstract intent into validated state. Existing IBN compilers often rely on complex, monolithic template engines (like Jinja2) that are structurally blind to the underlying topology of the network; they generate text, not state. Furthermore, current SoT databases are fundamentally flat relational models; they lack native graph traversal capabilities and struggle to model the recursive, multi-layered dependencies of modern routing protocols. Our algebraic model directly addresses this limitation. By unifying the structural connectivity of the network with the flat attribute data of a traditional SoT, the Plan Topology acts as the ultimate vendor-neutral "narrow waist" intermediate representation (IR), safely bridging the gap between high-level intent engines and low-level provisioning tools.

1.2 Mathematical Preliminaries

This report uses constructions from graph theory, multilayer network theory, and category theory. The following glossary provides working definitions aimed at network engineers; readers already familiar with these concepts may skip ahead.

> **Design Decision 3: Key Mathematical Concepts**
>
> *Appendix A provides detailed explanations, worked networking examples, and illustrated diagrams for each concept below.*

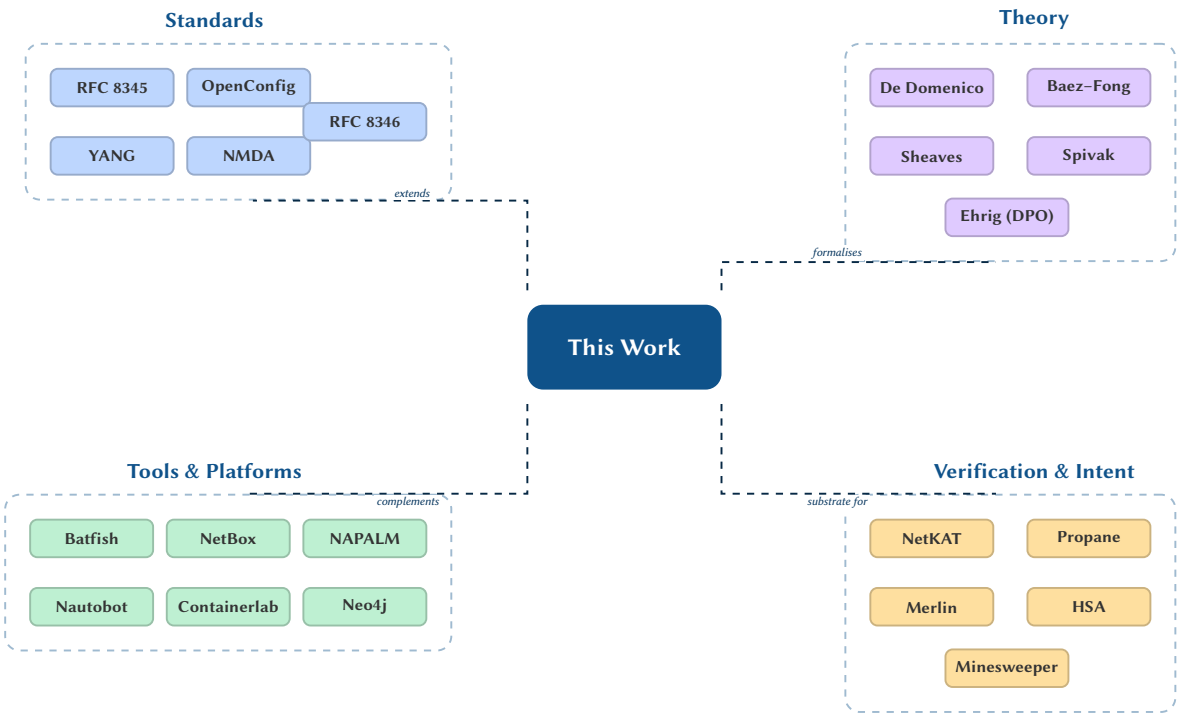


Figure 6. Related work landscape: positioning this model among existing standards, multilayer network theory, operational tooling, and formal verification approaches.

Term	Working Definition
Graph	A set of *vertices* (nodes) connected by *edges* (links). Our vertices are devices and interfaces; our edges are cables, protocol adjacencies, and ownership relations.
Hyperedge	An edge that connects *more than two* vertices simultaneously. A VLAN spanning 48 switch ports is naturally a single hyperedge rather than $\binom{48}{2}$ pairwise edges. See Section 14.6 for practical alternatives.
Property graph	A graph in which both vertices and edges carry key-value *properties* (attributes). Adding "hyperedge" gives us a *property hypergraph*.
Tensor	A multi-dimensional array that generalises matrices. A rank-4 tensor $M^{i\alpha}_{j\beta}$ is simply a matrix indexed by two pairs—here, (node, layer)—allowing one data structure to encode both intra-layer and inter-layer relationships.
Category	A collection of *objects* and *morphisms* (structure-preserving maps between objects) that compose associatively. Think of it as a type system for transformations: the objects are "kinds of graph" and the morphisms are "valid graph transformations."
Functor	A mapping between two categories that preserves structure. In this report, each layer functor $\mathcal{F}_\ell$ is a deterministic procedure that takes a plan topology (an object in the **Plan** category) and produces an overlay graph (an object in the **Overlay** category). Calling it a "functor" rather than a "function" ensures that composing two plan transformations and then computing the overlay gives the same result as computing overlays separately and composing them (the *composition law*).
Morphism	A structure-preserving map between objects in a category. For graphs, a morphism maps vertices to vertices and edges to edges while preserving adjacency—an attribute-preserving graph homomorphism.
DPO (Double Pushout)	A pattern-match-and-replace framework for graphs, analogous to find-and-replace in a text editor. A DPO rule specifies a *pattern* L to find, a *glue* K to preserve, and a *replacement* R to produce. We use DPO rules to describe how the overlay mapping transforms plan-topology fragments into protocol-specific edges.
Cospan	A diagram $A \to S \leftarrow B$ modelling an open system with boundary interfaces A and B and internal structure S. Two cospans compose by gluing along shared boundaries—useful for modular network verification ("verify each domain, then compose results").
Operad	An algebraic structure that formalises the composition of operations with typed inputs and outputs. In networking, each device is an operation whose "ports" (inputs/outputs) are typed interfaces, and the network topology is the wiring diagram connecting them.

2 The OSI Reference Model as a Graph Scaffold

The Open Systems Interconnection (OSI) reference model [35] defines seven layers of protocol abstraction. Each layer provides services to the layer above via *(N)-service access points* (SAPs) and maintains *(N)-connections* between peer entities.

We extend the classical seven layers with *sublayers* that reflect modern network engineering practice:

Insight:
The OSI SAP concept maps directly to inter-layer edges in a multilayer graph.

Table 1. Extended Layer Model with Sublayers.

Layer	Name	Sublayers	Key Protocols
1	Physical	PHY, Optics, Timing	802.3, SyncE, PTP
2	Data Link	MAC, VLAN, STP, LAG	802.1Q, 802.1AX, LLDP
2.5	Label Switching	MPLS, SR, EVPN	RFC 3031, 8402, 7432
3	Network	IP, VRF, Multicast	RFC 791, 8200, 4364
3.5	Routing	OSPF, IS-IS, BGP	RFC 2328, 1195, 4271
4	Transport	TCP, UDP, QUIC	RFC 9293, 768, 9000 [36]
5	Overlay	VXLAN, GRE, IPsec, Geneve	RFC 7348, 2784, 4301, 8926
6	Security	TLS, MACsec, 802.1X	RFC 8446, 802.1AE
7	Management	SNMP, NETCONF, gNMI	RFC 3411, 6241

Axiom 1 (Layer Independence). *Each layer in the extended model forms a* self-contained subgraph *with its own node set, edge set, and attribute schema. Inter-layer dependencies are represented exclusively by typed inter-layer edges, never by shared mutable state.* Note: *"independence" means no shared mutable state—layers may still have read-only data dependencies (e.g. BGP's next-hop resolution consults the IGP RIB). These dependencies are captured by the functor dependency DAG (Section 17.5.1).*

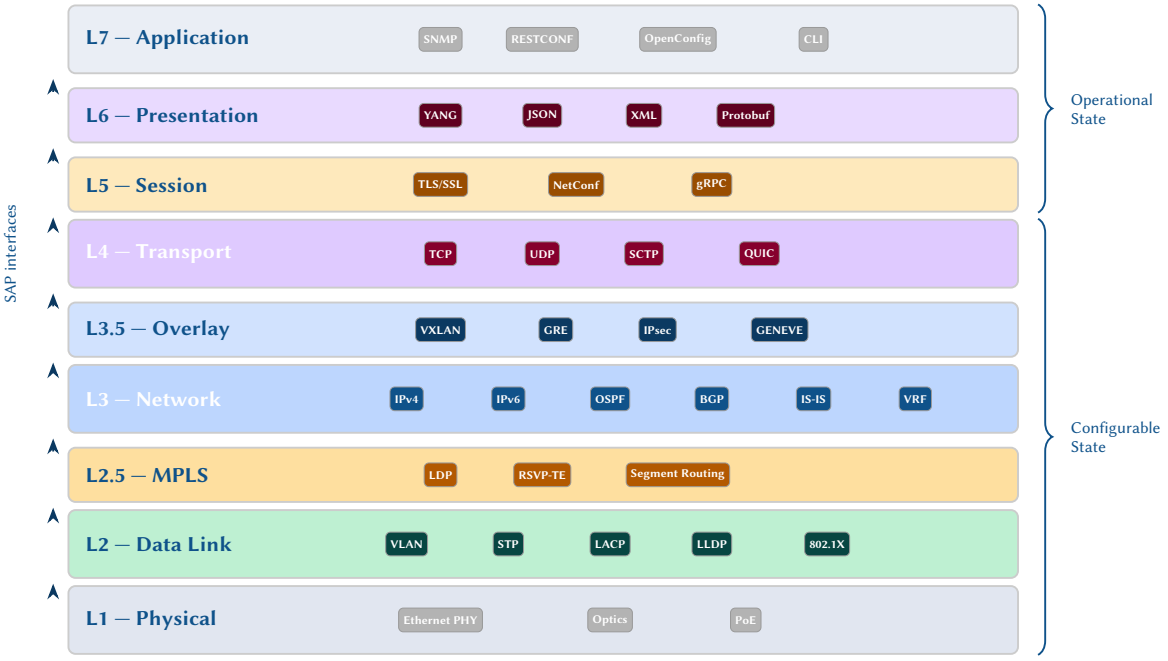


Figure 7. OSI layer stack with sublayers, protocol mappings, and state classification. Configurable state spans the lower layers (L1–L4), while operational state is derived at higher layers (L5–L7).

Design Rule 4: Table Notation

Attribute tables in Part II show *logical* types (struct, set, list, map). The NTE storage layer flattens these to scalar columns per the conventions in Section 1.

Exhaustive Protocol State Survey

Survey Organisation

This survey is organised in three tiers based on configuration importance for enterprise and service-provider networks. **Tier 1** covers the core routing, switching, and addressing protocols that define network forwarding

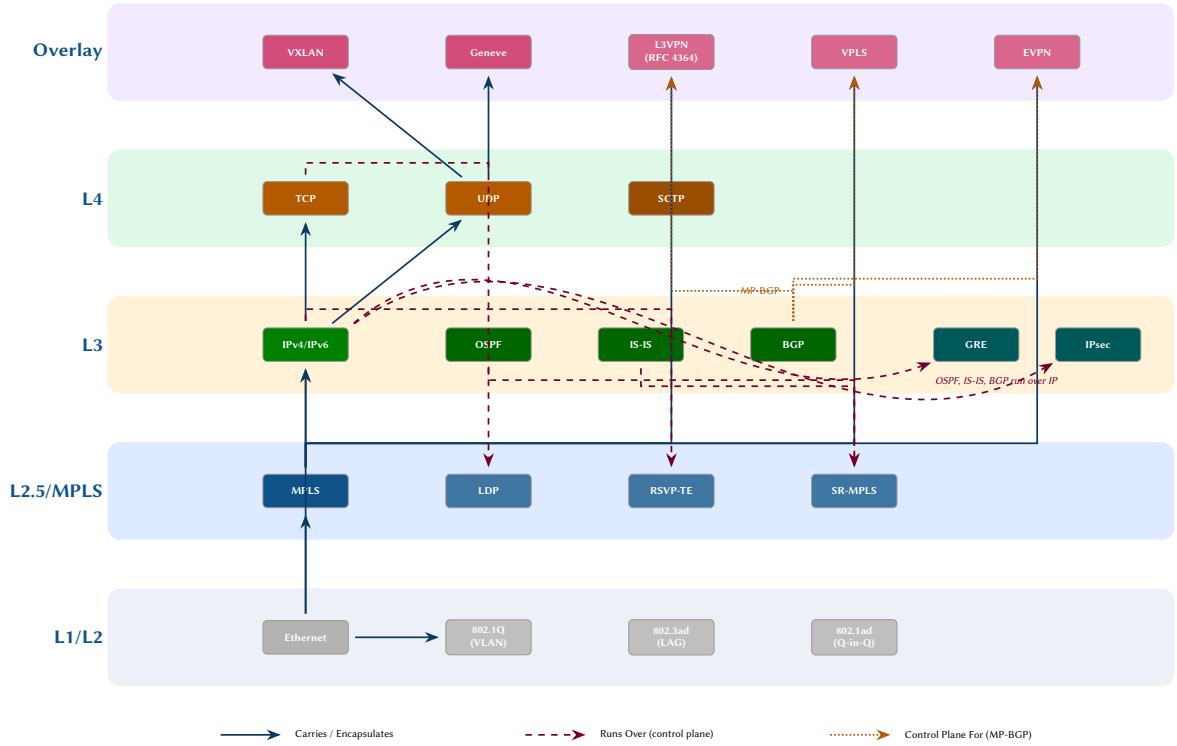


Figure 8. Protocol stacking and encapsulation relationships across the network stack. Solid arrows indicate data-plane encapsulation ("carries"), dashed arrows show control-plane dependencies ("runs over"), and dotted arrows indicate BGP-based signalling relationships. Protocols are colour-coded by layer: grey (L1/L2), blue (MPLS), green (L3), orange (L4), and purple (overlay/VPN).

behaviour. **Tier 2** covers overlay, transport, and multicast protocols that extend the core fabric. **Tier 3** covers supporting services, security mechanisms, and management protocols.

Tier 1: Core Routing and Switching

3 Layer 3: Network

3.1 IP Addressing

Every routed interface carries IPv4 and/or IPv6 addressing state. The plan topology stores addresses as flat attributes on endpoints, with subnet membership, secondary addresses, and anycast designations captured as additional keys. VRF membership (Section 3.2) determines the routing context in which each address participates.

Design Decision 4: IP Addressing Endpoint Attributes

Attribute	Type	Description
`ipv4_address`	ipv4/prefix	Primary IPv4 address [37]
`ipv4_secondary`	list	Secondary IPv4 addresses
`ipv6_address`	ipv6/prefix	Primary IPv6 address [38]; NDP [39]
`ipv6_link_local`	ipv6	Link-local IPv6 address (auto or manual)
`ipv6_autoconf`	bool	SLAAC enabled
`ip_mtu`	u16	IP MTU (distinct from L2 MTU)
`ip_directed_bcast`	bool	Directed broadcast forwarding
`proxy_arp`	bool	Proxy ARP [40] enabled
`ip_unnumbered_src`	str	Unnumbered interface source

Design Decision 5: ECMP / Load Balancing Node Attributes

Attribute	Type	Description
`ecmp_hash_algo`	enum	crc, xor, random, resilient
`ecmp_hash_fields`	set	src-ip, dst-ip, src-port, dst-port, protocol, ingress-port
`ecmp_hash_seed`	u32	Hash seed for polarisation avoidance
`ecmp_resilient`	bool	Resilient hashing enabled (minimise flow reassignment)
`ecmp_max_paths`	u8	Global maximum ECMP paths

Design Decision 6: Sub-interface Endpoint Attributes

Attribute	Type	Description
`subif_parent`	str	Parent physical interface name (e.g. `eth1`)
`subif_encap`	enum	Encapsulation type: `dot1q` or `qinq`
`subif_vlan_id`	u16	Outer VLAN tag (802.1Q VLAN identifier)
`subif_inner_vlan`	u16	Inner VLAN tag (QinQ only; 0 if single-tagged)

Design Decision 7: IRB/SVI Endpoint Attributes

Attribute	Type	Description
`irb_vlan`	u16	Associated VLAN identifier
`irb_anycast_gateway`	ip	Distributed anycast gateway IP (EVPN fabric)
`irb_mac`	mac	Physical MAC address of the IRB/SVI
`irb_virtual_mac`	mac	Virtual MAC address (shared across leaf switches)

3.2 VRF

Virtual Routing and Forwarding (VRF) [41] provides L3 routing isolation.

Design Decision 8: VRF Attributes

Attribute	Type	Description
Node-level (per VRF instance):		
`vrf_name`	str	VRF name (or "default" for global table)
`vrf_rd`	str	Route Distinguisher (Type 0: ASN:NN, Type 1: IP:NN)
`vrf_rt_import`	list	Import Route Targets
`vrf_rt_export`	list	Export Route Targets
`vrf_description`	str	Administrative description
`vrf_route_leaking`	list	Leaked routes from other VRFs
Endpoint-level:		
`vrf_membership`	str	VRF this interface belongs to

3.3 First-Hop Redundancy

Virtual Router Redundancy Protocol (VRRP) [42] and vendor equivalents provide gateway redundancy.

Insight:
Route Distinguishers (RDs) make prefixes unique across VRFs inside BGP; Route Targets (RTs) control which VRFs import which routes. Three encoding types exist: ***Type 0*** *(2-byte ASN : 4-byte NN),* ***Type 1*** *(IPv4 : 2-byte NN),* ***Type 2*** *(4-byte ASN : 2-byte NN). RT import/export lists on each VRF drive route leaking: a VRF exports routes tagged with its export RT, and imports any route carrying a matching import RT. In* hub-and-spoke *designs, spoke VRFs export with an RT that only the hub imports, while the hub exports with an RT all spokes import—enforcing traffic tromboning through a central policy point. In* full-mesh *designs, all VRFs share the same import and export RT, granting any-to-any reachability.*

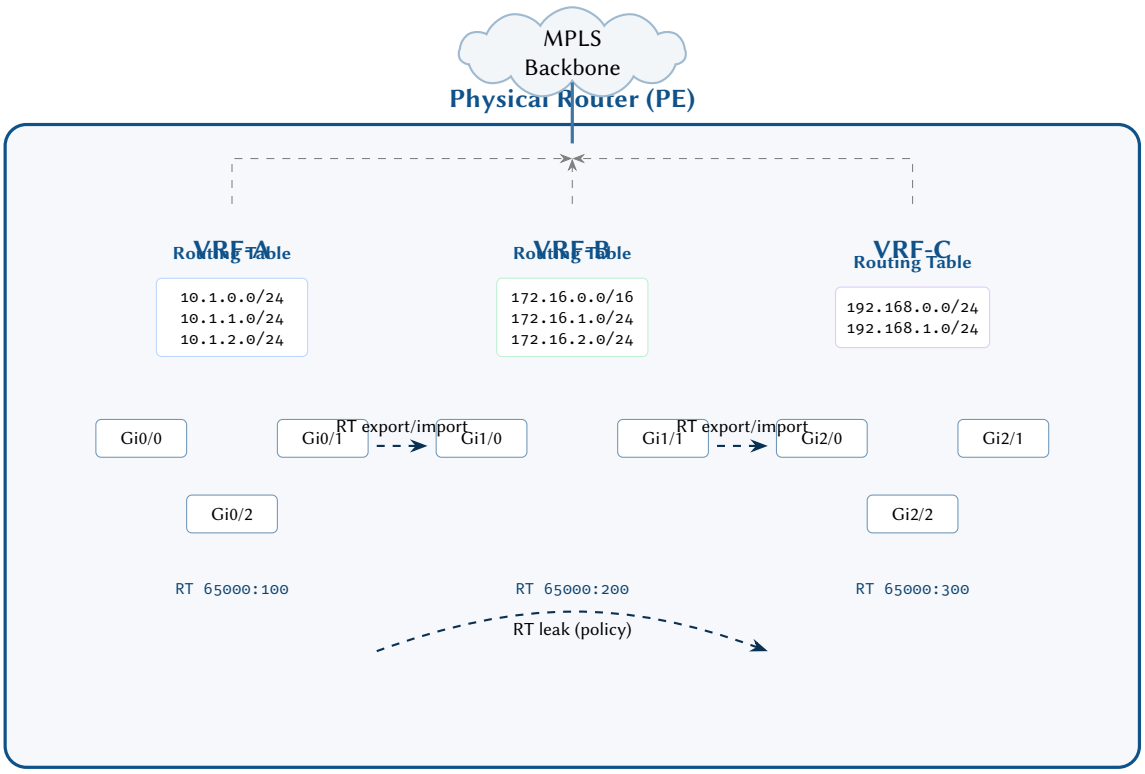


Figure 9. VRF isolation within a single physical PE router. Each VRF maintains an independent routing table and interface set. Route Target (RT) import/export policies control selective route leaking between VRFs, with MPLS providing backbone connectivity.

Design Decision 9: FHRP Endpoint Attributes

Attribute	Type	Description
`vrrp_group_id`	u8	VRRP group number (0–255)
`vrrp_virtual_ip`	ip	Virtual IP address
`vrrp_priority`	u8	Priority (1–254, default 100)
`vrrp_preempt`	bool	Preemption enabled
`vrrp_adv_interval`	u16	Advertisement interval (centiseconds)
`vrrp_state`	enum	initialize, backup, master
`vrrp_tracked_intf`	list	Tracked interfaces with decrement values
`vrrp_version`	enum	v2, v3

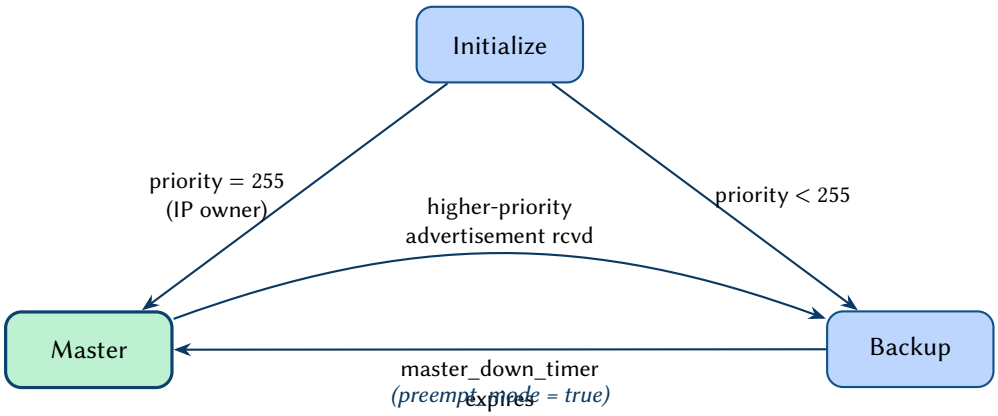


Figure 10. VRRP state machine (RFC 5798). A router in the `Initialize` state transitions directly to `Master` if it owns the virtual IP (priority 255), otherwise to `Backup`. The `Master` state (green) is relinquished when a higher-priority advertisement is received and preemption is enabled.

3.4 BFD

Bidirectional Forwarding Detection (BFD) [43] provides sub-second failure detection.

Design Decision 10: BFD Endpoint Attributes

Attribute	Type	Description
`bfd_enabled`	bool	BFD enabled on this interface
`bfd_min_tx`	u32	Minimum TX interval (μs)
`bfd_min_rx`	u32	Minimum RX interval (μs)
`bfd_multiplier`	u8	Detection multiplier
`bfd_mode`	enum	async, demand
`bfd_echo`	bool	Echo mode enabled
`bfd_state`	enum	admin-down, down, init, up
`bfd_multihop`	bool	Multihop BFD session

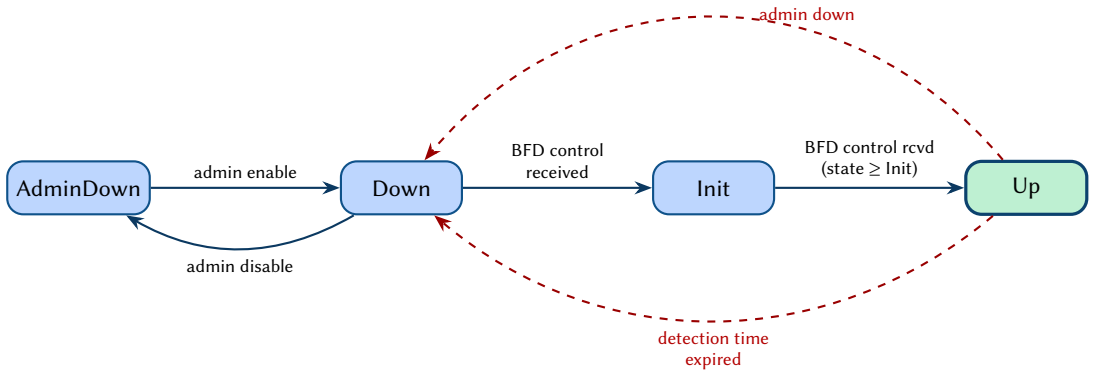


Figure 11. BFD session state machine (RFC 5880). The four states progress from `Down` through `Init` to the operational `Up` state (green). Dashed red arcs indicate failure transitions triggered by detection-time expiry or administrative shutdown.

3.5 NAT

Network Address Translation (NAT) [44] translates addresses between network domains.

Design Decision 11: NAT Endpoint Attributes

Attribute	Type	Description
`nat_enabled`	bool	NAT enabled on this interface
`nat_direction`	enum	inside, outside
`nat_type`	enum	static, dynamic, pat (port address translation)
`nat_pool_name`	str	NAT pool name (dynamic NAT)
`nat_pool_start`	ip	Pool start address
`nat_pool_end`	ip	Pool end address
`nat_overload`	bool	PAT / many-to-one overload
`nat_acl`	str	ACL defining NAT-eligible traffic
`nat64_enabled`	bool	NAT64 (IPv6-to-IPv4 translation)
`nat64_prefix`	prefix	NAT64 well-known or configured prefix

3.6 DHCP

The Dynamic Host Configuration Protocol (DHCP) automates IP address assignment for hosts. From the plan topology's perspective, the relevant configuration state comprises relay agent settings on infrastructure interfaces and server scope definitions on DHCP servers. DHCP lease tables are operational state—addresses discovered via DHCP are reflected in the IP addressing attributes of the receiving endpoint.

Design Decision 12: DHCP Endpoint Attributes

Attribute	Type	Description
`dhcp_relay_enabled`	bool	DHCP relay agent enabled
`dhcp_relay_servers`	list	Relay destination server addresses
`dhcp_server_pool`	str	DHCP pool name (if acting as server)
`dhcpv6_relay`	bool	DHCPv6 relay enabled

3.7 Forwarding Tables

Layer 2 and Layer 3 forwarding tables represent operational state derived at runtime by protocol operation (see Design Rule 1). They are included here for survey completeness; the overlay functors $\mathcal{F}_\ell$ compute equivalent forwarding state from configuration inputs.

Design Decision 13: MAC Address Table (Forwarding Database)

Attribute	Type	Description
`mac_address`	str	MAC address (e.g. 00:11:22:33:44:55)
`mac_vlan`	u16	VLAN in which the MAC was learned
`mac_port`	str	Port/interface where the MAC resides
`mac_type`	enum	dynamic, static, secure, system
`mac_age_seconds`	u32	Time since last seen (dynamic entries)

Design Decision 14: ARP / NDP Neighbor Table

Attribute	Type	Description
`neighbor_ip`	ip	Neighbour IP address
`neighbor_mac`	str	Resolved MAC address
`neighbor_interface`	str	Interface where neighbour was learned
`neighbor_state`	enum	reachable, stale, delay, probe, incomplete (NDP)
`neighbor_type`	enum	dynamic, static
`neighbor_vrf`	str	VRF context
`neighbor_age`	u32	Age in seconds

Design Decision 15: FIB / TCAM Utilization Node Attributes

Attribute	Type	Description
`fib_ipv4_used`	u32	IPv4 FIB entries in hardware
`fib_ipv4_max`	u32	IPv4 FIB capacity
`fib_ipv6_used`	u32	IPv6 FIB entries in hardware
`fib_ipv6_max`	u32	IPv6 FIB capacity
`tcam_acl_used`	u32	ACL TCAM entries used
`tcam_acl_max`	u32	ACL TCAM capacity
`lfib_used`	u32	MPLS LFIB entries used
`lfib_max`	u32	MPLS LFIB capacity
`mac_table_used`	u32	MAC table entries used
`mac_table_max`	u32	MAC table capacity

3.8 Multicast (PIM, IGMP/MLD)

Protocol Independent Multicast (PIM) [45] provides multicast routing independent of the underlying unicast protocol. Internet Group Management Protocol (IGMP) [46] (IPv4) and Multicast Listener Discovery (MLD) [47] (IPv6) manage group membership on local segments. Multicast Source Discovery Protocol (MSDP) [48] enables inter-domain multicast source discovery.

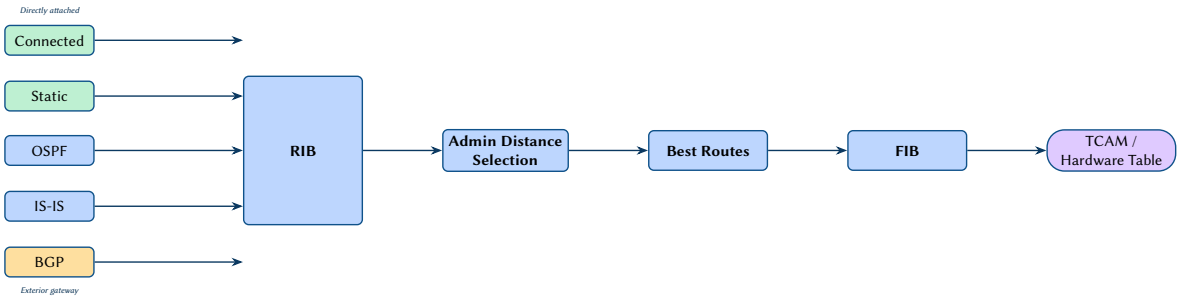


Figure 12. RIB/FIB route-selection pipeline. Multiple routing protocol sources (colour-coded by type) inject prefixes into the RIB. Administrative-distance selection and best-path computation produce the FIB, which is programmed into hardware forwarding tables (TCAM).

Design Decision 16: PIM Node Attributes

Attribute	Type	Description
`pim_enabled`	bool	PIM enabled globally
`pim_rp_address`	ip	Rendezvous Point address (static or BSR/Auto-RP)
`pim_rp_mode`	enum	static, bsr, auto-rp
`pim_ssm_range`	str	Source-Specific Multicast group range
`pim_spt_threshold`	enum	0 (immediate), infinity (never), kbps value
`msdp_enabled`	bool	MSDP enabled
`msdp_peer`	ip	MSDP peer address

Design Decision 17: PIM Endpoint Attributes

Attribute	Type	Description
`pim_intf_enabled`	bool	PIM enabled on this interface
`pim_mode`	enum	sparse, dense, sparse-dense, ssm
`pim_dr_priority`	u32	Designated Router election priority
`pim_hello_interval`	u16	Hello interval (seconds)
`pim_neighbor_count`	u16	Number of PIM neighbours (operational)
`pim_bsr_border`	bool	BSR boundary on this interface

Design Decision 18: IGMP/MLD Endpoint Attributes

Attribute	Type	Description
`igmp_enabled`	bool	IGMP enabled on this interface
`igmp_version`	enum	v1, v2, v3 [46]
`igmp_query_interval`	u16	Query interval (seconds)
`igmp_max_response`	u16	Max query response time (tenths of seconds)
`igmp_snooping`	bool	IGMP snooping enabled (L2)
`mld_enabled`	bool	MLD enabled (IPv6) [47]
`mld_version`	enum	v1, v2

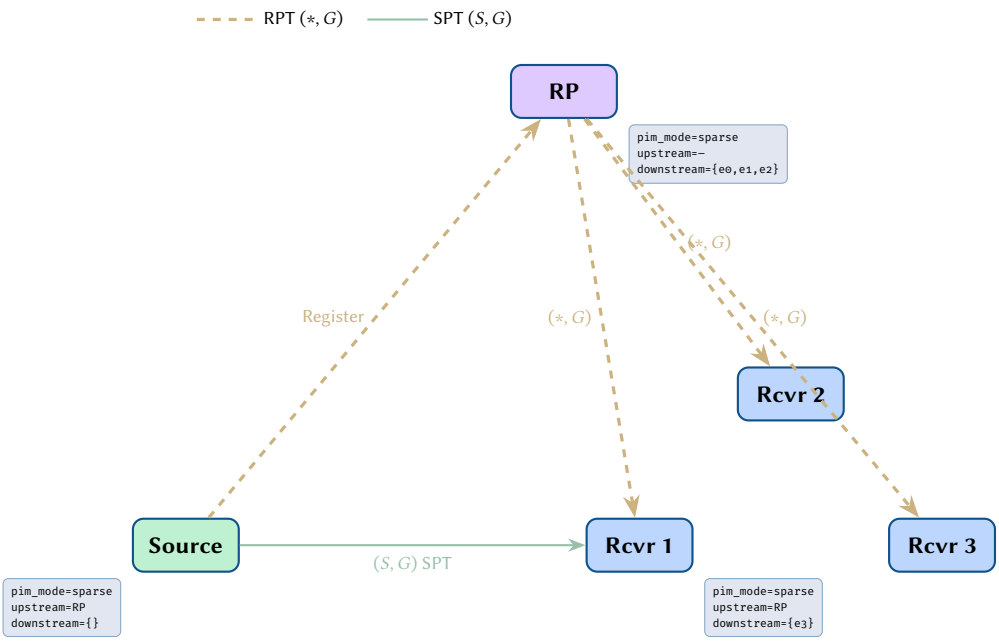


Figure 13. Multicast tree state: RPT $(*, G)$ dashed vs SPT (S, G) solid paths with PIM router attributes.

4 Layer 3.5: Routing Protocols

4.1 OSPF

Open Shortest Path First (OSPF) [49, 50] is a link-state IGP that computes shortest paths from a synchronized link-state database.

Design Decision 19: OSPF Node Attributes

Attribute	Type	Description
`ospf_enabled`	bool	OSPF process enabled
`ospf_version`	enum	v2 [49], v3 [50]
`ospf_router_id`	ipv4	OSPF router ID
`ospf_process_id`	u16	Process/instance ID
`ospf_ref_bandwidth`	u32	Reference bandwidth for cost calculation (Mbps)
`ospf_spf_delay`	u32	SPF computation delay (ms)
`ospf_graceful_restart`	bool	Graceful restart enabled
`ospf_max_paths`	u8	Maximum ECMP paths
`ospf_default_originate`	enum	none, always, conditional
`ospf_redistribute`	list	Redistributed protocols with route-map

Design Decision 20: OSPF Endpoint Attributes

Attribute	Type	Description
`ospf_area`	u32/dotted	Area ID (e.g. 0, 0.0.0.1)
`ospf_area_type`	enum	normal, stub, totally-stub, nssa, totally-nssa
`ospf_network_type`	enum	broadcast, nbma, p2p, p2mp
`ospf_cost`	u16	Interface cost (manual override)
`ospf_priority`	u8	DR election priority (0 = never DR)
`ospf_hello_interval`	u16	Hello interval (seconds)
`ospf_dead_interval`	u16	Dead interval (seconds)
`ospf_retransmit_interval`	u16	Retransmit interval (seconds)
`ospf_passive`	bool	Passive interface (no hellos)
`ospf_authentication`	enum	none, simple, md5, keychain
`ospf_bfd`	bool	BFD-enabled for OSPF on this interface
`ospf_neighbor_state`	enum	down, attempt, init, 2-way, exstart, exchange, loading, full (operational)
`ospf_dr`	ipv4	Designated Router (operational)
`ospf_bdr`	ipv4	Backup Designated Router (operational)

OSPF Multi-Area with Stub Configuration

A three-router topology demonstrates OSPF multi-area operation. Router `R1` (`ospf_router_id = 10.0.0.1`) sits in Area 0 (backbone) with `ospf_area_type = normal`. Router `R2` is an ABR connecting Area 0 and Area 51. Router `R3` resides solely in Area 51, configured as `ospf_area_type = totally-stub`.

- On the `R1`–`R2` link (Area 0), both endpoints carry `ospf_area = 0`, `ospf_network_type = p2p`, and `ospf_cost = 10` (derived from `ospf_ref_bandwidth = 100000` and a 10 Gbps link).
- `R2`'s interface toward `R3` carries `ospf_area = 51`. As an ABR, `R2` generates Type-3 summary LSAs into Area 51—but because Area 51 is totally-stub, it injects only a default route.
- `R3` has `ospf_priority = 0` (never DR) on its single p2p link and receives all external reachability via the ABR's default.

The plan topology captures this as endpoint-level attributes (`ospf_area`, `ospf_cost`, `ospf_area_type`) without modelling LSA flooding—that is derived state computed by the $\mathcal{F}_{\text{OSPF}}$ overlay functor.

4.2 IS-IS

Intermediate System to Intermediate System (IS-IS) [51, 52] is the other major link-state IGP, preferred in large SP networks.

Design Decision 21: IS-IS Node Attributes

Attribute	Type	Description
`isis_enabled`	bool	IS-IS enabled
`isis_net`	str	Network Entity Title (NET), e.g. 49.0001.1921.6800.1001.00
`isis_level`	enum	L1, L2, L1L2
`isis_overload_bit`	bool	Overload bit set (ISO 10589)
`isis_metric_style`	enum	narrow, wide, transition
`isis_max_paths`	u8	Maximum ECMP paths
`isis_sr_enabled`	bool	IS-IS for SR [53]

Design Decision 22: IS-IS Endpoint Attributes

Attribute	Type	Description
`isis_circuit_type`	enum	L1, L2, L1L2
`isis_metric`	u32	Interface metric (wide)
`isis_hello_interval`	u16	Hello interval (seconds)
`isis_hello_multiplier`	u8	Hold time = hello × multiplier
`isis_network_type`	enum	broadcast, p2p
`isis_passive`	bool	Passive interface
`isis_csnp_interval`	u16	CSNP interval (seconds)
`isis_priority`	u8	DIS election priority
`isis_authentication`	enum	none, md5, keychain
`isis_bfd`	bool	BFD-enabled for IS-IS

4.3 BGP

Border Gateway Protocol (BGP) [54] is the inter-domain path-vector routing protocol used for both Internet routing and data-centre overlays.

Design Decision 23: BGP Node Attributes

Attribute	Type	Description
`bgp_enabled`	bool	BGP process enabled
`bgp_asn`	u32	Local AS number (2-byte or 4-byte)
`bgp_router_id`	ipv4	BGP router ID
`bgp_cluster_id`	ipv4	Route reflector [55] cluster ID
`bgp_confederation_id`	u32	Confederation identifier [56]
`bgp_confederation_peers`	list	Confederation member ASNs
`bgp_bestpath_compare_routerid`	bool	Tiebreak on router ID
`bgp_bestpath_med_always_compare`	bool	Compare MED across ASes
`bgp_graceful_restart`	bool	Graceful restart [57]
`bgp_max_paths_ebgp`	u8	ECMP for eBGP
`bgp_max_paths_ibgp`	u8	ECMP for iBGP
`bgp_default_originate`	bool	Originate default route
`bgp_redistribute`	list	Redistributed protocols

BGP peer state is represented as attributes on a logical endpoint representing the peering session:

Design Decision 24: BGP Peer Endpoint Attributes

Attribute	Type	Description
`bgp_peer_address`	ip	Neighbour IP address
`bgp_peer_asn`	u32	Neighbour AS number
`bgp_peer_type`	enum	ibgp, ebgp
`bgp_peer_state`	enum	idle, connect, active, opensent, openconfirm, established (operational)
`bgp_peer_group`	str	Peer-group template name
`bgp_peer_update_source`	str	Update-source interface
`bgp_peer_multihop_ttl`	u8	eBGP multihop TTL
`bgp_peer_password`	str	TCP-MD5 / TCP-AO password (hashed)
`bgp_peer_hold_time`	u16	Hold timer (seconds)
`bgp_peer_keepalive`	u16	Keepalive timer (seconds)
`bgp_peer_rr_client`	bool	Route reflector client
`bgp_peer_next_hop_self`	bool	Set next-hop to self
`bgp_peer_remove_private_as`	bool	Remove private AS
`bgp_peer_soft_reconfig`	bool	Soft reconfiguration inbound
`bgp_peer_add_paths`	enum	send, receive, both
`bgp_peer_bfd`	bool	BFD for this BGP session
Per address-family on peer [58]:		
`bgp_afi_safi`	list	Enabled AFI/SAFI: ipv4-unicast, ipv6-unicast, vpnv4, vpnv6, evpn, l2vpn, flowspec [59]
`bgp_peer_route_map_in`	str	Inbound route-map name
`bgp_peer_route_map_out`	str	Outbound route-map name
`bgp_peer_prefix_list_in`	str	Inbound prefix-list
`bgp_peer_prefix_list_out`	str	Outbound prefix-list
`bgp_peer_max_prefixes`	u32	Maximum prefix limit
`bgp_peer_default_originate`	bool	Send default route to this peer
`bgp_peer_send_community`	enum	standard, extended, large, all

BGP Route Reflection with Cluster ID

A route reflector `RR1` (`bgp_router_id = 10.0.0.1`, `bgp_cluster_id = 10.0.0.1`) serves three iBGP clients: `PE1`, `PE2`, and `PE3`, all in AS 65000.

- Each client has a BGP session endpoint with `bgp_peer_type = internal` and `bgp_peer_rr_client = true` on `RR1`'s side.
- When `PE1` advertises prefix `10.100.0.0/16` with `bgp_peer_send_community = all`, `RR1` reflects the route to `PE2` and `PE3`, appending its `bgp_cluster_id` to the CLUSTER_LIST and setting the ORIGINATOR_ID to `PE1`'s router-id.
- `PE2` receives the reflected route and runs the best-path algorithm (Figure 88B): Weight → LOCAL_PREF → locally-originated → AS_PATH length → ORIGIN → MED → eBGP-preference → IGP cost to next-hop.
- If `PE2` also has an eBGP path to the same prefix, the "eBGP over iBGP" tiebreaker at step 7 selects the eBGP path.

In the plan topology, the RR relationship is captured by `bgp_peer_rr_client` on the session endpoint and `bgp_cluster_id` on the node—no separate "RR cluster" entity is required.

4.4 Static Routes and Policy

Static routes provide manually configured forwarding entries that override or supplement dynamic routing protocol decisions. They are commonly used for default routes, summary routes injected at redistribution boundaries, and null routes for blackhole filtering. Route-maps, the primary policy control mechanism, are referenced by name throughout the routing configuration and covered in full detail in Section 7.

Design Decision 25: Static Route and Policy Node Attributes

Attribute	Type	Description
`static_routes`	list	List of {prefix, next-hop, metric, VRF, tag, BFD}
`route_maps`	list	Named route-map sequences with match/set clauses
`prefix_lists`	list	Named prefix-list entries (permit/deny, le/ge)
`community_lists`	list	Community match lists
`as_path_filters`	list	AS-path regular expression filters
`pbr_policies`	list	Policy-based routing: match → set next-hop

Design Decision 26: Route-Map Entry Attributes (per sequence)

Insight: *Route policy constructs (route-maps, prefix-lists, community-lists) are complex multi-entry structures. In the flat model, each entry is indexed: e.g. `route_map_PEER_IN_seq_10_match = prefix-list PL1`, `route_map_PEER_IN_seq_10_set = local-preference 200`.*

Attribute	Type	Description
`rmap_name`	str	Route-map name
`rmap_seq`	u16	Sequence number
`rmap_action`	enum	permit, deny
`rmap_match_prefix`	str	Match prefix-list name
`rmap_match_community`	str	Match community-list name
`rmap_match_as_path`	str	Match AS-path filter name
`rmap_set_localpref`	u32	Set local-preference
`rmap_set_med`	u32	Set MED
`rmap_set_community`	str	Set community value
`rmap_set_nexthop`	ip	Set next-hop address
`rmap_set_weight`	u16	Set weight (Cisco-specific, but widely used)

5 Layer 2: Data Link

5.1 VLAN State (IEEE 802.1Q)

IEEE 802.1Q-2022 [60] defines VLAN tagging, trunking, and the consolidated spanning tree protocols.

Design Decision 27: VLAN Endpoint Attributes

Attribute	Type	Description
`switchport_mode`	enum	access, trunk, hybrid, dot1q-tunnel
`access_vlan`	u16	Access VLAN ID (1–4094)
`native_vlan`	u16	Native/untagged VLAN on trunk
`allowed_vlans`	set	Set of allowed VLAN IDs on trunk
`voice_vlan`	u16	Voice VLAN for IP telephony
`pvlan_type`	enum	primary, isolated, community (RFC 5517) [61]
`pvlan_association`	u16	Associated private VLAN
`qinq_outer_vlan`	u16	Outer S-tag for 802.1ad provider bridging

5.2 Spanning Tree State

The Spanning Tree Protocol (STP) family [60, 62] prevents Layer 2 loops.

Insight: *VLANs are naturally modelled as hyperedges connecting all member endpoints, not as attributes on individual edges. In the plan topology, VLAN membership is a flat attribute on each endpoint. In implementations that use binary graph storage, each VLAN hyperedge maps to a junction node connected via `MEMBER_OF` edges (see Section 14.6).*

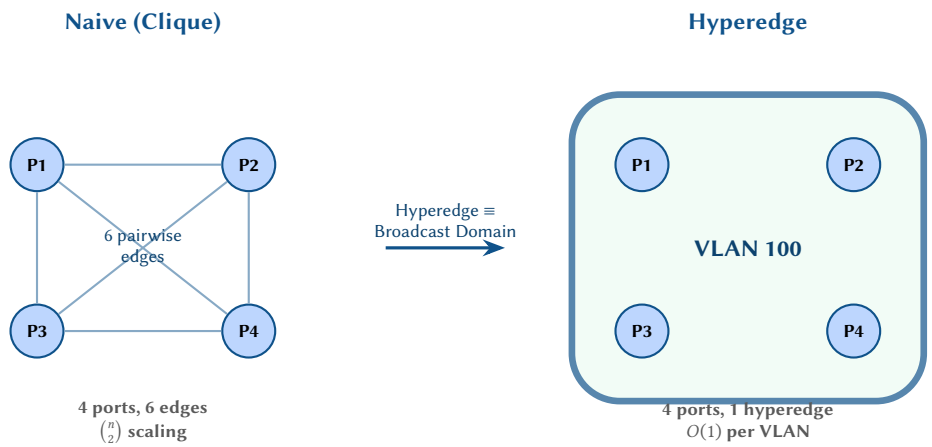


Figure 14. A VLAN modelled as a clique of pairwise edges (left) versus a single hyperedge (right). The hypergraph representation captures the broadcast-domain semantics directly: one hyperedge per VLAN avoids the $\binom{n}{2}$ edge explosion and preserves the set-membership relationship between ports and their VLAN.

Design Decision 28: STP Endpoint Attributes

Attribute	Type	Description
`stp_mode`	enum	stp, rstp, mstp, none
`stp_port_state`	enum	disabled, blocking, listening, learning, forwarding (STP) (operational – not configuration) or discarding, learning, forwarding (RSTP)
`stp_port_role`	enum	root, designated, alternate, backup, disabled (operational – not configuration)
`stp_port_priority`	u8	0–240 (multiples of 16)
`stp_port_cost`	u32	Path cost (auto or manual)
`stp_guard`	enum	none, root-guard, bpdu-guard, loop-guard
`stp_portfast`	bool	Edge port / PortFast enabled

Node-level STP attributes:

Design Decision 29: STP Node Attributes

Attribute	Type	Description
`stp_bridge_priority`	u16	Bridge priority (0–61440, multiples of 4096)
`stp_root_bridge`	bool	Is this bridge the current root?
`stp_root_path_cost`	u32	Cost to reach the root bridge
`mstp_region_name`	str	MSTP region name
`mstp_revision`	u16	MSTP region revision
`mstp_instance_map`	map	MSTI → VLAN set mapping

5.3 Link Aggregation (IEEE 802.1AX)

Link Aggregation Group (LAG)/Link Aggregation Control Protocol (LACP) [63] bundles multiple physical links.

Design Decision 30: LAG/LACP Attributes

Attribute	Type	Description
On the LAG logical endpoint:		
`lag_id`	u16	LAG/port-channel number
`lag_mode`	enum	static, lacp-active, lacp-passive
`lag_min_links`	u8	Minimum active links before bundle goes down
`lag_hash_algo`	enum	src-dst-ip, src-dst-mac, src-dst-port, …
`lag_system_priority`	u16	LACP system priority
On each member endpoint:		
`lag_member_of`	u16	Parent LAG ID
`lacp_port_priority`	u16	LACP port priority
`lacp_rate`	enum	slow (30s), fast (1s)
`lacp_actor_state`	u8	LACP state flags (activity, timeout, aggregation, …)
`lacp_partner_state`	u8	Peer's LACP state flags

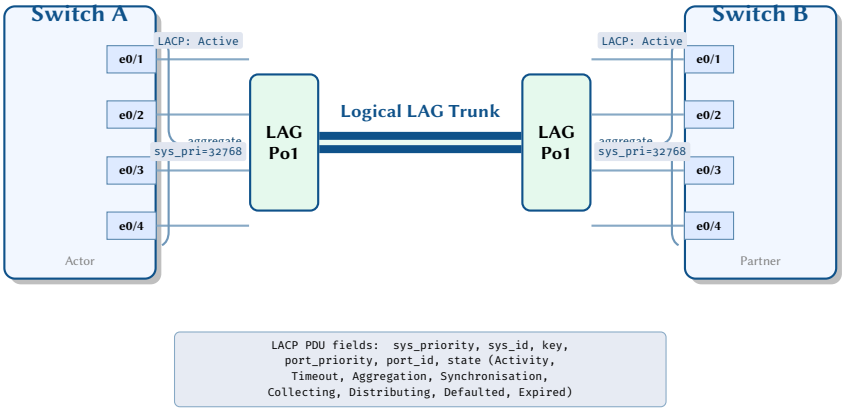


Figure 15. Link Aggregation (LAG) with LACP. Four physical member links on each switch are aggregated into a single logical port-channel (Po1). LACP PDUs exchanged on each member negotiate actor/partner state to ensure both ends agree on aggregation membership before enabling traffic distribution.

5.4 LLDP (IEEE 802.1AB)

Link Layer Discovery Protocol (LLDP) [64] provides neighbour discovery.

Design Decision 31: LLDP Endpoint Attributes

Attribute	Type	Description
`lldp_enabled`	bool	LLDP transmit/receive enabled
`lldp_neighbor_chassis`	str	Discovered neighbour chassis ID
`lldp_neighbor_port`	str	Discovered neighbour port ID
`lldp_neighbor_sysname`	str	Discovered neighbour system name
`lldp_capabilities`	set	bridge, router, telephone, …
`lldp_mgmt_address`	str	Neighbour management address

5.5 Layer 2 Security

Port-level security state:

Design Decision 32: L2 Security Endpoint Attributes

Attribute	Type	Description
`port_security_enabled`	bool	MAC-based port security
`port_security_max_mac`	u16	Maximum allowed MAC addresses
`port_security_violation`	enum	protect, restrict, shutdown
`dot1x_enabled`	bool	802.1X port-based NAC [65]
`dot1x_role`	enum	authenticator, supplicant
`dot1x_auth_state`	enum	unauthorized, authorized, force-authorized
`dot1x_host_mode`	enum	single-host, multi-host, multi-domain, multi-auth
`dot1x_eap_method`	enum	md5, tls, peap, ttls, fast
`dot1x_reauth_enabled`	bool	Periodic reauthentication
`dot1x_reauth_period`	u32	Reauthentication interval (seconds)
`dot1x_guest_vlan`	u16	VLAN for unauthenticated hosts
`dot1x_authfail_vlan`	u16	VLAN on authentication failure
`dot1x_critical_vlan`	u16	VLAN when RADIUS unreachable
`dot1x_mab_enabled`	bool	MAC Authentication Bypass fallback
`dhcp_snooping_trust`	bool	DHCP snooping trust state
`dai_enabled`	bool	Dynamic ARP Inspection
`ipsg_enabled`	bool	IP Source Guard
`storm_ctrl_bcast_pps`	u32	Broadcast storm control threshold
`storm_ctrl_mcast_pps`	u32	Multicast storm control threshold
`storm_ctrl_ucast_pps`	u32	Unknown unicast storm control threshold
`macsec_enabled`	bool	MACsec (802.1AE) [66]
`macsec_cipher`	enum	GCM-AES-128, GCM-AES-256, GCM-AES-XPN-256

5.6 Ethernet OAM

CFM (802.1ag) [67] and Y.1731 provide fault and performance management:

Design Decision 33: Ethernet OAM Endpoint Attributes

Attribute	Type	Description
`cfm_md_name`	str	Maintenance Domain name
`cfm_md_level`	u8	MD level (0–7)
`cfm_ma_name`	str	Maintenance Association name
`cfm_mep_id`	u16	Maintenance Endpoint ID
`cfm_direction`	enum	up, down
`cfm_ccm_interval`	enum	3.3ms, 10ms, 100ms, 1s, 10s, 1min, 10min
`y1731_dm_enabled`	bool	Delay measurement
`y1731_lm_enabled`	bool	Loss measurement

6 Layer 2.5: Label Switching

6.1 MPLS

The Multiprotocol Label Switching (MPLS) architecture [68, 69] creates label-switched overlay paths. LSP Ping [70] provides MPLS data-plane failure detection.

Design Decision 34: MPLS Node Attributes

Attribute	Type	Description
`mpls_enabled`	bool	MPLS forwarding enabled globally
`mpls_router_id`	ipv4	MPLS router ID (usually loopback)
`mpls_label_range`	range	Static/dynamic label range (e.g. 16–1048575)
`mpls_ttl_propagate`	bool	Propagate IP TTL into MPLS header
`mpls_php_enabled`	bool	Penultimate Hop Popping

Design Decision 35: MPLS Endpoint Attributes

Attribute	Type	Description
`mpls_interface_enabled`	bool	MPLS enabled on this interface
`mpls_mtu`	u16	MPLS MTU (IP MTU + label stack)

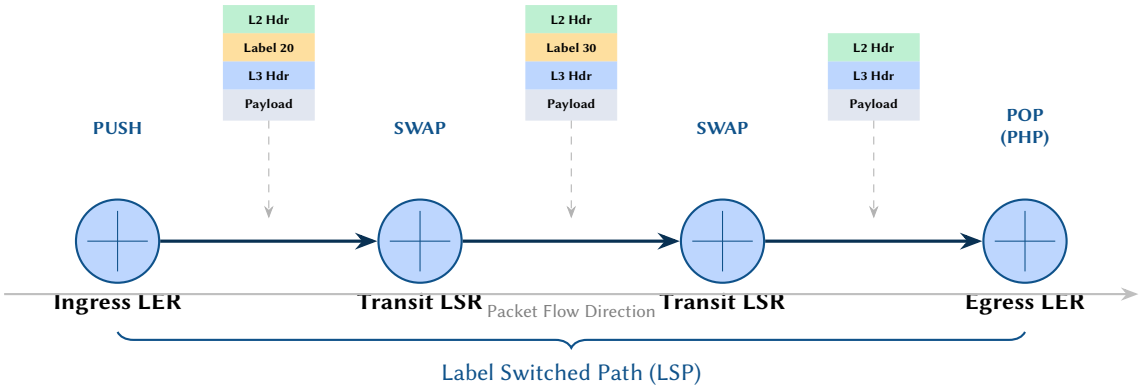


Figure 16. MPLS label stack operations along a Label Switched Path (LSP). The ingress LER pushes a label, transit LSRs swap labels, and the egress LER pops the final label via Penultimate Hop Popping (PHP).

6.2 LDP

The Label Distribution Protocol (LDP) [71] automatically assigns and distributes MPLS labels for IGP-learned prefixes. See Appendix B for detailed protocol mechanics.

Design Decision 36: LDP Attributes

Attribute	Type	Description
Node-level:		
`ldp_enabled`	bool	LDP enabled
`ldp_router_id`	ipv4	LDP router ID
`ldp_session_holdtime`	u16	Session keepalive holdtime (seconds)
`ldp_label_retention`	enum	liberal, conservative
`ldp_label_control`	enum	ordered, independent
Endpoint-level:		
`ldp_interface_enabled`	bool	LDP hello on this interface
`ldp_hello_interval`	u16	Hello interval (seconds)
`ldp_hello_holdtime`	u16	Hello holdtime (seconds)
`ldp_transport_address`	ip	Transport address for TCP session

6.3 RSVP-TE

Resource Reservation Protocol with Traffic Engineering extensions (RSVP-TE) [72] establishes explicitly routed LSPs with guaranteed bandwidth reservation. See Appendix B for detailed protocol mechanics.

Design Decision 37: RSVP-TE Attributes

Attribute	Type	Description
Node-level:		
`rsvp_enabled`	bool	RSVP-TE enabled
`rsvp_graceful_restart`	bool	Graceful restart / helper mode
Endpoint-level:		
`rsvp_interface_enabled`	bool	RSVP enabled on interface
`rsvp_bandwidth_mbps`	u32	Reservable bandwidth
`rsvp_hello_interval`	u16	Hello interval (ms)
`rsvp_refresh_interval`	u16	Path/Resv refresh (seconds)
`rsvp_te_metric`	u32	TE metric override
`rsvp_admin_groups`	u32	Admin group / affinity bitmask
`rsvp_srlg`	list	Shared Risk Link Groups
`rsvp_frr_enabled`	bool	Fast Reroute (facility/one-to-one)
`rsvp_frr_bandwidth`	enum	none, sub-pool, node-protect

Design Decision 38: MPLS TE Tunnel Attributes

Attribute	Type	Description
`te_tunnel_id`	u32	Locally significant tunnel identifier
`te_tunnel_name`	str	Administrative name for the TE tunnel
`te_destination`	ip	Tail-end router address (tunnel destination)
`te_bandwidth`	u64	Reserved bandwidth in bits per second
`te_setup_priority`	u8	Setup priority (0–7, lower is higher priority)
`te_hold_priority`	u8	Hold priority (0–7, lower is higher priority)
`te_affinity`	u32	Administrative affinity bitmask for link colouring
`te_path_option`	enum	Path computation: `explicit` or `dynamic`
`te_protection`	enum	FRR protection type: `none`, `facility`, or `one-to-one`
`te_autoroute`	bool	Inject tunnel into IGP shortest-path computation
`te_forwarding_adjacency`	bool	Advertise tunnel as an IGP adjacency

6.4 Segment Routing

Insight:
Setup priority ≤ hold priority ensures that a newly signalled LSP cannot preempt an existing LSP of equal priority, preventing oscillation.

Segment Routing (SR) [73, 74] eliminates per-flow state in the network by encoding paths in the packet header.

Design Decision 39: SR/SRv6 Attributes

Attribute	Type	Description
Node-level:		
`sr_enabled`	bool	Segment Routing enabled
`sr_dataplane`	enum	mpls [75], srv6 [76]
`sr_global_block`	range	SRGB label range (e.g. 16000–23999)
`sr_local_block`	range	SRLB label range for local SIDs
`sr_node_sid`	u32	Node SID (prefix-SID on loopback)
`srv6_locator`	prefix	SRv6 locator prefix
`srv6_encap_source`	ipv6	SRv6 encapsulation source address
Endpoint-level:		
`sr_adjacency_sid`	u32	Adjacency SID for this link
`sr_adj_sid_protected`	bool	Protected adjacency SID
`sr_te_metric`	u32	TE metric for SR path computation

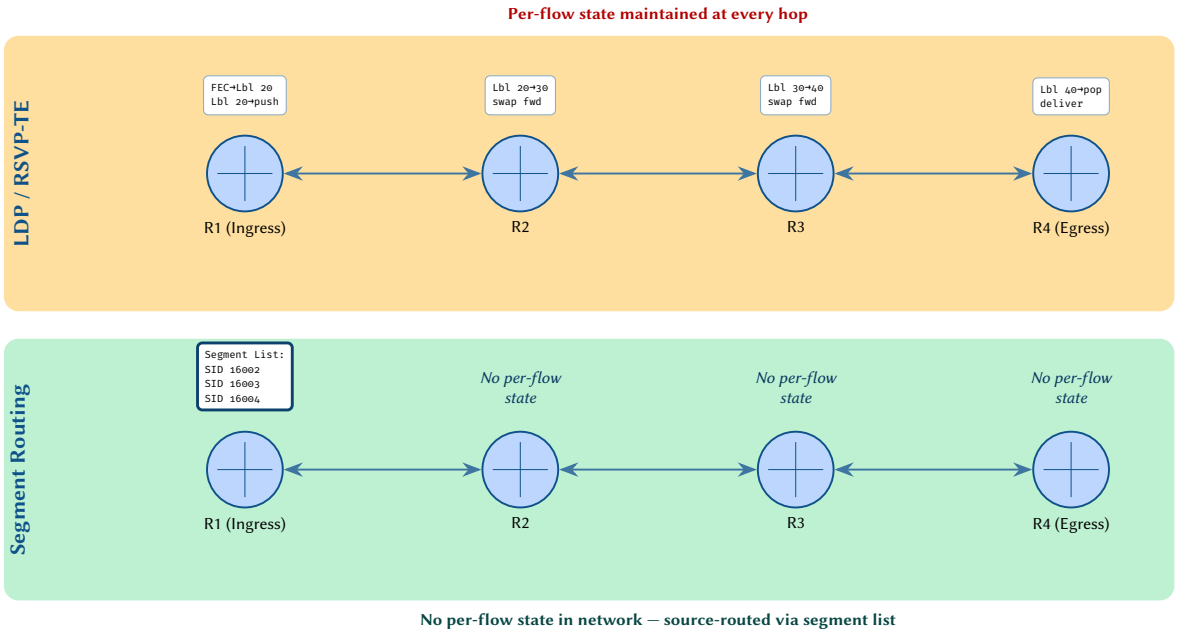


Figure 17. Comparison of LDP/RSVP-TE vs. Segment Routing state distribution. Traditional MPLS signalling maintains per-flow label bindings at every hop, whereas Segment Routing encodes the full path as a segment list at ingress, eliminating per-flow state from transit nodes.

SR-MPLS Label Stack Across a 4-Router Path

Four routers form a linear path: `R1–R2–R3–R4`. Each has a node-SID from the global block (`sr_global_block = 16000–23999`):

Router	`sr_node_sid`	Adj-SID to next	`sr_adj_sid`
R1	16001	R1→R2	24001
R2	16002	R2→R3	24002
R3	16003	R3→R4	24003
R4	16004	—	—

When `R1` sends traffic to `R4` using the shortest IGP path, it pushes a single node-SID label: $\langle 16004 \rangle$. Each transit router swaps the label using its SRGB offset. For a traffic-engineered path that must traverse the explicit sequence R1→R2→R3→R4, the label stack at `R1` is $\langle 24001, 24002, 24003 \rangle$ (adj-SIDs), which pops one label per hop.

In the plan topology, `sr_node_sid` is a node attribute and `sr_adjacency_sid` is an endpoint attribute. The $\mathcal{F}_{\text{SR}}$ functor computes reachability by combining the SRGB with IGP shortest paths, verifying label uniqueness via a pre-operation check.

6.5 EVPN

Ethernet Virtual Private Network (EVPN) [77, 78] provides control-plane-based MAC/IP learning for L2/L3 VPN services.

Design Decision 40: EVPN Attributes

Attribute	Type	Description
Node-level:		
`evpn_enabled`	bool	EVPN enabled
`evpn_rd`	str	EVPN route distinguisher
`evpn_rt_import`	list	Import route targets
`evpn_rt_export`	list	Export route targets
`evpn_vni_map`	map	VLAN → VNI mapping
`evpn_arp_suppress`	bool	ARP suppression in IRB
`evpn_mac_mobility`	bool	MAC mobility detection
`evpn_multihoming_mode`	enum	single-active, all-active
Endpoint-level (on VTEP/NVE):		
`evpn_esi`	str	10-byte Ethernet Segment Identifier
`evpn_esi_df_election`	enum	modulo, preference
`vtep_source_ip`	ip	VTEP source (NVE) interface IP

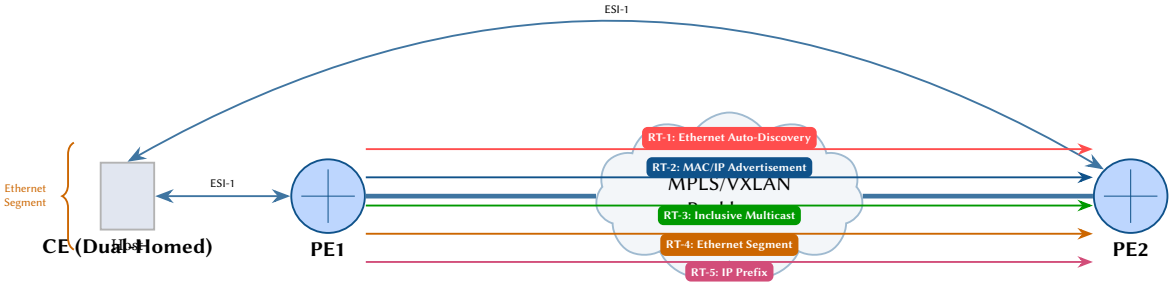


Figure 18. EVPN route types exchanged between PE routers over an MPLS/VXLAN backbone. A dual-homed CE is connected to both PEs via a shared Ethernet Segment Identifier (ESI). Five route types carry MAC/IP reachability, multicast, and segment coordination information.

EVPN-VXLAN Fabric with ESI Multi-Homing

Consider two leaf switches `leaf-01` and `leaf-02` dual-homing a server via an Ethernet Segment (ESI `00:11:22:33:44:55:66:77:88:01`). Both leaves share `evpn_multihoming_mode = all-active` and advertise RT-1 (Ethernet Auto-Discovery) routes to the spines. When the server sends a frame to MAC `aa:bb:cc:dd:ee:01`:

1. `leaf-01` learns the MAC locally and originates an RT-2 (MAC/IP Advertisement) with VNI 10100 and its VTEP source IP (`vtep_source_ip = 10.255.0.11`).
2. The spine route reflector distributes the RT-2 to `leaf-03` (remote leaf), which installs the MAC in its EVPN table keyed by `evpn_vni_map[100] = 10100`.
3. `leaf-03` encapsulates return traffic in a VXLAN header (outer dst = `10.255.0.11`, VNI = 10100) over an SR-MPLS underlay using the node-SID of `leaf-01`.
4. RT-4 (Ethernet Segment) routes coordinate DF election between `leaf-01` and `leaf-02` using `evpn_esi_df_election = modulo`, ensuring BUM traffic is forwarded by exactly one leaf.

In the plan topology, all state resides as flat attributes: the ESI on the NVE endpoint, VNI mappings on the node, and RT import/export controlling inter-leaf route distribution.

6.6 VPLS and Pseudowires

VPLS [79, 80] creates a multipoint Layer 2 VPN service by interconnecting customer sites across an MPLS backbone. Each participating Provider Edge (PE) router maintains a VPLS instance that bridges customer Ethernet frames over pseudowire (PW) tunnels to every other PE in the same service. BGP-based autodiscovery [79] or targeted LDP signalling [80] establishes the full mesh of pseudowires.

In the graph model, a VPLS instance is a set of attributes on the PE device node, while each pseudowire is an endpoint vertex owned by that node. The PW endpoint carries its own attribute scope (`pw_*`), and a connectivity edge links the PW endpoints on the two PEs.

Insight:
VPLS node attributes live on PE device vertices; pseudowire attributes live on endpoint vertices representing each PW tunnel.

Design Decision 41: VPLS Node Attributes

Attribute	Type	Description
`vpls_instance`	str	VPLS instance name or identifier
`vpls_rd`	str	Route distinguisher (Type 0/1/2)
`vpls_rt_import`	list	Route target communities to import
`vpls_rt_export`	list	Route target communities to export
`vpls_signaling`	enum	Signalling protocol: `bgp` or `ldp`
`vpls_mac_limit`	u32	Maximum MAC addresses in the forwarding table
`vpls_mac_withdraw`	bool	Whether MAC withdrawal is enabled [80]

Design Decision 42: Pseudowire Endpoint Attributes

Attribute	Type	Description
`pw_id`	u32	Pseudowire identifier (PW ID)
`pw_type`	enum	Encapsulation type: `vlan` or `ethernet`
`pw_peer`	ip	Remote PE loopback address (PW peer)
`pw_label_local`	u32	Locally assigned MPLS label
`pw_label_remote`	u32	Remotely assigned MPLS label
`pw_mtu`	u16	Pseudowire MTU negotiated between PEs
`pw_status`	enum	Operational status: `up`, `down`, or `standby`
`pw_control_word`	bool	Whether the control word is enabled [81]

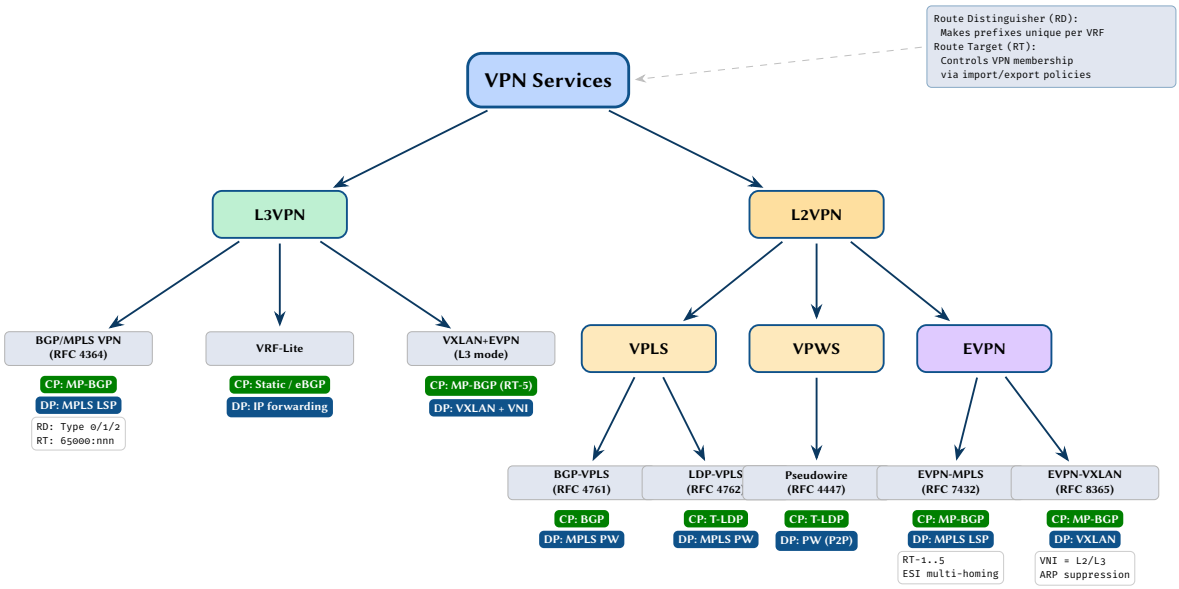


Figure 19. Taxonomy of VPN service types. L3VPN services provide routed IP connectivity using per-VRF routing tables, while L2VPN services extend Layer 2 domains across the backbone. Each leaf shows its control-plane signalling protocol and data-plane encapsulation. Route Distinguishers (RDs) ensure prefix uniqueness, and Route Targets (RTs) govern VPN membership via import/export policies.

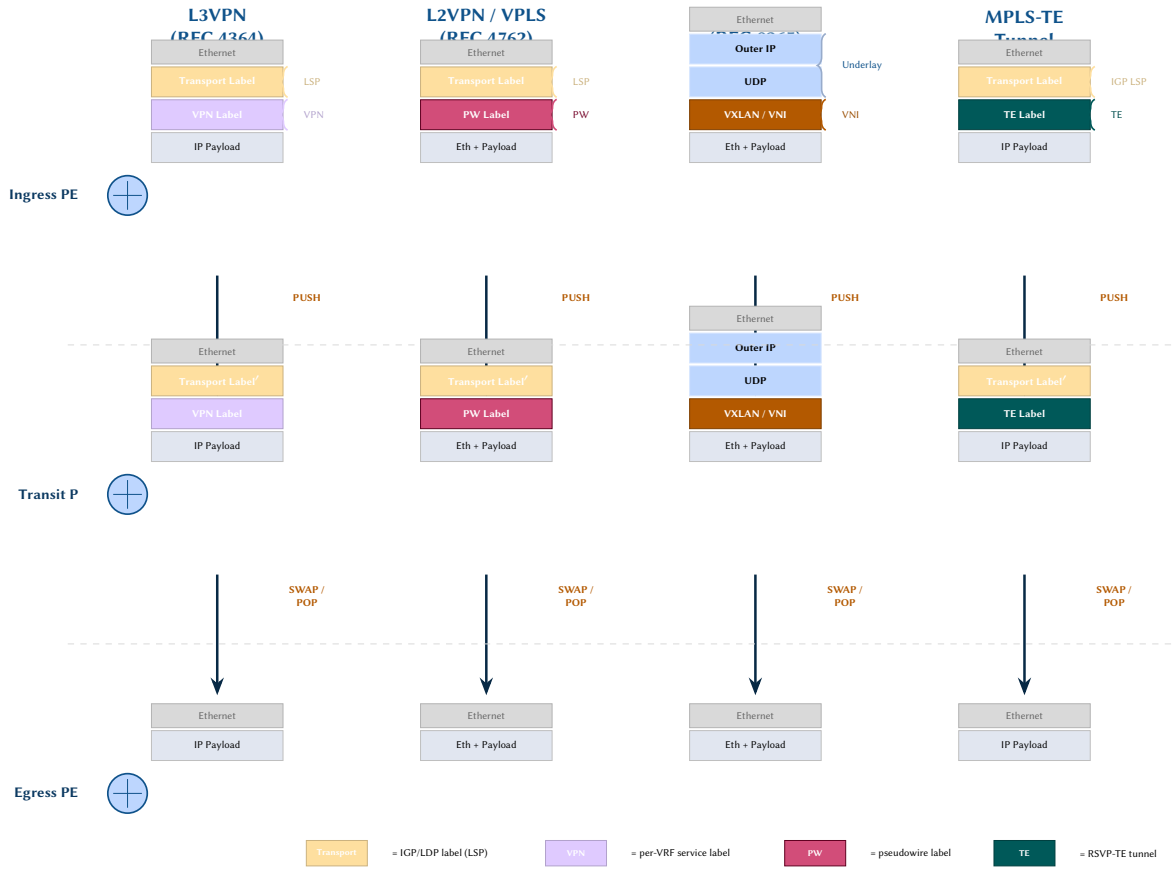


Figure 20. MPLS label stacks for four service types shown at each hop along the Label Switched Path. The ingress PE pushes service-specific and transport labels; transit P routers swap the outer transport label; the egress PE pops all labels to recover the original payload. EVPN-VXLAN uses IP/UDP encapsulation with a VXLAN Network Identifier (VNI) instead of MPLS labels.

7 Policy and Access Control

Network policy encompasses the mechanisms by which operators constrain, filter, and manipulate traffic flows and routing information. While the preceding sections modelled protocol *state*—interface counters, neighbour adjacencies, forwarding entries—this section turns to the *control* dimension: the rules that govern which packets are forwarded, how routes are selected and redistributed, and where traffic is steered. In the property hypergraph formalism, every policy construct reduces to flat key-value attributes attached to nodes (device-scoped policy) or endpoints (interface-scoped policy).

Access Control Lists (ACLs) remain the foundational filtering primitive across virtually all network operating systems. Despite decades of vendor-specific syntax, the semantics are remarkably stable: an ordered sequence of match–action rules evaluated top-to-bottom against packet headers. RFC 8519 [82] provides a vendor-neutral YANG data model for ACLs that we adopt as a reference point, while retaining IOS-style wildcard masks and operational counters that the IETF model omits.

7.1 ACL Rule Structure

Each ACL comprises an ordered list of *Access Control Entries* (ACEs). In the flat attribute model, every field of every ACE becomes an independently addressable attribute, indexed by ACL name and sequence number. The table below enumerates the complete per-entry attribute set.

Design Decision 43: ACL Entry Attributes (per-sequence)

Attribute	Type	Description
`acl_name`	str	ACL name identifier
`acl_seq`	u32	Sequence number (determines evaluation order)
`acl_action`	enum	permit, deny, or remark
`acl_protocol`	enum	ip, tcp, udp, icmp, any (per RFC 8519 [82])
`acl_src_ip`	prefix	Source IP prefix with mask
`acl_src_wildcard`	prefix	Source wildcard mask (IOS-style inverse mask)
`acl_dst_ip`	prefix	Destination IP prefix
`acl_dst_wildcard`	prefix	Destination wildcard mask
`acl_src_port_op`	enum	eq, gt, lt, range
`acl_src_port`	u16 or range	Source port number or range
`acl_dst_port_op`	enum	eq, gt, lt, range
`acl_dst_port`	u16 or range	Destination port number or range
`acl_dscp_match`	u6	Match on DSCP value (RFC 2474 [83])
`acl_icmp_type`	u8	ICMP type match
`acl_icmp_code`	u8	ICMP code match
`acl_established`	bool	Match established TCP connections (ACK/RST set)
`acl_log`	bool	Log matching packets
`acl_counter_packets`	u64	Packet hit counter (operational state)
`acl_counter_bytes`	u64	Byte hit counter (operational state)

Several design choices deserve comment. The `acl_src_wildcard` and `acl_dst_wildcard` attributes preserve IOS-style inverse masks alongside the more natural prefix notation used by RFC 8519. This dual representation is deliberate: many operational tools and audit scripts still reference wildcard masks, and the flat model can carry both without ambiguity. The `acl_dscp_match` field connects ACLs to the Differentiated Services architecture (RFC 2474 [83]), enabling classification-based filtering within the same rule structure. Operational counters (`acl_counter_packets`, `acl_counter_bytes`) are marked as operational state: they are read from the device but never pushed as configuration.

The following example demonstrates the full encoding for a realistic three-rule ACL applied to a border interface.

Insight:
The flat model explodes each ACL entry into individually indexed attributes. For an ACL named `DENY_WEB` with sequence number 10, every field becomes a separate key: `acl_DENY_WEB_seq_10_action`, `acl_DENY_WEB_seq_10_protocol`, `acl_DENY_WEB_seq_10_src_ip`, and so on. This encoding trades compactness for queryability: a columnar store can answer "which devices have a deny rule matching 10.0.0.0/8?" with a single predicate scan, without parsing nested ACL structures.

ACL Encoding Example

Consider a standard edge ACL named `EDGE_IN` with three rules:

1. `permit tcp any host 10.0.0.1 eq 443`
2. `deny ip 192.168.0.0 0.0.255.255 any`
3. `permit ip any any`

In the flat attribute model on the ingress endpoint:

Flat Key	Value
`acl_EDGE_IN_seq_10_action`	permit
`acl_EDGE_IN_seq_10_protocol`	tcp
`acl_EDGE_IN_seq_10_src_ip`	0.0.0.0/0 (any)
`acl_EDGE_IN_seq_10_dst_ip`	10.0.0.1/32
`acl_EDGE_IN_seq_10_dst_port_op`	eq
`acl_EDGE_IN_seq_10_dst_port`	443
`acl_EDGE_IN_seq_20_action`	deny
`acl_EDGE_IN_seq_20_protocol`	ip
`acl_EDGE_IN_seq_20_src_ip`	192.168.0.0/16
`acl_EDGE_IN_seq_20_src_wildcard`	0.0.255.255
`acl_EDGE_IN_seq_20_dst_ip`	0.0.0.0/0 (any)
`acl_EDGE_IN_seq_30_action`	permit
`acl_EDGE_IN_seq_30_protocol`	ip
`acl_EDGE_IN_seq_30_src_ip`	0.0.0.0/0 (any)
`acl_EDGE_IN_seq_30_dst_ip`	0.0.0.0/0 (any)

The endpoint also carries `acl_in = EDGE_IN` to bind the ACL. Operational counters (e.g. `acl_EDGE_IN_seq_20_counter_packets = 148203`) are populated by the collection pipeline and stored alongside configuration attributes.

7.2 Firewall Filter Chains

While ACLs operate statelessly on individual packets, modern network devices increasingly support stateful firewalling directly on the forwarding plane. Stateful inspection maintains a connection-tracking table that permits return traffic for established sessions without requiring explicit ACL entries. This distinction matters for the attribute model: stateless ACL attributes describe *rules*, whereas firewall attributes describe both rules and *runtime state* (connection table size, timeout values).

Firewall filter chains follow the `input`/`output`/ `forward` trichotomy inherited from Netfilter and adopted by most vendor implementations. The *chain* determines at which point in the packet path the filter is evaluated: `input` applies to traffic destined for the device's own control plane, `output` to locally generated traffic, and `forward` to transit traffic. The default action for each chain (accept, drop, or reject) establishes the security posture: a "default-deny" stance requires explicit permit rules for all desired traffic.

Design Decision 44: Firewall Attributes

Attribute	Type	Description
`fw_conntrack_enabled`	bool	Connection tracking enabled
`fw_state_table_size`	u32	Maximum connection-tracking table entries
`fw_state_timeout`	u32	Default connection state timeout (seconds)
`fw_chain`	enum	input, output, forward
`fw_default_action`	enum	accept, drop, reject

When connection tracking is enabled, ACL rules can reference `acl_established` to match return traffic for TCP sessions already present in the state table. This interplay between stateless ACL attributes and stateful firewall attributes is resolved at evaluation time: the ACL engine consults the connection-tracking table when `fw_conntrack_enabled = true` and the rule specifies `acl_established = true`.

7.3 Control Plane Policing (CoPP)

The control plane of a network device—where routing protocol processes, management daemons, and keepalive mechanisms execute—is a shared resource vulnerable to denial-of-service attacks and traffic-induced overload. Control Plane Policing (CoPP) protects this resource by rate-limiting traffic destined for the CPU. CoPP policies classify control-plane traffic into classes (BGP, OSPF, ICMP, SSH, SNMP, and others) and apply per-class rate limits, ensuring that a flood of one traffic type cannot starve the others.

CoPP is logically distinct from interface-level ACLs: it operates at the boundary between the forwarding plane and the control plane, after the routing lookup has determined that a packet is "for us." In the property

hypergraph, CoPP attributes attach to the *node* rather than to an endpoint, reflecting their device-global scope.

Design Decision 45: CoPP Attributes

Attribute	Type	Description
`copp_class`	str	Traffic class name
`copp_rate_limit`	u64	Rate limit in bits per second
`copp_burst`	u32	Burst size in bytes
`copp_protocol_match`	enum	bgp, ospf, icmp, ssh, snmp, all
`copp_action`	enum	permit, police, drop

A typical CoPP deployment defines classes for each critical protocol: BGP sessions receive a generous rate limit (to avoid session flaps during convergence events), while ICMP and broadcast traffic receive more restrictive policing. The flat model indexes CoPP entries by class name (e.g. `copp_BGP_rate_limit = 500000`, `copp_ICMP_rate_limit = 64000`), following the same flattening pattern used for ACL entries.

7.4 Extended Route-Map Attributes

Route-maps are the universal policy tool in IP routing. They appear at every redistribution boundary, in every BGP peer policy, and at the interface of every PBR configuration. Section 4 introduced the core route-map attributes (`rmap_name` through `rmap_set_weight`). This section extends the attribute set to cover the full range of match and set clauses encountered in production networks, following the BGP path attribute semantics defined in RFC 4271 [54].

The extended match clauses enable route-maps to filter on properties beyond prefix and community: interface membership, IGP metric, route tags, next-hop reachability (via prefix-list indirection), and BGP origin type. The extended set clauses provide the corresponding manipulation primitives: origin rewriting, AS-path prepending (the primary tool for outbound traffic engineering in BGP), tag assignment for redistribution control, and route dampening activation.

Design Decision 46: Additional Route-Map Clauses

Attribute	Type	Description
`rmap_match_tag`	u32	Match on route tag value
`rmap_match_interface`	str	Match on receiving interface
`rmap_match_metric`	u32	Match on IGP metric
`rmap_match_next_hop_pl`	str	Match next-hop against a named prefix-list
`rmap_match_origin`	enum	igp, egp, incomplete
`rmap_set_origin`	enum	igp, egp, incomplete
`rmap_set_as_path_prepend`	str	AS path prepend value (e.g. “65000 65000 65000”)
`rmap_set_tag`	u32	Set route tag value
`rmap_set_dampening`	bool	Enable route dampening for matched prefixes

AS-path prepending deserves particular attention: the `rmap_set_as_path_prepend` attribute stores the literal prepend string rather than a count, because operators frequently prepend different ASNs or repeat their own ASN a variable number of times. In the flat model, a route-map sequence that prepends AS 65000 three times is recorded as `rmap_OUTBOUND_seq_10_set_as_path_prepend = 65000 65000 65000`.

7.5 Prefix-List and Community-List Structures

Prefix-lists and community-lists are the named match objects referenced by route-map clauses. Both follow the same structural pattern as ACLs: an ordered sequence of entries, each with a sequence number, an action (permit or deny), and a match predicate. In the flat model, each entry is indexed by list name and sequence number.

Design Decision 47: Prefix-List Entry Attributes

Attribute	Type	Description
`pl_name`	str	Prefix-list name
`pl_seq`	u32	Sequence number
`pl_action`	enum	permit, deny
`pl_prefix`	prefix	IP prefix to match
`pl_ge`	u8	Minimum prefix length (ge constraint)
`pl_le`	u8	Maximum prefix length (le constraint)

The `ge` and `le` modifiers provide range matching on prefix lengths: a prefix-list entry `10.0.0.0/8 ge 16 le 24` matches any prefix within 10.0.0.0/8 whose mask length is between /16 and /24. This is the primary mechanism for implementing minimum allocation size filters on BGP sessions.

Community-lists come in three flavours: standard (matching exact community values), expanded (matching via regular expressions), and large (RFC 8092 large communities with three 32-bit fields).

Design Decision 48: Community-List Attributes

Attribute	Type	Description
`cl_name`	str	Community-list name
`cl_seq`	u32	Sequence number
`cl_type`	enum	standard, expanded, large
`cl_match`	str	Community value or regular expression

Extended communities (RFC 4360) and large communities (RFC 8092) require additional structure. Extended communities carry a type field distinguishing Route Targets (RT) from Sites of Origin (SoO), while large communities provide three independent 32-bit fields for flexible tagging in multi-provider environments.

Design Decision 49: Extended and Large Community Attributes

Attribute	Type	Description
`cl_ext_type`	enum	rt (Route Target), soo (Site of Origin)
`cl_ext_value`	str	Extended community value (e.g. 65000:100)
`cl_large_global`	u32	Large community global administrator field
`cl_large_local1`	u32	Large community local data part 1
`cl_large_local2`	u32	Large community local data part 2

7.6 Policy-Based Routing

Policy-Based Routing (PBR) overrides the normal destination-based forwarding decision by steering packets that match specified criteria to an alternate next-hop, interface, or VRF. PBR is typically implemented as a special route-map applied to an ingress interface: the match clause references an ACL (or other classifier), and the set clause specifies the forwarding override.

PBR occupies a critical position in the packet processing pipeline (Figure 21): it is evaluated *after* the ingress ACL has filtered the packet and *after* the standard routing lookup, but *before* the egress ACL. When a PBR match occurs, the routing lookup result is overridden; when no match occurs, normal forwarding proceeds.

Design Decision 50: PBR Attributes

Attribute	Type	Description
`pbr_name`	str	PBR policy name
`pbr_seq`	u32	Sequence number
`pbr_match_acl`	str	Match ACL name (classifier)
`pbr_set_nexthop`	ip	Override next-hop address
`pbr_set_interface`	str	Override output interface
`pbr_set_vrf`	str	Override VRF for lookup
`pbr_set_dscp`	u6	Set DSCP marking (RFC 2474 [83])
`pbr_default_action`	enum	route (normal forwarding), drop

The `pbr_set_vrf` attribute enables inter-VRF policy routing, where traffic matching a specific class is redirected into a different routing domain—a pattern commonly used for service chaining and traffic inspection architectures. The `pbr_set_dscp` attribute allows PBR to perform simultaneous steering and classification, combining forwarding-plane and QoS functions in a single policy.

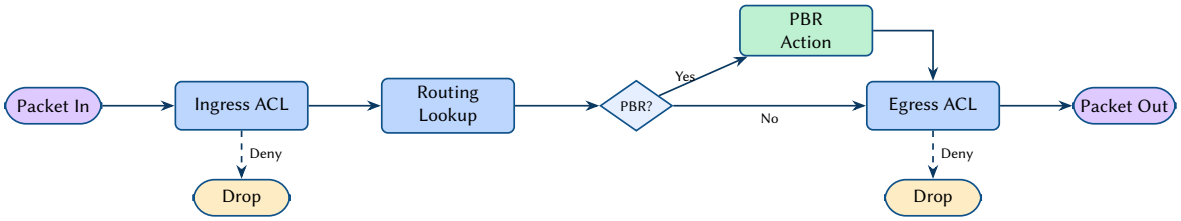


Figure 21. Packet processing pipeline showing ingress ACL, routing lookup, optional policy-based routing, and egress ACL stages. Dashed arrows indicate packets dropped by deny rules.

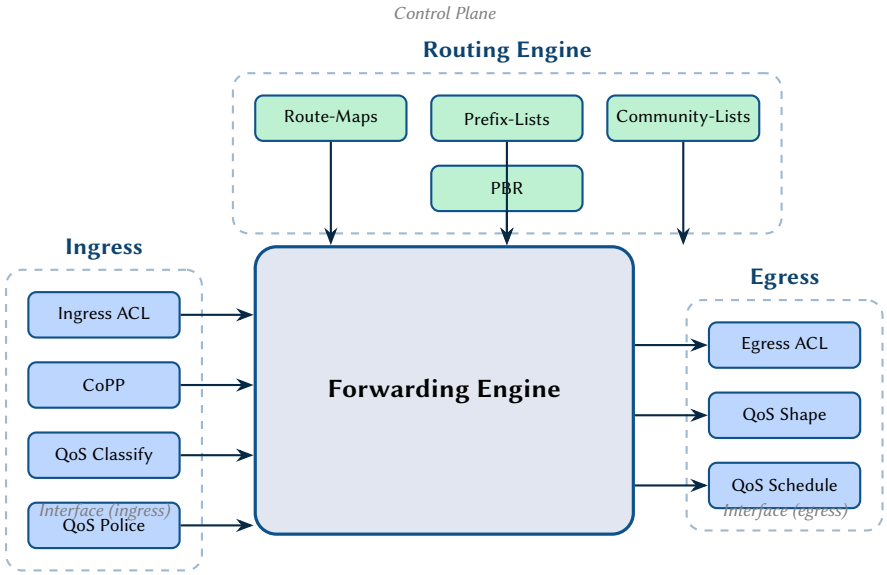


Figure 22. Policy application points on a router, showing ingress, routing engine, and egress attachment points for ACLs, QoS policies, route-maps, and policy-based routing.

Tier 2: Overlay, Transport, and Multicast

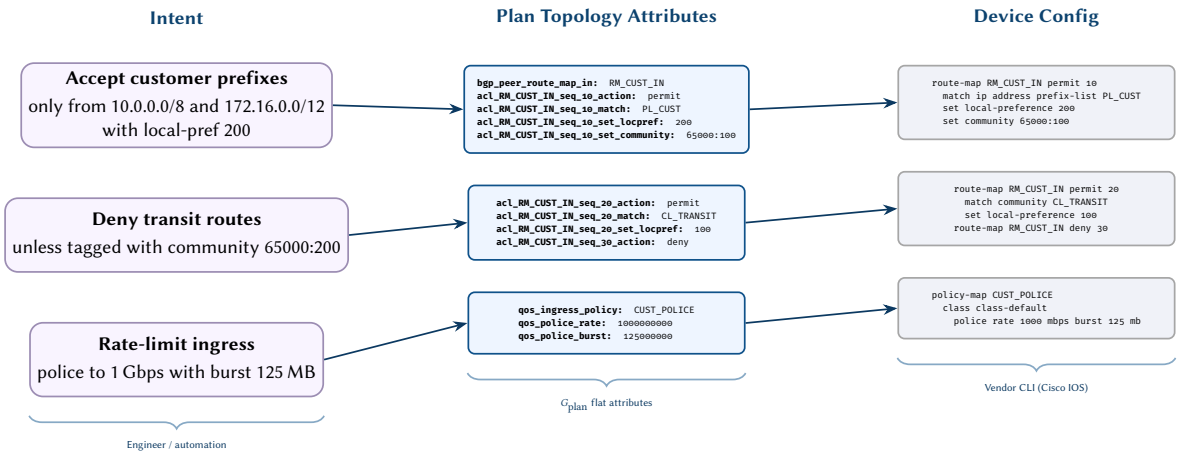


Figure 23. Policy intent mapping: high-level intent (left) maps to flat plan topology attributes (centre), which generate vendor-specific device configuration (right). Each policy construct—route-maps, prefix-lists, QoS policies—follows the same intent → attribute → config pipeline.

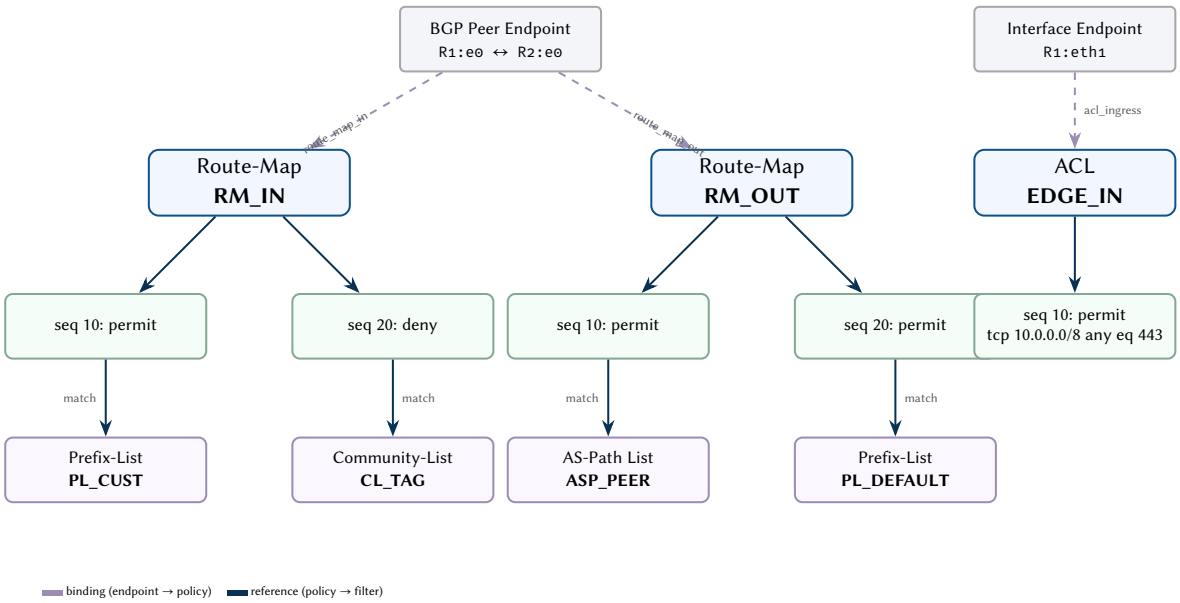


Figure 24. Policy object relationship graph. BGP peer endpoints bind to route-maps via name attributes; route-map entries reference prefix-lists, community-lists, and AS-path lists. Interface endpoints bind to ACLs. All objects are stored as flat attributes in G_{plan}.

8 Layer 5: Overlays

8.1 VXLAN

Virtual eXtensible Local Area Network (VXLAN) [84] provides Layer 2 overlay connectivity using UDP encapsulation with a 24-bit VNI. The VTEP (VXLAN Tunnel Endpoint) is the logical device that performs encapsulation and decapsulation; the attributes below configure its identity, learning mode, and BUM traffic handling.

Design Decision 51: VXLAN Attributes

Attribute	Type	Description
`vxlan_enabled`	bool	VXLAN enabled
`vxlan_source_intf`	str	Source interface for VTEP (typically a loopback)
`vxlan_udp_port`	u16	UDP destination port (default 4789)
`vxlan_vni_map`	map	VLAN → VNI mapping
`vxlan_vrf_vni_map`	map	VRF → L3VNI mapping
`vxlan_flood_vteps`	list	Static flood list (if no EVPN)
`vxlan_learning`	enum	data-plane, control-plane, flood-and-learn
`vxlan_arp_suppression`	bool	Suppress ARP flooding by answering from local cache
`vxlan_multicast_group`	ipv4	Multicast group for BUM traffic (flood-and-learn mode)
`vxlan_replication_mode`	enum	multicast, ingress-replication, static

The `vxlan_learning` attribute is a mode switch that selects the control-plane architecture: `data-plane` learns MACs from received frames, `control-plane` relies on EVPN, and `flood-and-learn` broadcasts BUM traffic to all VTEPs. When `vxlan_arp_suppression` is enabled, the VTEP intercepts ARP requests and replies locally from its MAC/IP binding cache, reducing broadcast traffic across the overlay fabric. The `vxlan_replication_mode` determines how BUM traffic is forwarded: `multicast` uses the group in `vxlan_multicast_group`, `ingress-replication` unicasts to each remote VTEP, and `static` uses the explicit `vxlan_flood_vteps` list. See Appendix B for detailed protocol mechanics.

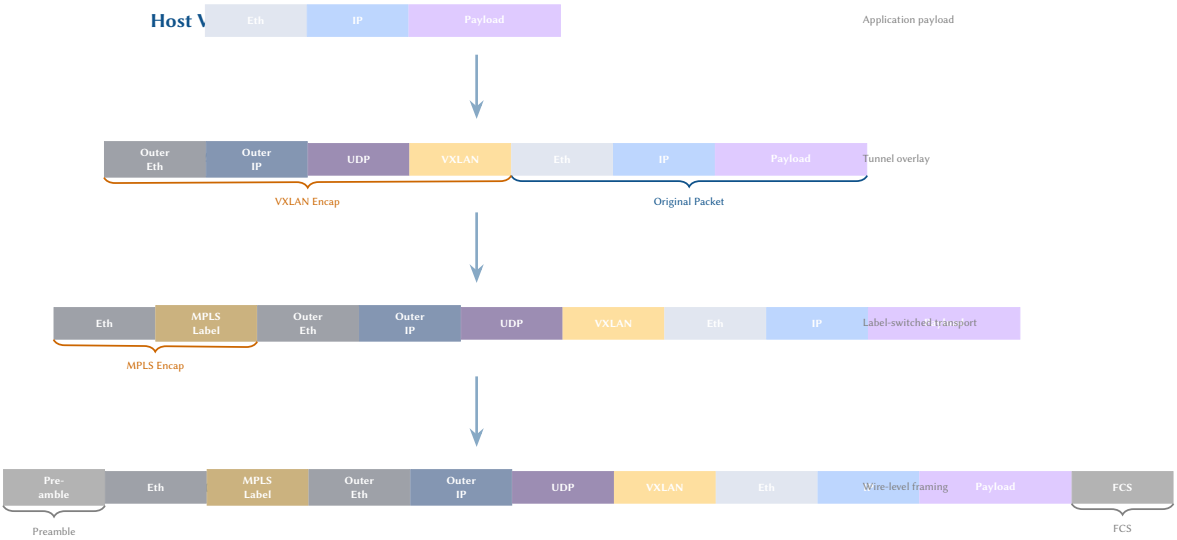


Figure 25. Full L3 overlay encapsulation stack showing progressive header addition from the host VM through VXLAN encapsulation, MPLS transport labelling, and physical-layer framing. Each layer adds its own headers around the inner payload.

Insight: *The `vxlan_learning` attribute is a mode switch that determines the entire control-plane architecture: changing it from `flood-and-learn` to `evpn` replaces static flood lists with BGP-distributed host reachability, enabling ARP suppression, automatic VTEP discovery, and deterministic MAC mobility handling.*

8.2 GRE

GRE [85] provides simple IP-in-IP tunnelling for point-to-point overlays. The attributes below define the tunnel endpoints and operational parameters configured on each device.

Design Decision 52: GRE Tunnel Attributes

Attribute	Type	Description
`gre_source`	ipv4	Tunnel source address
`gre_destination`	ipv4	Tunnel destination address
`gre_key`	u32	Optional GRE key for demultiplexing
`gre_keepalive_interval`	u32	Keepalive probe interval (seconds)
`gre_mtu`	u32	Tunnel MTU (accounts for encap overhead)
`gre_ttl`	u32	Outer IP TTL (default 255)
`gre_mode`	enum	gre, mgre (multipoint), nvgre

The `gre_mode` attribute selects the tunnel variant: `gre` creates a point-to-point tunnel, `mgre` enables multipoint GRE used in DMVPN deployments, and `nvgre` uses the GRE key field as a virtual subnet identifier for data-centre overlays. The `gre_mtu` should be set to account for the encapsulation overhead (typically 24 bytes for GRE, more with optional fields).

8.3 IPsec

IPsec [86] adds confidentiality and integrity via ESP/AH encapsulation, used for site-to-site VPNs and transport-mode protection. The configuration attributes below define what cryptographic parameters to apply; see Appendix B for IKE negotiation mechanics.

Design Decision 53: IPsec Transform Set Attributes

Attribute	Type	Description
`ipsec_transform_encryption`	enum	aes-128-cbc, aes-256-cbc, aes-gcm-128, aes-gcm-256
`ipsec_transform_integrity`	enum	sha-256, sha-384, sha-512 (ignored for GCM)
`ipsec_transform_mode`	enum	tunnel, transport
`ipsec_sa_lifetime`	u32	SA lifetime (seconds)
`ipsec_pfs_group`	enum	none, group14, group19, group20

Design Decision 54: IKE Policy Attributes

Attribute	Type	Description
`ike_version`	enum	v1, v2
`ike_encryption`	enum	aes-128, aes-256, aes-gcm-256
`ike_integrity`	enum	sha-256, sha-384, sha-512
`ike_dh_group`	enum	group14, group19, group20, group21
`ike_lifetime`	u32	IKE SA lifetime (seconds, default 86400)
`ike_auth_method`	enum	psk, certificate

Design Decision 55: IPsec Peer and Crypto Map Attributes

Attribute	Type	Description
`ipsec_crypto_map`	str	Crypto map or IPsec profile name
`ipsec_peer_address`	ipv4	Remote peer address
`ike_cert_dn`	str	Peer certificate distinguished name
`ike_traffic_selector_local`	str	Local traffic selector (prefix/proto/port)
`ike_traffic_selector_remote`	str	Remote traffic selector
`ipsec_dpd_interval`	u16	Dead peer detection probe interval (seconds)
`ipsec_dpd_retries`	u8	Consecutive DPD failures before teardown
`ipsec_rekey_margin`	u32	Seconds before expiry to initiate rekey
`ipsec_rekey_fuzz`	u8	Random jitter percentage for rekey timing

The transform set defines the per-packet cryptographic treatment applied to protected traffic. The IKE policy attributes configure the parameters used to establish the control-channel SA. The crypto map binds these together and associates them with a specific peer address and traffic selectors.

8.4 Geneve

Geneve [87] is an extensible tunnel format designed to supersede VXLAN and NVGRE with variable-length metadata TLVs. Each tunnel is modelled as a logical endpoint with encapsulation attributes.

Design Decision 56: Geneve Tunnel Attributes

Attribute	Type	Description
`geneve_vni`	u24	Geneve virtual network identifier
`geneve_udp_port`	u16	UDP destination port (default 6081)
`geneve_option_class`	u16	IANA option class identifier
`geneve_option_type`	u8	Option type within class
`geneve_critical_bit`	bool	Critical bit; drop packet if option unrecognised
`geneve_int_enabled`	bool	Enable in-band network telemetry via Geneve options

Geneve's variable-length option TLVs enable use cases such as in-band network telemetry (INT), where switch ASICs insert per-hop metadata (queue depth, latency, ingress port) directly into the tunnel header. The `geneve_critical_bit` ensures that endpoints which cannot parse a given option will drop the packet rather than silently ignore it. Unlike VXLAN's fixed header, Geneve's extensibility makes it the preferred encapsulation for programmable data planes and network function virtualisation platforms that require in-band metadata exchange.

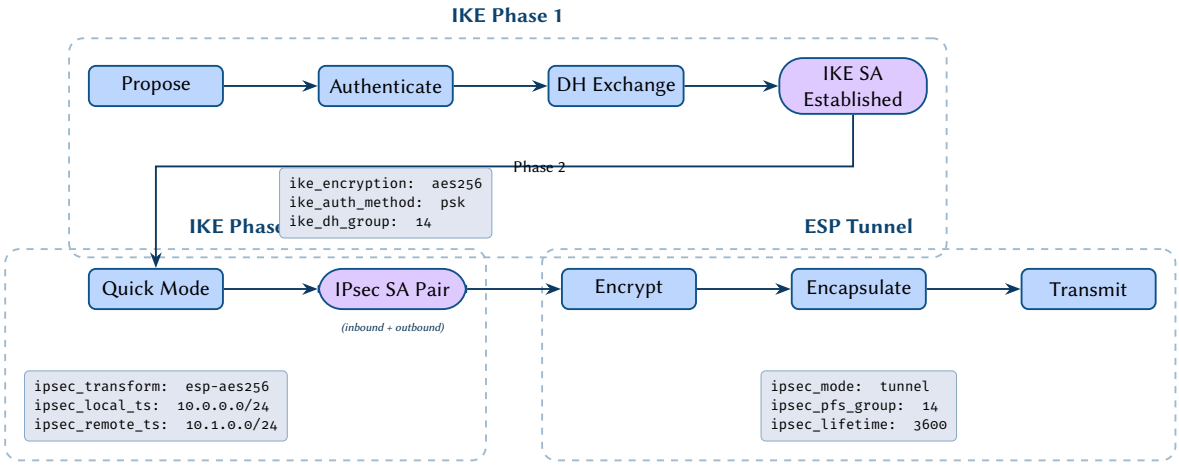


Figure 26. IKE/IPsec establishment phases. Phase 1 negotiates cryptographic parameters and establishes the IKE SA (purple). Phase 2 uses Quick Mode to create an IPsec SA pair. The resulting ESP tunnel encrypts, encapsulates, and transmits protected traffic.

9 Layer 4: Transport

Transport layer state is typically *derived* rather than configured on network infrastructure devices. TCP [88] and UDP [89] state machines operate on end hosts. However, infrastructure devices expose transport-relevant configuration that influences how flows are handled, policed, and optimised as they traverse the network fabric. This section enumerates the configurable attributes for TCP, UDP, and QUIC, together with transport-layer security mechanisms that protect control-plane sessions.

9.1 TCP Configuration State

TCP remains the dominant transport for infrastructure control planes: BGP [54], gNMI [90], NETCONF [91], and PCEP [92] all rely on TCP sessions between network devices. Consequently, a range of TCP parameters are directly configurable on routers and switches.

Design Decision 57: TCP Configuration Attributes

Attribute	Type	Description
`tcp_mss_clamp`	u16	Maximum Segment Size clamping value (bytes)
`tcp_window_scaling`	bool	Enable RFC 7323 window scaling option
`tcp_sack`	bool	Selective Acknowledgement support [93]
`tcp_timestamps`	bool	TCP timestamps option (RFC 7323)
`tcp_ecn`	enum	Explicit Congestion Notification mode: `off`, `passive`, `active` [94]
`tcp_congestion_algo`	enum	Congestion control algorithm: `cubic`, `bbr`, `reno`, `newreno`
`tcp_keepalive_interval`	u16	Keepalive probe interval in seconds
`tcp_keepalive_probes`	u8	Number of unanswered probes before declaring dead
`tcp_syn_flood_protection`	bool	Enable SYN cookie or SYN proxy defence
`tcp_connection_limit`	u32	Maximum concurrent TCP connections per context

The `tcp_ecn` attribute warrants special attention. In `passive` mode the device honours incoming ECN-capable packets but does not initiate ECN negotiation; in `active` mode the device sets the ECE and CWR flags during the three-way handshake. Modern data-centre fabrics increasingly deploy active ECN in conjunction with DCTCP [95] to achieve low-latency congestion signalling.

Insight: *MSS clamping is critical in tunnelled environments: when the outer encapsulation reduces the effective MTU, clamping the TCP MSS prevents fragmentation along the path. A correct MSS clamp value must account for all encapsulation headers (e.g. VXLAN adds 50 bytes, GRE adds 24 bytes).*

9.2 UDP State

UDP is stateless by design, yet infrastructure devices still carry configuration that governs UDP behaviour. Source port randomisation mitigates off-path injection attacks against DNS, RADIUS, and syslog flows. The IPv4 UDP checksum is historically optional (RFC 768), whereas IPv6 mandates it [38].

Design Decision 58: UDP Configuration Attributes

Attribute	Type	Description
`udp_source_port_randomisation`	bool	Randomise ephemeral source ports for outgoing UDP flows
`udp_checksum`	bool	Enable UDP checksum on IPv4 (always enabled for IPv6)

Insight: *Disabling the IPv4 UDP checksum can improve performance for encapsulated traffic (e.g. VXLAN inner packets) but eliminates the only integrity check below the application layer. Operators should weigh the performance gain against the risk of silent corruption.*

9.3 QUIC Configuration

QUIC [36] operates over UDP and provides its own congestion control, multiplexing, and TLS 1.3 integration. On infrastructure devices, QUIC is relevant primarily as a transport for gNMI and streaming telemetry, but also as a next-generation substrate for control-plane protocols. As QUIC adoption grows in network infrastructure, devices expose the following configuration knobs:

Design Decision 59: QUIC Configuration Attributes

Attribute	Type	Description
`quic_enabled`	bool	Enable QUIC transport on the device
`quic_connection_id_length`	u8	Length of the connection ID field (bytes)
`quic_version`	enum	Negotiated QUIC version (`v1`, `v2`)
`quic_zero_rtt`	bool	Allow 0-RTT early data [96]
`quic_max_streams_bidi`	u32	Maximum concurrent bidirectional streams
`quic_max_streams_uni`	u32	Maximum concurrent unidirectional streams
`quic_idle_timeout`	u32	Idle timeout in milliseconds before connection teardown
`quic_preferred_address`	ip	Server preferred address for connection migration
`quic_retry_token`	bool	Require address validation via Retry packets

The `quic_zero_rtt` attribute enables 0-RTT connection resumption, which reduces handshake latency but introduces replay risk. In infrastructure contexts where telemetry streams are long-lived, the latency benefit of 0-RTT is marginal, so conservative deployments disable it.

Insight: *QUIC's built-in encryption renders traditional deep-packet inspection ineffective. Network operators who rely on middlebox visibility into transport headers must adapt monitoring strategies—for example, by leveraging the QUIC spin bit for latency measurement or by deploying endpoint-based telemetry.*

9.4 Transport Security

Control-plane TCP sessions between routers have historically been protected by TCP MD5 signatures [97]. This mechanism, while widely deployed for BGP, suffers from known weaknesses: it uses a single static key, offers no key rollover mechanism, and relies on the cryptographically broken MD5 hash.

TCP Authentication Option (TCP-AO) [98] supersedes TCP MD5 with support for multiple algorithms, key identifiers that enable hitless key rotation, and a next-key-id field that coordinates rollover between peers.

Design Decision 60: Transport Security Attributes

Attribute	Type	Description
`tcp_md5_enabled`	bool	Legacy TCP MD5 authentication [97]
`tcp_ao_enabled`	bool	TCP-AO authentication [98]
`tcp_ao_key_id`	u8	Current outgoing key identifier
`tcp_ao_rnext_key_id`	u8	Requested next receive key identifier
`tcp_ao_algorithm`	enum	MAC algorithm: `hmac-sha-1-96`, `aes-128-cmac-96`
`macsec_transport_mode`	enum	MACsec operational mode: `transport`, `tunnel`

The `macsec_transport_mode` attribute governs whether MACsec operates in transport mode (authenticating and optionally encrypting the payload between adjacent L2 hops) or tunnel mode (preserving the outer Ethernet header for transit across intermediate switches). While MACsec is fundamentally a Layer 2 mechanism, its mode selection directly influences how transport-layer flows are protected end-to-end, warranting its inclusion in this section.

Insight: *TCP MD5 and TCP-AO are mutually exclusive on a given session. Operators migrating from MD5 to AO must coordinate the transition with all peers, as a mismatch in authentication method causes immediate session failure. The `tcp_ao_rnext_key_id` field enables graceful key rollover without traffic disruption.*

9.5 Transport State Model

Figure 27 presents a consolidated view of the TCP, UDP, and QUIC attribute families, illustrating how each protocol's configuration attributes relate to the common transport endpoint abstraction.

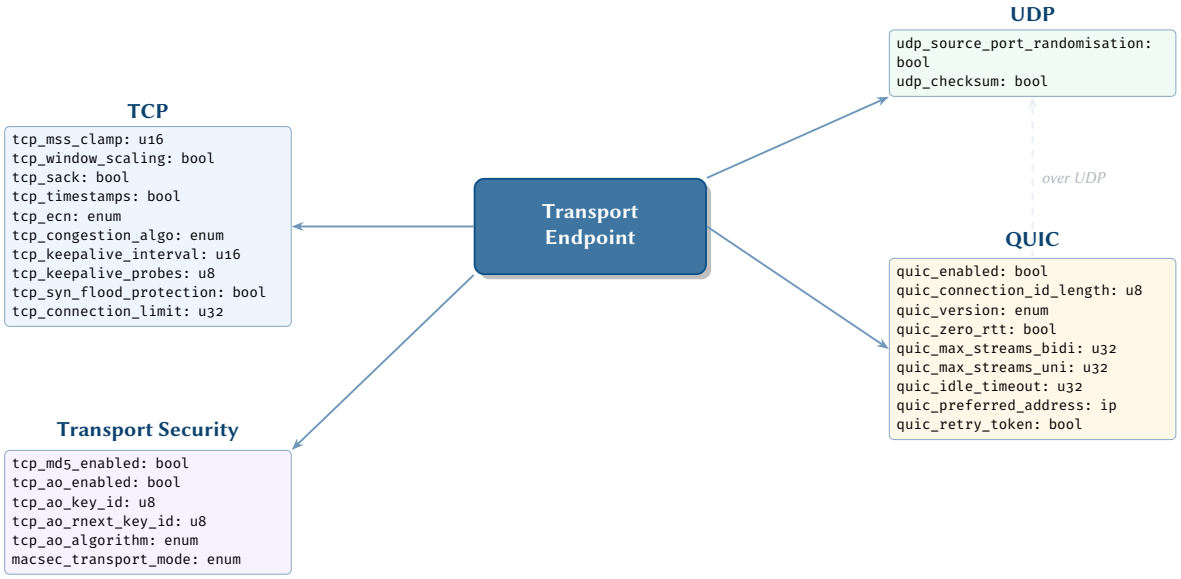


Figure 27. Transport state model: TCP, UDP, and QUIC attribute families radiate from a common transport endpoint abstraction. The dashed arrow indicates that QUIC operates over UDP.

Tier 3: Physical, Security, and Management

10 Layer 1: Physical

10.1 IEEE 802.3 Ethernet PHY

The physical layer defines the electrical, optical, and mechanical properties of network links. IEEE 802.3-2022 [99] consolidates all Ethernet PHY variants from 10BASE-T through 400GBASE-*.

Design Decision 61: PHY Endpoint Attributes

Attribute	Type	Source / Notes
`phy_type`	enum	1000BASE-T, 10GBASE-SR, 100GBASE-LR4, …
`speed_mbps`	u32	Negotiated or configured speed
`duplex`	enum	full, half
`autoneg_state`	enum	disabled, enabled, complete, failed (Clause 28/73)
`fec_mode`	enum	none, FC-FEC, RS-FEC (Clause 91/108)
`link_training`	enum	not-started, in-progress, complete, failed
`medium`	enum	copper, mmf, smf, dac, aoc
`connector`	enum	RJ45, SFP, SFP+, QSFP28, QSFP-DD, OSFP, …
`breakout_mode`	string	e.g. `4x25G`, `2x100G`
`mtu_bytes`	u16	Physical MTU (default 1518, jumbo 9216)
`eee_enabled`	bool	Energy Efficient Ethernet (802.3az)
`admin_state`	enum	up, down, testing
`oper_state`	enum	up, down, testing, dormant, not-present (operational)
`error_counters`	struct	CRC, runts, giants, alignment, symbol errors (operational)

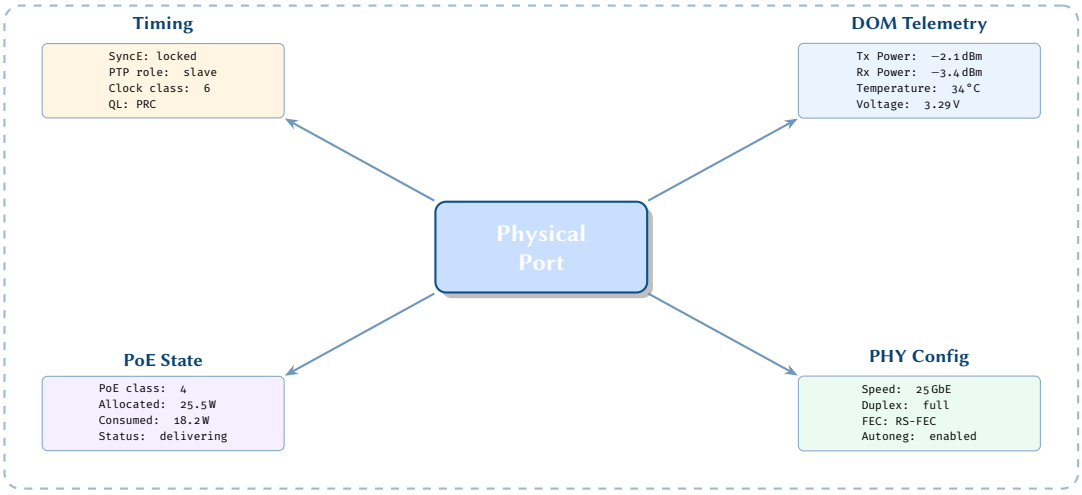


Figure 28. Physical-layer attribute groups radiating from a single port. Each cluster represents an independently pollable state domain: DOM optics telemetry, PHY configuration, timing/synchronisation, and Power-over-Ethernet delivery status.

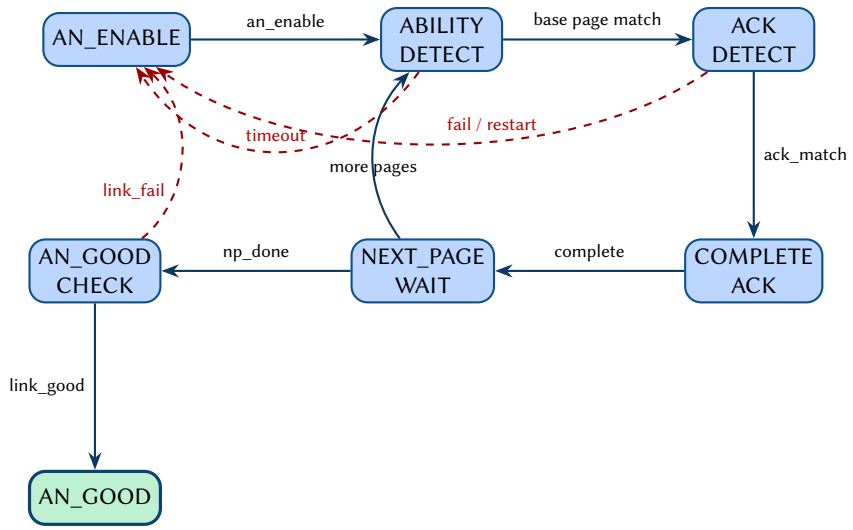


Figure 29. IEEE 802.3 Clause 28 autonegotiation finite state machine. The forward path proceeds from `AN_ENABLE` through ability and acknowledgement detection to `AN_GOOD` (green). Dashed red arcs indicate failure/timeout transitions that reset the process.

10.2 Digital Optical Monitoring (DOM)

Transceivers expose real-time telemetry. These attributes are operational—they are read from hardware, not configured—but are included in the survey for completeness (see Design Rule 1):

Design Decision 62: DOM / Transceiver Attributes

Attribute	Type	Description
`dom_temperature_c`	f64	Module temperature (°C)
`dom_voltage_v`	f64	Supply voltage (V)
`dom_tx_power_dbm`	f64	Transmit optical power per lane (dBm)
`dom_rx_power_dbm`	f64	Receive optical power per lane (dBm)
`dom_laser_bias_ma`	f64	Laser bias current (mA)
`dom_alarm_flags`	u16	Bitmask of high/low alarm/warning states
`dom_rx_power_thresh_high`	f64	Rx power high alarm threshold (dBm)
`dom_rx_power_thresh_low`	f64	Rx power low alarm threshold (dBm)
`dom_tx_power_thresh_high`	f64	Tx power high alarm threshold (dBm)
`dom_tx_power_thresh_low`	f64	Tx power low alarm threshold (dBm)
`dom_temp_thresh_high`	f64	Temperature high alarm threshold (°C)
`dom_temp_thresh_low`	f64	Temperature low alarm threshold (°C)
`dom_voltage_thresh_high`	f64	Voltage high alarm threshold (V)
`dom_voltage_thresh_low`	f64	Voltage low alarm threshold (V)

Threshold attributes are configurable on many transceiver platforms and define the bounds at which alarm flags are raised. When a monitored value crosses a threshold, the corresponding bit in `dom_alarm_flags` is set, enabling automated remediation or operator alerting.

10.3 PHY Training and Auto-Negotiation

Auto-negotiation and link-training attributes control the initial handshake between PHY endpoints. These are configured per interface and determine which speeds and encodings are advertised to the link partner.

Design Decision 63: PHY Training / Auto-Negotiation Attributes

Attribute	Type	Description
`phy_auto_negotiation`	bool	Enable IEEE 802.3 Clause 28/73 auto-negotiation
`phy_speed_advertise`	set	Set of speeds to advertise (e.g. 1G, 10G, 25G, 100G)
`phy_fec_mode`	enum	none, rs-fec, fc-fec, auto
`phy_link_training`	bool	Enable link training (Clause 72/93)
`phy_energy_efficient_ethernet`	bool	Enable EEE (IEEE 802.3az) low-power idle

When `phy_auto_negotiation` is enabled, the set of speeds in `phy_speed_advertise` is offered to the link partner during the negotiation exchange. The `phy_fec_mode` attribute selects the forward error correction scheme; `auto` defers to the negotiated result. EEE allows the PHY to enter a low-power idle state during gaps in traffic, reducing energy consumption on access ports.

10.4 Cable and Connector

Cable and connector attributes describe the physical medium attached to an interface. These are particularly relevant for breakout configurations on high-speed QSFP ports, where a single physical connector can be split into multiple logical interfaces.

Design Decision 64: Cable / Connector Attributes

Attribute	Type	Description
`phy_connector_type`	enum	rj45, sfp, sfp+, sfp28, qsfp28, qsfp-dd, cfp, cfp2
`phy_cable_length`	u32	Cable length in metres (passive copper DAC)
`phy_breakout_mode`	enum	none, 4x10g, 4x25g, 2x50g, 2x100g, 4x100g

The `phy_breakout_mode` attribute triggers port channelisation on the switch ASIC, creating multiple logical sub-interfaces from a single physical port. Changing this attribute typically requires a port-level reset.

10.5 Timing and Synchronisation

Synchronous Ethernet (SyncE) [100] and Precision Time Protocol (PTP) [101] provide frequency and phase synchronisation for applications such as mobile fronthaul and financial trading. The attributes below control per-port timing behaviour.

Design Decision 65: PTP Attributes (IEEE 1588)

Attribute	Type	Description
`ptp_enabled`	bool	Enable PTP on this interface
`ptp_profile`	enum	default-1588, G.8275.1 [102] (telecom), 802.1AS (gPTP)
`ptp_domain`	u32	PTP domain number (0–127)
`ptp_role`	enum	grandmaster, boundary-clock, slave, transparent
`ptp_clock_class`	u32	ITU-T G.8275.1 clock class
`ptp_announce_interval`	i32	Log_2 announce interval (−3 to +4)
`ptp_sync_interval`	i32	Log_2 sync message interval (−7 to +1)
`ptp_delay_mechanism`	enum	e2e (end-to-end), p2p (peer-to-peer)

Design Decision 66: SyncE Attributes (ITU-T G.8261)

Attribute	Type	Description
`synce_enabled`	bool	Enable SyncE on this port
`synce_quality_level`	enum	PRC, SSU-A, SSU-B, SEC, DNU
`synce_ssm_enabled`	bool	Enable SSM (Synchronisation Status Messages)

The `ptp_delay_mechanism` selects how path delay is measured: end-to-end uses delay-request/response exchanges with the grandmaster, while peer-to-peer measures delay on each hop independently. The `synce_ssm_enabled` attribute controls whether the port transmits and processes quality-level information in Ethernet Synchronisation Messaging Channel (ESMC) frames.

10.6 Power over Ethernet

IEEE 802.3af/at/bt defines Power over Ethernet (PoE) state:

Design Decision 67: PoE Attributes

Attribute	Type	Description
`poe_enabled`	bool	PoE sourcing enabled
`poe_standard`	enum	802.3af (15.4W), 802.3at (30W), 802.3bt Type 3/4 (60/90W)
`poe_power_class`	u8	Negotiated power class (0–8)
`poe_allocated_w`	f32	Allocated power in watts
`poe_consumed_w`	f32	Measured power draw
`poe_priority`	enum	critical, high, low

11 Layer 6–7: Security and Management

11.1 TLS/MACsec

Security state relevant to infrastructure devices:

Design Decision 68: Security Attributes

Attribute	Type	Description
`tls_min_version`	enum	1.2, 1.3 [103]
`tls_cipher_suites`	list	Allowed cipher suites
`tls_cert_cn`	str	Certificate common name

Beyond baseline TLS, infrastructure links increasingly require additional certificate lifecycle and transport-layer security attributes. TLS certificate chains allow validation up to a trusted root, while OCSP stapling reduces

reliance on external revocation checks during the handshake. DTLS extends the same security guarantees to datagram-oriented management protocols such as SNMP over UDP.

Design Decision 69: Extended TLS/DTLS Attributes

Attribute	Type	Description
`tls_cert_chain`	list	Ordered certificate chain to root CA
`tls_ocsp_stapling`	bool	OCSP stapling enabled [104]
`tls_session_resumption`	bool	Session ticket / ID resumption
`tls_mutual_auth`	bool	Mutual (client + server) authentication
`tls_policy_profile`	str	Named TLS policy profile reference
`dtls_enabled`	bool	DTLS transport enabled [105]
`dtls_version`	enum	1.2, 1.3

11.1.1 MACsec (802.1AE)

IEEE 802.1AE MACsec provides hop-by-hop Layer 2 encryption and integrity protection. The MKA protocol manages key distribution within each connectivity association. See Appendix B for the MACsec key hierarchy.

Design Decision 70: MACsec / MKA Attributes

Attribute	Type	Description
`macsec_enabled`	bool	MACsec enabled on interface
`macsec_cipher_suite`	enum	gcm-aes-128, gcm-aes-256, gcm-aes-xpn-128, gcm-aes-xpn-256
`macsec_confidentiality_offset`	enum	0, 30, 50 (bytes of unencrypted header)
`macsec_include_sci`	bool	Include SCI in SecTAG
`macsec_replay_protection`	bool	Anti-replay protection enabled
`macsec_replay_window`	u32	Replay window size (packets)
`mka_key_server_priority`	u8	MKA key server election priority
`mka_cak`	str	Connectivity Association Key (hex)
`mka_ckn`	str	Connectivity Association Key Name (hex)

11.1.2 802.1X Port-Based Access Control

IEEE 802.1X provides port-based network access control, relaying EAP authentication between supplicants and a RADIUS back-end.

Design Decision 71: 802.1X / RADIUS Attributes

Attribute	Type	Description
`dot1x_enabled`	bool	802.1X authenticator enabled
`dot1x_eap_method`	enum	peap, eap-tls, eap-fast, eap-ttls
`dot1x_reauth_period`	u32	Re-authentication interval (seconds)
`dot1x_max_reauth_req`	u8	Maximum re-authentication requests
`dot1x_guest_vlan`	u16	VLAN for unauthenticated hosts
`dot1x_auth_fail_vlan`	u16	VLAN on authentication failure
`dot1x_critical_vlan`	u16	VLAN when RADIUS is unreachable
`dot1x_mab_enabled`	bool	MAC Authentication Bypass fallback
`radius_dynamic_vlan`	bool	Accept RADIUS-assigned VLAN
`radius_coa_enabled`	bool	RADIUS Change-of-Authorization [106]

11.2 Management Plane

The management plane encompasses all protocols and services used to configure, monitor, and maintain network devices. SNMP provides polling-based monitoring, NETCONF and RESTCONF offer transactional configuration management via YANG models, and gNMI (gRPC Network Management Interface) supports streaming telemetry and configuration at scale. These attributes are node-level properties describing each device's management access configuration.

Design Decision 72: Management Node Attributes

Attribute	Type	Description
`hostname`	str	System hostname
`domain_name`	str	DNS domain name
`mgmt_address`	ip	Out-of-band management IP
`ntp_servers`	list	NTP server addresses [107]
`ntp_source_intf`	str	NTP source interface
`dns_servers`	list	DNS resolver addresses [108]
`syslog_servers`	list	Syslog destination addresses
`syslog_facility`	enum	local0–local7
`snmp_community`	str	SNMPv2c community [109]
`snmp_v3_users`	list	SNMPv3 USM users
`snmp_trap_targets`	list	SNMP trap/inform destinations
`netconf_enabled`	bool	NETCONF [91] enabled
`gnmi_enabled`	bool	gNMI enabled
`restconf_enabled`	bool	RESTCONF enabled
`aaa_method_authn`	list	Authentication method list (local, radius, tacacs)
`aaa_method_authz`	list	Authorization method list
`aaa_method_acct`	list	Accounting method list
`radius_servers`	list	RADIUS server addresses
`tacacs_servers`	list	TACACS+ server addresses
`ssh_version`	enum	v2
`ssh_port`	u16	SSH listening port
`banner_motd`	str	Login banner

11.3 QoS

Quality of Service (QoS) policy applies across layers:

Design Decision 73: QoS Endpoint Attributes

Attribute	Type	Description
`qos_policy_in`	str	Ingress QoS policy name
`qos_policy_out`	str	Egress QoS policy name
`qos_trust_mode`	enum	dscp, cos, untrusted
`qos_default_dscp`	u6	Default DSCP marking [83]
`qos_default_cos`	u3	Default 802.1p CoS
`qos_scheduler`	enum	strict-priority, wrr, dwrr, wfq
`qos_shaping_rate`	u64	Egress shaping rate (bps)
`qos_policing_rate`	u64	Ingress policing rate (bps)
`qos_policing_burst`	u32	Policer burst size (bytes)

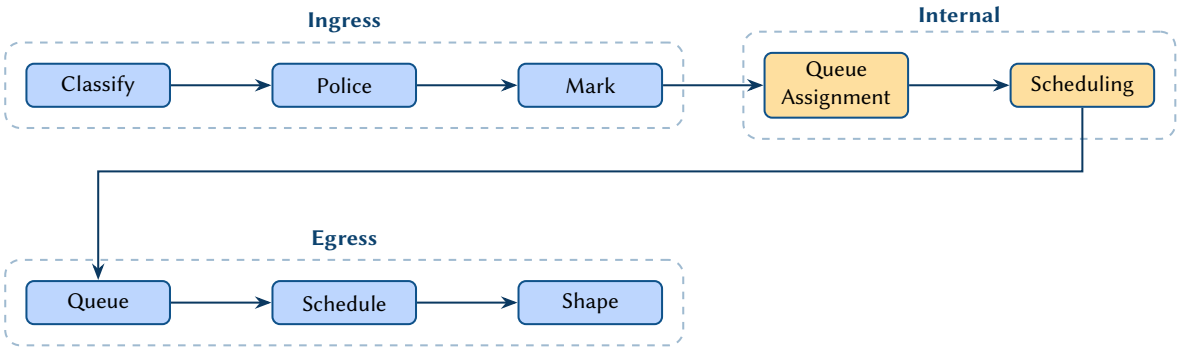


Figure 30. QoS processing pipeline. Ingress stages classify, police, and mark traffic before internal queue assignment and scheduling. Egress stages perform per-queue scheduling and traffic shaping before transmission.

11.4 ACLs

The interface-level ACL binding attributes are:

Design Decision 74: ACL Endpoint Attributes

Attribute	Type	Description
`acl_in`	str	Ingress ACL name [82]
`acl_out`	str	Egress ACL name

The full ACL rule structure, firewall chains, control plane policing, route-map extensions, prefix-lists, community-lists, and policy-based routing are covered in Section 7.

11.5 Hardware and Inventory

Hardware inventory attributes describe the physical platform on which the network operating system runs. These properties—serial numbers, installed modules, power supply status, environmental readings—are essential for asset management, capacity planning, and lifecycle tracking. In the plan topology they attach as node-level attributes to each device.

Design Decision 75: Hardware Node Attributes

Attribute	Type	Description
`vendor`	str	Device vendor (for inventory, not config)
`model`	str	Device model
`serial_number`	str	Chassis serial number
`os_type`	str	NOS type (e.g. eos, junos, iosxr, sros, sonic)
`os_version`	str	NOS version string
`platform`	str	Hardware platform / ASIC family
`role`	enum	spine, leaf, border-leaf, pe, p, ce, rr, …
`site`	str	Physical site / data centre
`rack`	str	Rack identifier
`rack_position`	u8	Rack unit position

Mathematical Foundations

12 The Rosetta Stone: Bridging Math and Network Operations

The mathematical framework presented in this section utilises Category Theory, Algebraic Graph Rewriting, and Tensors. While these rigorous formalisms are essential for enabling the Network Topology Engine (NTE) to guarantee *correctness-by-construction* and perform microsecond blast-radius calculations, they are deliberately abstracted away from the network operator during daily workflows.

To bridge the gap between pure mathematics and practical network engineering, Table 2 provides a translation of the core mathematical terminology into their operational networking equivalents.

Insight:
As a general rule: ***The Operator writes the Intent, and the Mathematics generates the State.*** *An engineer does not need a degree in Category Theory to use the system; they simply assign a VLAN or SGT, and the Functors rigorously compile the resulting routing and security matrices.*

Mathematical Concept	Definition in Pure Math	Network Engineering Equivalent
Functor ($\mathcal{F}$)	A structure-preserving mapping between two categories.	**Protocol State Machine / Control Plane.** E.g., The OSPF process that reads interface config and computes adjacencies.
Double Pushout (DPO)	A strict algebraic rule for finding and replacing subgraphs ($L \leftarrow K \rightarrow R$).	**Policy Application / Route Redistribution.** E.g., Matching an ACL to an interface, or transforming an OSPF route into a BGP route.
Supra-adjacency Tensor	A multidimensional matrix storing node relationships across multiple interconnected layers.	**Cross-Layer Dependency Map.** E.g., The combined structural truth of `show cdp neighbors`, `show ip ospf neighbor`, and `show bgp summary` evaluated simultaneously.
Boolean Semiring	An algebraic structure where addition is Logical OR ($\vee$) and multiplication is Logical AND ($\wedge$).	**Path Redundancy / Fate-Sharing.** E.g., Calculating whether ECMP multipath ($\vee$) survives a shared physical conduit cut ($\wedge$).
Directed Acyclic Graph (DAG)	A graph with directed edges and no cycles.	**Loop-Free Routing Topology / Execution Order.** E.g., Ensuring BGP routes do not redistribute back into OSPF, preventing a routing loop.

Table 2. The Rosetta Stone: Mapping pure mathematical concepts to daily network operations.

13 Multi-Layer Network Formalism

13.1 The Supra-Adjacency Tensor

Following De Domenico et al. [13] and Kivelä et al. [14], we model the network as a *multilayer network* $\mathcal{M} = (\mathcal{G}, C)$ where $\mathcal{G} = \{G_1, G_2, \dots, G_L\}$ is a family of layer graphs and C is the set of inter-layer couplings.

Insight: *The supra-adjacency tensor $M_{j\beta}^{i\alpha}$ provides the natural mathematical home for multi-protocol network state.*

Design Decision 76: Supra-Adjacency Tensor

The rank-4 adjacency tensor $M_{j\beta}^{i\alpha}$ encodes directed, weighted connections between node i in layer α and node j in layer β, where $i, j = 1, \dots, N$ (nodes) and $\alpha, \beta = 1, \dots, L$ (layers). When $\alpha = \beta$, entries represent *intra-layer* edges. When $\alpha \neq \beta$, entries represent *inter-layer* couplings.

Design Decision 77: Supra-Adjacency Matrix

Flattening the tensor yields the $NL \times NL$ supra-adjacency matrix:

$$\mathbf{A} = \begin{pmatrix} A^{11} & C^{12} & \cdots & C^{1L} \\ C^{21} & A^{22} & \cdots & C^{2L} \\ \vdots & \vdots & \ddots & \vdots \\ C^{L1} & C^{L2} & \cdots & A^{LL} \end{pmatrix}$$

where $A^{\alpha\alpha}$ is the $N \times N$ intra-layer adjacency matrix for layer α, and $C^{\alpha\beta}$ is the $N \times N$ inter-layer coupling matrix.

For our network model, the layers correspond to the extended OSI layers from Table 1. The coupling matrices $C^{\alpha\beta}$ encode "runs-on" or "depends-on" relationships: an OSPF adjacency in layer 3.5 *depends on* an IP-addressed interface in layer 3, which depends on an Ethernet link in layer 2, which depends on a physical port in layer 1.

The coupling matrices admit two canonical forms, depending on whether the inter-layer relationship connects a device to *itself* across layers or to a *different* device.

Design Decision 78: Coupling Matrix Classification

Let $C^{\alpha\beta} \in \mathbb{R}^{N \times N}$ be the coupling matrix from layer α to layer β. We distinguish:

1. **Identity coupling.** $C_{ij}^{\alpha\beta} = \delta_{ij} \cdot \mathbf{1}[i \in V_\alpha \cap V_\beta]$, where δ_{ij} is the Kronecker delta and V_α is the node set of layer α. This form asserts that device i in layer α *is the same physical device* as device i in layer β. The resulting coupling is a diagonal matrix whose nonzero entries correspond exactly to devices that participate in both layers.
2. **Dependency coupling.** $C_{ij}^{\alpha\beta} = w_{ij}$, where $w_{ij} \in \mathbb{R}_{\geq 0}$ is a weight that quantifies the strength or capacity of the dependency from node i in layer α to node j in layer β. This generalisation accommodates cross-device dependencies such as control-plane peerings that traverse intermediate infrastructure.

In the multiplex case (Axiom 2 below), all coupling matrices are of identity type.

In the specific network model developed in this report, the layers form a strict dependency chain:

$$L_1 \longrightarrow L_2 \longrightarrow L_{2.5} \longrightarrow L_3 \longrightarrow L_{3.5}$$

where the arrow $L_\alpha \to L_\beta$ denotes "layer β runs on top of layer α." Each arrow contributes a coupling matrix $C^{\beta\alpha}$ (encoding the downward dependency) that is of identity type. Consequently, the supra-adjacency matrix is *block-bidiagonal*: the only nonzero off-diagonal blocks are the coupling matrices between adjacent layers in the chain. Formally, $C^{\alpha\beta} = \mathbf{0}$ whenever $|\alpha - \beta| > 1$ in the layer ordering. This sparsity is not merely a convenience—it reflects the engineering reality that protocol stacks are composed layer by layer, without skip connections.

The dependency chain also induces a partial order on layers: layer α *depends on* layer β if there exists a directed path from L_β to L_α in the chain. A failure at layer β therefore propagates upward to every layer α with $\alpha > \beta$, a phenomenon that the supra-adjacency representation makes explicit through the composition of coupling matrices.

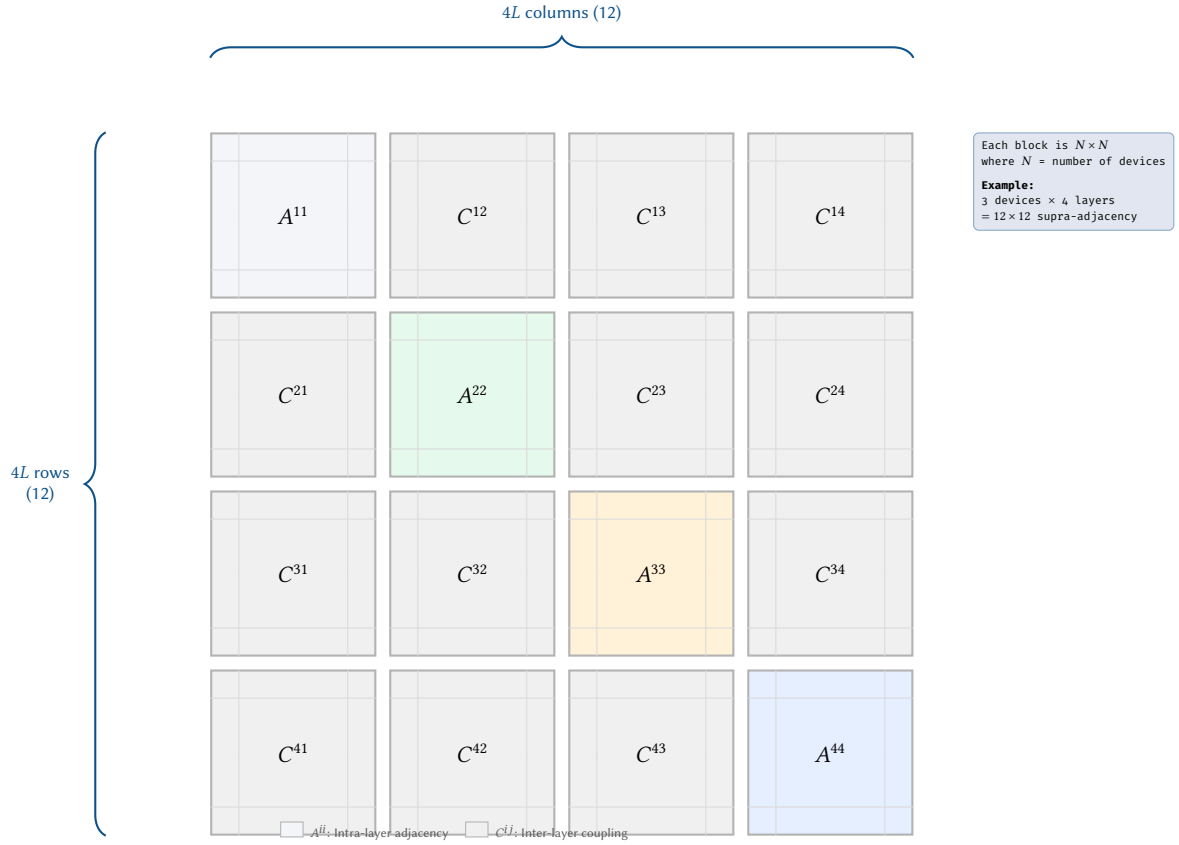


Figure 31. Supra-adjacency matrix structure for a four-layer multiplex network. Diagonal blocks A^{ii} encode intra-layer adjacency (coloured by layer), while off-diagonal blocks C^{ij} encode inter-layer coupling. For N devices and L layers the full matrix is $NL \times NL$.

13.2 Multiplex vs Interconnected Structure

Our network model is a *multiplex* network [15]: the same physical device appears across multiple layers (as a router in layer 3.5, as an MPLS LSR in layer 2.5, as a switch in layer 2, etc.). Inter-layer edges are *identity couplings* that connect the same device across layers.

Axiom 2 (Multiplex Identity Coupling). *For every device node v present in layers α and β, there exists an inter-layer edge (v, α, v, β) with weight 1. No cross-device inter-layer edges are permitted.*

This ensures that the multi-layer model decomposes cleanly into per-device "pillars" connected by identity couplings, with intra-layer edges representing protocol adjacencies within each layer.

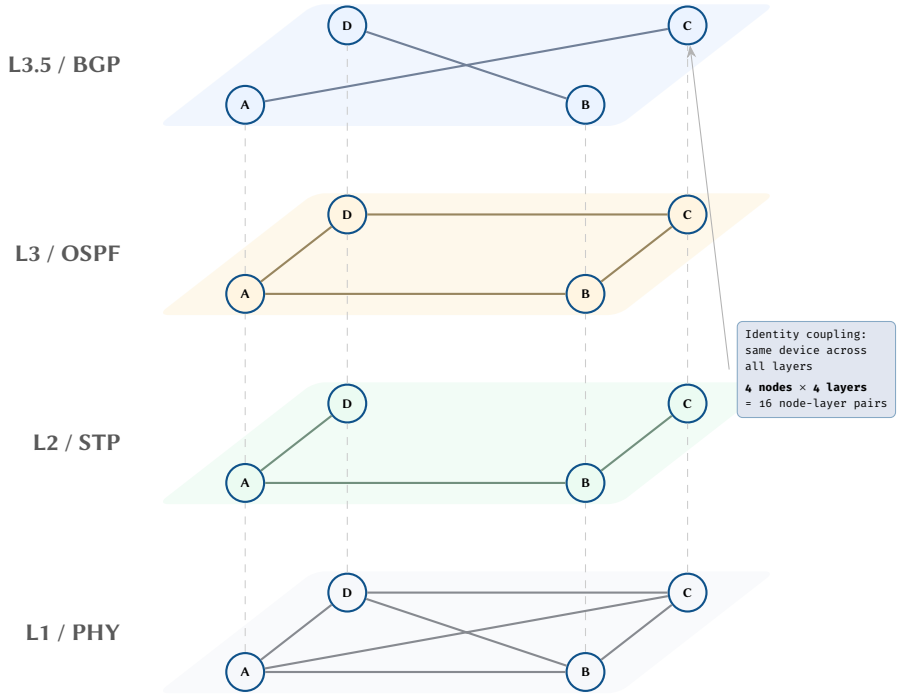


Figure 32. Multiplex network with four protocol layers. Each horizontal plane hosts the same four devices but with layer-specific adjacencies: a full mesh at L1, a spanning tree at L2, an OSPF ring at L3, and sparse iBGP sessions at L3.5. Dashed vertical lines represent identity coupling between layers.

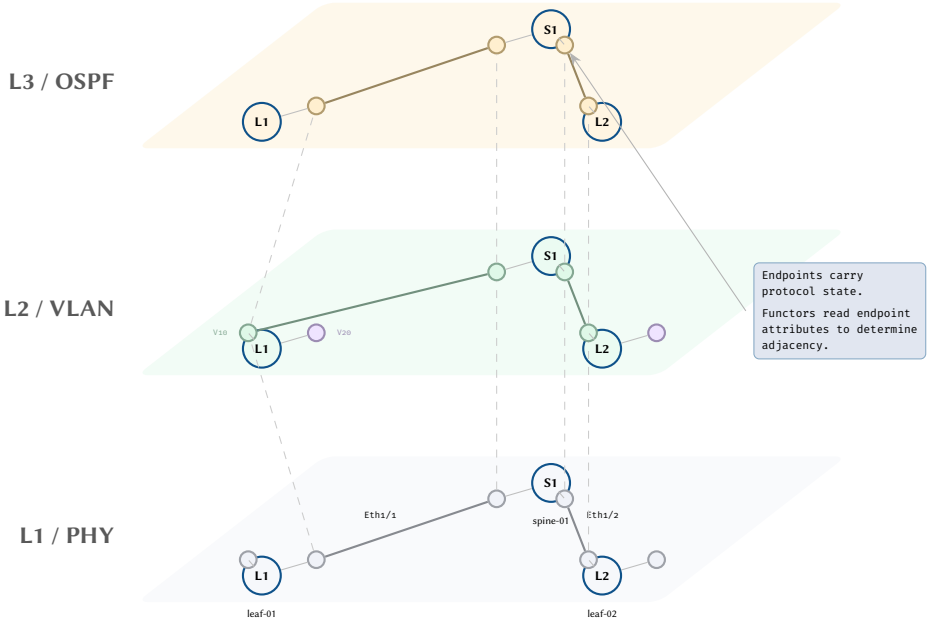


Figure 33. Multi-layer view with endpoint granularity. Each device's interfaces appear as endpoint vertices within each layer plane, and adjacency edges connect endpoints, not devices.

13.3 Layer Participation and Projection

Given the multilayer network $\mathcal{M} = (\mathcal{G}, \mathcal{C})$, it is frequently necessary to reason about individual layers in isolation and about the degree to which each device is involved in the overall protocol stack. We formalise both notions below.

Layer projection. The *projection operator* Π_α extracts the subgraph corresponding to a single layer:

$$\Pi_\alpha(\mathcal{M}) \;=\; G_\alpha \;=\; (V_\alpha,\, E_\alpha,\, A^{\alpha\alpha})$$

where $V_\alpha \subseteq V$ is the set of nodes active in layer α, E_α is the intra-layer edge set, and $A^{\alpha\alpha}$ is the intra-layer adjacency matrix. The projection discards all inter-layer couplings and all nodes not present in layer α. Applying Π_α to the supra-adjacency matrix is equivalent to selecting the α-th diagonal block.

Node participation vector. For each node $i \in V$, define the *participation vector*

$$\mathbf{b}_i \;=\; (\beta_1,\, \beta_2,\, \ldots,\, \beta_L) \qquad \text{where} \quad \beta_\alpha \;=\; \begin{cases} 1 & \text{if } i \in V_\alpha, \\ 0 & \text{otherwise.} \end{cases}$$

The participation vector is a binary indicator that records which protocol layers a given device participates in. Its ℓ_1-norm $\|\mathbf{b}_i\|_1 = \sum_\alpha \beta_\alpha$ equals the *cross-layer degree* of node i (defined formally in Section 13.4 below).

Not every device appears in every layer. A pure layer-2 access switch operates only in layers 1 and 2; it has no IP-addressed interfaces (layer 3), no routing-protocol adjacencies (layer 3.5), and no MPLS label-switching entries (layer 2.5). Conversely, a core router typically participates in all five layers. The participation vector makes these distinctions precise and computable.

Participation Vectors by Device Role

Consider a campus network with three device roles and the layer ordering $L_1, L_2, L_{2.5}, L_3, L_{3.5}$:

1. **Access switch** (pure L2, no routing): $\mathbf{b}_{\text{access}} = (1,\, 1,\, 0,\, 0,\, 0)$. The switch has physical ports (L1) and Ethernet/VLAN forwarding (L2), but performs no IP routing or MPLS switching.
2. **Distribution/spine switch** (L3 capable, no MPLS): $\mathbf{b}_{\text{spine}} = (1,\, 1,\, 0,\, 1,\, 1)$. The spine switch participates in L1, L2, L3 (IP forwarding), and L3.5 (OSPF or BGP adjacencies), but does not run MPLS (L2.5).
3. **Core router** (full stack including MPLS): $\mathbf{b}_{\text{core}} = (1,\, 1,\, 1,\, 1,\, 1)$. The core router participates in every layer, including MPLS label-switched paths at L2.5.

The cross-layer degrees are $\|\mathbf{b}_{\text{access}}\|_1 = 2$, $\|\mathbf{b}_{\text{spine}}\|_1 = 4$, and $\|\mathbf{b}_{\text{core}}\|_1 = 5$. Higher cross-layer degree implies greater configuration complexity and a larger blast radius upon device failure.

The participation vector also serves as an input to automated validation: if a device's declared role implies participation in layer α but $\beta_\alpha = 0$ (i.e. no edges exist in that layer), the model can flag a potential misconfiguration before deployment.

13.4 Aggregation and Cross-Layer Metrics

While the full supra-adjacency tensor preserves per-layer detail, several useful summary statistics can be derived by aggregating across layers. These metrics quantify the structural complexity of the network's protocol stack and provide actionable indicators for capacity planning, risk assessment, and change-impact analysis.

Aggregate adjacency matrix. The *aggregate* (or *flattened*) adjacency matrix collapses all intra-layer adjacency information into a single $N \times N$ matrix:

$$A^*_{ij} \;=\; \sum_{\alpha=1}^{L} A^{\alpha\alpha}_{ij}.$$

An entry $A^*_{ij} > 0$ indicates that nodes i and j share at least one protocol-layer adjacency. The value of A^*_{ij} counts (or, in the weighted case, sums the weights of) the distinct layers in which i and j are neighbours. The aggregate matrix is the adjacency matrix of the *overlay graph* obtained by projecting the multilayer network onto a single layer.

Design Decision 79: Cross-Layer Aggregate Metrics

Let $\mathcal{M}$ be a multilayer network with N nodes and L layers.

1. **Cross-layer degree.** The cross-layer degree of node i is $d_i^{\times} = \|\mathbf{b}_i\|_1 = \sum_{\alpha=1}^{L} \beta_\alpha$, the number of layers in which node i participates.
2. **Aggregate degree.** The aggregate degree of node i is $d_i^{*} = \sum_j \mathbf{1}[A_{ij}^{*} > 0]$, the number of distinct neighbours of i across all layers.
3. **Inter-layer edge density.** The fraction of realised inter-layer couplings relative to the theoretical maximum:
$$\rho_{\text{inter}} = \frac{\sum_{\alpha \neq \beta} \sum_{i,j} \mathbf{1}[C_{ij}^{\alpha\beta} > 0]}{N^2 \cdot L(L-1)}.$$
In the multiplex case with identity coupling, ρ_{inter} simplifies to $\frac{1}{N \cdot L(L-1)} \sum_{\alpha \neq \beta} |V_\alpha \cap V_\beta|$.
4. **Layer redundancy index.** The mean cross-layer degree normalised by L: $\bar{r} = \frac{1}{NL} \sum_i d_i^{\times}$. A value of $\bar{r} = 1$ indicates that every node participates in every layer; $\bar{r} \ll 1$ indicates a sparse, role-differentiated network.

These metrics have direct engineering interpretations. The cross-layer degree $d_i^{\times}$ of a device determines the number of protocol configurations that must be generated and validated for that device—higher values imply larger configuration files, more failure modes, and longer maintenance windows. The inter-layer edge density ρ_{inter} measures how tightly coupled the layers are; a high density suggests that changes in one layer are likely to cascade. The layer redundancy index $\bar{r}$ characterises the overall role diversity: a network populated entirely by identical full-stack routers has $\bar{r} \approx 1$, while a hierarchical campus design with differentiated access, distribution, and core tiers will exhibit $\bar{r} < 1$.

13.5 Relationship to the Plan Topology

The preceding subsections developed the multilayer network $\mathcal{M}$ as an abstract tensor object. We now connect this formalism to the concrete *plan topology* G_{plan} introduced in Section 16.

The plan topology G_{plan} is, by design, a *flattened* representation: it stores all protocol-layer attributes—interface addresses, VLAN assignments, OSPF costs, BGP policies, MPLS label bindings—as node and edge attributes within a single attributed graph. The multilayer structure is implicit in these attributes rather than explicit in the graph topology. The overlay-mapping functor $\mathcal{F}$ defined in Section 17 serves precisely to *reconstruct* the per-layer views from this flattened representation.

Concretely, each layer functor $\mathcal{F}_\alpha$ (e.g. $\mathcal{F}_{\text{OSPF}}$, $\mathcal{F}_{\text{BGP}}$, $\mathcal{F}_{\text{VLAN}}$) reads the relevant attributes from G_{plan} and produces a layer graph G_α. The collection of all such functors yields the full multilayer decomposition:

$$\mathcal{F}(G_{\text{plan}}) = \big(\mathcal{F}_1(G_{\text{plan}}),\ \mathcal{F}_2(G_{\text{plan}}),\ \ldots,\ \mathcal{F}_L(G_{\text{plan}})\big) = \mathcal{G}.$$

Combined with the identity coupling matrices C derived from node co-occurrence across layers, this gives the fundamental reconstruction result:

Axiom 3 (Plan–Multilayer Reconstruction). *Let G_{plan} be a well-formed plan topology and let $\mathcal{F} = \{\mathcal{F}_\alpha\}_{\alpha=1}^{L}$ be the family of layer functors. Then*

$$\mathcal{F}(G_{\text{plan}}) = \mathcal{M} = (\mathcal{G}, C)$$

where $\mathcal{G} = \{G_\alpha\}$ is the family of layer graphs produced by the functors and C is the set of identity coupling matrices induced by $C_{ij}^{\alpha\beta} = \delta_{ij} \cdot \mathbf{1}[i \in V_\alpha \cap V_\beta]$.

This reconstruction is *deterministic* and *total*: given a well-formed plan topology, the multilayer network $\mathcal{M}$ is uniquely determined. The practical consequence is significant. Engineers work with G_{plan}—a single, versionable, diffable attributed graph—rather than with L separate layer graphs and $\binom{L}{2}$ coupling matrices. The tensor formalism developed in this section provides the analytical machinery for reasoning about cross-layer properties (failure propagation, dependency depth, layer participation), while the plan topology provides the operational data structure for storage, modification, and configuration generation.

Insight: *The plan topology G_{plan} is the "source of truth" from which the entire multilayer tensor can be deterministically reconstructed via the layer functors. This means that storing and versioning a single flat graph suffices to capture the full multi-protocol state of the network.*

The two representations are thus complementary: G_{plan} is optimised for *engineering workflows* (editing, committing, reviewing), while $\mathcal{M}$ is optimised for *mathematical analysis* (spectral methods, percolation theory, centrality measures on multilayer networks). The functor $\mathcal{F}$ is the bridge that allows practitioners to move freely between the two views without loss of information.

14 Alternative Graph Representations

Before committing to the property hypergraph formalism, it is instructive to examine the principal competing graph representations and evaluate their suitability for encoding multi-layer network state. Each representation brings genuine strengths—mature tooling, reasoning engines, algebraic compositionality—but also structural limitations that motivate the design choices made in this report. This section surveys four alternatives (labeled property graphs, RDF/OWL triple stores, relational schemas, and category-theoretic approaches) before synthesising the rationale for a property hypergraph model.

14.1 Labeled Property Graph (LPG)

The Labeled Property Graph (LPG) model, popularised by Neo4j [110], represents data as a directed graph in which both nodes and relationships (edges) carry an arbitrary set of key-value *properties*, and each node and relationship bears one or more *labels* or *types*. The natural mapping for network state is immediate: a router becomes a node labeled `:Device` with properties such as `hostname`, `role`, and `mgmt_ip`; an OSPF adjacency becomes a relationship of type `:OSPF_ADJ` from one device to another, decorated with properties `area_id`, `cost`, and `state`.

Querying is performed in the Cypher declarative language, which provides concise pattern-matching syntax for graph traversals. A typical two-hop path query that finds all devices reachable within two OSPF hops of a given router can be expressed as:

Insight: *The LPG model excels at point-to-point relationships but struggles with set-membership constructs such as VLANs, multicast groups, and broadcast domains that are inherently hyperedge-like.*

```
MATCH (src:Device {hostname: 'pe01'})-[:OSPF_ADJ*1..2]-(dst:Device)
RETURN dst.hostname, dst.role
```

The strength of Cypher lies in its declarative traversal semantics: variable-length path patterns, optional matches, and aggregation functions allow complex reachability and shortest-path queries without explicit loop constructs. Neo4j's index-free adjacency—where each node holds a physical pointer to its neighbours—yields $O(1)$ per-hop traversal cost regardless of total graph size, a significant advantage over join-based relational engines for deep path queries.

However, the LPG model has a fundamental structural limitation: it supports only binary edges. A VLAN broadcast domain that spans n ports cannot be represented as a single hyperedge; it must be decomposed. Two decomposition strategies are common:

1. **Clique expansion**: create $\binom{n}{2}$ pairwise edges of type `:SAME_VLAN` between every pair of member ports. This preserves traversability but introduces quadratic edge blow-up, obscures the set-membership semantics, and makes VLAN deletion an $O(n^2)$ operation.
2. **Junction node**: introduce an auxiliary node labeled `:VLAN` (or `:BroadcastDomain`) and connect each member port to it via a `:MEMBER_OF` relationship. This restores linear edge count but conflates two distinct entity types—physical ports and logical broadcast domains—into the same node namespace, complicating schema evolution and query optimisation.

Design Decision 80: LPG Encoding: VLAN as Junction Node

In the junction-node approach, a VLAN broadcast domain with n member ports is encoded as:

Entity	Label	Properties
Junction node	`:VLAN`	`vlan_id`: 100, `name`: "Server"
Member edge 1	`:MEMBER_OF`	`tagging`: "tagged"
Member edge 2	`:MEMBER_OF`	`tagging`: "untagged"
…	`:MEMBER_OF`	(one edge per member port)

This encoding uses n edges (linear in port count) but introduces an auxiliary node that is neither a device nor an interface, requiring schema conventions to distinguish "real" network elements from structural artifacts.

A second limitation arises from Neo4j's lack of native support for multi-layer indexing. Representing the same device across OSI layers requires either duplicating the node with per-layer labels (breaking identity) or adding layer-discriminating properties to every relationship (increasing query complexity). Neither approach recovers the clean separation of intra-layer and inter-layer edges provided by the supra-adjacency tensor of Section 13.

14.2 RDF/OWL Triple Store

The Resource Description Framework (RDF) [111] models knowledge as a set of *triples* (subject, predicate, object), where each element is identified by a URI. In the network domain, a triple such as ⟨`:pe01`, `:hasInterface`, `:pe01-eth0`⟩ asserts that device `pe01` has an interface `pe01-eth0`. Scalar attributes are expressed as triples with literal objects: ⟨`:pe01-eth0`, `:mtu`, "9216"⟩.

The primary strength of the RDF ecosystem lies in *ontological reasoning*. Using OWL (Web Ontology Language) class hierarchies and inference rules, a reasoner can derive facts not explicitly stated in the data. For example, an ontology might declare that any device with `bgp_router_id` ≠ null is an instance of class `:BGPSpeaker`, and that all instances of `:BGPSpeaker` are also instances of `:L3Device`. SPARQL queries can then match on inferred classes without enumerating the classification logic at query time.

Design Decision 81: RDF Encoding: OSPF Adjacency

An OSPF adjacency between two routers, expressed as RDF triples in Turtle syntax:

Subject	Predicate	Object
`:pe01`	`rdf:type`	`:Router`
`:pe02`	`rdf:type`	`:Router`
`:ospf_adj_01`	`rdf:type`	`:OSPFAdjacency`
`:ospf_adj_01`	`:source`	`:pe01`
`:ospf_adj_01`	`:target`	`:pe02`
`:ospf_adj_01`	`:areaId`	`"0.0.0.0"`
`:ospf_adj_01`	`:cost`	`"10"`
`:ospf_adj_01`	`:state`	`:Full`

Note the reification pattern: the adjacency itself becomes a node (`:ospf_adj_01`) with its own triples, because RDF triples cannot carry properties natively.

The reification overhead visible in the example above is the central limitation of RDF for network state modelling. Every edge that carries properties—and in a network graph, *most* edges carry properties (cost, state, metric, area, AS number)—must be decomposed into a blank or named node with additional triples for each property. An OSPF adjacency that would be a single attributed edge in an LPG becomes 6–8 triples in RDF. For a data-centre fabric with ~10,000 protocol adjacencies, reification can triple the triple count (an unfortunate tautology), degrading both storage efficiency and query performance.

SPARQL, while Turing-complete and well-standardised, imposes verbose query patterns compared to Cypher. A two-hop OSPF traversal that takes two lines in Cypher requires explicit triple-pattern joins across the reified adjacency nodes. The verbosity is not merely aesthetic; it increases the cognitive load on network engineers who are the primary consumers of the model.

OWL reasoning remains the compelling differentiator. In Section 14.5 we argue that the property hypergraph can recover much of this reasoning capability through typed constraint validators operating on flat attributes, without the storage overhead of full RDF reification.

14.3 Relational/Tabular Model

The relational model remains the dominant paradigm for network inventory and IP address management systems such as NetBox, Device42, and Nautobot. A normalised schema for network state typically employs four core tables linked by foreign keys:

Design Decision 82: Relational Schema: Network State Tables

Table	Primary Key	Columns
`nodes`	`node_id`	`hostname`, `role`, `vendor`, `model`, `mgmt_ip`, `site_id`
`endpoints`	`ep_id`	`node_id` (FK), `name`, `if_type`, `admin_status`, `oper_status`, `speed`, `mtu`
`attributes`	`attr_id`	`owner_type` (node/ep), `owner_id` (FK), `key`, `value`, `layer`
`edges`	`edge_id`	`src_ep_id` (FK), `dst_ep_id` (FK), `edge_type`, `layer`

Protocol-specific attributes are stored in the entity-attribute-value (EAV) `attributes` table, avoiding wide-table schemas that would require ALTER TABLE for each new protocol. Edges reference source and destination endpoints via foreign keys.

The relational model offers several practical advantages: ACID transactions, mature indexing (B-tree, GiST for IP ranges), SQL as a lingua franca understood by virtually every engineering team, and straightforward integration with business intelligence and reporting tools.

However, graph traversal queries are fundamentally expensive in a relational engine. Computing the transitive closure of a routing adjacency—"find all routers reachable from `pe01` via OSPF"—requires either recursive common table expressions (CTEs) or iterative self-joins on the `edges` table. Each hop adds a join, and the query

planner must materialise intermediate result sets that grow combinatorially with path depth. PostgreSQL's `WITH RECURSIVE` mitigates this syntactically but does not change the underlying $O(|V| + |E|)$ per-level cost, and query plans for variable-depth traversals are notoriously difficult to optimise.

A second limitation is schema rigidity. The EAV pattern used in the `attributes` table recovers some flexibility, but at the cost of type safety (all values stored as text), query complexity (every attribute access requires a join or subquery), and index effectiveness (a single `(key, value)` index cannot be specialised per attribute type). Adding a new protocol layer—say, SRv6—requires no schema migration but degrades query performance as the EAV table grows.

The relational model also shares the LPG's limitation on hyperedges: VLAN membership must be modelled either as a junction table (`vlan_members`) or as rows in the `edges` table pointing to a synthetic VLAN node. Neither representation captures the broadcast-domain semantics natively.

14.4 Category-Theoretic Approaches

Category theory provides the most mathematically rigorous framework for compositional system modelling. For readers unfamiliar with the formalism, Section 1.2 provides brief working definitions of the key terms (category, functor, morphism, operad, cospan). Two categorical constructions have direct relevance to network state: operads (in the sense of Spivak [21]) and decorated cospans (Baez and Fong [19]).

14.4.1 Operads and Wiring Diagrams

Spivak's typed operads [21] model compositional assembly via *wiring diagrams*: each operation (morphism) takes a tuple of typed inputs and produces a tuple of typed outputs, and operations compose by connecting outputs of one box to inputs of another. In the network domain, a router can be modelled as an operation whose inputs and outputs are typed interfaces (Ethernet, MPLS, loopback), and the network topology emerges from wiring these interfaces together according to compatibility constraints.

The operad framework naturally enforces type safety—an Ethernet port cannot be wired to a serial interface without an explicit adapter morphism—and supports hierarchical composition: a campus network is an operation that internally composes building-level operations, which in turn compose floor-level switches. Fong and Spivak [112] extend this framework to *hypergraph categories*, providing categorical semantics for systems with shared interfaces.

However, the operad approach is fundamentally *structural*: it models how components connect, not what configuration state they carry. Encoding the configuration attributes of an OSPF interface (area, cost, hello/dead intervals, authentication) within an operadic framework requires extending each operation's type signature with state-carrying parameters, producing unwieldy type expressions for any realistic protocol stack.

14.4.2 Decorated Cospans

Baez and Fong's decorated cospans [19] address exactly this gap between structure and state. A cospan $A \to S \leftarrow B$ in a category $\mathbf{C}$ models an open system with boundary ("feet") A and B and internal structure S. The *decoration* is a functor $F : \mathbf{C} \to \mathbf{Set}$ that assigns to each internal structure its state space—a set of possible internal states. Two open systems compose by gluing along shared boundaries, and the decoration functor ensures that internal states are compatible across the join.

Decorated Cospan: OSPF Domain

Consider a 3-router OSPF domain modelled as a decorated cospan. Let $A = \{r_1\}$ and $B = \{r_3\}$ be the boundary (ingress/egress) routers, and $S = \{r_1, r_2, r_3\}$ the internal structure. The feet are the cospan legs $\iota_A : A \hookrightarrow S$ and $\iota_B : B \hookrightarrow S$.
The decoration $F(S)$ assigns to S the state space containing all valid OSPF configurations: area assignments, interface costs, hello/dead intervals, adjacency states, and the resulting shortest-path tree. Formally, $F(S) = \prod_{(i,j)\in E(S)} \Sigma_{\text{OSPF}}$ where Σ_{OSPF} is the set of valid OSPF adjacency state tuples.
Composing this cospan with an adjacent BGP domain (sharing boundary router r_3) yields a larger cospan whose decoration captures the combined OSPF+BGP state, with redistribution constraints enforced at the composition boundary.

The decorated cospan framework provides exactly the compositional semantics needed for modular network verification: verify each domain independently, then compose the results along shared boundaries. However, the practical barriers are substantial. No production-grade graph database or query engine implements categorical composition natively. The abstraction barrier is steep: expressing a routine query like "find all interfaces in OSPF area 0 with cost > 100" requires navigating the cospan-decoration structure rather than a simple attribute filter. And the framework, while mathematically elegant, offers no inherent advantage for the columnar SIMD queries that drive performance in the NTE implementation (Section 25).

14.5 Why Property Hypergraph

The preceding survey reveals that each representation excels along one axis but falters along others. The LPG provides efficient traversal but cannot express hyperedges natively. RDF enables ontological reasoning but imposes reification overhead for edge properties. The relational model offers transactional guarantees and mature tooling but performs poorly on graph traversals and lacks native hyperedge support. Category-theoretic frameworks provide compositional rigour but lack practical tooling and impose a steep abstraction barrier.

Design Rule 5: Representation Selection

The property hypergraph is selected as the canonical representation for network state because it satisfies three requirements simultaneously: (i) **hyperedges** for broadcast domains, multicast groups, and VLAN membership without clique expansion or junction nodes; (ii) **flat key-value properties** on nodes and endpoints for columnar SIMD processing in the NTE's Arrow-backed dataframes; (iii) **layer indexing** via the supra-adjacency tensor for clean multiplex network decomposition. No single alternative satisfies all three.

The following comparison table summarises the trade-offs across seven evaluation criteria:

Insight: *The property hypergraph can be viewed as a pragmatic synthesis: LPG-style properties, RDF-style typed relationships, relational-style columnar storage for attributes, and category-theoretic compositionality via the functorial overlay mapping $\mathcal{F}$.*

Design Decision 83: Representation Comparison

Criterion	LPG	RDF	Relational	Cat-Theory	Prop-HG
Hyperedge support	—	—	—	✓	✓
Edge properties	✓	reify	join	functor	✓
Query language	Cypher	SPARQL	SQL	—	filter DSL
Reasoning	—	OWL	triggers	proof	validators
Tool ecosystem	✓	✓	✓	—	emerging
SIMD-friendliness	—	—	✓	—	✓
Formalisability	informal	✓	✓	✓	✓

The property hypergraph is the only representation that achieves ✓ marks in all of hyperedge support, edge properties, SIMD friendliness, and formalisability. The "emerging" tool ecosystem is a genuine limitation, addressed in this report by providing a complete formal specification and reference implementation architecture (Section 25).

The comparison diagram in Figure 34 illustrates these differences concretely by encoding the same three-router, two-VLAN network fragment in each of the four primary representations.

The following diagrams illustrate how key network constructs—IP addressing, ACL/firewall policy, resource allocation, and route policy—map concretely into the property hypergraph model.

14.6 Implementing Hyperedges: True Hyperedges vs Junction Nodes

The property hypergraph formalism uses true hyperedges to represent set-membership constructs such as VLAN broadcast domains and multicast groups. However, most practical graph engines—including the NTE—store only binary edges. This creates a design question: should the implementation provide native hyperedge primitives, or should it encode hyperedges using *junction nodes* (also called intermediate or auxiliary nodes)?

14.6.1 The Junction Node Encoding

A junction node is a synthetic graph node that represents the set itself. Each member of the set connects to the junction node via a binary `MEMBER_OF` edge. For a VLAN broadcast domain with n member endpoints:

- **True hyperedge**: one hyperedge incident on all n endpoints, carrying properties `vlan_id`, `name`, `stp_root_bridge`, etc.
- **Junction node**: one node labeled `BroadcastDomain` with properties `vlan_id`, `name`, etc., plus n binary edges of type `MEMBER_OF`, each optionally carrying per-member properties such as `tagging` (tagged/untagged).

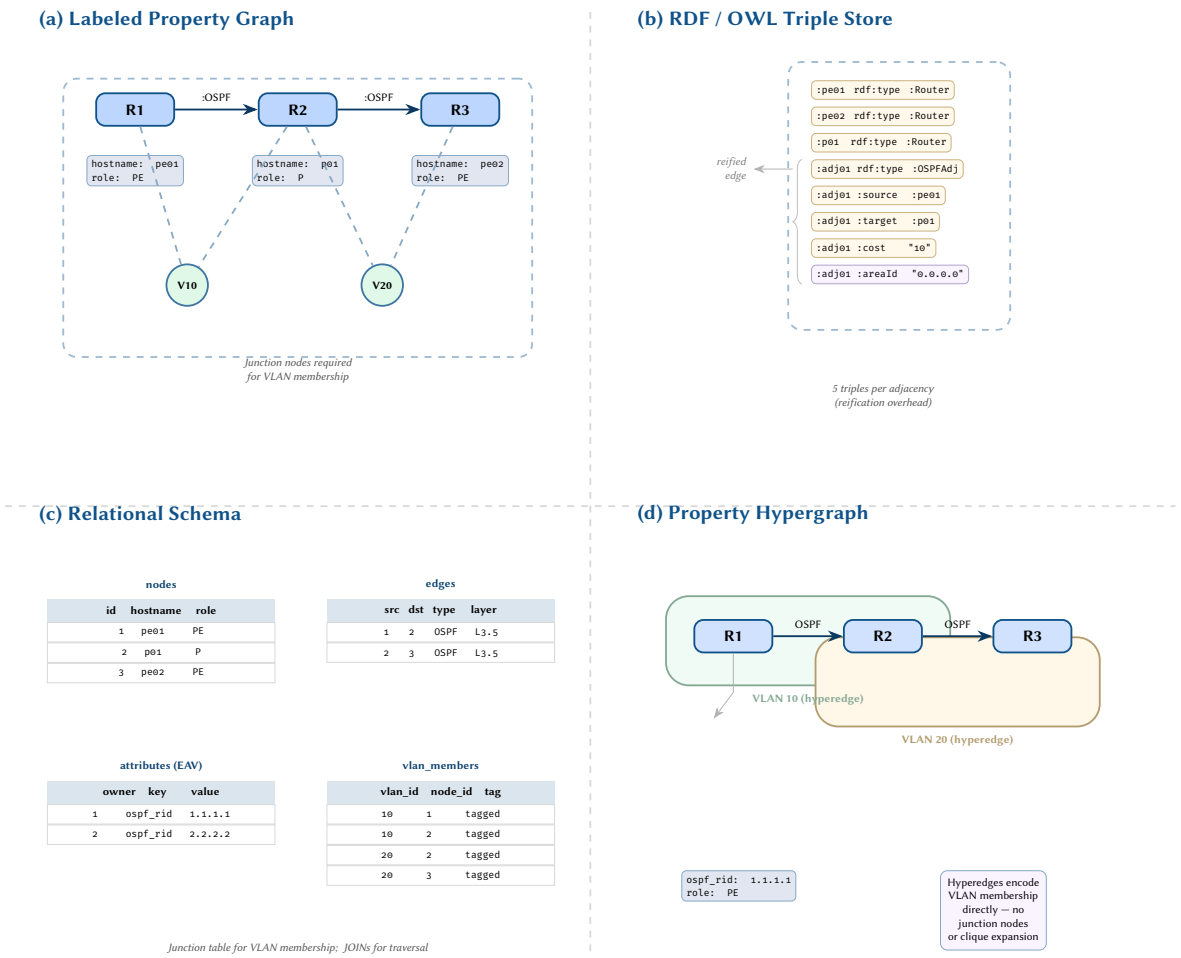


Figure 34. The same 3-router, 2-VLAN network fragment represented in four graph models. (a) The LPG requires junction nodes `V10` and `V20` to encode VLAN membership. (b) The RDF triple store reifies each OSPF adjacency into multiple triples. (c) The relational schema uses separate tables linked by foreign keys, with a junction table for VLAN membership. (d) The property hypergraph encodes VLANs as shaded hyperedges enclosing their member nodes directly, with flat properties on nodes and binary edges for point-to-point adjacencies.

Design Decision 84: Hyperedge vs Junction Node Comparison

Criterion	True Hyperedge	Junction Node
Edge count	1	n
Node count	0 additional	1 additional
Per-member properties	awkward	natural (on each edge)
Query: "all members of VLAN 100"	direct incidence	one-hop traversal
Query: "which VLANs does port p belong to?"	scan hyperedges	one-hop traversal
Binary graph engine support	requires extension	native
Schema clarity	pure semantics	requires label conventions

The junction node encoding has an additional advantage for the plan topology: per-member properties become edge attributes on the individual `MEMBER_OF` edges rather than requiring indexed attribute keys on the hyperedge. For example, a port's VLAN tagging mode (`tagged`/`untagged`) is a natural property of the membership relationship, not of the broadcast domain itself or of the port in isolation.

Insight:
Junction nodes are the pragmatic choice for implementations that use binary graph storage. The NTE uses junction nodes labeled `BroadcastDomain`, `MulticastGroup`, or `SecurityGroup` connected via typed `MEMBER_OF` edges. The formal model retains hyperedge notation for mathematical clarity, with the understanding that each hyperedge maps to a junction node in the implementation.

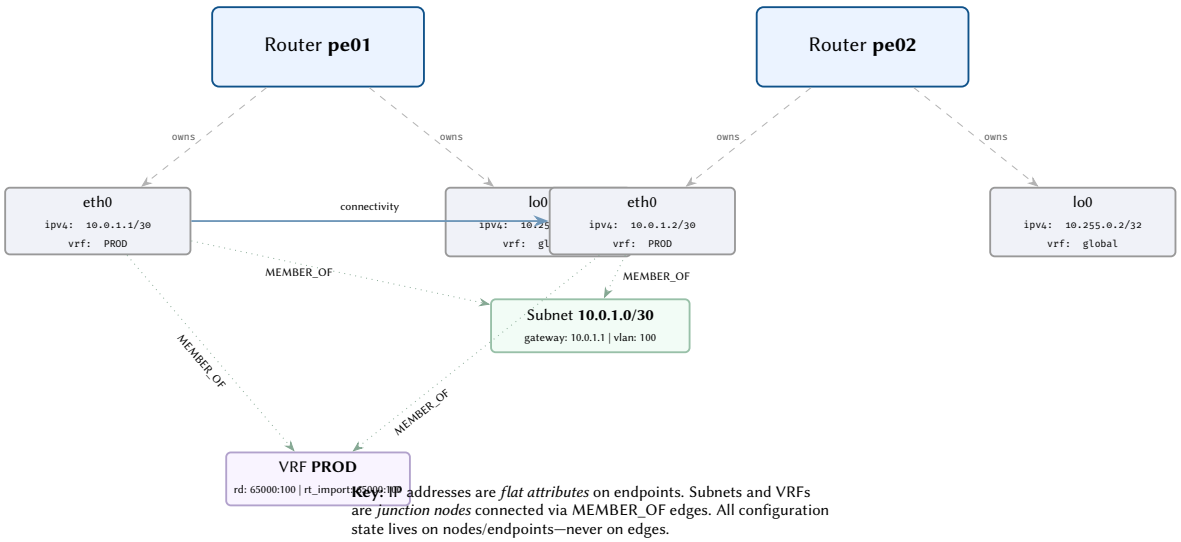


Figure 35. IP addressing mapped to the property hypergraph: addresses as endpoint attributes, subnets and VRFs as junction nodes.

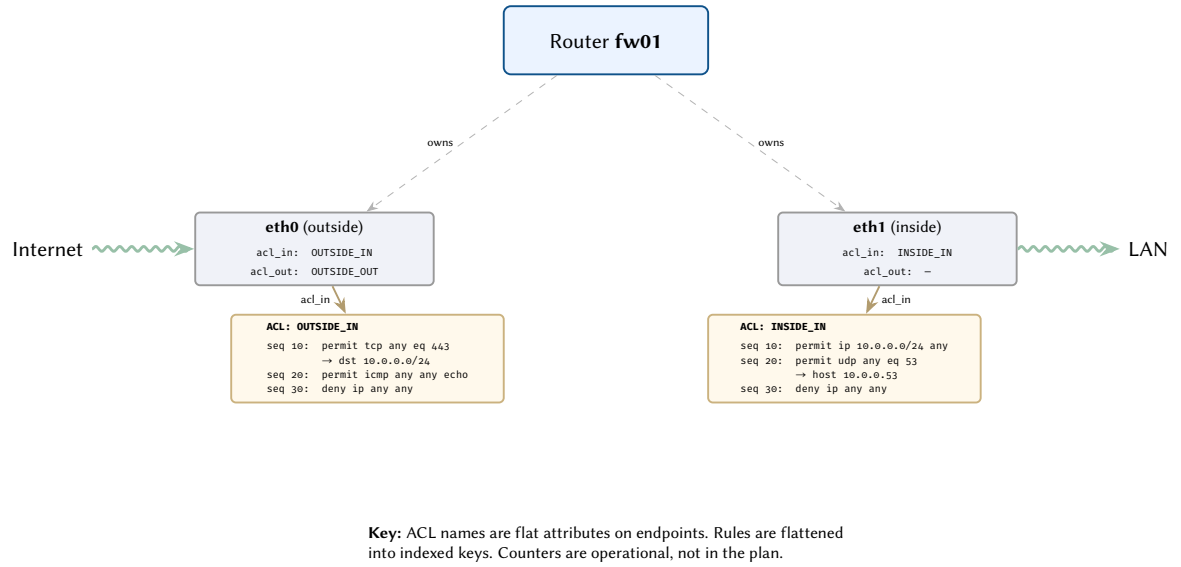


Figure 36. ACL/firewall policy in the property hypergraph: ACL bindings as endpoint attributes, rules as flattened indexed keys.

Design Decision 85: NTE Implementation: Junction Node for VLAN

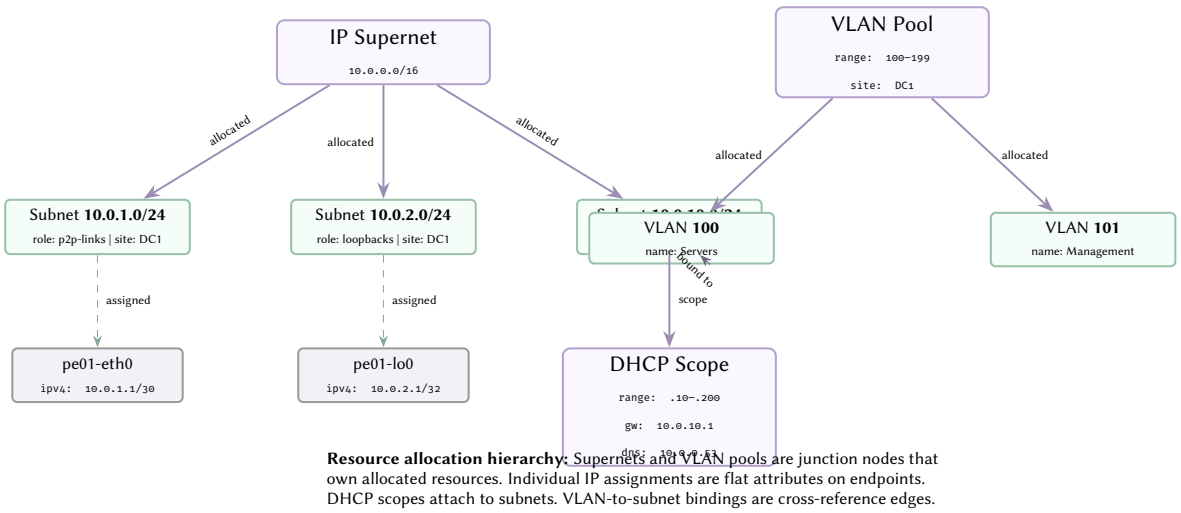
A VLAN 100 broadcast domain with three member ports is stored as:

From	Edge Type	To	Edge Properties
`sw1-eth0`	`MEMBER_OF`	`VLAN_100`	`tagging: untagged`
`sw1-eth1`	`MEMBER_OF`	`VLAN_100`	`tagging: tagged`
`sw2-eth0`	`MEMBER_OF`	`VLAN_100`	`tagging: tagged`

The junction node `VLAN_100` (type: `BroadcastDomain`) carries domain-level properties: `vlan_id: 100`, `name: "Server"`, `stp_root_bridge: "sw1"`. This encoding uses $n + 1$ entities (n edges + 1 node) compared to a single hyperedge, but is natively supported by any binary graph engine.

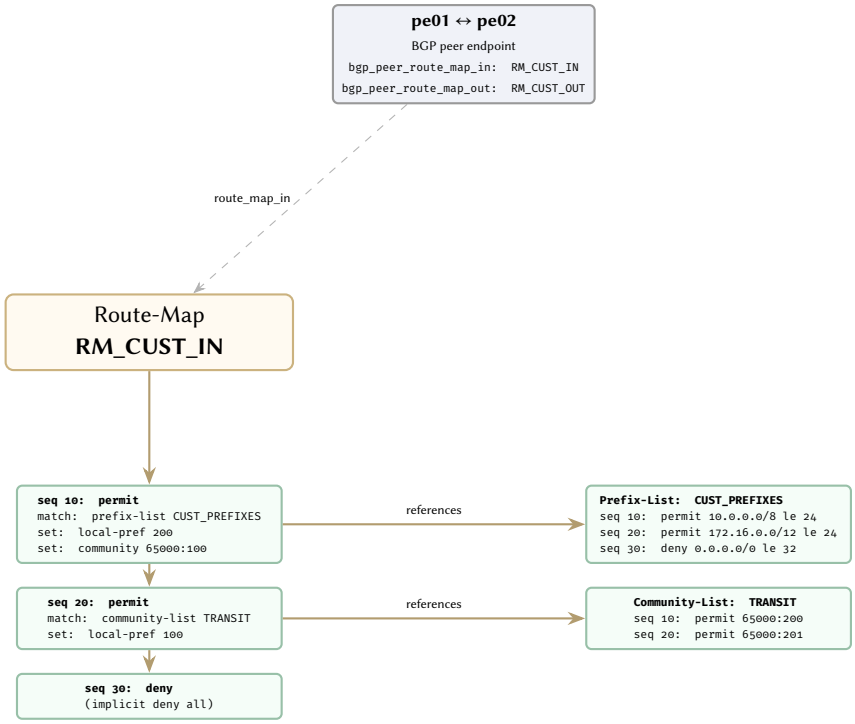
14.6.2 Notational Convention

Throughout this report, we use hyperedge notation in formal definitions and diagrams for mathematical conciseness. The following equivalence holds for implementation:



Resource allocation hierarchy: Supernets and VLAN pools are junction nodes that own allocated resources. Individual IP assignments are flat attributes on endpoints. DHCP scopes attach to subnets. VLAN-to-subnet bindings are cross-reference edges.

Figure 37. Resource allocation in the property hypergraph: IP supernets, VLAN pools, and DHCP scopes modelled as junction nodes with allocation edges.



Policy chain: Route-map names are flat attributes on BGP peer endpoints. Route-map entries, prefix-lists, and community-lists are modelled as ordered sequences of flattened attributes. Cross-references between policies use name-based lookups resolved by the $\mathcal{F}_{\text{BGP}}$ functor.

Figure 38. Route policy in the property hypergraph: route-maps reference prefix-lists and community-lists via named bindings.

Design Rule 6: Hyperedge–Junction Node Equivalence

Every hyperedge h with member set $M(h) = \{e_1, \ldots, e_n\}$ and property map $\pi(h)$ is equivalent to a junction node j_h with $\pi(j_h) = \pi(h)$ and n binary edges $\{(e_i, \texttt{MEMBER\_OF}, j_h) \mid e_i \in M(h)\}$. The formal results (functorial composition, DPO rules, complexity bounds) hold under either encoding.

15 Node and Endpoint Architecture

15.1 Interface Hierarchy

Every network device is modelled as a *node vertex* that owns zero or more *endpoint vertices*. An endpoint represents any named attachment point on the device: a physical port, a logical aggregation, a virtual tunnel, or a protocol session. Ownership is expressed as a directed edge from the device node to each of its endpoints, forming a strict 1:N relationship—every endpoint belongs to exactly one device.

Endpoints fall into three broad categories. **Physical interfaces** (e.g. `eth1`, `eth2`) correspond to hardware ports and carry PHY-layer attributes such as speed, duplex, and media type. **Logical interfaces** derive from one or more physical parents. A LAG endpoint aggregates several physical members; a sub-interface partitions a physical port by VLAN encapsulation. These parent–child relationships are captured by directed edges *between* endpoint vertices, distinct from the ownership edges that connect device to endpoint. **Virtual interfaces** (loopbacks, SVIs/IRBs, tunnels, NVE/VTEPs, BGP sessions) have no physical parent but still appear as endpoint vertices owned by the device node.

Figure 39 illustrates the full structure for a spine switch. Figure 40 shows how attributes cascade through the parent–child chain: each level contributes attributes within its own prefix scope, and no attribute key is shared across scopes.

Insight: *The two edge types—ownership (device→endpoint) and parent–child (endpoint→endpoint)—together define a forest rooted at each device node, enabling efficient subtree queries such as "all endpoints derived from eth1."*

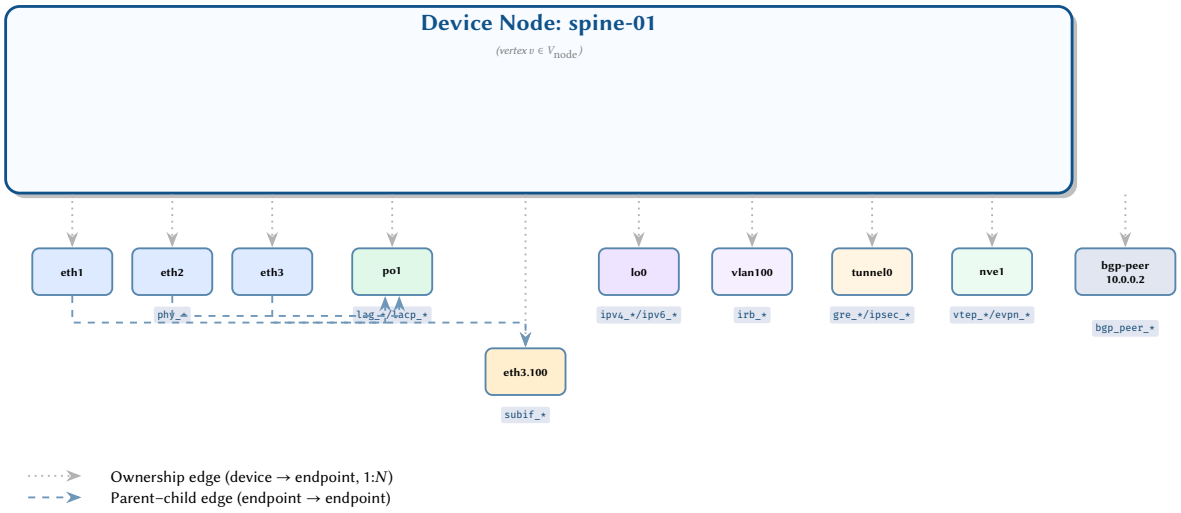


Figure 39. Node/endpoint internal structure. Device node `spine-01` owns multiple endpoint vertices via ownership edges (dotted). Parent–child edges (dashed) capture LAG membership and sub-interface derivation. Each endpoint type carries attributes within its own prefix scope.

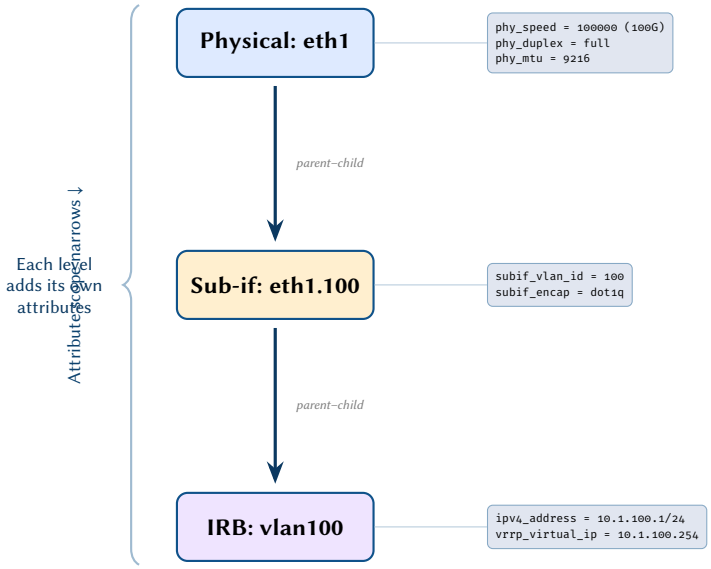


Figure 40. Concrete interface hierarchy showing attribute cascading. Physical interface `eth1` carries PHY-layer attributes; sub-interface `eth1.100` adds encapsulation attributes; IRB `vlan100` adds L3 and VRRP attributes. Each level contributes attributes within its own prefix scope.

15.2 Endpoint Type Taxonomy

Each endpoint type defines an attribute prefix scope. The prefix partitions the flat attribute namespace so that physical attributes (`phy_*`) never collide with sub-interface attributes (`subif_*`) or tunnel attributes (`gre_*`), even when multiple endpoint types coexist on the same device. Table 15.2 enumerates the canonical types.

Design Decision 86: Endpoint Type Taxonomy

Type	Parent Relationship	Attribute Prefix Scope
Physical	None	`phy_*`
LAG	Physical members	`lag_*`, `lacp_*`
Sub-interface	Physical parent	`subif_*`
Loopback	None	`ipv4_*`, `ipv6_*`
SVI/IRB	VLAN	`irb_*`
Tunnel	None	`gre_*`, `ipsec_*`, `ike_*`
NVE/VTEP	None	`vtep_*`, `evpn_*`
BGP Session	None	`bgp_peer_*`

Design Rule 7: Prefix Scope Isolation

No two endpoint types shall share an attribute prefix. This invariant enables type-safe filtering: selecting all attributes matching `subif_*` is guaranteed to return only sub-interface state, regardless of which device or endpoint vertex is queried.

15.3 Formal Ownership Model

The informal "every endpoint belongs to exactly one device" invariant from Section 15.1 can be stated precisely. Let V denote the set of device (node) vertices and $\mathcal{E}$ the set of endpoint vertices in the plan topology (Section 16).

Axiom 4 (Endpoint Ownership). *The* ownership function

$$\text{own} : \mathcal{E} \to V$$

is a total function. Every endpoint $e \in \mathcal{E}$ maps to exactly one device $v = \text{own}(e)$. There are no unowned endpoints and no endpoint is shared between devices.

The inverse image gives the set of endpoints belonging to a device:

$$\mathcal{E}(v) \;=\; \text{own}^{-1}(v) \;=\; \{\, e \in \mathcal{E} \mid \text{own}(e) = v \,\}.$$

$\mathcal{E}(v)$ may be empty (a device with no interfaces) but is always finite.

To capture the parent–child relationships described in Section 15.1, we define a partial function over endpoints:

$$\text{parent} : \mathcal{E} \to \mathcal{E} \cup \{\bot\}$$

where $\text{parent}(e) = \bot$ indicates that e has no parent endpoint.

Design Decision 87: Endpoint Hierarchy Invariants

Ownership consistency. If $\text{parent}(e) = e'$ with $e' \neq \bot$, then $\text{own}(e) = \text{own}(e')$—a child endpoint is always owned by the same device as its parent.

Acyclicity. The directed graph $G_{\text{hier}} = (\mathcal{E}, \{(e, \text{parent}(e)) \mid \text{parent}(e) \neq \bot\})$ is a *forest*: it contains no directed cycles.

Root classification. If $\text{parent}(e) = \bot$, then e is either a physical interface or a virtual interface (loopback, tunnel, BGP session, etc.). Logical interfaces such as sub-interfaces and LAG bundles always have a non-$\bot$ parent.

Together, the ownership edges and parent–child edges define a two-level forest for each device: the device vertex is the super-root, physical and virtual interfaces are tree roots, and logical interfaces are interior or leaf nodes. This structure is exploited by the overlay mapping functors described in Section 17.

15.4 Connectivity Edge Semantics

Devices in the plan topology are never directly adjacent. All inter-device connectivity is mediated through endpoints. We define the set of *connectivity edges*:

$$E_{\text{conn}} \;\subseteq\; \mathcal{E} \times \mathcal{E},$$

subject to the constraint that every connectivity edge links endpoints on *different* devices:

$$(e_i, e_j) \in E_{\text{conn}} \implies \text{own}(e_i) \neq \text{own}(e_j).$$

Connectivity edges are symmetric. If $(e_i, e_j) \in E_{\text{conn}}$, then $(e_j, e_i) \in E_{\text{conn}}$. This reflects the physical reality that a cable connects two ports bidirectionally and extends to logical constructs such as tunnel endpoints that are negotiated by both parties.

The set E_{conn} partitions naturally into two subsets:

- **Physical connectivity edges** represent cables, patch-panel cross-connects, and optical fibres. Each physical endpoint participates in at most one physical connectivity edge.
- **Logical connectivity edges** represent tunnel sessions (GRE, IPsec, VXLAN), protocol peerings (BGP, LDP), and other constructs that do not correspond to a single physical medium. A given endpoint may participate in multiple logical connectivity edges (e.g. a loopback used as the source for several BGP sessions).

From E_{conn} we derive the *induced device adjacency*: devices u and v are adjacent if and only if there exist endpoints $e_u \in \mathcal{E}(u)$ and $e_v \in \mathcal{E}(v)$ such that $(e_u, e_v) \in E_{\text{conn}}$. The device-level adjacency graph is therefore a projection of the endpoint-level connectivity graph.

Design Rule 8: Connectivity Edge Invariants

1. No connectivity edge may link two endpoints on the same device. Intra-device forwarding is implicit in the device's role as a node vertex.
2. Every connectivity edge must reference endpoints that exist in the plan topology; dangling references are a validation error.
3. Physical connectivity edges are injective: each physical endpoint is the source or target of at most one such edge.

15.5 Node Roles and Classification

Every device node carries a `device_role` attribute drawn from a controlled vocabulary. We formalise this as a role function:

$$\rho : V \to \mathcal{P}(\mathcal{R}),$$

where $\mathcal{R}$ is the universe of recognised roles. The power-set codomain allows a single device to hold multiple roles (e.g. a border-leaf that also acts as a route-reflector).

The standard role set $\mathcal{R}$ includes:

> `spine`, `leaf`, `border-leaf`, `super-spine`, `access`, `distribution`, `core`, `PE`, `CE`, `P`, `route-reflector`, `firewall`, `load-balancer`.

Implementations may extend $\mathcal{R}$ with site-specific roles; the framework imposes no closed-world assumption.

Roles are *advisory*—they do not alter the graph structure—but they constrain the expected endpoint types and protocol participation. A spine node is expected to maintain BGP sessions (endpoint type `bgp_peer_*`) to every leaf in its pod. A PE node is expected to own VRF-associated endpoints carrying `vrf_*` attributes. A route-reflector is expected to hold iBGP sessions with `bgp_peer_rr_client = true` attributes on its peering endpoints.

Role-Driven Attribute Expectations

Consider three devices in an EVPN–VXLAN fabric:

Device	Role	Expected Attributes
`spine-01`	`spine`	Loopback with `ipv4_addr`, BGP session endpoints to each leaf with `bgp_peer_asn`, `bgp_peer_type = ebgp`.
`leaf-01`	`leaf`	Physical endpoints with `phy_speed`, SVI/IRB endpoints with `irb_vlan_id`, VTEP endpoint with `vtep_vni`.
`brdr-01`	`border-leaf`, `route-reflector`	All leaf attributes plus BGP session endpoints with `bgp_peer_rr_client = true` and external peerings with `bgp_peer_type = ebgp` toward upstream providers.

Validation rules keyed on $\rho(v)$ can flag missing or unexpected attributes—for example, a spine with no BGP session endpoints, or a leaf carrying `bgp_peer_rr_client` attributes.

15.6 Endpoint Capacity and Scaling

Real devices impose finite limits on the number of physical ports. We capture this as a capacity bound:

$$|\mathcal{E}_{\text{phy}}(v)| \;\leq\; C_v,$$

where $\mathcal{E}_{\text{phy}}(v) \subseteq \mathcal{E}(v)$ is the subset of physical endpoints and C_v is the hardware port count of device v. This bound is strict: no plan may assign more physical interfaces than the device can physically accommodate.

Logical and virtual endpoints are not subject to the same hardware ceiling. A single physical interface may host hundreds of sub-interfaces (one per VLAN), and a device may maintain thousands of BGP sessions—each modelled as its own endpoint vertex. The total endpoint count is therefore:

$$|\mathcal{E}(v)| \;=\; |\mathcal{E}_{\text{phy}}(v)| + |\mathcal{E}_{\text{log}}(v)| + |\mathcal{E}_{\text{virt}}(v)|,$$

where $|\mathcal{E}_{\text{log}}(v)|$ and $|\mathcal{E}_{\text{virt}}(v)|$ are bounded only by control-plane resources (memory, session limits) rather than by physical port counts.

The plan topology makes no structural distinction between physical, logical, and virtual endpoints—all are endpoint vertices with ownership edges and attribute maps. This uniformity is deliberate. A device running 1,000 BGP sessions will contribute 1,000-plus endpoint vertices to the plan topology. While this inflates vertex counts, it preserves the invariant that every configurable attachment point is individually addressable, queryable, and validatable.

16 The Plan Topology

16.1 Design Principles

The plan topology G_{plan} is a single-layer graph that serves as the *source of truth* for all network state. It captures the architect's intent without reference to protocol-specific overlay structures.

Insight: *Treating every attachment point—physical port, sub-interface, tunnel, or BGP session—as a first-class endpoint vertex trades graph compactness for* uniform reachability*: any analysis that works on a physical interface works identically on a VXLAN tunnel or a BGP peering, with no special-case logic. The scaling cost is linear in the number of configured attachment points, which is acceptable because the plan topology is a design-time artefact, not a runtime data structure.*

Design Rule 9: Single Graph, All State

G_{plan} is a labeled property hypergraph $G_{\text{plan}} = (V, E, H, \pi_V, \pi_E)$ where:

- V is the set of **nodes** (devices),
- $E \subseteq V \times V$ is the set of **edges** (physical links),
- $H \subseteq \mathcal{P}(V)$ is the set of **hyperedges** (broadcast domains, VLANs, multicast groups),
- $\pi_V : V \to \mathbb{A}$ assigns flat attribute maps to nodes and their endpoints,
- $\pi_E : E \to \mathbb{A}$ assigns *structural-only* attributes to edges (no protocol state).

Design Rule 10: Endpoint Locality

Each node $v \in V$ contains a set of **endpoints** $\mathcal{E}(v) = \{e_1, e_2, \ldots, e_k\}$ representing physical and logical interfaces. All protocol state is stored as flat attributes on either the node or its endpoints:

- **Node attributes**: global device state (hostname, router-id, ASN, OSPF process config, VRF definitions, management config).

- **Endpoint attributes**: per-interface state (IP address, VLAN, OSPF area/cost, BGP peer config, MPLS/SR SID, QoS policy).

No protocol state resides on edges. Edges carry only connectivity semantics.

16.2 Formal Definition

Design Decision 88: Plan Topology

The plan topology is a tuple:

$$G_{\text{plan}} = (V, \mathcal{E}, E_{\text{conn}}, E_{\text{own}}, \pi_V, \pi_{\mathcal{E}})$$

where:

1. V is the set of device nodes.
2. $\mathcal{E} = \bigcup_{v \in V} \mathcal{E}(v)$ is the set of all endpoints across all devices.
3. $E_{\text{conn}} \subseteq \mathcal{E} \times \mathcal{E}$ is the set of connectivity edges (endpoint-to-endpoint links).
4. $E_{\text{own}} \subseteq V \times \mathcal{E}$ is the set of ownership edges (v owns endpoint e).
5. $\pi_V : V \to \mathbb{A}$ assigns a flat attribute map to each node.
6. $\pi_{\mathcal{E}} : \mathcal{E} \to \mathbb{A}$ assigns a flat attribute map to each endpoint.

The attribute space $\mathbb{A} = \{k_1 : T_1, k_2 : T_2, \ldots\}$ is a flat key-value map where each key is a string and each value is a typed primitive (bool, integer, float, string, IP address, enum). Complex types (struct, set, list, map) are flattened per Section 1.

16.3 Example Plan Topology

Insight: *The plan topology mirrors the NTE data model: nodes and endpoints are graph vertices with Polars DataFrames providing typed columnar storage for flat attributes.*

A small plan topology demonstrating how all protocol state lives as flat attributes on nodes and endpoints:

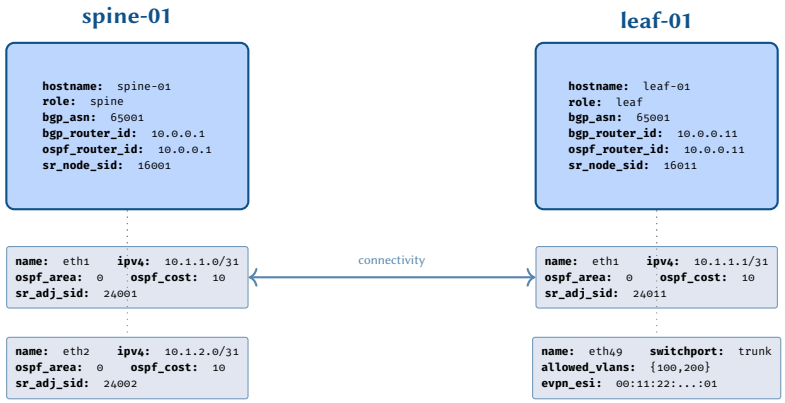


Figure 41. Plan topology fragment: all protocol state as flat attributes on nodes and endpoints. Edges carry only structural semantics.

16.4 G_{plan} as a Unified Input

The central claim of this model is that a *single* plan topology G_{plan} is sufficient to derive *all* protocol-specific overlay graphs. This subsection makes that claim precise and discusses what it requires.

16.4.1 What G_{plan} Must Capture

For the overlay functors $\mathcal{F}_\ell$ to be pure functions of the plan topology, G_{plan} must contain every configuration parameter that any functor reads. Concretely, the plan topology must capture:

1. **Physical topology**: which devices exist, which interfaces each device has, and which interfaces are physically connected. This is the graph structure itself (V, $\mathcal{E}$, E_{conn}, E_{own}).
2. **Layer 2 configuration**: switchport mode, VLAN membership, trunk allowed VLANs, STP priority, LAG membership, LLDP settings. These are flat attributes on endpoints.
3. **IP addressing and VRF**: IPv4/IPv6 addresses, subnet masks, VRF membership, VRRP/HSRP virtual IPs, DHCP relay configuration.
4. **Routing protocol configuration**: OSPF area, cost, network type, hello/dead intervals; IS-IS metric, level, NET; BGP ASN, peer address, peer type, route-map bindings, address-family activation.
5. **Label switching**: MPLS enabled, LDP/RSVP-TE/SR configuration, segment routing SIDs, EVPN ESI, VNI, route targets.

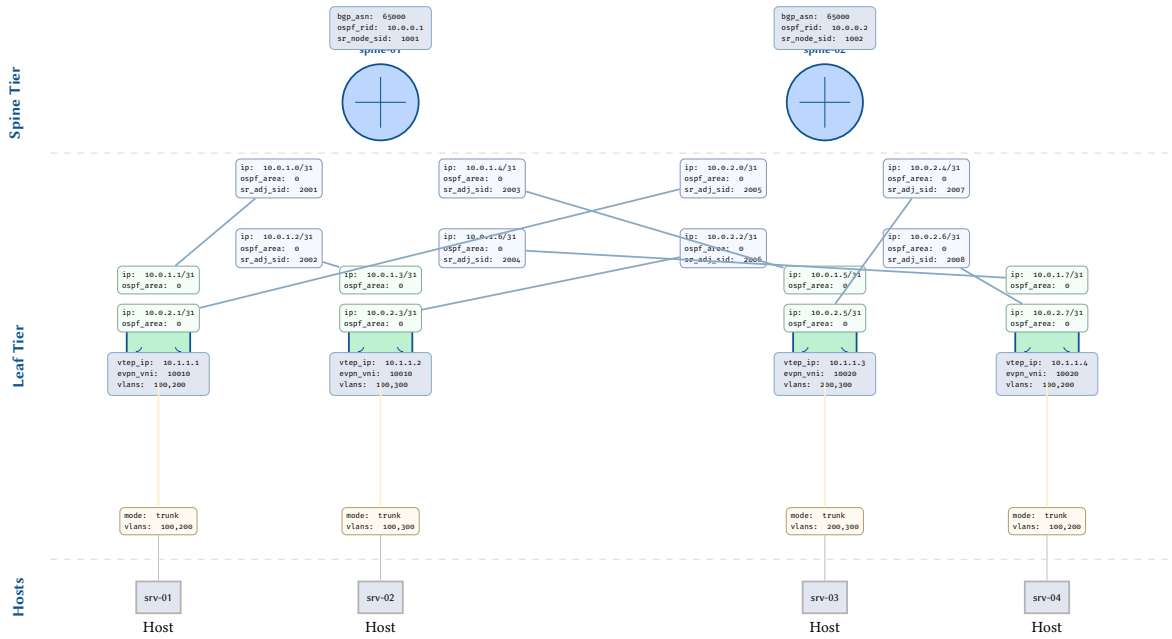


Figure 42. Plan topology for a leaf-spine Clos fabric. Each device node carries protocol-specific attributes (BGP ASN, OSPF router-id, SR node-SID for spines; VXLAN VTEP IP, EVPN VNI for leaves). Endpoints expose per-interface state: IP addressing and OSPF area on uplinks, switchport mode and VLANs on server-facing downlinks. This topology serves as the running example throughout the paper.

6. **Policy and access control**: ACL rules (flattened), route-map entries, prefix-lists, community-lists, PBR rules, CoPP classes.
7. **Security and segmentation**: SGT assignments, SXP peers, SGACL policies, 802.1X configuration, MACsec settings.
8. **QoS**: DSCP classification, queue mappings, scheduler configuration, shaping/policing rates.

16.4.2 From One Input to All Overlays

Insight: *If a configuration parameter influences any protocol behaviour, it belongs in G_{plan}. The completeness of the plan topology directly determines the fidelity of the computed overlays. The survey in Part II provides the exhaustive catalogue of such parameters.*

Given a complete G_{plan}, the full overlay is computed by applying each layer functor in dependency order:

Design Decision 89: Overlay Generation Pipeline

1. **Load** G_{plan} from the configuration source (device configs, source-of-truth database, or intent system).
2. **Validate** structural constraints: every endpoint has exactly one owner, connectivity edges connect same-layer endpoints, required attributes are present.
3. **Evaluate** the functors in topological order of the dependency DAG (Section 17.5.1):

$$\mathcal{F}_{\text{PHY}} \to \mathcal{F}_{\text{L2}} \to \mathcal{F}_{\text{STP}} \to \mathcal{F}_{\text{MPLS}} \to \mathcal{F}_{\text{IGP}} \to \mathcal{F}_{\text{BGP}} \to \mathcal{F}_{\text{EVPN}}$$

   with cross-layer functors ($\mathcal{F}_{\text{QoS}}$, $\mathcal{F}_{\text{ACL}}$, $\mathcal{F}_{\text{Security}}$) evaluated after physical and L2 resolution.
4. **Compose** the per-layer results into the full multilayer overlay $G_{\text{overlay}} = \bigoplus_{\ell} \mathcal{F}_{\ell}(G_{\text{plan}})$.

The power of this approach is that the *same* G_{plan} answers fundamentally different questions: "what are the OSPF adjacencies?" ($\mathcal{F}_{\text{IGP}}$), "which paths carry MPLS labels?" ($\mathcal{F}_{\text{MPLS}}$), "what firewall rules apply to traffic between host A and host B?" ($\mathcal{F}_{\text{ACL}}$). Each question is answered by the appropriate functor reading the relevant attribute prefixes from the same underlying graph.

16.4.3 Populating G_{plan}

In practice, G_{plan} can be populated from multiple sources:

- **Configuration parsing**: Vendor configs (IOS, JunOS, EOS, NX-OS) are parsed into flat attribute maps. Each parsed config produces a node with its endpoints and their attributes.
- **Source of truth**: Systems like NetBox or Nautobot export inventory, IP addressing, and circuit data that map directly to plan topology attributes.
- **Intent systems**: High-level intent declarations ("deploy OSPF in area 0 on all spine links") translate to attribute assignments on matching endpoints.

- **Hybrid**: The most practical approach combines all three—inventory from a source of truth, protocol configuration from parsed device configs, and policy intent from an automation system.

Design Rule 11: Plan Topology Completeness

The plan topology is **complete** if, for every active protocol layer ℓ, every attribute required by $\mathcal{F}_\ell$ is present on the relevant nodes and endpoints. An incomplete plan topology produces a partial overlay—valid but potentially missing adjacencies or state that depends on the absent attributes.

16.5 Plan Topology Validation

Before any functor evaluation $\mathcal{F}_\ell(G_{\text{plan}})$ is invoked, the plan topology must pass a sequence of structural and protocol-level validation checks. Validation serves as a *pre-condition guard*: it prevents ill-formed inputs from propagating through the functor pipeline, where they would produce undefined or nonsensical overlay outputs. By catching errors early—at the plan level—the system provides precise, actionable diagnostics rather than cryptic failures deep inside a protocol functor.

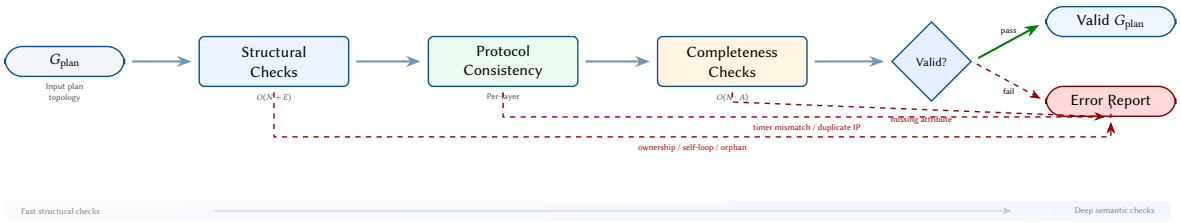


Figure 43. Validation pipeline for G_{plan}. The three stages execute in order of increasing cost: structural checks verify graph invariants in $O(N + E)$, protocol consistency validates per-layer constraints on connected endpoints, and completeness checks ensure all required attributes are present ($O(N \cdot A)$). Any stage may emit errors that halt the pipeline and produce a diagnostic report.

16.5.1 Structural Constraints

The structural constraints enforce invariants on the graph topology itself, independent of any protocol semantics. These are fast to verify—$O(|V| + |\mathcal{E}| + |E_{\text{conn}}|)$—and must hold before any protocol-level checks proceed.

Design Decision 90: Structural Invariants

Let $G_{\text{plan}} = (V, \mathcal{E}, E_{\text{conn}}, E_{\text{own}}, \pi_V, \pi_{\mathcal{E}})$. The following must hold:

1. **Unique ownership.** Every endpoint has exactly one owning device:
$$\forall e \in \mathcal{E},\ |\{v \in V \mid (v, e) \in E_{\text{own}}\}| = 1$$
2. **No self-loops.** Connectivity edges never connect endpoints on the same device:
$$\forall (e_i, e_j) \in E_{\text{conn}},\ \text{owner}(e_i) \neq \text{owner}(e_j)$$
3. **Bidirectional connectivity.** If a connectivity edge exists in one direction, it must exist in the other:
$$(e_i, e_j) \in E_{\text{conn}} \iff (e_j, e_i) \in E_{\text{conn}}$$
4. **No orphan nodes.** Every device has at least one endpoint:
$$\forall v \in V,\ |\mathcal{E}(v)| \geq 1$$

16.5.2 Protocol Consistency Rules

Insight: *Structural validation is purely graph-theoretic. It can be implemented as a single pass over the ownership and connectivity edge sets, making it the cheapest phase of the pipeline.*

Protocol consistency rules validate that the attribute values on connected endpoints are mutually compatible. Unlike structural checks (which examine the graph shape), consistency rules read attribute values and compare them across connectivity edges.

- **OSPF timer agreement.** Hello and dead intervals must match on connected endpoints that share an OSPF area:
$$\forall (e_i, e_j) \in E_{\text{conn}} : \texttt{ospf\_area}(e_i) = \texttt{ospf\_area}(e_j) \implies \texttt{ospf\_hello}(e_i) = \texttt{ospf\_hello}(e_j) \wedge \texttt{ospf\_dead}(e_i) = \texttt{ospf\_dead}(e_j)$$

- **BGP peer reciprocity.** Every BGP peer endpoint must reference a peer address that resolves to a corresponding endpoint on the remote device. If endpoint e_i declares $\texttt{bgp\_peer\_addr}(e_i) = a$, then there must exist an endpoint e_j on the intended remote device with $\texttt{ipv4}(e_j) = a$ and a reciprocal peer declaration.
- **VLAN trunk overlap.** Connected trunk ports must have a non-empty intersection of allowed VLANs:

$$\forall (e_i, e_j) \in E_{\text{conn}} : \texttt{switchport}(e_i) = \texttt{trunk} \land \texttt{switchport}(e_j) = \texttt{trunk} \implies \texttt{allowed\_vlans}(e_i) \cap \texttt{allowed\_vlans}(e_j) \neq \emptyset$$

- **IP uniqueness.** No two endpoints within the same VRF may share an IP address:

$$\forall e_i, e_j \in \mathcal{E},\ e_i \neq e_j : \texttt{vrf}(e_i) = \texttt{vrf}(e_j) \implies \texttt{ipv4}(e_i) \neq \texttt{ipv4}(e_j)$$

- **STP bridge priority.** The bridge priority must be a valid multiple of 4096 within the permitted range:

$$\texttt{stp\_priority}(v) \in \{0, 4096, 8192, \ldots, 61440\}$$

- **MPLS label range separation.** Label ranges configured on the same device must not overlap, to prevent label collisions in the forwarding plane:

$$\forall r_i, r_j \in \texttt{label\_ranges}(v),\ r_i \neq r_j \implies r_i \cap r_j = \emptyset$$

- **IS-IS consistency.** The Network Entity Title (NET) must be unique per IS-IS process on each device. Additionally, IS-IS hello intervals must be consistent on endpoints sharing a common IS-IS segment, analogous to the OSPF timer rule.

16.5.3 Completeness Checks

Completeness validation ensures that every active protocol instance has all required attributes populated. A protocol is considered *active* on a node if its enabling flag is set (e.g., `ospf_enabled = true`). Activation without the corresponding required attributes is an error.

Table 3. Required attributes per active protocol.

Protocol	Enable Flag	Required Attributes
OSPF	`ospf_enabled`	`ospf_router_id`, `ospf_area` (on ≥ 1 endpoint)
BGP	`bgp_enabled`	`bgp_asn`, `bgp_router_id`
IS-IS	`isis_enabled`	`isis_net`, `isis_level`
MPLS	`mpls_enabled`	`mpls_label_range`
EVPN	`evpn_enabled`	`evpn_vni`, `evpn_rd`, `evpn_rt`
STP	`stp_enabled`	`stp_priority`, `stp_mode`
VRRP	`vrrp_enabled`	`vrrp_group`, `vrrp_virtual_ip`, `vrrp_priority`

The completeness check iterates over every node and, for each active protocol flag, verifies that the corresponding required attributes are non-null. For endpoint-level attributes (e.g., `ospf_area`), at least one endpoint on the device must carry the attribute.

16.5.4 Validation Execution Order

The three validation phases execute in a fixed sequence, ordered by increasing computational cost and diagnostic specificity:

1. **Structural checks** run first. They verify graph-level invariants (ownership, self-loops, bidirectionality, orphans) and execute in $O(|V| + |\mathcal{E}| + |E_{\text{conn}}|)$. Because they touch only the graph skeleton, they are the fastest phase and catch gross errors (e.g., an endpoint without an owner) before any attribute inspection occurs.
2. **Protocol consistency checks** run second, on a per-layer basis. Each rule may query attributes on both endpoints of a connectivity edge, making the cost proportional to the number of active edges per layer. Errors here indicate configuration mismatches between adjacent devices (timer mismatches, missing peers, VLAN misalignment).
3. **Completeness checks** run last, iterating over every node and its active protocol flags. The cost is $O(|V| \cdot K)$ where K is the number of protocol-specific attribute keys. This phase catches missing required attributes that would cause a functor to produce incomplete output.

If any phase produces errors, the pipeline halts and emits a structured error report. Downstream phases are not executed, because structural violations may cause protocol checks to fail spuriously, and protocol inconsistencies may mask completeness issues.

Design Decision 91: Validation Constraints Summary

The plan topology G_{plan} is **valid** if and only if all of the following hold:

$$\textsc{Ownership}: \quad \forall e \in \mathcal{E},\ |\{v \in V \mid (v, e) \in E_{\text{own}}\}| = 1 \tag{1}$$

$$\textsc{No-loops}: \quad \forall (e_i, e_j) \in E_{\text{conn}},\ \text{owner}(e_i) \neq \text{owner}(e_j) \tag{2}$$

$$\textsc{IP-unique}: \quad \forall e_i \neq e_j \in \mathcal{E},\ \texttt{vrf}(e_i) = \texttt{vrf}(e_j) \implies \texttt{ipv4}(e_i) \neq \texttt{ipv4}(e_j) \tag{3}$$

$$\textsc{Complete}: \quad \forall v \in V,\ \forall \ell \in \Lambda(v),\ \text{req}(\ell) \subseteq \text{keys}(\pi_V(v)) \cup \bigcup_{e \in \mathcal{E}(v)} \text{keys}(\pi_{\mathcal{E}}(e)) \tag{4}$$

where $\Lambda(v)$ is the set of active protocol layers on node v and $\text{req}(\ell)$ is the set of required attribute keys for layer ℓ (Table 3).

16.6 Design Patterns in the Plan Topology

A critical property of the plan topology is that it captures *design intent* without prescribing a single topology pattern. The same attribute schema accommodates fundamentally different architectural choices—the functors produce different overlay graphs depending on the attribute values, not on any hard-coded topology assumption.

This subsection illustrates the principle with three representative design patterns. A comprehensive catalogue of design patterns is the subject of a companion reference document [113].

16.6.1 BGP Peering Topologies

Consider three common iBGP peering designs for a 4-router autonomous system. Each uses the same attribute schema; only the attribute *values* differ.

Design Decision 92: BGP Design Patterns

Pattern	Plan Topology Attributes
Full mesh	Every router pair has a BGP endpoint with `bgp_peer_type = ibgp` pointing at the remote loopback. For n routers this yields $n(n-1)/2$ connectivity edges in the BGP overlay. No route-reflector attributes are set.
Single route reflector	One router (e.g., `spine-01`) carries `bgp_rr_cluster_id = 1.1.1.1`. Its BGP endpoints to each client carry `bgp_rr_client = true`. Clients peer only with the reflector—no inter-client mesh. $\mathcal{F}_{\text{BGP}}$ emits reflected route edges from the RR to each client.
Hierarchical RR	Two tiers: Tier-1 RRs peer in a full mesh and carry `bgp_rr_cluster_id = 1.1.1.1`. Tier-2 RRs peer with Tier-1 and carry `bgp_rr_cluster_id = 2.2.2.{1,2}`. Leaf routers peer only with their local Tier-2 RR. The functor resolves the hierarchy via cluster-ID nesting.

In every case the BGP functor $\mathcal{F}_{\text{BGP}}$ reads `bgp_*` attributes and produces the correct overlay. The plan topology *encodes* the design choice; the functor *derives* the consequences.

16.6.2 IGP Area Design

OSPF multi-area and IS-IS multi-level designs are captured purely through endpoint attributes:

- **Single-area OSPF.** All endpoints carry `ospf_area = 0`. $\mathcal{F}_{\text{OSPF}}$ produces a single connected overlay.
- **Multi-area OSPF.** Core endpoints carry `ospf_area = 0`; distribution endpoints carry `ospf_area = 1` (or 2, etc.). ABR devices have endpoints in both areas. The functor partitions the overlay by area and creates inter-area summary edges at ABRs.
- **Stub / NSSA areas.** Setting `ospf_area_type = stub` or `nssa` on endpoints in a non-backbone area instructs the functor to suppress external route injection (Type-5 LSAs) into that area's overlay partition.

The IS-IS equivalent uses `isis_level` (L1, L2, or L1L2) and `isis_metric_style` (narrow or wide) in place of `ospf_area` and `ospf_cost`, respectively. The structural principle is identical: the plan topology captures the engineer's hierarchical intent, and the appropriate IGP functor realises it.

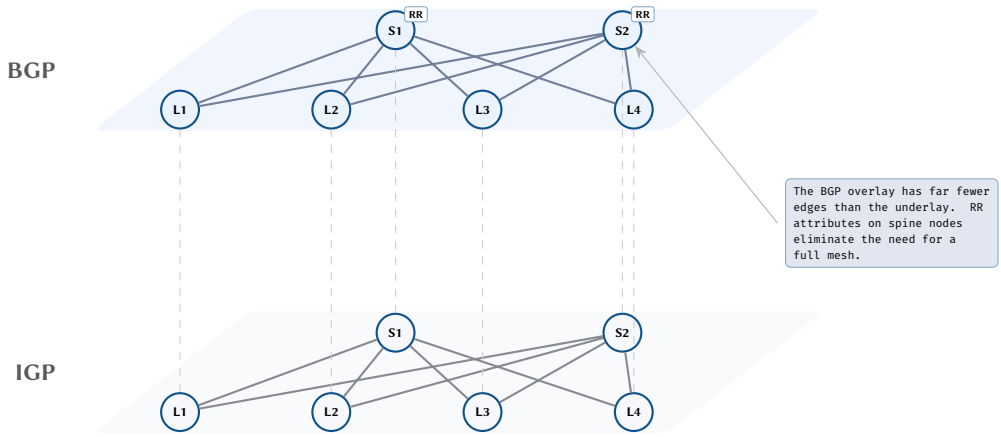


Figure 44. BGP route reflector topology: the BGP overlay (top) is sparser than the underlay IGP (bottom), with spines serving as route reflectors.

16.6.3 EVPN Fabric Variants

EVPN-VXLAN fabrics present at least two major design choices:

1. **Centralised gateway** vs. **distributed gateway** (anycast). In centralised mode, a pair of border-leaf devices carry `evpn_gw_role = centralised` and a shared `evpn_anycast_mac`. In distributed mode, every leaf carries `evpn_gw_role = distributed` with `evpn_anycast_ip` matching across the fabric.
2. **Symmetric IRB** vs. **asymmetric IRB.** Symmetric routing uses a per-VRF "transit" VNI (`evpn_l3vni`) on every leaf; asymmetric routing requires every leaf to have all VLANs and their L2 VNIs (`evpn_vni`) locally configured. The attribute `evpn_irb_mode = symmetric | asymmetric` controls which path $\mathcal{F}_{\text{EVPN}}$ takes.

These choices have cascading effects on the overlay graph: symmetric IRB produces fewer L2 VNI edges but adds L3 VNI edges; distributed gateway removes the centralised hairpin paths. The plan topology captures the design intent; the functor computes the structural consequences.

17 The Overlay Mapping $\mathcal{F}(G)$

17.1 Functorial Interpretation

The mapping $\mathcal{F} : G_{\text{plan}} \to G_{\text{overlay}}$ transforms the plan topology into a multilayer overlay structure. In engineering terms, $\mathcal{F}$ is a deterministic procedure that reads the plan topology and produces the per-layer overlay graphs; the categorical language (see Section 1.2 for a brief glossary) formalises the guarantees this procedure provides. Following the category-theoretic framework of Baez and Fong [19] and the algebraic graph transformation theory of Ehrig et al. [114]:

Insight: *Design patterns are not hard-coded into the model. They emerge from the* attribute values *an engineer places on the plan topology. This makes the model inherently extensible: new design patterns require no schema changes—only new combinations of existing attributes.*

Design Decision 93: Overlay Functor

$\mathcal{F}$ is a functor from the category **Plan** of plan topologies to the category **Overlay** of multilayer overlay graphs. For a plan topology G_{plan}:

$$\mathcal{F}(G_{\text{plan}}) = \bigoplus_{\ell \in \mathcal{L}} \mathcal{F}_\ell(G_{\text{plan}})$$

where $\mathcal{L}$ is the set of protocol layers and $\mathcal{F}_\ell$ is the layer-specific transformation functor. The direct sum $\bigoplus$ denotes the multilayer composition with identity couplings.

17.2 Layer Extraction Rules

Each layer functor $\mathcal{F}_\ell$ operates via *attribute-driven extraction*:

Insight: *Each protocol layer has its own functor $\mathcal{F}_\ell$ that extracts relevant attributes from the plan and constructs the layer-specific graph. The full overlay is the composition of all layer functors.*

Design Decision 94: Layer Extraction

For layer ℓ with attribute prefix p_ℓ, the layer functor is:

$$V_\ell = \{v \in V \mid \exists\, k \in \pi_V(v) \text{ with } k.\text{prefix} = p_\ell\} \tag{5}$$

$$\mathcal{E}_\ell = \{e \in \mathcal{E} \mid \exists\, k \in \pi_{\mathcal{E}}(e) \text{ with } k.\text{prefix} = p_\ell\} \tag{6}$$

$$E_\ell = \{(e_i, e_j) \in E_{\text{conn}} \mid e_i, e_j \in \mathcal{E}_\ell\} \tag{7}$$

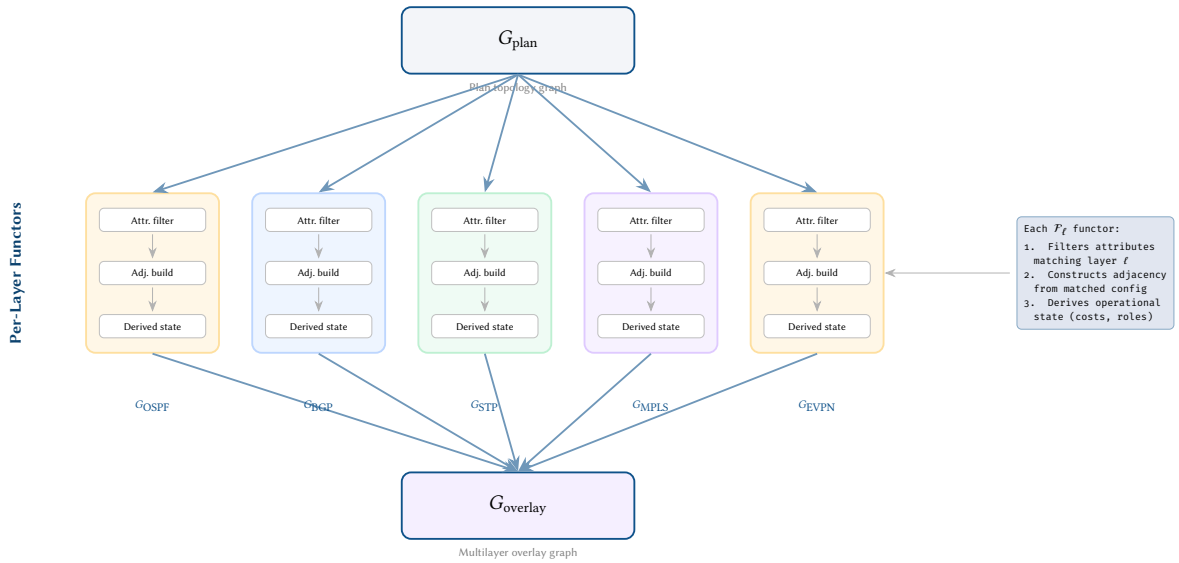


Figure 45. The $\mathcal{F}(G)$ transformation pipeline. A single plan topology G_{plan} fans out to per-protocol functors $\mathcal{F}_\ell$, each of which filters relevant attributes, constructs layer-specific adjacency, and derives operational state. The resulting layer graphs merge into the multilayer overlay G_{overlay}.

> Nodes and endpoints with no attributes matching the layer prefix are excluded from that layer's graph.

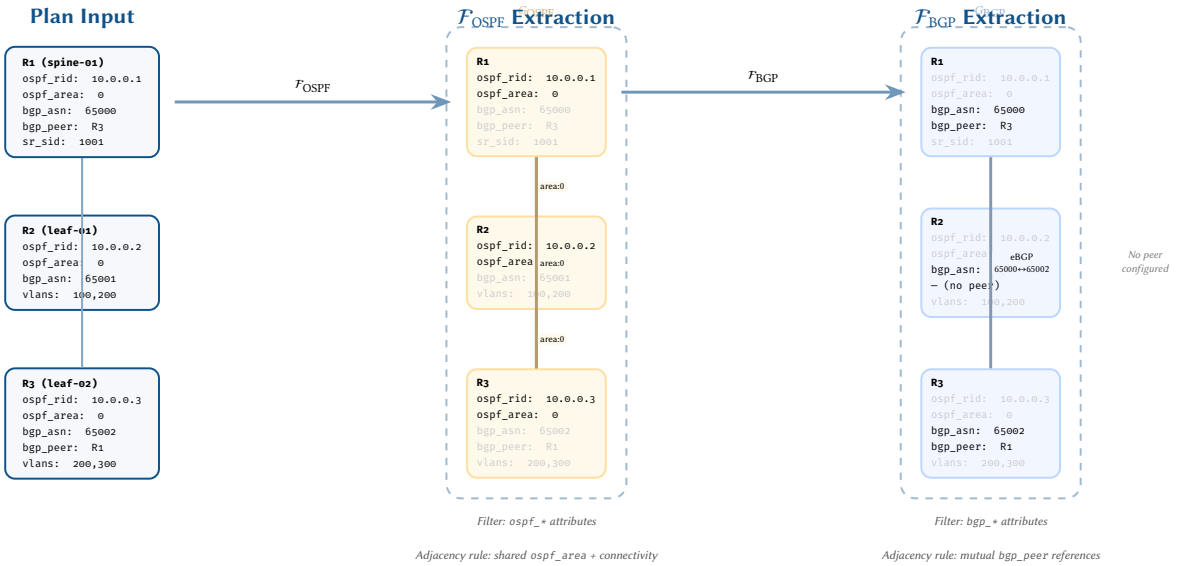


Figure 46. Layer extraction example. From a plan topology with mixed attributes, $\mathcal{F}_{\text{OSPF}}$ retains `ospf_*` attributes and forms adjacencies where `ospf_area` values match across a connectivity edge, producing a full mesh. $\mathcal{F}_{\text{BGP}}$ retains `bgp_*` attributes and forms sessions only where mutual `bgp_peer` references exist, yielding a single eBGP edge. Greyed-out attributes are filtered by each functor.

17.3 Derived State Computation

Beyond extraction, some layer functors must *compute* derived state. Each functor has a well-defined type signature:

Design Decision 95: Functor Type Signatures

Functor	Signature
$\mathcal{F}_{\text{OSPF}}$	$G_{\text{plan}} \to (V_{\text{OSPF}}, E_{\text{adj}}, \text{RIB}_{\text{OSPF}}, \text{DR/BDR})$
$\mathcal{F}_{\text{ISIS}}$	$G_{\text{plan}} \to (V_{\text{ISIS}}, E_{\text{adj}}, \text{RIB}_{\text{ISIS}}, \text{DIS})$
$\mathcal{F}_{\text{BGP}}$	$G_{\text{plan}} \to (V_{\text{BGP}}, E_{\text{session}}, \text{Adj-RIB-In, Loc-RIB, Adj-RIB-Out})$
$\mathcal{F}_{\text{STP}}$	$G_{\text{plan}} \to (V_{\text{STP}}, E_{\text{bridge}}, \text{root, port\_roles})$
$\mathcal{F}_{\text{MPLS}}$	$G_{\text{plan}} \to (V_{\text{MPLS}}, E_{\text{lsp}}, \text{LFIB})$
$\mathcal{F}_{\text{EVPN}}$	$G_{\text{plan}} \to (V_{\text{EVPN}}, E_{\text{bgp}}, \text{MAC/IP, VNI})$
$\mathcal{F}_{\text{QoS}}$	$G_{\text{plan}} \to (\text{queue\_map, scheduler\_config, DSCP\_tables})$
$\mathcal{F}_{\text{ACL}}$	$G_{\text{plan}} \to (\text{TCAM\_entries, interface\_bindings})$
$\mathcal{F}_{\text{Security}}$	$G_{\text{plan}} \to (\text{SGT\_assignments, SXP\_bindings, SGACL\_policies})$

Example: $\mathcal{F}_{\text{OSPF}}$ computation. The OSPF functor is a pure function on a static plan snapshot. The algorithm filters endpoints by `ospf_*` prefix, groups them by area, builds per-area adjacency graphs with DR/BDR election, runs Dijkstra SPF, and emits RIB entries. Full pseudocode appears in Appendix C.1.

All derived-state functors share this structure: filter by attribute prefix, group by relevant partitioning key, build the layer-specific adjacency graph, run the protocol algorithm, and emit the overlay subgraph. Each functor is a pure function on a static snapshot of G_{plan}, ensuring deterministic and reproducible results.

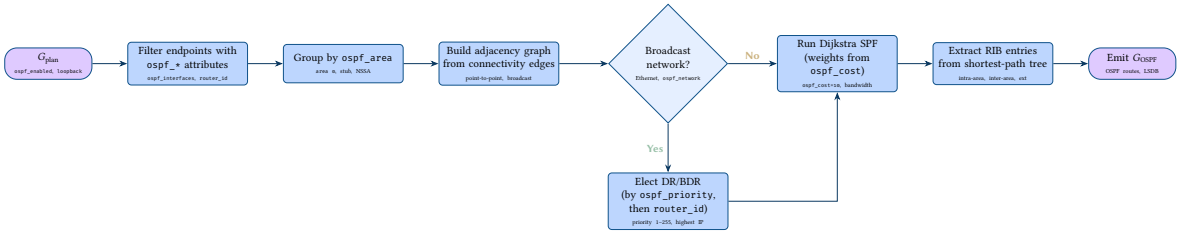


Figure 47. $\mathcal{F}_{\text{OSPF}}$ flowchart: from plan attributes to OSPF overlay graph.

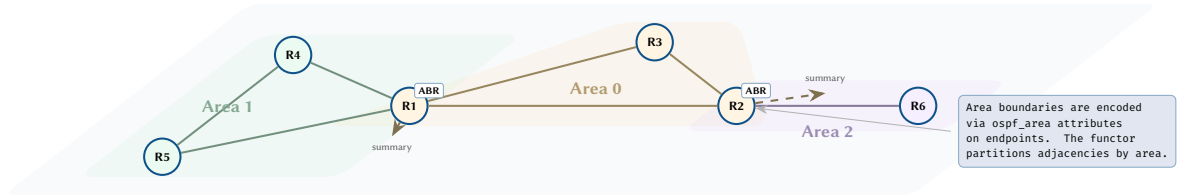


Figure 48. OSPF multi-area topology. Area membership is determined by endpoint attributes; ABR routers have endpoints in multiple areas.

17.4 Graph Rewriting Rules

Following the Double Pushout (DPO) approach [114], each derived state computation can be expressed as a graph rewriting rule $p : L \xleftarrow{l} K \xrightarrow{r} R$. Informally, a DPO rule is a "find-and-replace" for graphs (see Section 1.2 for a brief glossary): L is the pattern to match, K is the part to keep, and R is what to produce. The diagram below shows the two "pushout" squares that guarantee the rewrite is well-defined:

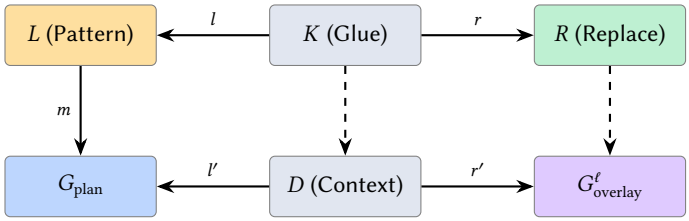


Figure 49. Double-pushout (DPO) graph rewriting for overlay construction.

Example rule: "Deploy OSPF adjacency"

- L: Two endpoints e_i, e_j connected by a connectivity edge, both with `ospf_area` attribute matching.
- K: The two endpoints (preserved).
- R: The two endpoints plus a new *OSPF adjacency* semantic edge with attributes (area, cost, network-type, neighbour-state=`full`).

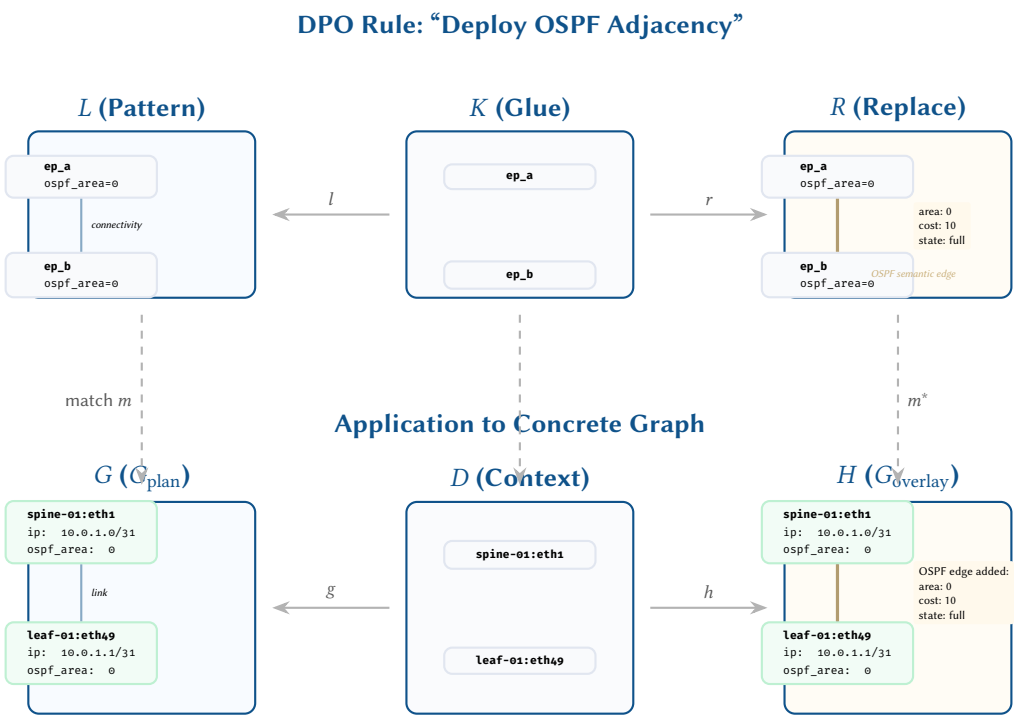


Figure 50. Double-Pushout (DPO) graph rewriting applied to OSPF adjacency deployment. The rule $L \leftarrow K \rightarrow R$ matches two endpoints sharing `ospf_area=0` connected by a connectivity edge, preserves the endpoints through K, and produces a new OSPF semantic edge with derived attributes in R. Below, the rule is applied to the concrete spine-01/leaf-01 interface pair in G_{plan}, yielding G_{overlay} with the added OSPF adjacency.

17.5 Composition and Verification

The full overlay is the composition of all layer functors:

$$G_{\text{overlay}} = \mathcal{F}(G_{\text{plan}}) = \bigoplus_{\ell \in \mathcal{L}} \mathcal{F}_\ell(G_{\text{plan}})$$

This overlay can then be verified using existing tools:

- **Batfish** [4]: Vendor-neutral configuration verification against the computed forwarding state.
- **NetKAT** [27]: Algebraic verification of forwarding correctness (reachability, isolation).
- **Minesweeper** [28]: SMT-based control-plane convergence verification.

17.5.1 Functor Dependency Ordering

The layer functors are not independent: $\mathcal{F}_{\text{BGP}}$ requires the IGP RIB for next-hop resolution, and $\mathcal{F}_{\text{MPLS}}$ requires IGP-computed shortest paths for label binding. We define a dependency DAG over functors:

Design Decision 96: Category Plan

Component	Definition
Objects	Plan topologies G_{plan} (node-endpoint graphs with flat attribute maps)
Morphisms	Attribute-preserving graph homomorphisms $f : G \to G'$
Identity	id_G: the identity homomorphism on G
Composition	Standard function composition: $g \circ f$

Design Decision 97: Category Overlay

Component	Definition
Objects	Multilayer overlay graphs G_{overlay} (directed graphs with layer labels and inter-layer couplings)
Morphisms	Layer-respecting graph homomorphisms $g : G_{\text{ov}} \to G'_{\text{ov}}$
Identity	$\text{id}_{G_{\text{ov}}}$
Composition	Standard function composition

Design Decision 98: Functor Law Verification

The overlay mapping $\mathcal{F} : \textbf{Plan} \to \textbf{Overlay}$ satisfies:

1. **Identity preservation:** $\mathcal{F}(\mathrm{id}_G) = \mathrm{id}_{\mathcal{F}(G)}$. The identity homomorphism on the plan maps to the identity on the overlay—no spurious state is created.
2. **Composition preservation:** $\mathcal{F}(g \circ f) = \mathcal{F}(g) \circ \mathcal{F}(f)$. Composing two plan transformations and then computing the overlay yields the same result as computing overlays independently and composing them.

The dependency DAG has the following primary chain (read as "must be evaluated before"):

$$\mathcal{F}_{\mathrm{PHY}} \to \mathcal{F}_{\mathrm{L2}} \to \mathcal{F}_{\mathrm{MPLS}} \to \underbrace{\mathcal{F}_{\mathrm{OSPF}} \mid \mathcal{F}_{\mathrm{ISIS}}}_{\mathrm{IGP}} \to \mathcal{F}_{\mathrm{BGP}} \to \mathcal{F}_{\mathrm{EVPN}}$$

The IGP stage offers two alternative functors: $\mathcal{F}_{\mathrm{OSPF}}$ (Section 17) and $\mathcal{F}_{\mathrm{ISIS}}$ (Section 18.3). A network may activate one or both; the dependency DAG evaluates whichever functors have matching attributes present in G_{plan}. Additionally, $\mathcal{F}_{\mathrm{BGP}}$ depends on the IGP functors for next-hop resolution, and $\mathcal{F}_{\mathrm{MPLS}}$ depends on the IGP functors for label binding. The three cross-layer functors ($\mathcal{F}_{\mathrm{QoS}}$, $\mathcal{F}_{\mathrm{ACL}}$, $\mathcal{F}_{\mathrm{Security}}$; see Section 19) read directly from G_{plan} and depend only on the physical and L2 layers for interface resolution. The evaluation order is any topological sort of this DAG.

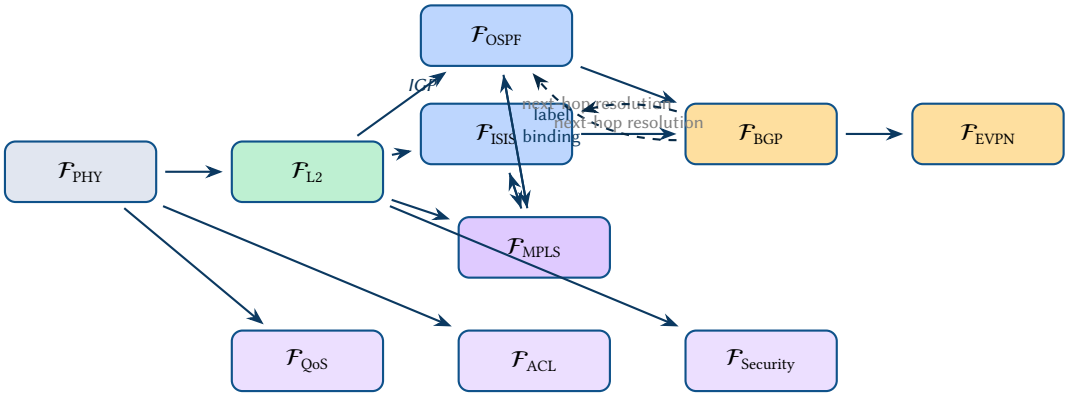


Figure 51. Functor dependency DAG: directed edges show 'depends-on' relationships between layer functors. The IGP layer offers two alternative functors ($\mathcal{F}_{\mathrm{OSPF}}$ and $\mathcal{F}_{\mathrm{ISIS}}$) which feed into $\mathcal{F}_{\mathrm{BGP}}$ and $\mathcal{F}_{\mathrm{MPLS}}$. Cross-layer functors ($\mathcal{F}_{\mathrm{QoS}}$, $\mathcal{F}_{\mathrm{ACL}}$, $\mathcal{F}_{\mathrm{Security}}$) read from the plan topology and physical/L2 layers.

Proposition 1 (Composition Preservation). *If G_{plan} satisfies the NTE validation rules (same-layer connectivity, no orphan endpoints, acyclic layer hierarchy), then $\mathcal{F}(G_{\mathrm{plan}})$ produces a well-formed multilayer overlay where each layer graph is internally consistent and inter-layer couplings respect the dependency ordering.*

Proof. The NTE validator enforces: (1) connectivity edges connect endpoints in the same layer, preventing cross-layer short-circuits; (2) every endpoint has exactly one owner, ensuring the multiplex identity coupling is well-defined; (3) the layer dependency graph is acyclic, guaranteeing a valid topological ordering for functor composition. Each $\mathcal{F}_\ell$ operates only on p_ℓ-prefixed attributes, so layer extraction is independent. The DPO gluing condition is satisfied because connectivity edges are preserved (not deleted) during overlay construction. □

17.6 Complexity and Scalability

The plan topology's state space grows linearly with network size:

Design Decision 99: State Space Complexity

Component	Complexity	Notes
Plan topology size	$O(N+E)$	N = nodes + endpoints, E = connectivity edges
Per-functor extraction	$O(N)$	Linear scan of attribute-prefixed columns
$\mathcal{F}_{\text{OSPF}}$	$O((N+E)\log N)$	Dijkstra SPF per area
$\mathcal{F}_{\text{BGP}}$	$O(P \cdot R)$	P = peers, R = routes in best-path selection
$\mathcal{F}_{\text{STP}}$	$O(N+E)$	Root election + port role assignment
Full composition	$O(L \cdot (N+E)\log N)$	L = number of active layers
Memory (plan)	$O(N \cdot A)$	A = average attributes per node/endpoint

Practical bounds. For a 1000-node Clos fabric with an average of 48 endpoints per node (leaf switches) and 8 active protocol layers, the plan topology contains approximately $N = 1000 + 48000 = 49000$ vertices and $E \approx 17000$ connectivity edges. The full overlay adds approximately 17000 overlay vertices across all layers, for a total of ~66000 overlay vertices—well within the capacity of a single NTE instance running on commodity hardware (sub-second $\mathcal{F}(G)$ evaluation with SIMD-accelerated DataFrame queries).

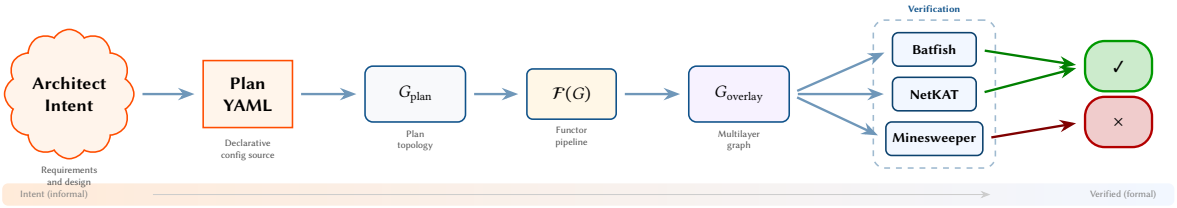


Figure 52. End-to-end verification pipeline. Architect intent is encoded as declarative Plan YAML, parsed into G_{plan}, transformed by the functor pipeline $\mathcal{F}(G)$ into the multilayer G_{overlay}, and then validated by complementary verification tools (Batfish for configuration analysis, NetKAT for header-space algebra, Minesweeper for SMT-based property checking). The pipeline progresses from informal intent to formally verified state.

18 Detailed Layer Functor Specifications

The preceding sections established the plan topology G_{plan} as a single flat graph carrying all network state, and described the general mechanism by which layer functors $\mathcal{F}_\ell$ extract protocol-specific overlay graphs. In Section 17 we gave a complete specification for $\mathcal{F}_{\text{OSPF}}$. We now turn to the remaining major functors, providing for each one (i) the input attributes consumed from G_{plan}, (ii) the algorithmic procedure expressed as pseudocode, (iii) the corresponding DPO rewriting rule, and (iv) a flowchart diagram. The goal is to demonstrate that every widely-deployed control-plane protocol can be captured within the functor framework without introducing protocol-specific graph structures or ad-hoc extensions.

18.1 $\mathcal{F}_{\text{BGP}}$: BGP Overlay Construction

The Border Gateway Protocol is the sole exterior routing protocol in production networks and, via its multi-address-family extensions (MP-BGP), has become the control plane for MPLS VPNs, EVPN, and segment routing policy. The BGP functor $\mathcal{F}_{\text{BGP}}$ is the most attribute-rich of all layer functors, consuming a large set of peer-level and node-level attributes from G_{plan}.

18.1.1 Input Attributes

Design Decision 100: BGP Input Attributes

Attribute	Type	Semantics
`bgp_enabled`	bool	BGP process active on this node
`bgp_asn`	u32	Autonomous system number
`bgp_router_id`	ipv4	BGP router identifier
`bgp_cluster_id`	ipv4	Route reflector cluster ID
`bgp_peer_address`	ip	Peer remote address
`bgp_peer_asn`	u32	Peer remote ASN
`bgp_peer_update_source`	str	Source interface/address for session
`bgp_peer_multihop`	u8	eBGP multihop TTL (0 = disabled)
`bgp_peer_password`	str	TCP-MD5 authentication key
`bgp_peer_afi_safi`	list	Enabled address families
`bgp_peer_route_map_in`	str	Inbound route-map name
`bgp_peer_route_map_out`	str	Outbound route-map name
`bgp_peer_default_originate`	bool	Advertise default route to peer
`bgp_peer_next_hop_self`	bool	Override next-hop to self
`bgp_peer_rr_client`	bool	Peer is a route reflector client
`bgp_peer_send_community`	enum	Community propagation (standard/extended/both/none)
`bgp_confederation_id`	u32	BGP confederation identifier
`bgp_confederation_peers`	list	Member-AS list within confederation

The attributes are split between node-level properties (`bgp_enabled`, `bgp_asn`, `bgp_router_id`, `bgp_cluster_id`, `bgp_confederation_*`) and endpoint-level properties (`bgp_peer_*`). Each configured BGP neighbour relationship produces one endpoint on each side of the session; the functor's first task is to resolve these endpoint pairs into sessions.

18.1.2 Session Resolution

A BGP session exists between endpoints e_i and e_j if and only if $\texttt{bgp\_peer\_address}(e_i)$ matches the IP address bound to the interface owning e_j, and vice versa. This bidirectional address resolution is the plan-topology analogue of TCP connection establishment. The functor must also validate:

- **ASN consistency**: $\texttt{bgp\_peer\_asn}(e_i) = \texttt{bgp\_asn}(\text{owner}(e_j))$, and symmetrically.
- **Multihop reachability**: if $\texttt{bgp\_peer\_multihop} > 0$, an IGP path must exist between the endpoints (requiring $\mathcal{F}_{\text{IGP}}$ output).
- **Authentication**: if `bgp_peer_password` is set on one side, the other must carry a matching value.
- **AFI/SAFI intersection**: the session is established only for address families present in both peers' `bgp_peer_afi_safi` lists.

Sessions are classified as iBGP ($\texttt{bgp\_asn}(e_i) = \texttt{bgp\_asn}(e_j)$) or eBGP. For iBGP sessions, the functor checks `bgp_peer_rr_client` to determine route-reflector topology: if e_i marks e_j as an RR client, routes received from e_j are reflected to all other clients and non-client peers according to the standard reflection rules (RFC 4456).

18.1.3 Best-Path and RIB Computation

Once sessions are resolved, the functor computes the three RIB stages for each BGP speaker. The algorithm resolves peer sessions by address matching, applies inbound route-maps to populate Adj-RIB-In, runs the twelve-step best-path decision process to build Loc-RIB, applies outbound route-maps for Adj-RIB-Out, and handles next-hop-self rewriting and route reflection per RFC 4456. Full pseudocode appears in Appendix C.2.

The route-map evaluation deserves particular attention. Each route-map is a named, ordered list of match/set clauses that filters and transforms routes. In the plan topology, route-maps are referenced by name via `bgp_peer_route_map_in` and `bgp_peer_route_map_out`; the route-map definitions themselves are stored as node-level attributes with a `route_map_*` prefix (see Section 7). The functor resolves these references at evaluation time, applying match conditions (prefix-list, community, AS-path regex) and set actions (local-preference, MED, community, next-hop) in sequence.

Insight: *The BGP best-path algorithm is the most complex derived-state computation in the functor pipeline: it involves a twelve-step tie-breaking sequence, route-map policy evaluation with set/match clauses, and conditional route reflection. Despite this complexity, it remains a pure function on G_{plan} attributes and the IGP RIB from $\mathcal{F}_{\text{IGP}}$.*

18.1.4 DPO Rewriting Rule

Design Decision 101: DPO Rule: Deploy BGP Session

The BGP session deployment rule $p_{\text{BGP}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): Two endpoints e_i, e_j on distinct nodes, where `bgp_peer_address`(e_i) resolves to the interface owning e_j and vice versa, with compatible ASN, AFI/SAFI, and authentication.
- K (preserve): Both endpoints and their owner nodes.
- R (produce): The preserved elements plus a new directed *BGP session* edge (e_i, e_j) labeled with:
  - `session_type`: iBGP or eBGP,
  - `afi_safi`: negotiated address families,
  - `adj_rib_in`, `loc_rib`, `adj_rib_out`: computed RIB contents.

For route-reflector topologies, additional edges from the RR to each client encode the reflection relationship.

The DPO rule is applied once per resolved session pair. Because BGP sessions are point-to-point, the matching is deterministic: each pair of endpoints with mutually consistent `bgp_peer_address` values produces exactly one session edge. The functor's non-trivial output is the RIB content attached to each session, which encodes the full convergence outcome of the BGP decision process.

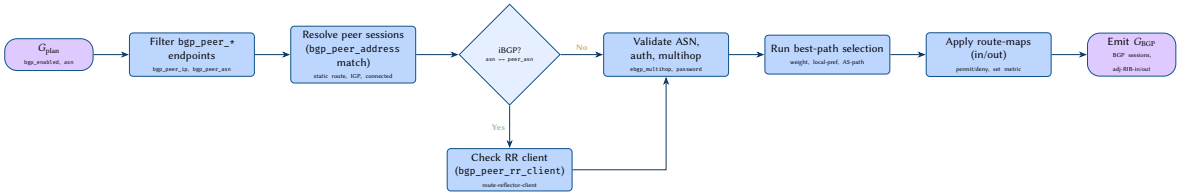


Figure 53. $\mathcal{F}_{\text{BGP}}$ flowchart: from plan attributes to BGP overlay graph with session resolution and best-path computation.

18.1.5 Confederation Support

BGP confederations (RFC 5065) partition a single AS into multiple sub-ASes to reduce the iBGP full-mesh requirement. The functor handles confederations by treating intra-confederation eBGP sessions as a hybrid: the AS-path uses confederation-specific segment types (AS_CONFED_SEQ, AS_CONFED_SET), and the local-preference and MED attributes are preserved across confederation boundaries, unlike true eBGP where MED comparison is restricted to routes from the same neighbouring AS.

When `bgp_confederation_id` is set on a node, the functor maps the node's `bgp_asn` to a member-AS within the confederation identified by `bgp_confederation_id`. Sessions between nodes sharing the same `bgp_confederation_id` but different `bgp_asn` values are classified as confederation-eBGP.

18.2 $\mathcal{F}_{\text{STP}}$: Spanning Tree Overlay

The Spanning Tree Protocol family (STP, RSTP, MSTP) prevents forwarding loops in bridged Ethernet networks by selectively blocking redundant links. Unlike the routing-protocol functors that *add* forwarding state, $\mathcal{F}_{\text{STP}}$ *removes* edges from the L2 topology: its output is a pruned subgraph of the broadcast domain where every VLAN has a loop-free spanning tree.

18.2.1 Input Attributes

Design Decision 102: STP Input Attributes

Attribute	Type	Semantics
`stp_enabled`	bool	STP participation on this bridge
`stp_mode`	enum	Protocol variant: stp, rstp, mstp
`stp_bridge_priority`	u16	Bridge priority (0–61440, step 4096)
`stp_port_priority`	u8	Port priority for tie-breaking
`stp_port_cost`	u32	Path cost to traverse this port
`stp_guard`	enum	Guard mode: root, loop, bpdu, none
`stp_portfast`	bool	Skip listening/learning states
`stp_bpdu_filter`	bool	Suppress BPDU transmission
`stp_msti_mapping`	map	MSTP instance-to-VLAN mapping

Node-level attributes include `stp_enabled`, `stp_mode`, and `stp_bridge_priority`. Endpoint-level attributes capture per-port parameters: cost, priority, guard mode, and PortFast configuration. The MSTP mapping attribute is a flattened representation of the VLAN-to-instance table (flattened per Section 1).

18.2.2 Root Election and Port Role Assignment

The STP functor operates per VLAN (in classic STP/RSTP mode) or per MSTI (in MSTP mode). For each spanning tree instance, the algorithm proceeds in three phases:

1. **Root election.** The bridge with the lowest (priority, bridge-ID) tuple is elected root. The bridge-ID is typically the base MAC address of the switch, which in the plan topology is represented as a node attribute.
2. **Root-path computation.** For each non-root bridge, the functor computes the shortest path to the root using `stp_port_cost` as the edge weight. The port on each bridge that lies on its shortest path to the root becomes the *root port*.
3. **Designated and blocked port assignment.** On each network segment (link), the port with the lowest root-path cost is the *designated port*. All other ports on that segment become *alternate* (RSTP) or *blocked* (classic STP) ports. Backup ports arise when multiple ports on the same bridge connect to the same segment.

The algorithm elects a root bridge per VLAN (or MSTI instance) by lowest priority and bridge ID, computes shortest-path costs to the root for each port, assigns root, designated, and alternate/blocked port roles per segment, and returns the pruned spanning tree. Full pseudocode appears in Appendix C.3.

18.2.3 Guard and Protection Features

Insight: *STP is the only functor that* subtracts *forwarding capacity: its output is a subset of the L2 connectivity graph with redundant edges removed. This makes STP's interaction with other functors particularly important—the L2 overlay that feeds into MPLS and IGP must incorporate STP port states to avoid computing routes over blocked links.*

The plan topology captures several STP protection mechanisms through endpoint attributes:

- **Root guard** (`stp_guard=root`): prevents a port from becoming a root port. If a superior BPDU is received, the port enters root-inconsistent state rather than accepting the new root. In the functor, this is modelled as a constraint: during root-path computation, root-guarded ports cannot be assigned the root port role.
- **BPDU guard** (`stp_guard=bpdu`): places the port in err-disabled state upon receiving any BPDU. The functor marks such ports as unconditionally blocked unless paired with PortFast.
- **PortFast** (`stp_portfast=true`): the port transitions directly to forwarding without passing through listening/learning. In the static plan model, PortFast ports are treated as edge ports that do not participate in the spanning tree computation.

18.2.4 DPO Rewriting Rule

Design Decision 103: DPO Rule: Elect STP Root

The STP root election rule $p_{\text{STP}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): A set of bridge nodes $\{b_1, \ldots, b_n\}$ with `stp_enabled=true`, connected by L2 connectivity edges, all participating in the same VLAN or MSTI.
- K (preserve): All bridge nodes and connectivity edges.
- R (produce): The preserved elements plus:
  - A *root designation* edge from the elected root to a virtual root vertex.
  - *Port role* labels on each endpoint: root, designated, alternate, or backup.
  - *Blocked* annotations on alternate/backup ports.

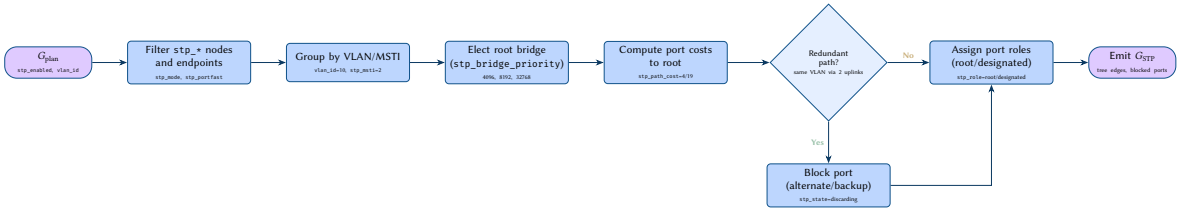


Figure 54. $\mathcal{F}_{\text{STP}}$ flowchart: from plan attributes to Spanning Tree overlay graph with root election and port role assignment.

18.3 $\mathcal{F}_{\text{ISIS}}$: IS-IS Overlay Construction

The Intermediate System to Intermediate System protocol (IS-IS, RFC 1195) is a link-state IGP that operates natively over the CLNS data link layer rather than IP. Originally designed for OSI routing, IS-IS was extended

to carry IPv4 and IPv6 reachability information via its TLV-based PDU encoding, making it highly extensible without protocol version changes. IS-IS is widely deployed in ISP backbone networks and large enterprise environments, where its independence from the IP layer and its proven convergence characteristics make it a preferred alternative to OSPF. The IS-IS functor $\mathcal{F}_{\text{ISIS}}$ constructs the IS-IS adjacency graph and computes per-level SPF trees from plan-topology attributes.

18.3.1 Input Attributes

Design Decision 104: IS-IS Input Attributes

Attribute	Type	Semantics
`isis_enabled`	bool	IS-IS process active on this node
`isis_net`	str	Network Entity Title (e.g. `49.0001.0010.0000.0001.00`)
`isis_circuit_type`	enum	Circuit type: level-1, level-2, level-1-2
`isis_metric`	u32	Interface metric for SPF computation
`isis_metric_style`	enum	Metric style: narrow (6-bit), wide (24-bit), transition
`isis_level`	enum	Router level: level-1, level-2, level-1-2
`isis_overload_bit`	bool	Set overload (OL) bit in LSPs
`isis_auth_type`	enum	Authentication: none, text, md5, hmac-sha
`isis_auth_key`	str	Authentication key string
`isis_interface_type`	enum	Interface mode: broadcast, point-to-point
`isis_priority`	u8	DIS election priority (0–127)
`isis_csnp_interval`	u16	CSNP transmission interval (seconds)
`isis_hello_interval`	u16	IIH PDU transmission interval (seconds)
`isis_hello_multiplier`	u8	Hold-time multiplier for adjacency timeout
`isis_lsp_interval`	u16	Minimum interval between LSP transmissions (ms)

Node-level attributes include `isis_enabled`, `isis_net`, `isis_level`, and `isis_overload_bit`. Endpoint-level attributes capture per-interface parameters: metric, circuit type, interface type, DIS priority, and timer settings. The Network Entity Title (NET) encodes the area address, system identifier, and selector byte, which the functor parses to determine area membership and system identity.

18.3.2 Adjacency Formation

IS-IS forms adjacencies through the exchange of IS-IS Hello (IIH) PDUs. The functor models adjacency formation by matching endpoints that share a common link in the plan topology, verifying:

- **Level compatibility**: the circuit types of both endpoints must have at least one level in common (e.g. a level-1-only circuit cannot form an adjacency with a level-2-only circuit).
- **Area match for L1**: Level-1 adjacencies require that both endpoints share the same area address (extracted from `isis_net`).
- **Authentication**: if `isis_auth_type` is set, both sides must agree on the type and carry matching `isis_auth_key` values.
- **Metric style**: endpoints using incompatible metric styles (narrow vs. wide without transition mode) cannot correctly exchange link-state information.

On broadcast segments (`isis_interface_type=broadcast`), IS-IS elects a Designated Intermediate System (DIS) rather than a DR/BDR pair. The DIS election uses the highest `isis_priority` value, with ties broken by the highest Subnetwork Point of Attachment (SNPA, typically the MAC address). Unlike OSPF, the DIS election is preemptive: a new router with higher priority immediately takes over. On point-to-point links (`isis_interface_type=point-to-point`), the functor models the three-way handshake (TLV 240) that ensures both sides have confirmed the adjacency before transitioning to the Up state.

18.3.3 SPF Computation and Route Leaking

IS-IS maintains separate Link-State Databases (LSDBs) for Level-1 and Level-2. The functor partitions the adjacency graph by level and runs Dijkstra's SPF algorithm independently on each:

- **Level-1 SPF**: computes shortest paths within a single area. Level-1 routers install a default route toward the nearest Level-1-2 router (the *attached* bit in L2 LSPs signals inter-area reachability).
- **Level-2 SPF**: computes shortest paths across the backbone, connecting all areas. Only routers with `isis_level` $\in$ {level-2, level-1-2} participate.

Route leaking between levels is handled by Level-1-2 routers. By default, Level-1 routes are leaked into the Level-2 LSDB, providing inter-area reachability. The reverse direction—Level-2 routes leaked into Level-1—requires explicit policy (a distribute-list or route-map filter) to prevent routing loops.

The algorithm filters `isis_*` endpoints, partitions them by level (Level-1, Level-2), builds per-level adjacency graphs with DIS election, runs Dijkstra SPF using `isis_metric` costs, and handles L1→L2 route leaking for Level-1-2 routers. Full pseudocode appears in Appendix C.4.

18.3.4 DPO Rewriting Rule

Design Decision 105: DPO Rule: Deploy IS-IS Adjacency

The IS-IS adjacency deployment rule $p_{\text{ISIS}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): Two endpoints e_i, e_j on distinct nodes, both with `isis_enabled=true`, connected by a physical or logical link in G_{plan}, with compatible circuit types, matching area addresses (for L1), and consistent authentication.
- K (preserve): Both endpoints, their owner nodes, and the underlying connectivity edge.
- R (produce): The preserved elements plus a new undirected *IS-IS adjacency* edge (e_i, e_j) labeled with:
  - `adj_level`: the negotiated level (L1, L2, or both),
  - `adj_metric`: the `isis_metric` value,
  - `adj_state`: Up (three-way handshake complete),
  - `dis`: the elected DIS system ID (broadcast segments only).

For broadcast segments, an additional pseudonode vertex is created, with edges from the DIS to the pseudonode encoding the LAN topology in the LSDB.

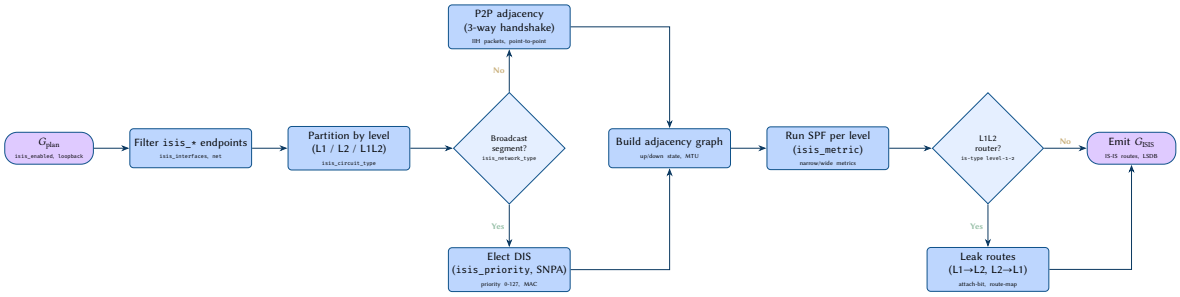


Figure 55. $\mathcal{F}_{\text{ISIS}}$ flowchart: from plan attributes to IS-IS overlay graph with level partitioning, DIS election, and SPF computation.

Insight: *IS-IS and OSPF are both link-state IGPs, but differ in several structurally significant ways. IS-IS uses a two-level hierarchy (L1/L2) rather than OSPF's multi-area model with stub/NSSA variants. IS-IS metrics are flat (narrow: 6-bit, wide: 24-bit) whereas OSPF assigns per-interface costs. The TLV-based PDU encoding of IS-IS allows new features to be added without protocol version changes, in contrast to OSPF's fixed header format that required OSPFv3 for IPv6. DIS election in IS-IS is preemptive and does not require a BDR, unlike OSPF's DR/BDR pair with non-preemptive election. Finally, IS-IS runs directly over the data link layer (CLNS), making it independent of IP, whereas OSPF is encapsulated in IP (protocol 89).*

18.4 $\mathcal{F}_{\text{MPLS}}$: Label Path Construction

The MPLS functor is unique among the layer functors in that it must reconcile three distinct label-distribution mechanisms—LDP, Segment Routing (SR-MPLS), and RSVP-TE—into a single Label Forwarding Information Base (LFIB). Each mechanism binds Forwarding Equivalence Classes (FECs) to labels using different signalling paradigms, but the resulting LFIB entries share a common structure: an incoming label, an outgoing label (or label stack), and a next-hop.

18.4.1 Input Attributes

Design Decision 106: MPLS Input Attributes

Attribute	Type	Semantics
`mpls_enabled`	bool	MPLS forwarding enabled on this node
`mpls_label_range_min`	u32	Dynamic label range lower bound
`mpls_label_range_max`	u32	Dynamic label range upper bound
`ldp_enabled`	bool	LDP label distribution active
`ldp_router_id`	ipv4	LDP router identifier
`ldp_transport_address`	ipv4	LDP session transport address
`ldp_session_holdtime`	u16	LDP session keepalive (seconds)
`sr_enabled`	bool	Segment Routing enabled
`sr_global_block_min`	u32	SRGB lower bound
`sr_global_block_max`	u32	SRGB upper bound
`sr_node_sid`	u32	Node segment identifier
`sr_adj_sid_enabled`	bool	Adjacency SID allocation
`rsvp_enabled`	bool	RSVP-TE signalling active
`rsvp_bandwidth`	u64	Reservable bandwidth (bps)
`rsvp_tunnel_name`	str	TE tunnel identifier
`rsvp_tunnel_dest`	ipv4	TE tunnel tail-end address
`rsvp_setup_priority`	u8	Tunnel setup priority (0–7)
`rsvp_hold_priority`	u8	Tunnel hold priority (0–7)
`rsvp_explicit_path`	list	Explicit route hops

The attribute set reflects the three label-distribution protocols. Node-level attributes (`mpls_enabled`, `mpls_label_range_*`) control the global MPLS data plane. LDP attributes are primarily node-level (router-ID, transport address), while SR attributes mix node-level (SRGB range) and endpoint-level (node-SID, adjacency-SID) properties. RSVP-TE attributes are associated with tunnel endpoints.

18.4.2 Label Distribution Mechanisms

LDP (Label Distribution Protocol). LDP establishes label bindings via downstream unsolicited distribution: each LSR independently assigns a local label to each FEC (typically an IGP-learned prefix) and advertises the binding to all LDP neighbours. The functor models this by:

1. Identifying LDP-enabled node pairs connected by IGP adjacencies.
2. For each IGP prefix reachable through a given next-hop, allocating a label from the dynamic range and creating a label mapping message.
3. Building the LFIB entry: swap the incoming label for the next-hop's advertised label, forward to the next-hop interface.

Segment Routing (SR-MPLS). SR eliminates the need for a label-distribution protocol by encoding forwarding instructions directly in the label stack. The functor handles two SID types:

- **Node SID**: a globally significant label (SRGB base + `sr_node_sid`) that identifies a destination prefix. Every SR-capable node in the domain installs an LFIB entry for every other node's prefix SID.
- **Adjacency SID**: a locally significant label that steers traffic to a specific adjacency. Used for traffic engineering and strict forwarding paths.

RSVP-TE. RSVP-TE establishes label-switched paths (LSPs) via explicit signalling. The functor processes each configured tunnel by:

1. Computing the path: either the explicit route in `rsvp_explicit_path`, or a CSPF (Constrained SPF) path respecting bandwidth and priority constraints.
2. Allocating labels at each hop along the path.
3. Installing LFIB entries with the appropriate label stack.

18.4.3 LFIB Merge and Pseudocode

The algorithm processes three label-distribution protocols in turn—LDP (per-FEC label allocation along IGP adjacencies), Segment Routing (prefix-SID and adjacency-SID computation from the SRGB), and RSVP-TE (explicit or CSPF path computation with per-hop label allocation)—then merges the resulting bindings

into a unified LFIB with configurable priority (default: SR > RSVP-TE > LDP). Full pseudocode appears in Appendix C.5.

The merge step resolves conflicts when multiple protocols bind labels to the same FEC. The default priority order (SR, then RSVP-TE, then LDP) reflects common deployment practice, but the plan topology can encode an explicit priority via a node-level attribute.

18.4.4 DPO Rewriting Rule

Design Decision 107: DPO Rule: Bind MPLS Label to FEC

The MPLS label-binding rule $p_{\text{MPLS}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): Two adjacent LSR nodes n_i, n_j with `mpls_enabled=true` and a shared IGP adjacency. A FEC F is reachable via n_j.
- K (preserve): Both nodes, their endpoints, and the IGP adjacency edge.
- R (produce): The preserved elements plus a new *label-switched path* edge from n_i to n_j annotated with:
  - `in_label`: the locally allocated label,
  - `out_label`: the downstream-advertised label,
  - `fec`: the bound prefix or tunnel,
  - `protocol`: LDP, SR, or RSVP-TE.

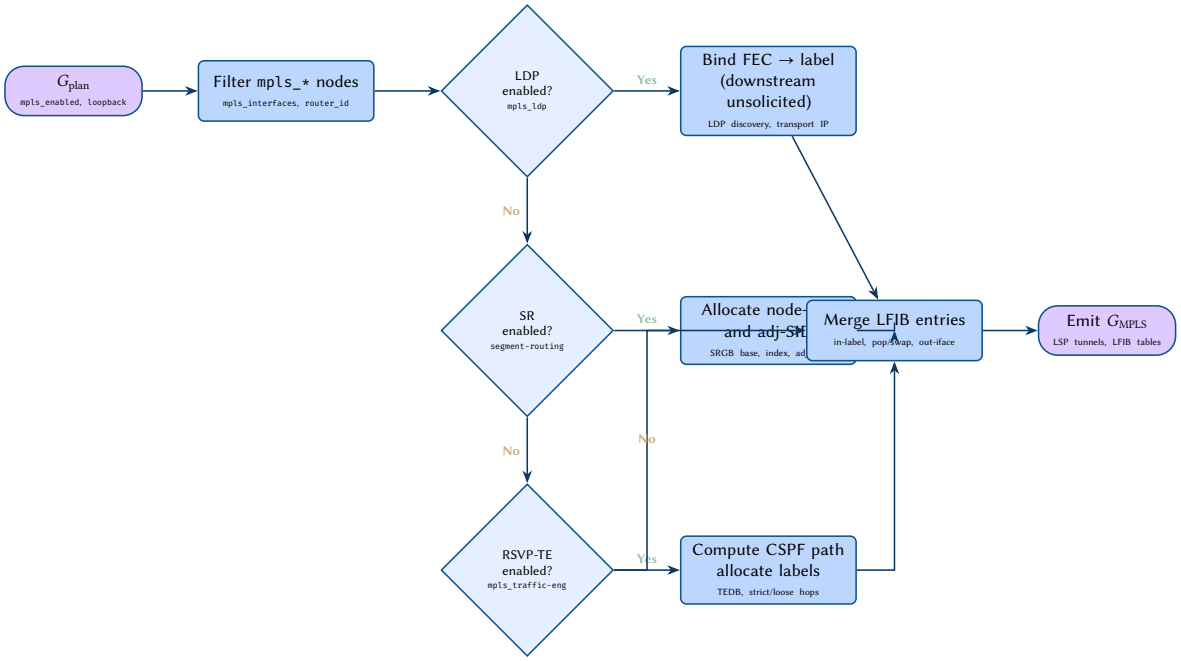


Figure 56. $\mathcal{F}_{\text{MPLS}}$ flowchart: parallel label protocol branches (LDP, SR, RSVP-TE) merging into a unified LFIB.

Insight:
The MPLS functor depends on $\mathcal{F}_{\text{IGP}}$ for two reasons: LDP requires IGP adjacencies to discover neighbours, and SR requires IGP shortest paths for prefix-SID forwarding. This dependency is encoded in the functor DAG (Section 17.5.1).

18.5 $\mathcal{F}_{\text{EVPN}}$: EVPN Overlay Construction

Ethernet VPN (EVPN, RFC 7432) has emerged as the dominant overlay technology for data-centre fabrics, replacing VPLS and providing MAC/IP learning via BGP rather than data-plane flooding. The EVPN functor is tightly coupled to $\mathcal{F}_{\text{BGP}}$ (for the control plane) and $\mathcal{F}_{\text{MPLS}}$ (for the VXLAN or MPLS data plane), making it one of the highest-altitude functors in the dependency DAG.

18.5.1 Input Attributes

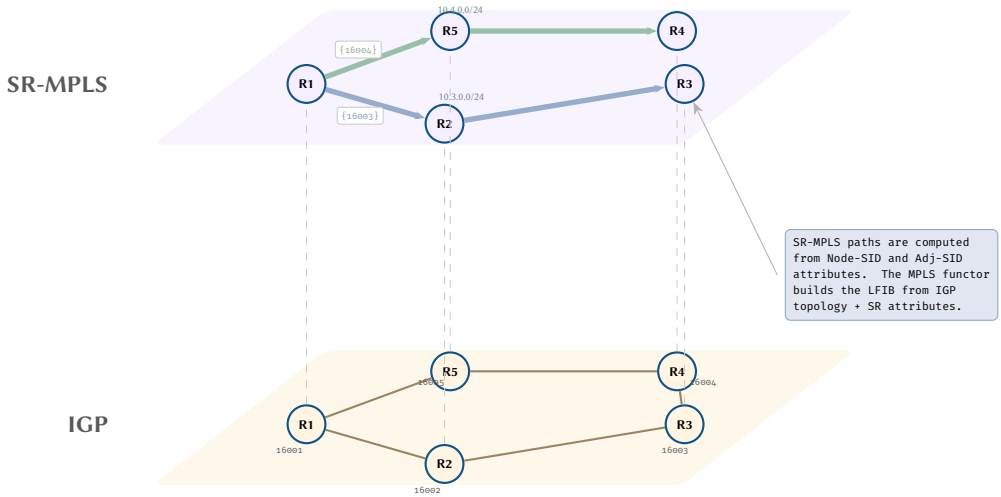


Figure 57. SR-MPLS overlay: label-switched paths computed from segment routing attributes overlaid on the IGP topology.

Design Decision 108: EVPN Input Attributes

Attribute	Type	Semantics
`evpn_enabled`	bool	EVPN service active on this node
`evpn_evi`	u32	EVPN instance identifier
`evpn_rd`	str	Route distinguisher for the EVI
`evpn_rt_import`	list	Import route targets
`evpn_rt_export`	list	Export route targets
`evpn_arp_suppress`	bool	ARP suppression (proxy-ARP)
`vxlan_enabled`	bool	VXLAN encapsulation active
`vxlan_vni`	u32	VXLAN Network Identifier
`vxlan_source_interface`	str	VTEP source interface name
`vtep_address`	ip	VTEP IP address
`evpn_multihoming`	bool	Multi-homing (ESI LAG) enabled
`evpn_esi`	str	Ethernet Segment Identifier
`evpn_df_election`	enum	DF election method: modulo or preference

EVPN attributes span the control plane (`evpn_*`), the data plane (`vxlan_*`), and multi-homing (`evpn_esi`, `evpn_df_election`). Node-level attributes include `evpn_enabled` and `vtep_address`; most other attributes are per-EVI and therefore attach to endpoints representing VXLAN tunnel interfaces.

18.5.2 Route Type Processing

EVPN defines five route types, of which the functor processes three in detail:

- **Type-2 (MAC/IP Advertisement):** each VTEP advertises the MAC and optionally IP addresses learned on its locally attached segments. The functor builds a distributed MAC/IP table by importing Type-2 routes whose route targets match the EVI's `evpn_rt_import`.
- **Type-3 (Inclusive Multicast Ethernet Tag):** each VTEP advertises its membership in a broadcast domain, enabling the construction of an ingress-replication list (or multicast tree) for BUM (broadcast, unknown unicast, multicast) traffic.
- **Type-1 (Ethernet Auto-Discovery):** used for multi-homing. VTEPs sharing an Ethernet Segment Identifier (ESI) advertise their presence, enabling fast failover and split-horizon filtering.

Type-4 (Ethernet Segment) and Type-5 (IP Prefix) routes are handled analogously but omitted from the pseudocode for brevity.

18.5.3 Multi-Homing and DF Election

When `evpn_multihoming=true`, multiple VTEPs attach to the same Ethernet segment (e.g., a dual-homed server connected via an ESI LAG). The functor must elect a *Designated Forwarder* (DF) for each (ESI, VLAN) pair to prevent duplicate frames:

- **Modulo-based** (`evpn_df_election=modulo`): DF = VTEP[VLAN mod |VTEPs in ESI|]. Simple but can create uneven load distribution.
- **Preference-based** (`evpn_df_election=preference`): each VTEP advertises a preference value; the highest preference wins. Provides deterministic DF placement.

18.5.4 Pseudocode

The algorithm filters EVPN-enabled VTEPs, exports Type-2 (MAC/IP) and Type-3 (BUM membership) routes per EVI using RD/RT attributes, imports routes by RT intersection, and builds per-EVI MAC/IP tables with VNI mappings. For multi-homing, it groups VTEPs by ESI and performs DF election (modulo or preference-based) per VLAN. Full pseudocode appears in Appendix C.6.

18.5.5 DPO Rewriting Rule

Design Decision 109: DPO Rule: Deploy EVPN Type-2 Route

The EVPN Type-2 route deployment rule $p_{\text{EVPN-T2}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): Two VTEP nodes v_i, v_j with overlapping route targets ($\texttt{evpn\_rt\_export}(v_i) \cap \texttt{evpn\_rt\_import}(v_j) \neq \emptyset$), connected by a BGP session (from $\mathcal{F}_{\text{BGP}}$).
- K (preserve): Both VTEPs, the BGP session edge, and existing EVI configuration.
- R (produce): The preserved elements plus a new *MAC/IP binding* edge from v_i to v_j annotated with:
  - `mac`: the learned MAC address,
  - `ip`: the associated IP (if known),
  - `vni`: the VXLAN Network Identifier,
  - `esi`: the Ethernet Segment Identifier (if multi-homed).

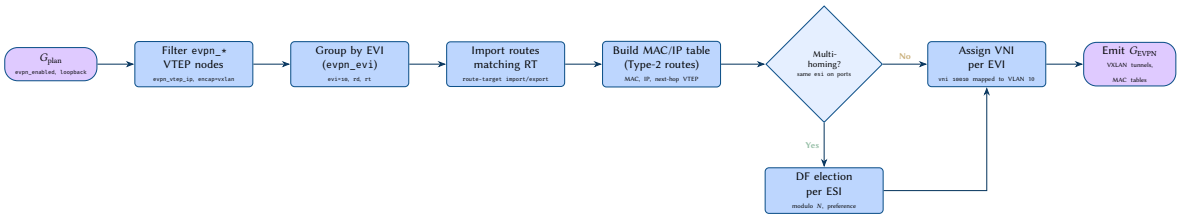


Figure 58. $\mathcal{F}_{\text{EVPN}}$ flowchart: from plan attributes to EVPN overlay graph with multi-homing support and VNI assignment.

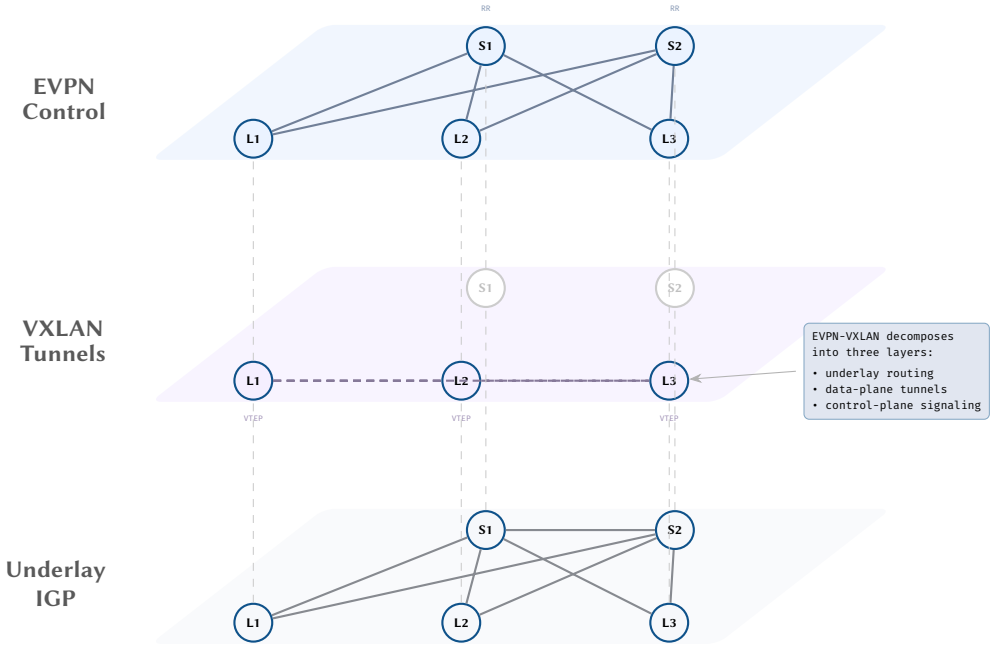


Figure 59. EVPN-VXLAN fabric decomposed into three overlay layers: underlay IGP routing, VXLAN data-plane tunnels, and EVPN control-plane sessions.

18.6 $\mathcal{F}_{\text{L2}}$: VLAN and Switching Overlay

The Layer 2 functor constructs the switching overlay by extracting switchport configuration from G_{plan} and integrating the STP port states produced by $\mathcal{F}_{\text{STP}}$. It is the bridge between the physical topology and the higher-layer protocol overlays.

The functor consumes `switchport_*` attributes: `switchport_mode` (access/trunk/hybrid), `switchport_access_vlan`, `switchport_trunk_allowed_vlans`, and `switchport_native_vlan`. From these attributes, it constructs VLAN membership hyperedges: each VLAN V produces a hyperedge $h_V \subseteq \mathcal{E}$ connecting all endpoints that participate in that VLAN.

The algorithm filters `switchport_*` endpoints, assigns each port to its VLAN membership set (access ports to their access VLAN, trunk ports to each allowed VLAN), then integrates STP port states by removing blocked ports from each VLAN's member set. Full pseudocode appears in Appendix C.7.

The key insight is that the L2 overlay is not simply the set of VLAN memberships, but the *STP-pruned* set: only ports in forwarding state for a given VLAN contribute to that VLAN's broadcast domain. This pruned L2 overlay then serves as the physical substrate over which $\mathcal{F}_{\text{IGP}}$ and $\mathcal{F}_{\text{MPLS}}$ compute their adjacencies and label bindings. A VLAN with all trunk ports blocked by STP is effectively partitioned, which may cause IGP adjacency loss in the layers above—a cross-layer effect captured by the functor dependency ordering.

18.7 Cross-Layer Dependency Walk-Through

The preceding subsections specified each functor in isolation. In practice, the functors interact through the dependency DAG: each functor's output enriches G_{plan} (or, equivalently, is made available to downstream functors via the overlay composition $\bigoplus$). We now trace the full evaluation pipeline through a concrete example: a leaf-spine IP fabric running VXLAN/EVPN over an eBGP underlay with SR-MPLS transport.

18.7.1 Fabric Topology

Consider a two-tier Clos fabric with 4 spine switches and 8 leaf switches. Each leaf has 48 server-facing ports and 4 uplinks to the spines. The plan topology G_{plan} contains:

- 12 nodes (4 spines, 8 leaves),
- $8 \times 48 + 8 \times 4 + 4 \times 8 = 448$ endpoints,
- $8 \times 4 = 32$ spine–leaf connectivity edges,
- Flat attributes spanning L1 through EVPN.

18.7.2 Evaluation Order

The functor DAG dictates the following topological sort:

1. $\mathcal{F}_{\text{PHY}}$: extract physical link attributes (speed, MTU, media type). Output: 32 physical edges with validated parameters.
2. $\mathcal{F}_{\text{L2}}$: extract switchport VLAN memberships. In a routed spine-leaf fabric, most inter-switch links are L3 (routed), so the L2 overlay is sparse—primarily the server-facing access ports grouped by VLAN. STP runs per-VLAN on the access segments.
3. $\mathcal{F}_{\text{MPLS}}$: allocate SR node-SIDs and adjacency-SIDs for each switch. The SRGB is uniform across the fabric (e.g., 16000–23999). Each leaf and spine receives a unique node-SID.
4. $\mathcal{F}_{\text{IGP}}$: in this design, eBGP is used as the underlay routing protocol (no OSPF/IS-IS). However, the IGP functor still computes the loopback reachability graph needed for BGP next-hop resolution. In a pure eBGP fabric, $\mathcal{F}_{\text{IGP}}$ degenerates to directly-connected route extraction.
5. $\mathcal{F}_{\text{BGP}}$: resolve eBGP sessions between each leaf–spine pair (32 sessions) and any leaf–leaf iBGP sessions for EVPN address family. Compute best-path for underlay prefixes (loopbacks) and overlay prefixes (EVPN Type-2/3/5).
6. $\mathcal{F}_{\text{EVPN}}$: import EVPN routes by RT, build the per-EVI MAC/IP tables, assign VNIs, and elect DFs for any multi-homed segments.

Each functor reads from G_{plan} (for its prefix-matched attributes) and from the output of upstream functors (for cross-layer dependencies). The result is a fully composed multilayer overlay G_{overlay} that captures the converged forwarding state of the entire fabric.

18.7.3 Failure Cascade Analysis

A significant advantage of the functor framework is its ability to model cross-layer failure propagation. When a physical link fails, the consequences ripple through the functor DAG:

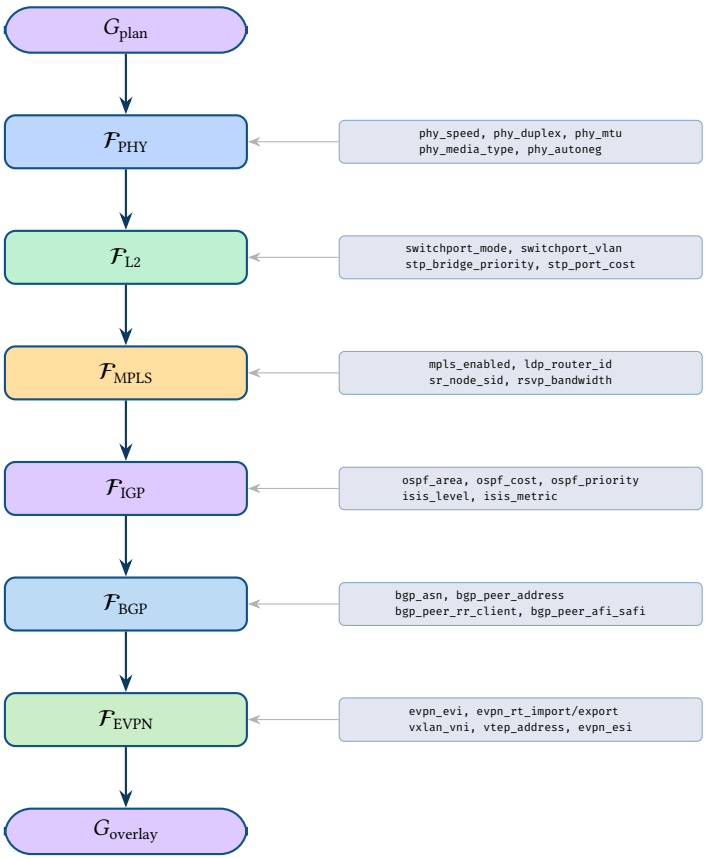


Figure 60. End-to-end attribute flow through the functor pipeline. Each functor consumes prefix-matched attributes from G_{plan} and emits its protocol-specific overlay subgraph.

Link-Failure Cascade

Consider a spine–leaf link failure in the fabric described above. The cascade proceeds through the functor pipeline:

1. **L1 (Physical):** The link-down event removes a connectivity edge from G_{plan}. $\mathcal{F}_{\text{PHY}}$ reflects this immediately.
2. **L2 (Switching):** If the failed link carried trunk VLANs, those VLANs lose a forwarding path. $\mathcal{F}_{\text{STP}}$ may reconverge, unblocking a previously alternate port.
3. **L2.5 (MPLS):** SR adjacency-SIDs associated with the failed interface are withdrawn. If RSVP-TE tunnels traversed the link, they must be rerouted via the make-before-break mechanism or CSPF recomputation.
4. **L3 (IGP/Underlay):** The eBGP session over the failed link goes down. The affected leaf loses one of its four ECMP paths to the spine tier, and BGP reconverges with the remaining three paths.
5. **L3.5 (BGP/EVPN Overlay):** If the failed link was the sole path to a spine carrying EVPN routes, those routes are withdrawn. Other leaves update their MAC/IP tables, potentially causing brief traffic black-holing until reconvergence completes.

Total reconvergence time: $\Delta_{\text{STP}} + \Delta_{\text{MPLS}} + \Delta_{\text{IGP}} + \Delta_{\text{BGP}} \approx$ 50ms + 50ms + 0ms (eBGP direct) + 3s (BGP hold timer) $\approx$ 3.1s without BFD, or $<$ 150ms with BFD-assisted fast detection.

The cascade analysis demonstrates a key benefit of the plan-topology approach: because all protocol state is co-located in a single flat graph, cross-layer effects become explicit. In traditional network management, these cascades are invisible—each protocol's tooling operates independently, and correlating a physical link failure with an EVPN route withdrawal requires manual cross-referencing of multiple protocol-specific databases. The functor framework makes the causal chain visible and computable: re-evaluating the functor DAG after modifying G_{plan} (removing the failed link) produces the new converged state of all layers simultaneously.

18.7.4 Incremental Re-Evaluation

For large networks, full re-evaluation of all functors after every change is wasteful. The dependency DAG enables *incremental evaluation*: only functors whose input attributes are affected by the change need to be re-run. A link failure affects $\mathcal{F}_{\text{PHY}}$, $\mathcal{F}_{\text{L2}}$, $\mathcal{F}_{\text{MPLS}}$, and $\mathcal{F}_{\text{BGP}}$, but not $\mathcal{F}_{\text{STP}}$ (if the failed link was a routed L3 link) or $\mathcal{F}_{\text{EVPN}}$ (if no EVPN routes were lost). Tracking attribute-level dependencies allows the evaluation engine to

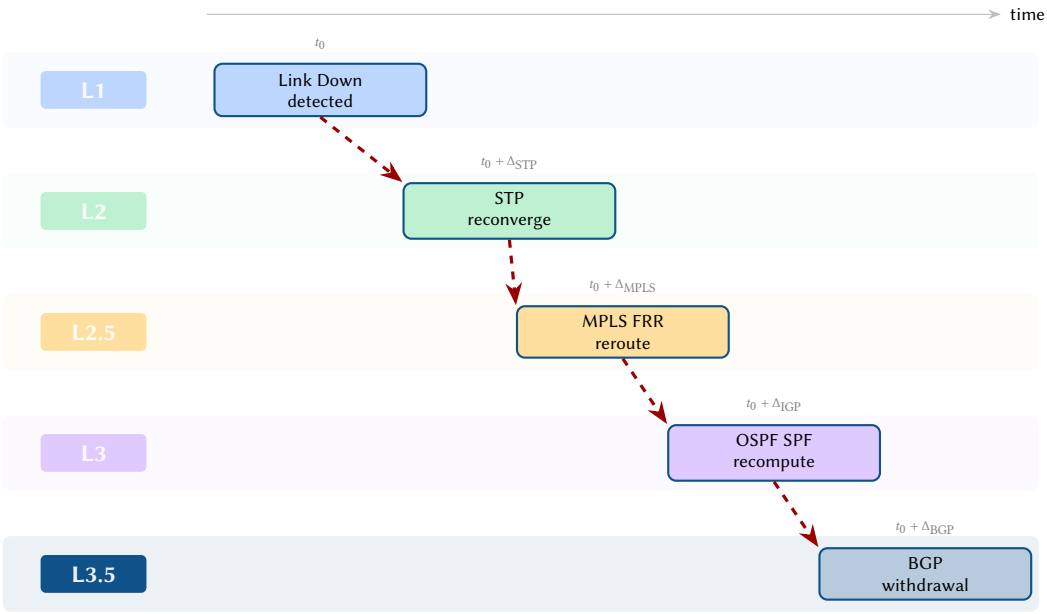


Figure 61. Cross-layer failure cascade: a single link-down event at L1 propagates through STP reconvergence, MPLS fast reroute, OSPF SPF recomputation, and BGP route withdrawal.

skip unaffected functors, reducing re-evaluation time from $O(L \cdot (N+E) \log N)$ to $O(k \cdot (N+E) \log N)$ where $k \ll L$ is the number of affected layers.

19 Cross-Layer Functor Extensions

The preceding section specified functors for the core protocol layers (BGP, STP, MPLS, EVPN, L2). Each of those functors consumes attributes from a single protocol family and produces an overlay graph whose edges encode the derived state of that protocol—adjacencies, forwarding entries, elected roles. The functor framework's power, however, lies not only in its ability to capture individual protocols but in its capacity to express *cross-layer* transformations that consume attributes spanning multiple protocol families simultaneously.

This section extends the functor framework with three cross-layer functors that consume the policy, QoS, and security group attributes defined in Sections 7 and D. Unlike the protocol-specific functors, which model control-plane convergence (neighbour discovery, route selection, tree election), these cross-layer functors model the compilation of operator intent into hardware-level forwarding and classification state. They sit downstream in the functor dependency DAG: $\mathcal{F}_{\text{QoS}}$ reads QoS policy attributes and produces queue/scheduler configurations, $\mathcal{F}_{\text{ACL}}$ compiles access control lists into TCAM entries, and $\mathcal{F}_{\text{Security}}$ resolves SGT assignments and SGACL policy matrices.

The three functors share a common architectural pattern. Each begins by filtering G_{plan} endpoints for a characteristic attribute prefix (`qos_*`, `acl_*`, `sgt_*`). Each then resolves named references—policy names, ACL names, group identifiers—against the flat attribute namespace. Finally, each compiles the resolved policy into a hardware-oriented representation: queue maps, TCAM tables, or enforcement matrices. This filter–resolve–compile pipeline reflects the actual processing stages in network operating systems, making the functor specifications both formally precise and operationally transparent.

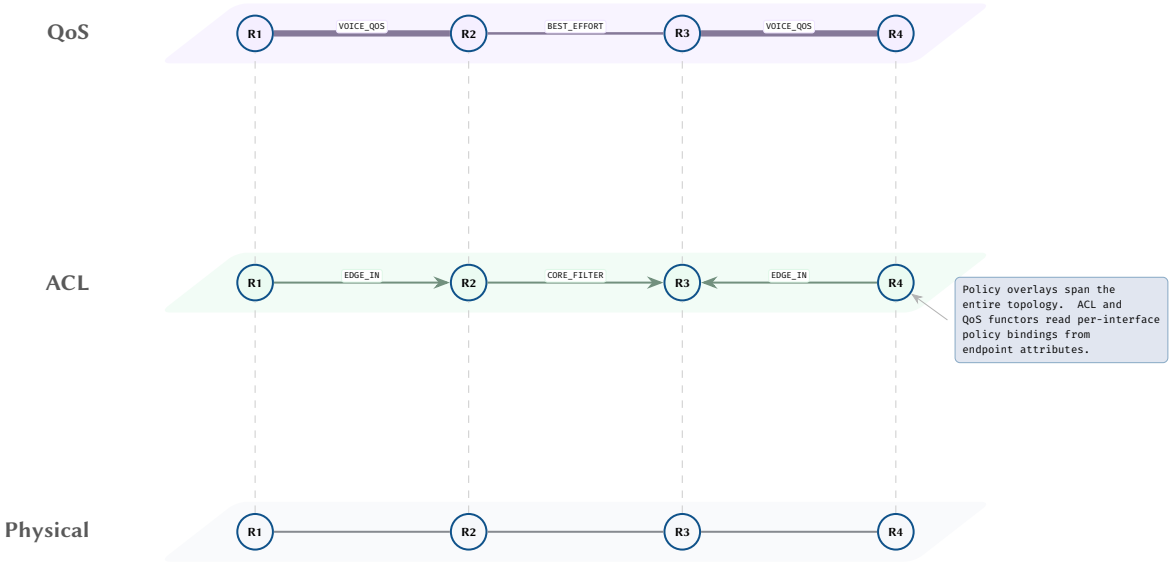


Figure 62. Cross-layer policy overlays: ACL filtering (middle) and QoS classification (top) applied across the physical topology (bottom).

19.1 $\mathcal{F}_{\text{QoS}}$: QoS Policy Compilation

Quality of Service policies translate high-level intent—service classes, bandwidth guarantees, latency targets—into device-specific queue configurations, DSCP markings, and scheduling parameters. In production networks, QoS configuration is among the most error-prone tasks: a single mismatched DSCP remark at a domain boundary can cause an entire traffic class to be treated as best-effort, silently degrading voice or video quality. The QoS functor $\mathcal{F}_{\text{QoS}}$ makes this transformation explicit and verifiable by reading `qos_*` endpoint attributes from G_{plan} and producing a compiled QoS overlay graph G_{QoS} containing queue maps, scheduler configurations, and DSCP rewrite tables.

19.1.1 Type Signature

The QoS functor maps the plan topology to a triple of QoS artefacts:

$$\mathcal{F}_{\text{QoS}} : G_{\text{plan}} \longrightarrow (\text{queue\_map},\ \text{scheduler\_config},\ \text{DSCP\_tables})$$

The *queue map* assigns each traffic class to a hardware queue on each interface. The *scheduler configuration* specifies the scheduling discipline (strict priority, weighted fair queuing, deficit round robin) and associated weights or bandwidth allocations per queue. The *DSCP tables* record per-interface DSCP-to-queue classification mappings and any DSCP rewrite actions applied at ingress or egress.

19.1.2 Input Attributes

Design Decision 110: QoS Input Attributes

Attribute	Type	Semantics
`qos_policy_in`	str	Inbound service-policy name
`qos_policy_out`	str	Outbound service-policy name
`qos_default_dscp`	u6	Default DSCP marking for unclassified traffic
`qos_trust`	enum	Trust boundary: dscp, cos, ip-precedence, none
`qos_scheduler`	enum	Scheduling discipline: strict, wrr, wfq, drr
`qos_bandwidth_pct`	u8	Bandwidth percentage allocated to this queue
`qos_priority_level`	u8	Strict-priority level (0 = not strict)
`qos_shaping_rate`	u64	Egress shaping rate in bits per second
`qos_policing_rate`	u64	Ingress policing rate in bits per second
`qos_policing_burst`	u32	Policing burst size in bytes
`qos_queue_limit`	u32	Queue depth limit (packets or bytes)
`qos_wred_enabled`	bool	Weighted Random Early Detection enabled

The attributes divide naturally into two groups. The *interface-level* attributes (`qos_policy_in`, `qos_policy_out`, `qos_trust`, `qos_default_dscp`) establish the QoS trust boundary and bind named service policies to specific interfaces. The *queue-level* attributes (`qos_scheduler`, `qos_bandwidth_pct`, `qos_priority_level`, `qos_shaping_rate`, `qos_policing_rate`) parameterise the queuing and scheduling behaviour within each bound policy.

19.1.3 Compilation Procedure

The functor processes each interface in five stages: classification, policing, marking, scheduling, and shaping. This pipeline mirrors the hardware processing order on modern ASICs, where ingress classification occurs before policing, and egress shaping occurs after scheduling.

The algorithm resolves inbound and outbound service policies per interface, then executes five stages in hardware processing order: classification (DSCP/CoS to queue mapping based on `qos_trust`), policing (ingress rate limiting), marking (DSCP rewriting), scheduling (queue weight computation for strict-priority, WRR, or WFQ), and shaping (egress token-bucket rate limiting). Full pseudocode appears in Appendix C.8.

The classification stage is the most architecturally significant. The `qos_trust` attribute determines where in the packet header the classifier looks for class-of-service information. When set to `dscp`, the classifier reads the six-bit DiffServ Code Point from the IP header (RFC 2474). When set to `cos`, it reads the three-bit IEEE 802.1p priority field from the VLAN tag—a common configuration on access ports connecting IP phones, where the phone marks voice traffic with CoS 5 and the switch trusts this marking. When set to `none`, all inbound traffic is classified into the default queue and may be marked with the value specified by `qos_default_dscp`.

The scheduling stage translates the abstract queue weights into concrete hardware parameters. For weighted round-robin (WRR) schedulers, each queue's `qos_bandwidth_pct` is normalised against the interface bandwidth to produce a deficit counter. For hierarchical QoS (HQoS) deployments—common in service-provider access networks—the shaping rate at the parent level constrains the aggregate bandwidth available to child queues, creating a two-tier scheduling hierarchy. The functor does not model HQoS nesting explicitly but records the shaping rate as an attribute on the interface endpoint, enabling downstream verification tools to check that the sum of child queue bandwidth allocations does not exceed the parent shaper rate.

19.1.4 DPO Rewriting Rule

Design Rule 12: Apply QoS Policy to Interface

The QoS policy application rule $p_{\text{QoS}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): An endpoint e on a node n where `qos_policy_in`(e) or `qos_policy_out`(e) is set, referencing a named service policy present in G_{plan}.
- K (preserve): The endpoint e and its owner node n, including all existing protocol-layer attributes.
- R (produce): The preserved elements plus a new *QoS binding* edge from n to e labeled with:
  - `queue_map`: mapping from DSCP/CoS values to hardware queue indices,
  - `scheduler_type`: the scheduling discipline and per-queue weights,
  - `dscp_rewrite`: the DSCP rewrite table (if marking is configured),

Insight: *The QoS functor is the only functor in the framework whose output is* entirely local*: each interface's queue map and scheduler configuration depends solely on its own `qos_*` attributes and the resolved service policy. There is no inter-device convergence step. This locality property means that $\mathcal{F}_{\text{QoS}}$ can be evaluated in parallel across all interfaces with no ordering constraints—a marked contrast to $\mathcal{F}_{\text{BGP}}$ or $\mathcal{F}_{\text{OSPF}}$, where route propagation creates global dependencies.*

- `shaping_rate`: the egress shaping rate (if configured).

The rule is applied once per interface with at least one QoS policy binding. Because QoS compilation is purely local, the rule application is idempotent and order-independent: applying the rule to interface e_1 neither enables nor inhibits its application to interface e_2. This property simplifies formal verification, as it guarantees that the set of QoS binding edges in G_{QoS} is uniquely determined regardless of evaluation order.

19.1.5 Flowchart

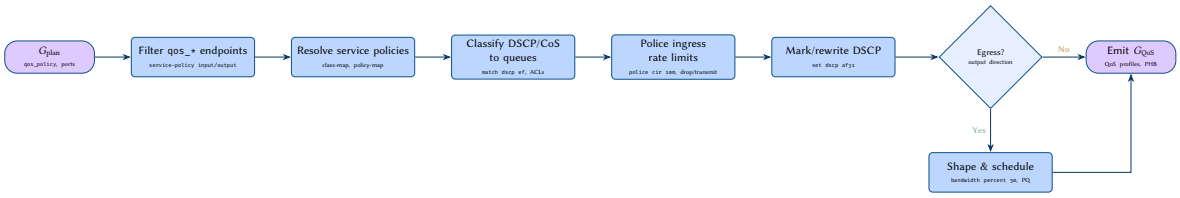


Figure 63. $\mathcal{F}_{\text{QoS}}$ flowchart: from plan attributes to QoS overlay graph.

19.1.6 Discussion

The separation of QoS compilation into a dedicated functor yields two practical benefits. First, it enables *end-to-end QoS path verification*: given a traffic flow traversing multiple hops, a verification tool can walk the sequence of G_{QoS} edges along the forwarding path and check that DSCP markings are consistent across trust boundaries. A common misconfiguration is a trust boundary mismatch, where an upstream router marks traffic with DSCP EF (Expedited Forwarding) but the downstream switch has `qos_trust` set to `cos`, causing the DSCP marking to be ignored and the traffic to be placed in the default queue. The functor's explicit DSCP rewrite tables make such mismatches detectable by static analysis.

Second, the functor supports *capacity planning*. By summing the `qos_bandwidth_pct` allocations across all queues on an interface and comparing against the interface bandwidth, a planning tool can detect oversubscription before deployment. The shaping rate at each interface provides an upper bound on the traffic that the interface will emit, enabling network-wide bandwidth modelling.

19.2 $\mathcal{F}_{\text{ACL}}$: ACL to TCAM Compilation

The ACL functor bridges policy intent and hardware forwarding. Access Control Lists, despite their conceptual simplicity, occupy a critical position in the network stack: they are the primary mechanism by which operators express security policy at the interface level, and their hardware implementation in Ternary Content-Addressable Memory (TCAM) determines the packet-per-second rate at which policy can be enforced. The functor $\mathcal{F}_{\text{ACL}}$ reads `acl_*` attributes from G_{plan}, resolves named ACL references from `acl_in`/`acl_out` endpoint bindings, expands rule entries, compiles wildcard masks into TCAM-compatible ternary entries, and generates per-interface TCAM programming tables.

19.2.1 Type Signature

$$\mathcal{F}_{\text{ACL}} : G_{\text{plan}} \longrightarrow (\text{TCAM\_entries},\ \text{interface\_bindings})$$

The *TCAM entries* are ternary match–action tuples indexed by interface and direction (ingress or egress). Each entry comprises a ternary bit vector (value bits and mask bits) specifying the packet-header fields to match, and an action (permit or deny) to apply on match. The *interface bindings* record which TCAM table is applied to which interface in which direction, providing the linkage between the compiled hardware state and the logical interface topology.

19.2.2 Input Attributes

The ACL functor consumes two classes of attributes. The *binding attributes* (`acl_in`, `acl_out`) are endpoint-level attributes that name the ACL applied to an interface in each direction. The *rule attributes* (`acl_<name>_seq_<n>_*`) are node-level attributes that define the individual ACEs within each named ACL. The complete per-entry attribute set was specified in Section 7.1; the functor consumes all fields listed there, including the match fields (source/destination IP, protocol, ports, DSCP), the action field, and the operational counters.

19.2.3 Compilation Procedure

The algorithm filters interfaces with `acl_in` or `acl_out` bindings, resolves named ACL entries from flat attributes, compiles each ACE into a ternary match–action TCAM entry (converting prefix and wildcard masks to ternary bit vectors with action, logging, and counter flags), and appends an implicit deny-all entry at the end of each per-interface TCAM table. Full pseudocode appears in Appendix C.9.

The compilation step—`compile_to_tcam`—deserves detailed attention, as it embodies the transformation from the operator's address-oriented mental model to the hardware's bit-oriented matching model. An ACL entry

specifying "permit tcp 10.0.0.0 0.0.0.255 any eq 443" contains four independent match dimensions: protocol (TCP = 6), source address (10.0.0.0/24, expressed as wildcard 0.0.0.255), destination address (any = 0.0.0.0/0.0.0.0 wildcard 255.255.255.255), and destination port (443). The compiler concatenates these fields into a single ternary bit vector where each bit position is one of three states: 0 (must be zero), 1 (must be one), or X (don't care, masked out).

The wildcard-to-TCAM conversion is direct for contiguous wildcards (standard subnet masks), but IOS-style ACLs permit non-contiguous wildcard masks—for example, `0.0.0.254` matches all even addresses in a /24. Non-contiguous wildcards map naturally to TCAM entries (each don't-care bit becomes an X in the ternary vector) but cannot be represented as a single prefix; this is a case where the TCAM representation is strictly more expressive than prefix-based longest-prefix match.

The implicit deny-all entry appended at the end of each TCAM table reflects the standard ACL semantics: any packet that does not match an explicit ACE is denied. This convention is universal across Cisco IOS, Junos, and EOS, though some platforms (notably Junos) require an explicit `then discard` term rather than relying on an implicit deny. The functor normalises this variation by always appending the deny-all entry, ensuring consistent behaviour regardless of the source platform's convention.

Insight:
The ACL functor is the bridge between policy-intent attributes (what the operator writes) and hardware forwarding tables (what the ASIC executes). The compilation step makes explicit the transformation that vendors perform implicitly in their CLI parsers.

19.2.4 TCAM Resource Modelling

A secondary benefit of explicit TCAM compilation is resource-consumption analysis. Each TCAM entry consumes one slot in the hardware TCAM bank; modern ASICs provide between 2 K and 64 K entries per bank, partitioned across multiple lookup stages. By summing the entries across all interfaces on a device, the functor's output enables a planning tool to estimate TCAM utilisation and flag devices approaching capacity limits. This is a non-trivial operational concern: TCAM exhaustion causes the switch to fall back to software forwarding for excess entries, reducing throughput by orders of magnitude.

TCAM Overflow Detection

Consider a data-centre leaf switch with 48 downlink ports, each with an ingress ACL of 50 entries and an egress ACL of 30 entries. The total TCAM consumption is $48 \times (50 + 30) = 3{,}840$ entries. If the switch's TCAM bank provides 4 096 entries, only 256 entries remain for system-reserved rules (ARP, DHCP snooping, control-plane policing). The functor's compiled output makes this arithmetic trivially computable, enabling pre-deployment validation that is impossible when ACLs exist only as CLI text.

19.2.5 DPO Rewriting Rule

Design Rule 13: Compile ACL to TCAM

The ACL compilation rule $p_{\text{ACL}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): An endpoint e on a node n where `acl_in`(e) or `acl_out`(e) references a named ACL, and the named ACL has one or more sequence entries (`acl_<name>_seq_<n>_action` exists in n's attributes).
- K (preserve): The endpoint e, its owner node n, and all ACL-defining attributes (the named ACL's sequence entries remain available for other interfaces that may reference the same ACL).
- R (produce): The preserved elements plus a new *TCAM binding* edge from n to e labeled with:
  - `direction`: ingress or egress,
  - `tcam_entries`: ordered list of compiled ternary match–action tuples,
  - `entry_count`: total TCAM entries consumed (for resource accounting).

A single named ACL may be referenced by multiple interfaces (a common pattern is a "deny RFC 1918" ACL applied to all WAN-facing interfaces). The DPO rule handles this correctly because the ACL-defining attributes reside on the node (in K) and are preserved across rule applications. Each interface receives its own TCAM binding edge, but the source ACL entries are shared. This mirrors hardware reality: each interface gets its own TCAM table instance even when the logical ACL definition is shared.

19.2.6 Flowchart

19.3 $\mathcal{F}_{\text{Security}}$: Group Policy Compilation

The security functor reads `sgt_*` and `sgacl_*` attributes to construct the security overlay graph G_{sec}, whose structure was defined in Section D.7. While the ACL functor operates on per-interface, address-based policy, the security functor operates on *identity-based* policy: traffic is classified not by source and destination IP address but by source and destination security group. This shift from address-centric to identity-centric access control is the defining characteristic of the SGT framework and the reason it requires a dedicated functor rather than being subsumed by $\mathcal{F}_{\text{ACL}}$.

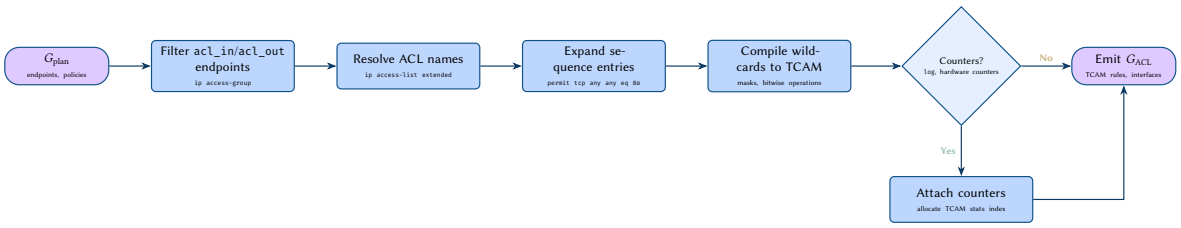


Figure 64. $\mathcal{F}_{\text{ACL}}$ flowchart: from plan attributes to TCAM-compiled ACL overlay graph.

The functor proceeds in four phases: SGT assignment resolution, SXP-mediated binding propagation, SGACL policy matrix construction, and per-enforcement-point policy compilation.

19.3.1 Type Signature

$$\mathcal{F}_{\text{Security}} : G_{\text{plan}} \longrightarrow (\text{SGT\_assignments},\ \text{SXP\_bindings},\ \text{SGACL\_policies})$$

The *SGT assignments* map IP addresses (or prefixes) to SGT values. The *SXP bindings* record the IP-to-SGT mappings propagated between SXP speakers and listeners. The *SGACL policies* form a matrix indexed by (source SGT, destination SGT) pairs, where each cell contains a permit/deny action with optional protocol and port constraints.

19.3.2 Input Attributes

The functor consumes three attribute families, all defined in Section D:

Design Decision 111: Security Functor Input Attributes

Attribute	Type	Semantics
SGT Assignment		
`sgt_value`	u16	SGT assigned to this endpoint
`sgt_assignment_method`	enum	Assignment method: static, dynamic, vlan
`sgt_propagation`	enum	Propagation mode: inline, sxp
SXP Topology		
`sxp_enabled`	bool	SXP protocol active on this node
`sxp_role`	enum	speaker, listener, both
`sxp_peer_address`	ip	SXP peer remote address
`sxp_source_ip`	ip	Source IP for SXP connections
`sxp_password`	str	SXP connection authentication
SGACL Policy		
`sgacl_name`	str	SGACL policy name
`sgacl_src_sgt`	u16	Source security group tag
`sgacl_dst_sgt`	u16	Destination security group tag
`sgacl_action`	enum	permit, deny, permit with logging
`sgacl_protocol`	enum	Protocol constraint (tcp, udp, ip, any)
`sgacl_port`	u16	Port constraint (if protocol is tcp/udp)

19.3.3 Compilation Procedure

The algorithm proceeds in four phases: (1) resolve IP-to-SGT assignments from `sgt_value` endpoint attributes, (2) propagate bindings across SXP speaker–listener topology with static-over-dynamic conflict resolution, (3) build the SGACL policy matrix indexed by (source SGT, destination SGT) pairs from `sgacl_*` attributes, and (4) compile per-enforcement-point policy subsets based on locally relevant destination SGTs. Full pseudocode appears in Appendix C.10.

Phase 1 builds the global IP-to-SGT mapping table from locally configured assignments. Each endpoint carrying an `sgt_value` attribute contributes one entry to this table. The assignment method (`sgt_assignment_method`) records provenance but does not affect the mapping itself: a static assignment and a dynamic assignment produce identical table entries, differing only in how the assignment was derived.

Phase 2 propagates these bindings across the network via the SGT Exchange Protocol (SXP). SXP is a control-plane protocol that carries IP-to-SGT bindings between devices that lack data-plane SGT capability (inline tagging) and devices that enforce SGT policy. The functor models SXP topology by resolving `sxp_peer_address` references between SXP speakers and listeners, then copying the speaker's local IP-to-SGT table to each connected listener. In networks where SXP forms a multi-hop chain (speaker → both → listener), the propagation

must be iterated until the binding tables converge—a fixed-point computation analogous to route convergence in $\mathcal{F}_{\text{OSPF}}$.

Phase 3 assembles the SGACL policy matrix from `sgacl_*` attributes. Each SGACL rule specifies a (source SGT, destination SGT) pair and an action to apply when traffic from the source group attempts to reach the destination group. The matrix is conceptually a two-dimensional table where rows are source SGTs, columns are destination SGTs, and each cell contains the enforcement action. In practice, the matrix is sparse: most SGT pairs are either implicitly denied (no entry) or implicitly permitted (default policy), with explicit entries only for inter-group flows that require specific treatment.

Phase 4 compiles the policy matrix into per-node enforcement tables. An enforcement point downloads only the subset of the policy matrix relevant to the SGTs present in its local domain. This is a critical optimisation: a campus access switch serving endpoints in SGTs 3 and 5 need not store policy entries for SGT pairs involving SGTs 10 through 100, which may reside entirely in the data-centre fabric.

Insight:
SXP propagation introduces the only inter-device dependency in the security functor. Without SXP, each enforcement point could resolve its policy independently from local SGT assignments alone. SXP creates a control-plane distribution graph for identity bindings, making the security functor's convergence behaviour structurally similar to that of a routing-protocol functor.

SGT Policy Compilation

Consider a campus network with three security groups: Employees (SGT 3), Contractors (SGT 5), and Servers (SGT 10). The SGACL policy matrix specifies:

- (3, 10) → permit tcp 443: Employees may access servers via HTTPS.
- (5, 10) → permit tcp 443, deny tcp 22: Contractors may access servers via HTTPS but not SSH.
- (5, 3) → deny ip: Contractors may not access Employee endpoints.

An access switch serving both Employees and Contractors downloads all three entries. A data-centre leaf switch serving only Servers downloads entries where `dst_sgt` = 10. The functor's per-node compilation ensures that each switch receives precisely the policy entries it needs to enforce.

19.3.4 DPO Rewriting Rule

Design Rule 14: Deploy SGT-Based Access Policy

The security policy deployment rule $p_{\text{Security}} : L \xleftarrow{l} K \xrightarrow{r} R$:

- L (match): Two endpoints e_s, e_d with `sgt_value`(e_s) and `sgt_value`(e_d) set, and a node n carrying attributes `sgacl_src_sgt` and `sgacl_dst_sgt` matching e_s and e_d respectively.
- K (preserve): Both endpoints e_s, e_d and node n, with all SGT and SGACL attributes retained.
- R (produce): The preserved elements plus a new *security policy* edge between e_s and e_d (via n) labeled with:
  - `action`: the enforcement action (permit/deny),
  - `protocol`: protocol constraint (if any),
  - `port`: port constraint (if any),
  - `enforcement_point`: the node n that enforces the policy.

The rule introduces a three-party match: source endpoint, destination endpoint, and enforcement node. This is more complex than the two-party matches in $\mathcal{F}_{\text{BGP}}$ (two peer endpoints) or the single-party match in $\mathcal{F}_{\text{QoS}}$ (one interface endpoint). The three-party structure reflects the architectural reality of SGT enforcement: the policy decision depends on the source identity (who is sending), the destination identity (who is receiving), and the enforcement point (where the policy is applied). In general, the enforcement point is the egress switch closest to the destination, but ingress enforcement is also possible when the source switch has the destination's SGT in its local binding table.

19.3.5 Flowchart

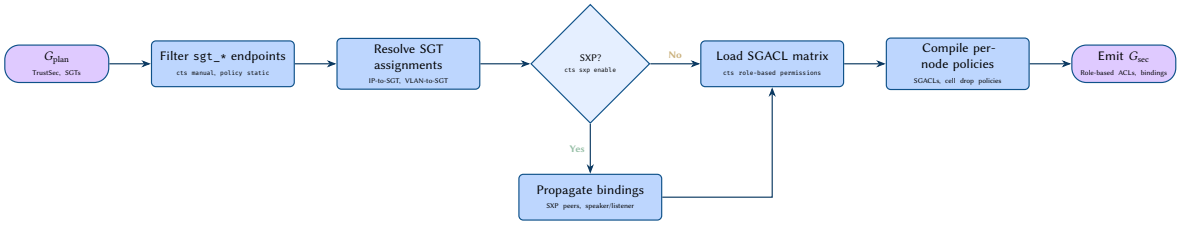


Figure 65. $\mathcal{F}_{\text{Security}}$ flowchart: from plan attributes to security group overlay graph.

19.3.6 Relationship to $\mathcal{F}_{\text{ACL}}$

The security functor and the ACL functor are complementary, not competing, access-control mechanisms. ACLs operate on Layer 3/4 header fields (IP addresses, ports, protocol numbers) and are bound to specific

interfaces. SGACLs operate on security group identifiers and are bound to enforcement points independently of interface topology. In networks that deploy both mechanisms, a packet may be subject to an interface ACL (processed by $\mathcal{F}_{\text{ACL}}$) *and* an SGACL (processed by $\mathcal{F}_{\text{Security}}$), with the most restrictive action prevailing.

The functor framework captures this interaction cleanly: both functors produce independent overlay graphs (G_{ACL} and G_{sec}), and a downstream composition step can merge them to compute the effective access policy at each hop. This compositionality is a direct benefit of modelling each policy dimension as a separate functor rather than attempting to capture all access-control mechanisms in a single monolithic transformation.

20 End-to-End Packet Trace

To demonstrate how the functor framework operates as an integrated system, we trace a single HTTPS flow through every processing stage. At each step we identify precisely which functor's output is consulted, showing that the forwarding decision is a *composition* of independent, layer-specific computations.

20.1 Scenario

Host A, attached to `leaf-01` port `eth1` (access port, VLAN 100, VRF PROD, IP address 10.100.1.10), sends an HTTPS request (TCP destination port 443) to Host B, attached to `leaf-03` port `eth5` (access port, VLAN 200, VRF PROD, IP address 10.200.1.20). The fabric uses VXLAN/EVPN as the overlay, eBGP as the underlay routing protocol, and SR-MPLS for transport optimisation across a leaf-spine topology with two spine switches.

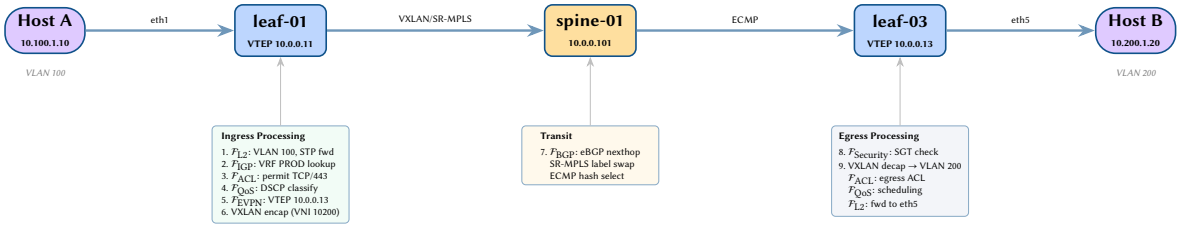


Figure 66. End-to-end packet trace overview. Host A's HTTPS flow traverses leaf-01 (ingress processing, steps 1–6), the spine tier (underlay transit, step 7), and leaf-03 (egress processing, steps 8–9) before reaching Host B. Each annotation indicates which functor output is consulted.

20.2 Processing Steps

20.2.1 Step 1: L2 Ingress ($\mathcal{F}_{\text{L2}}$)

Host A transmits an Ethernet frame with its source MAC address to `leaf-01` port `eth1`. The L2 functor output (Section 18.6) is consulted:

- **Port state:** $\mathcal{F}_{\text{STP}}$ confirms that `eth1` is in the *forwarding* state (Section 18.2). If the port were in *blocking* or *discarding* state, the frame would be dropped at this stage.
- **VLAN membership:** The VLAN membership hyperedge (Section 18.6) confirms that `eth1` participates in VLAN 100. The `switchport_mode` attribute is `access` and `switchport_vlan` is 100.
- **MAC learning:** Host A's source MAC is recorded in the local MAC table associated with VLAN 100.

Functor outputs read: $\mathcal{F}_{\text{L2}}$ (VLAN membership, switchport mode), $\mathcal{F}_{\text{STP}}$ (port forwarding state).

20.2.2 Step 2: L3/VRF Lookup ($\mathcal{F}_{\text{IGP}}$)

Since the destination address 10.200.1.20 resides in a different subnet from 10.100.1.0/24, `leaf-01` performs an L3 lookup within VRF PROD. The IGP functor (Section 17) provides the VRF routing table:

- ARP/ND resolution for the destination is unnecessary at this stage because the destination is remote. The VRF routing table is consulted for the longest-prefix match.
- An EVPN Type-5 route (IP prefix route) or a Type-2 route carrying both MAC and IP bindings provides the next-hop VTEP address.
- VRF membership is confirmed by the `vrf_name` attribute on `leaf-01`; the `vrf_rd` and `vrf_rt_import`/`vrf_rt_export` attributes (Section 3.2) govern route import and export.

20.2.3 Step 3: ACL Evaluation ($\mathcal{F}_{\text{ACL}}$)

Before forwarding, the ingress ACL bound to `eth1` is evaluated. The ACL functor's compiled TCAM entries (Section 19.2) are consulted for the 5-tuple:

Field	Value
Source IP	10.100.1.10
Destination IP	10.200.1.20
Protocol	TCP (6)
Source Port	*ephemeral*
Destination Port	443

A matching entry is found with action `permit`. The `acl_name`, `acl_direction` (ingress), and `acl_entries` attributes from G_{plan} were consumed by $\mathcal{F}_{\text{ACL}}$ to produce these compiled entries.

20.2.4 Step 4: QoS Classification ($\mathcal{F}_{\text{QoS}}$)

The QoS functor (Section 19.1) determines how the packet is queued and potentially remarked:

- The `qos_trust_mode` attribute on `eth1` is set to `dscp`, so the DSCP field in the IP header is trusted as the classification input.
- The packet's DSCP value is mapped to an internal traffic class via the ingress classification map.
- If the `qos_default_dscp` policy requires remarking, the DSCP field is rewritten before encapsulation.
- The packet is placed into the appropriate egress queue based on the scheduler and shaper hierarchy output by $\mathcal{F}_{\text{QoS}}$.

20.2.5 Step 5: EVPN Route Lookup ($\mathcal{F}_{\text{EVPN}}$)

The EVPN functor (Section 18.5) is consulted for the forwarding information required to reach Host B:

- A Type-2 MAC/IP advertisement route for Host B's MAC address is queried in the EVPN RIB.
- The route provides:
  - Destination VTEP address: `leaf-03`'s VTEP IP, 10.0.0.13.
  - VNI: 10200, corresponding to VLAN 200.
  - The `evpn_rt_import` attribute confirms the route target matches VRF PROD.
- The `evpn_evi`, `evpn_rt_export`, and `vxlan_vni` attributes from G_{plan} parameterise this lookup.

20.2.6 Step 6: VXLAN Encapsulation

The original Ethernet frame is encapsulated in a VXLAN header:

Design Decision 112: VXLAN Encapsulation Headers

Header Field	Value
Inner Ethernet	Original frame (Host A MAC → Host B MAC)
VXLAN VNI	10200 (mapped from VLAN 200)
Outer UDP dst	4789
Outer IP src	10.0.0.11 (`leaf-01` VTEP address)
Outer IP dst	10.0.0.13 (`leaf-03` VTEP address)

The VTEP address and VNI are derived from the EVPN functor output. The `vtep_source_interface` attribute on `leaf-01` identifies the loopback used as the outer source IP.

20.2.7 Step 7: Underlay Routing ($\mathcal{F}_{\text{IGP}}$ / $\mathcal{F}_{\text{BGP}}$)

The encapsulated packet must now traverse the underlay fabric to reach 10.0.0.13. Two functor outputs are consulted:

- $\mathcal{F}_{\text{BGP}}$ (Section 18.1): In this eBGP underlay fabric, the route to 10.0.0.13 was learned via eBGP sessions with the spine switches. The `bgp_asn`, `bgp_peer_address`, and `bgp_peer_afi_safi` attributes parameterise these sessions.
- **ECMP:** Multiple equal-cost paths exist across `spine-01` and `spine-02`. The underlay routing table installs both next-hops, and the forwarding plane selects one based on a hash of the flow 5-tuple.
- **SR-MPLS:** If segment routing is enabled (Section 18.4), an MPLS label stack is pushed. The `sr_node_sid` attribute on `leaf-03` provides the destination node SID, and the label stack encodes the path through the spine tier.

20.2.8 Step 8: Security Group Tag Check ($\mathcal{F}_{\text{Security}}$)

If SGT enforcement is enabled on the fabric (Section 19.3):

- The `sgt_value` attribute on Host A's port assigns a source SGT to the packet.

- At the egress enforcement point (`leaf-03`), the SGACL policy matrix is consulted for the (src_SGT, dst_SGT) pair.
- $\mathcal{F}_{\text{Security}}$ compiles the per-device policy matrix entries, ensuring that only the relevant (src_SGT, dst_SGT) pairs are installed on each enforcement switch.

20.2.9 Step 9: Egress Processing at `leaf-03`

When the encapsulated packet arrives at `leaf-03`, the following operations are performed, each reading from the appropriate functor output:

1. **VXLAN decapsulation:** The outer headers are stripped. The VNI 10200 is mapped to VLAN 200 using the `vxlan_vni` → `switchport_vlan` mapping from $\mathcal{F}_{\text{EVPN}}$.
2. **Egress ACL:** $\mathcal{F}_{\text{ACL}}$ is consulted for any egress ACL bound to Host B's port `eth5`.
3. **QoS scheduling:** $\mathcal{F}_{\text{QoS}}$ output determines the output queue and scheduling priority for `eth5`.
4. **L2 forwarding:** $\mathcal{F}_{\text{L2}}$ confirms that `eth5` is an access port in VLAN 200 and in the forwarding state.
5. **Frame delivery:** The inner Ethernet frame is transmitted to Host B on `eth5`.

20.3 Functor Composition as Forwarding

The packet trace makes three properties of the framework concrete:

1. **Attribute locality.** Each functor reads only its own prefix-scoped attributes from G_{plan}. $\mathcal{F}_{\text{ACL}}$ never examines `bgp_asn`; $\mathcal{F}_{\text{BGP}}$ never examines `acl_entries`.
2. **Evaluation independence.** The nine processing steps could, in principle, be evaluated in any order (or in parallel). The forwarding plane imposes a sequential *data-plane* pipeline, but the *control-plane* functor computations are independent.
3. **Incremental recomputation.** If Host B migrates to `leaf-04`, only $\mathcal{F}_{\text{EVPN}}$ and $\mathcal{F}_{\text{L2}}$ need recomputation. The ACL, QoS, BGP underlay, and security functor outputs remain valid.

Insight: *Each processing step reads from a different functor's output, yet no functor is aware of the others. The end-to-end forwarding decision emerges from the composition of independent functor evaluations:* $\mathcal{F}_{\text{L2}} \circ \mathcal{F}_{\text{STP}} \circ \mathcal{F}_{\text{IGP}} \circ \mathcal{F}_{\text{ACL}} \circ \mathcal{F}_{\text{QoS}} \circ \mathcal{F}_{\text{EVPN}} \circ \mathcal{F}_{\text{BGP}} \circ \mathcal{F}_{\text{Security}}$. *This is the central design property of the functor framework: layer independence enables parallel evaluation, incremental recomputation, and clean separation of concerns.*

20.4 Attribute Reads Per Step

Attribute Reads Per Processing Step

The following summarises every G_{plan} attribute read during the nine-step packet trace:

Step	Functor	Attributes Read
1	$\mathcal{F}_{\text{L2}}$, $\mathcal{F}_{\text{STP}}$	`switchport_mode`, `switchport_vlan`, `stp_port_state`, `stp_bridge_priority`
2	$\mathcal{F}_{\text{IGP}}$	`vrf_name`, `vrf_rd`, `vrf_rt_import`, `vrf_rt_export`
3	$\mathcal{F}_{\text{ACL}}$	`acl_name`, `acl_direction`, `acl_entries`
4	$\mathcal{F}_{\text{QoS}}$	`qos_trust_mode`, `qos_default_dscp`, `qos_policy_map`
5	$\mathcal{F}_{\text{EVPN}}$	`evpn_evi`, `evpn_rt_import`, `evpn_rt_export`, `vxlan_vni`, `vtep_source_interface`
6	(encap)	`vtep_source_interface`, `vxlan_vni`
7	$\mathcal{F}_{\text{BGP}}$, $\mathcal{F}_{\text{MPLS}}$	`bgp_asn`, `bgp_peer_address`, `sr_node_sid`, `sr_global_block`
8	$\mathcal{F}_{\text{Security}}$	`sgt_value`, `sgt_enforcement`, `sgacl_policy`
9	$\mathcal{F}_{\text{L2}}$, $\mathcal{F}_{\text{ACL}}$, $\mathcal{F}_{\text{QoS}}$	`switchport_mode`, `switchport_vlan`, `acl_name`, `qos_policy_map`

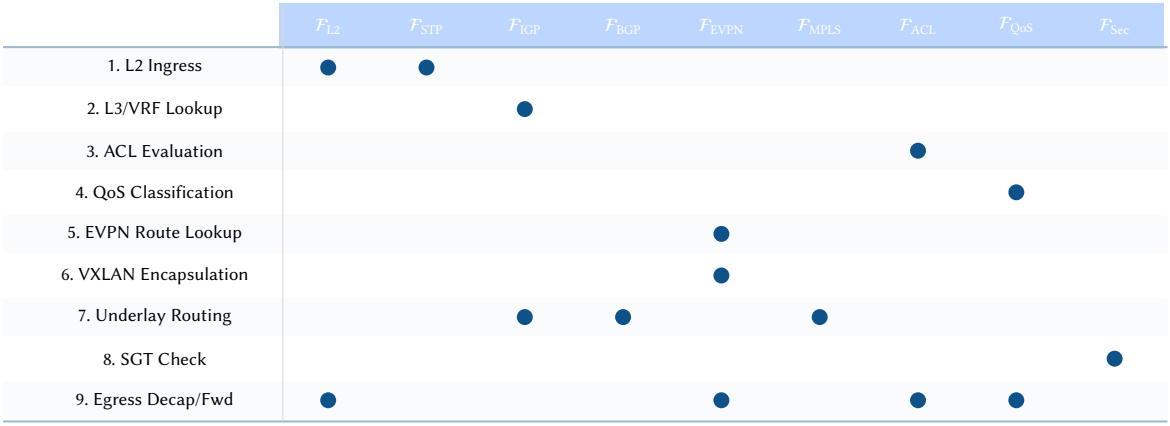


Figure 67. Functor read matrix for the packet trace. Each row represents a processing step; each column represents a layer functor. Filled circles indicate that the step consults the corresponding functor's output. Steps 7 and 9 read from the most functors, reflecting the complexity of underlay routing and egress processing respectively.

21 Advanced Network Graph Modeling Concepts

The abstract formalisms of multilayer network theory and algebraic graph rewriting provide strict foundational guarantees, but they must be proven against the rigorous demands of real-world network operations. This section demonstrates how the model resolves three notoriously difficult problems in carrier-grade networking: multipath fate-sharing, routing loop detection, and stateful directionality.

21.1 Multilayer Path Algebra and Blast-Radius Computation

Traditional logical network diagrams mask the underlying physical reality. A single logical connection at the BGP layer may traverse a multi-hop Layer 2 switched fabric, thereby sharing physical fate with numerous other seemingly unrelated BGP sessions. Furthermore, the presence of Link Aggregation Groups (LAG) at Layer 2 and Equal-Cost Multi-Path (ECMP) routing at Layer 3 means a single physical failure does not deterministically equal a logical service failure.

To compute the exact hardware blast-radius of any logical link, the NTE utilizes the supra-adjacency tensor $\mathcal{A}$. By defining adjacency using a Boolean semiring—where the addition operation ($\oplus$) represents logical OR (multipath resilience) and the multiplication operation ($\otimes$) represents logical AND (path dependency)—we reduce fate-sharing analysis to linear algebra.

By multiplying the Layer 3 Adjacency Matrix by the Layer 2 Adjacency Matrix and the Layer 1 Physical Matrix, the NTE statically and instantly computes cross-layer dependencies. As illustrated in Figure 68, a physical failure (L1) cascades vertically. Because the semiring addition evaluates the existence of alternate ECMP paths, the system algebraically proves that the top-level L5 BGP session survives the L1 fiber cut.

21.2 Algebraic Detection of Routing Loops

Mutual route redistribution—for example, injecting OSPF routes into BGP and simultaneously injecting BGP routes back into OSPF at a different boundary—is a primary cause of catastrophic network loops. Standard tree-based configuration models (e.g., YANG) validate local node syntax but are structurally blind to global topological routing loops.

In our model, the generation of a routing overlay is formalized as a functor $\mathcal{F}$. Consequently, route redistribution is modeled as the composition of functors across different routing domains.

When translating The Structural Baseline Plan into the algebraic graph, the NTE evaluates the composition of redistribution functors (e.g., $\mathcal{F}_{\text{OSPF}\to\text{BGP}} \circ \mathcal{F}_{\text{BGP}\to\text{OSPF}}$). If this composition yields a cyclic dependency in the underlying prefix-origin directed acyclic graph (DAG), it represents a mathematical fixed point that manifests as a routing loop. As shown in Figure 69, the compiler detects this cycle statically and rejects the plan, enforcing correctness-by-construction before any device configuration is generated.

21.3 Worked Example: The Boolean Semiring Calculation

To demystify the abstract algebra of the supra-adjacency tensor, consider a minimal, numerical worked example using the Boolean semiring for blast-radius calculation.

Scenario: We have three nodes: Router A, Switch B, and Router C.

- **Layer 1 (Physical):** A is connected to B via fiber 1. B is connected to C via fiber 2. A is *not* directly connected to C physically.
- **Layer 2 (Data Link):** A, B, and C are in the same VLAN. Thus, A can reach C logically at L2 (via B).

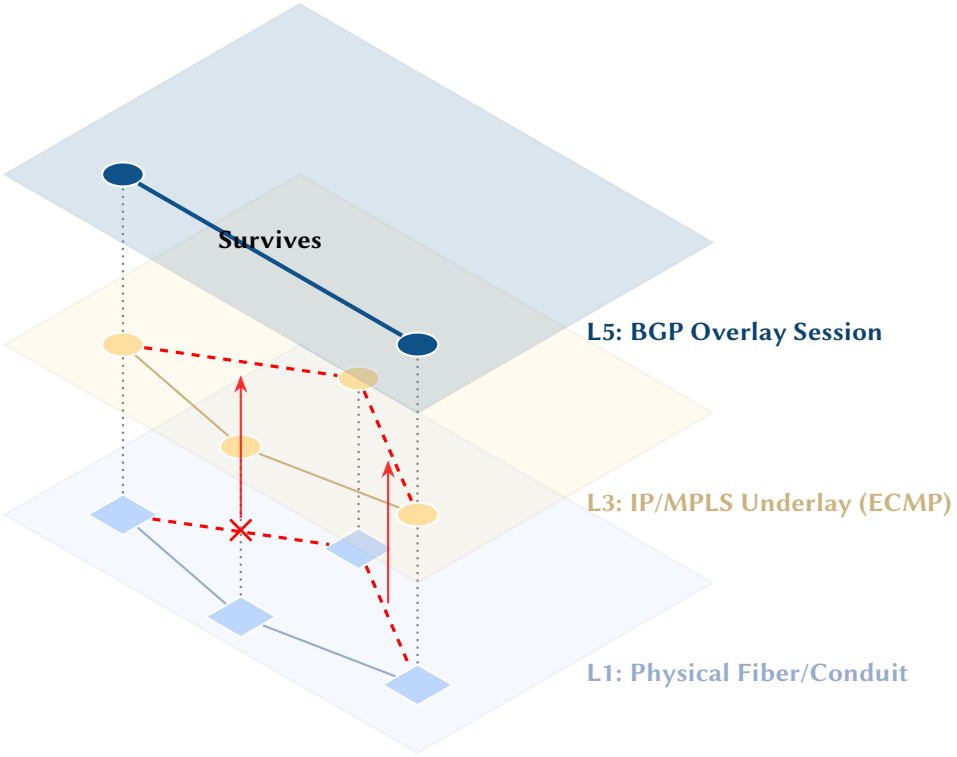


Figure 68. The Fate-Sharing Tensor: A physical fiber cut at L1 cascades vertically via the supra-adjacency tensor, destroying the upper ECMP path at L3. However, because the adjacency matrix semiring calculation evaluates multiple paths, the single logical BGP overlay session at L5 survives.

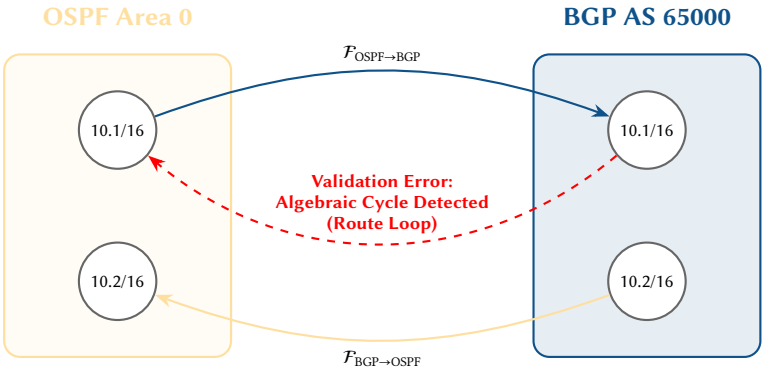


Figure 69. Algebraic Loop Detection: By treating route redistribution as the composition of functors, the NTE compiler algebraically analyzes the resulting prefix graph. A fixed point or cycle in the dependency graph triggers a compile-time rejection of the plan, preventing a real-world routing loop.

- **Layer 3 (Network):** A has an OSPF adjacency with C.

In standard linear algebra, matrix multiplication involves real number addition and multiplication. In our Boolean semiring, addition ($\oplus$) is the logical OR operator ($\vee$), and multiplication ($\otimes$) is the logical AND operator ($\wedge$).

Let $\mathbf{P}$ be the L1 Physical Adjacency Matrix:

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

To find if a physical path of length 2 exists between any nodes, we compute $\mathbf{P}^2$ using the semiring:

$$\mathbf{P}^2_{A,C} = (\mathbf{P}_{A,A} \wedge \mathbf{P}_{A,C}) \vee (\mathbf{P}_{A,B} \wedge \mathbf{P}_{B,C}) \vee (\mathbf{P}_{A,C} \wedge \mathbf{P}_{C,C})$$

$$\mathbf{P}^2_{A,C} = (0 \wedge 0) \vee (1 \wedge 1) \vee (0 \wedge 0) = 0 \vee 1 \vee 0 = 1$$

The result 1 mathematically proves that A and C share physical fate via a 2-hop sequence.

When a logical overlay matrix $\mathbf{L}_3$ (where A is directly adjacent to C) is multiplied by the transitive closure of the physical matrix $\mathbf{P}^*$, the NTE strictly guarantees that the OSPF session $\mathbf{L}_{3(A,C)}$ is dependent on the specific physical fibers $\mathbf{P}_{(A,B)}$ and $\mathbf{P}_{(B,C)}$. If $\mathbf{P}_{(B,C)}$ becomes 0 (fiber cut), the semiring calculation propagates the 0 upward, instantly recalculating the L3 adjacency to 0 unless an alternate path evaluates to 1 via the $\vee$ operation.

Alignment with Existing Standards

22 Relationship to RFC 8345

RFC 8345 [5] defines a YANG data model for network topologies with the same structural primitives as our model: networks, nodes, links, and termination points.

Insight: *Our plan topology is a property-graph realization of the RFC 8345 schema, with protocol state as flat attributes instead of YANG augmentations.*

Table 4. Mapping between RFC 8345 and the Plan Topology Model.

RFC 8345 Concept	Plan Topology	Notes
`network`	Layer graph G_ℓ	One network per protocol layer
`node`	Node $v \in V$	Device
`termination-point`	Endpoint $e \in \mathcal{E}$	Interface / port
`link`	E_{conn}	Connectivity edge
`supporting-network`	Inter-layer coupling $C^{\alpha\beta}$	Dependency
`supporting-node`	Identity coupling	Same device across layers
`supporting-tp`	Endpoint hierarchy	Same interface across layers

The key difference: RFC 8345 uses YANG augmentations (RFC 8346 for L3, RFC 8944 for L2, RFC 8795 for TE) that introduce nested containers. Our model flattens all state into the attribute map, enabling SIMD-friendly columnar queries via the NTE DataFrameStore.

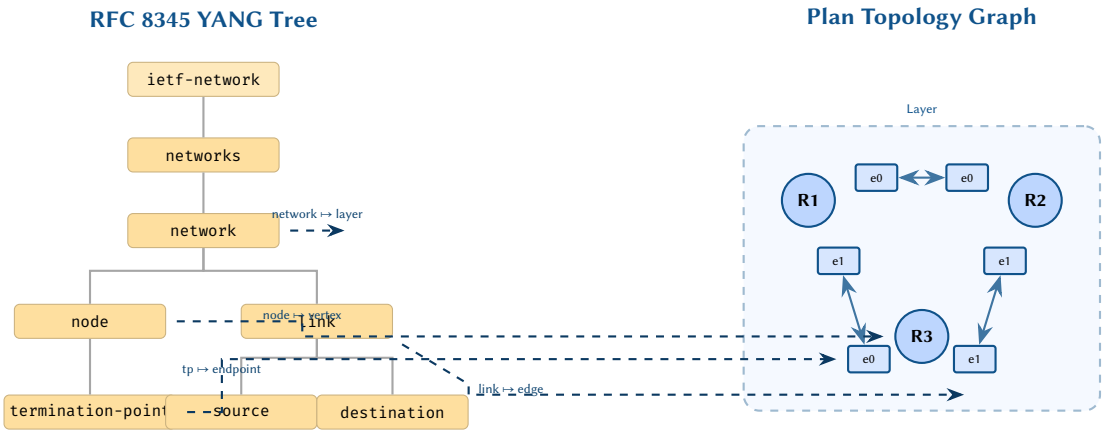


Figure 70. Structural mapping between RFC 8345 YANG data model (left) and Plan topology graph representation (right). Each YANG container maps to a graph primitive: `network` to a layer, `node` to a vertex, `termination-point` to an endpoint, and `link` to an edge connecting source and destination endpoints.

22.1 RFC 8346 (L3 Topology) Alignment

RFC 8346 [6] augments the base RFC 8345 topology model with L3-specific attributes. It does so via the `l3-unicast-topology` augmentation module. This augmentation introduces three attribute containers that map directly to our flat attribute space.

Table 5. RFC 8346 L3 topology augmentation mapping.

RFC 8346 Container	Plan Topology Attribute	Notes
`l3-node-attributes/router-id`	`ospf_router_id`	OSPF router identifier
`l3-node-attributes/router-id`	`bgp_router_id`	BGP router identifier
`l3-node-attributes/flags`	`igp_flags`	ABR, ASBR, stub flags
`l3-link-attributes/metric`	`ospf_cost`	OSPF interface cost
`l3-link-attributes/metric`	`isis_metric`	IS-IS interface metric
`l3-tp-attributes/ip-address`	`ipv4_address`	IPv4 interface address
`l3-tp-attributes/ip-address`	`ipv6_address`	IPv6 interface address
`l3-tp-attributes/ip-prefix-length`	`ipv4_mask`	Prefix length

The fundamental structural difference is that RFC 8346 nests these attributes inside YANG containers with list keys. For instance, `l3-node-attributes` is a container within a `node` list entry, itself nested inside a `network` list entry. Accessing a router ID requires traversing three levels of nesting in the YANG tree. Our model collapses this into a single flat lookup: $\pi(v, \texttt{ospf\_router\_id})$.

22.2 RFC 8944 (L2 Topology) Alignment

Insight: *RFC 8346's nested containers enforce a strict tree structure that prevents cross-referencing between L3 attributes and other protocol state without YANG augmentation. Our flat attribute map allows arbitrary cross-layer queries with no schema changes.*

RFC 8944 [7] provides the Layer 2 counterpart, augmenting RFC 8345 with Ethernet and bridging topology attributes. The mapping to our model's L2 layer attributes is given in Table 6.

Table 6. RFC 8944 L2 topology augmentation mapping.

RFC 8944 Container	Plan Topology Attribute	Notes
`l2-node-attributes/mgmt-address`	`mgmt_ipv4_address`	Out-of-band management
`l2-node-attributes/bridge-id`	`stp_bridge_id`	STP bridge identifier
`l2-node-attributes/flags`	`l2_flags`	Bridge type flags
`l2-link-attributes/rate`	`phy_speed`	Link speed (bps)
`l2-link-attributes/lag`	`lag_mode`	LAG membership
`l2-tp-attributes/mac-address`	`mac_address`	Interface MAC
`l2-tp-attributes/vlan-id-name`	`switchport_vlan`	Access/trunk VLAN
`l2-tp-attributes/encapsulation`	`encap_type`	802.1Q, QinQ

RFC 8944 also defines an `l2-network-attributes` container that carries network-wide properties such as the network name and flags. In our model these correspond to layer-level metadata stored as properties of the layer graph G_ℓ itself rather than as per-node or per-endpoint attributes.

A notable gap in RFC 8944 is the absence of EVPN-specific attributes; the L2 topology model predates widespread EVPN adoption and does not include route-target, route-distinguisher, or ESI attributes. Our model fills this gap with the `evpn_*` attribute prefix defined in the L2.5 label-switching layer.

22.3 RFC 8795 (TE Topology) Alignment

RFC 8795 [8] defines the most structurally complex augmentation in the RFC 8345 family, adding traffic engineering attributes for MPLS, RSVP-TE, and Segment Routing topologies. The TE topology model introduces several concepts that go beyond simple attribute containers: *connectivity matrices*, *path constraints*, *label restrictions*, and *bandwidth pools*.

Table 7. RFC 8795 TE topology augmentation mapping.

RFC 8795 Concept	Plan Topology Attribute	Notes
`te-link/admin-status`	`mpls_admin_state`	TE link state
`te-link/max-bandwidth`	`mpls_max_bw`	Maximum reservable BW
`te-link/te-metric`	`mpls_te_metric`	TE-specific metric
`te-link/srlg-values`	`srlg_groups`	Shared risk link groups
`te-node/sr-node-cap`	`sr_srgb_base`	SR Global Block base
`te-node/sr-node-cap`	`sr_srgb_range`	SR Global Block range
`te-tp/sr-adj-sid`	`sr_adj_sid`	Adjacency SID
`te-link/rsvp-te`	`rsvp_*`	RSVP-TE attributes

The most significant structural divergence lies in RFC 8795's *connectivity matrices*. These matrices encode which ingress termination points can reach which egress termination points through a TE node, forming an internal switching fabric model. Our model does not maintain per-node connectivity matrices; instead, we encode switching capability implicitly through the endpoint-to-endpoint adjacencies within a node.

Design Rule 15: TE Topology Flattening

RFC 8795 structures such as connectivity matrices, label restrictions, and bandwidth pools are *denormalized* into per-endpoint attributes in the plan topology. A TE link with n SRLG values produces n entries in the `srlg_groups` list attribute. Path constraints are expressed as attributes on the tunnel overlay layer rather than as nested container hierarchies.

23 Relationship to OpenConfig and YANG

OpenConfig [2] YANG models use a consistent `/config` and `/state` container pattern. Following the Network Management Datastore Architecture (NMDA) [115] architecture, our model supports the distinction:

Design Rule 16: NMDA-Aligned State Separation

Each attribute k in the plan topology implicitly has two values:

- $\pi_{\text{intended}}(k)$: The configured/intended value (from the plan).
- $\pi_{\text{operational}}(k)$: The observed/runtime value (from telemetry).

The plan topology stores intended state. Operational state is populated by the NTE event stream from live devices. Discrepancies between intended and operational state indicate configuration drift.

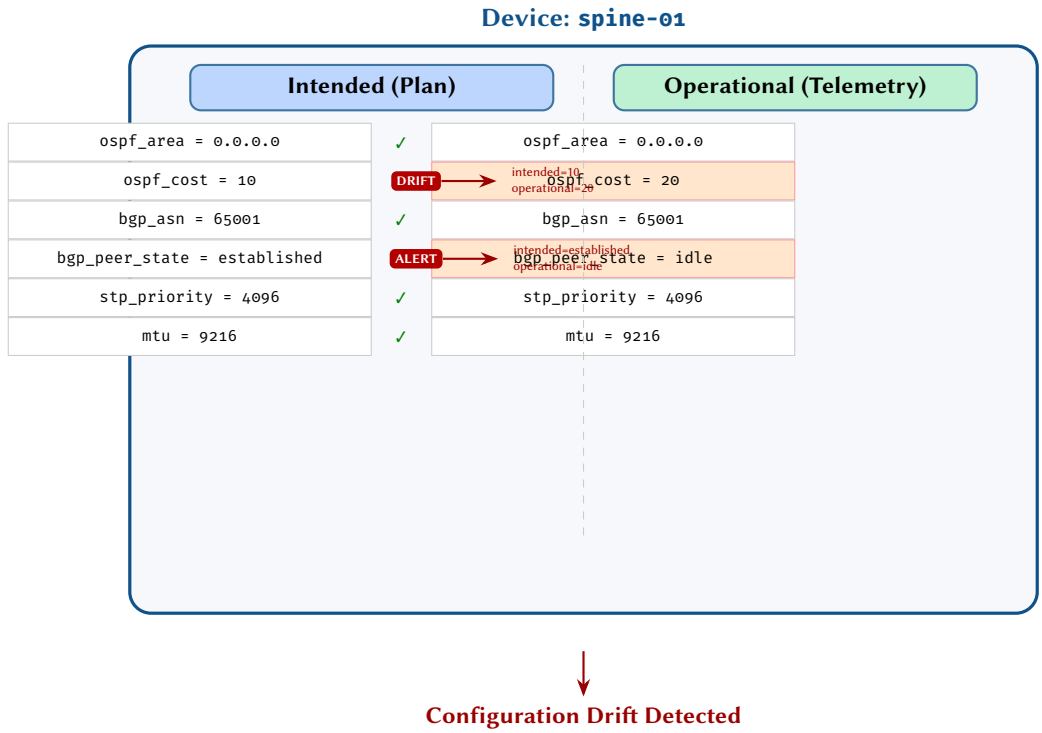


Figure 71. NMDA-style state separation for a single device, comparing intended configuration (from the plan) with operational state (from telemetry). Matching attributes are accepted; mismatches in `ospf_cost` and `bgp_peer_state` are flagged as configuration drift, enabling automated remediation or alerting.

23.1 OpenConfig Path Mapping

OpenConfig YANG models organize network state into deeply nested path hierarchies. A typical OpenConfig path traverses five or more levels of containers and lists before reaching a leaf value. Our model flattens these paths into single-key attribute lookups.

Design Decision 113: OpenConfig Path to Flat Attribute Mapping

OpenConfig YANG Path	Flat Attribute
`/interfaces/interface/config/name`	endpoint identity
`/interfaces/interface/config/description`	`description`
`/interfaces/interface/config/enabled`	`admin_status`
`/interfaces/interface/subinterfaces/subinterface/ipv4/addresses/address/config/ip`	`ipv4_address`
`/interfaces/interface/ethernet/config/aggregate-id`	`lag_member_of`
`/network-instances/network-instance/protocols/protocol/ospfv2/areas/area/interfaces/interface/config/metric`	`ospf_cost`
`/network-instances/network-instance/protocols/protocol/bgp/neighbors/neighbor/config/peer-as`	`bgp_peer_asn`
`/network-instances/network-instance/protocols/protocol/bgp/neighbors/neighbor/config/peer-type`	`bgp_peer_type`
`/network-instances/network-instance/protocols/protocol/isis/levels/level/interfaces/interface/config/metric`	`isis_metric`
`/network-instances/network-instance/vlans/vlan/config/vlan-id`	`switchport_vlan`

The flattening is systematic. An OpenConfig path of the form `/A/B[key]/C/D[key]/E/config/leaf` becomes a single attribute key derived from the protocol context (`A`, `C`) and the leaf name. The list keys (e.g., interface name, neighbor address) become part of the entity identity (node or endpoint) rather than part of the attribute key. This transformation eliminates the need for hierarchical path traversal at query time: selecting all OSPF costs across 10,000 interfaces is a single column scan rather than a nested-loop join across five levels of the YANG tree.

23.2 YANG Module Structure Comparison

YANG [1] organizes data into a tree of *containers*, *lists*, *leaf-lists*, *leaves*, *groupings*, and *augmentations*. Each container introduces a level of nesting; each list introduces a keyed collection; and augmentations allow external modules to graft new subtrees onto existing schemas. The result is a richly typed, deeply nested data model that faithfully mirrors the hierarchical nature of device configuration.

Our flat attribute approach trades structural fidelity for computational efficiency. The key differences are:

1. **Schema evolution.** Adding a new attribute to our model requires no schema change: a new key-value pair is simply inserted into the attribute map. In YANG, adding a new leaf requires either modifying the module (breaking backward compatibility) or publishing a new augmentation module.
2. **Query performance.** Flat attributes are stored in columnar DataFrames, enabling SIMD-vectorized scans, predicate pushdown, and zero-copy slicing. YANG's tree structure requires path-based traversal, which does not vectorize.
3. **Cross-protocol correlation.** Correlating OSPF cost with BGP peer-AS on the same interface is a simple join on endpoint identity in our model. In YANG, it requires navigating two disjoint subtrees under `/network-instances`.
4. **Type safety.** YANG provides strong typing via `typedef`, `range`, `pattern`, and `leafref` constraints. Our attribute map is dynamically typed; validation is deferred to the functor layer.
5. **Industry tooling.** YANG benefits from a mature ecosystem: `pyang`, `yanglint`, NETCONF/RESTCONF clients, and vendor-supported OpenConfig distributions. Our model has no external tooling dependency.

24 IETF Service and Network Models

The IETF has developed a three-tier hierarchy of YANG models that separates *customer-facing service intent* from *network-level orchestration* and *device-level configuration*. Understanding where our plan topology sits within this hierarchy is essential for integration with existing SDN and orchestration systems.

Insight: *The two representations are not in opposition. YANG serves as the external API for device interaction (via NETCONF, RESTCONF, or gNMI [90]), while the flat attribute map serves as the internal analytical representation. An ingest pipeline transforms YANG-structured telemetry into flat attributes on ingestion; an egress pipeline reverses the transformation when pushing intended state to devices.*

Design Decision 114: IETF Three-Tier Model Hierarchy

Tier	IETF Models	Plan Topology Role
Customer Service Model	RFC 8299 (L3SM) [11],RFC 8466 (L2SM) [12]	Input: service intent is translated into plan topology attributes
Network Model	RFC 9182 (L3NM) [116]	**Equivalent tier**: the plan topology encodes the same network-level abstraction
Device Model	OpenConfig [2], ietf-interfaces [9]	Output: plan topology attributes are rendered into per-device config

24.1 RFC 8299: L3VPN Service Model (L3SM)

The L3VPN Service Model [11] defines a customer-facing API for requesting L3VPN connectivity. Its key abstractions—VPN service profiles, site definitions, site-network-accesses—describe *what* the customer wants without specifying *how* the network implements it.

When an L3SM service request is processed, the orchestrator must translate service-level intent into network-level state. In our framework, this translation populates the VRF and BGP attributes of the plan topology:

Table 8. L3SM (RFC 8299) to plan topology attribute mapping.

L3SM Concept	Plan Topology Attribute	Notes
`vpn-service/vpn-id`	`vrf_name`	VPN instance identifier
`site/site-id`	Node identity	PE device selection
`site-network-access/ip-connection`	`ipv4_address`	CE-PE link address
`routing-protocol/bgp`	`bgp_peer_asn`	CE-PE BGP session
`vpn-service/vpn-topology`	Layer graph structure	Hub-spoke, any-to-any
`site/management/type`	`vrf_rd`	Route distinguisher
`vpn-policy/import-rt`	`vrf_rt_import`	Import route target
`vpn-policy/export-rt`	`vrf_rt_export`	Export route target

The L3SM operates at a higher level of abstraction: it does not specify which PE routers carry the VRF, which route reflectors propagate the VPNv4 prefixes, or what MPLS label allocation policy applies. These decisions are made during the translation to our plan topology, which encodes the complete network-level view.

24.2 RFC 8466: L2VPN Service Model (L2SM)

The L2VPN Service Model [12] provides an analogous customer-facing API for L2 connectivity services including VPLS, VPWS, and EVPN-based services. The mapping to plan topology attributes targets the L2.5 and L2 layers:

Table 9. L2SM (RFC 8466) to plan topology attribute mapping.

L2SM Concept	Plan Topology Attribute	Notes
`vpn-service/vpn-id`	`evpn_instance`	EVPN instance name
`vpn-service/vpn-type`	`evpn_type`	VPLS, VPWS, EVPN
`site/site-id`	Node identity	PE device selection
`site-network-access/bearer`	Endpoint identity	Physical attachment
`svc-mtu`	`mtu`	Service MTU
`vpn-policy/import-rt`	`evpn_rt_import`	Import route target
`vpn-policy/export-rt`	`evpn_rt_export`	Export route target
`ethernet/vlan-id`	`switchport_vlan`	Service-delimiting VLAN

24.3 RFC 9182: L3VPN Network Model (L3NM)

RFC 9182 [116] defines the L3VPN Network Model (L3NM), which occupies the middle tier between the customer-facing L3SM and the device-level configuration models. The L3NM is the closest IETF analog to our plan topology: it describes the network-level view of a VPN service, specifying which nodes participate, which interfaces carry VRFs, what BGP sessions are established, and what route targets are configured.

Design Rule 17: Plan Topology as Network Model

The plan topology's VRF, BGP, and MPLS attributes at the L3 and L2.5 layers collectively encode the same information as the IETF L3NM (RFC 9182). The plan topology generalizes the L3NM in two ways:

- It is not restricted to VPN services; it encodes the complete network state across all protocol layers.
- It supports multi-layer correlation: a VRF attribute on the L3 layer can be cross-referenced with the underlying MPLS transport on the L2.5 layer and the physical interface on the L1 layer through the inter-layer coupling structure.

The three-tier IETF architecture thus maps onto our framework as follows: the *Customer Service Model* (L3SM/L2SM) provides the input intent; the *plan topology* serves as the Network Model, encoding the network-wide realization of that intent; and the *device-level YANG models* (OpenConfig, ietf-interfaces) are the output format for per-device configuration rendering.

24.4 Network Slicing (RFC 9543)

RFC 9543 [117] introduces a framework for IETF network slices, which partition a shared physical infrastructure into isolated logical networks with guaranteed resource allocations. Each network slice has its own topology, bandwidth guarantees, and isolation properties.

In our model, a network slice maps naturally to a filtered subgraph of the plan topology. Slice membership is encoded as a node and endpoint attribute (`slice_id`), and the slice's resource guarantees are encoded as QoS attributes on the relevant endpoints. The inter-layer coupling structure ensures that a slice defined at the service layer can be traced down to its physical resource allocations through the L3, L2.5, and L1 layers—precisely the kind of cross-layer query that flat attributes and columnar storage are designed to support efficiently.

Implementation in NTE

25 Data Model Mapping

The plan topology and overlay functors described in this report are designed as a *client application* that uses the NTE as its graph and columnar storage engine. The model is not embedded within NTE itself; rather, it consumes NTE's APIs—graph mutation, DataFrame queries, and overlay builder hooks—to construct and query the plan topology. This separation means the model can evolve independently of the engine, and alternative storage backends could be substituted provided they support the same graph and columnar primitives.

The plan topology maps to NTE's existing architecture as follows:

- **Nodes** → `ExternalNode` (`GraphNodeKind`)
- **Endpoints** → `ExternalEndpoint` (`GraphNodeKind`)
- **Connectivity edges** → `NteExternal(Connectivity)` (`GraphEdgeKind`)
- **Ownership edges** → `NteExternal(Structural)` (`GraphEdgeKind`)
- **Flat attributes** → Polars DataFrames in `DataFrameStore`
- **Layer assignment** → `layers` DataFrame
- **Overlay graphs** → `NteExternal(Semantic(String))` edges (e.g. `Semantic("ospf")`, `Semantic("bgp")`)

The $\mathcal{F}_\ell$ functors are implemented as NTE *overlay builders*: functions that query the plan topology's DataFrames, extract layer-relevant attributes, and construct overlay subgraphs using `add_connectivity_edges` for intra-layer adjacencies and `add_parents` for inter-layer hierarchy.

Design Decision 115: Attribute Schema as DataFrames

Each attribute table from Sections 10–11 maps to a named DataFrame in the NTE DataFrameStore. For example:

- `ospf_config: (id: i32, ospf_area: str, ospf_cost: u16, ospf_priority: u8, ospf_passive: bool, ...)`
- `bgp_peers: (id: i32, bgp_peer_address: str, bgp_peer_asn: u32, bgp_peer_state: str, ...)`
- `ip_addressing: (id: i32, ipv4_address: str, ipv6_address: str, vrf_membership: str, ...)`

The `id` column joins to the graph's node/endpoint IDs, enabling $O(1)$ lookup and SIMD-accelerated bulk queries across all attributes.

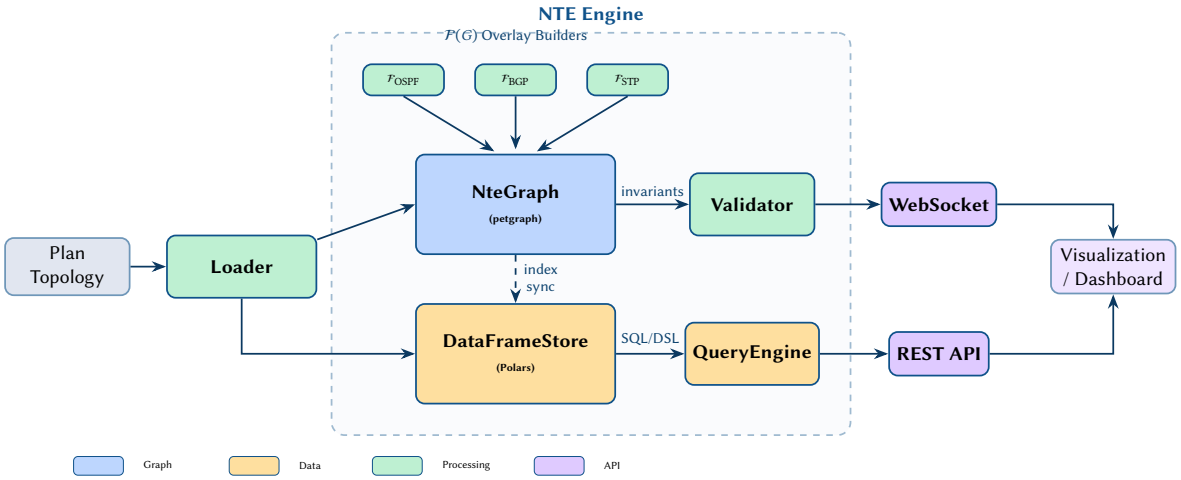


Figure 72. NTE engine architecture showing the four core components: the `NteGraph` (petgraph-based topology store), the `DataFrameStore` (Polars columnar store), the `Validator` (graph invariant checker), and the `QueryEngine` (SQL/DSL interface). Overlay builders $\mathcal{F}(G)$ extend the base graph with protocol-specific structure. External consumers access data through WebSocket and REST interfaces.

DataFrame: `ospf_config`

id	ospf_area	ospf_cost	ospf_priority	ospf_passive
i32	Utf8	u16	u8	bool
1	0.0.0.0	10	128	false
2	0.0.0.0	10	128	true
3	0.0.0.1	20	64	false
4	0.0.0.1	20	128	false
5	0.0.0.0	40	0	true

Join to graph node index

SIMD-columnar access

Figure 73. Polars DataFrame schema for OSPF configuration attributes. Each column is stored contiguously in memory, enabling SIMD-accelerated operations on individual attributes. The `id` column provides a foreign-key join to the graph node index, linking columnar data to the topology structure.

25.1 Transaction Semantics and MVCC

The plan topology may be accessed concurrently by multiple consumers: engineers querying device attributes, automation pipelines executing bulk mutations, and functor evaluation threads reading attribute DataFrames. To preserve correctness in this setting, the NTE provides *Multi-Version Concurrency Control* (MVCC) over the graph and columnar stores.

Transaction model. Every interaction with the plan topology occurs within a transaction. Read transactions acquire a *snapshot*—a logical timestamp that pins the version of every graph node, edge, and DataFrame row visible to that transaction. Write transactions accumulate mutations (attribute insertions, updates, deletions, edge additions) in a private write set that is invisible to concurrent readers until the transaction commits atomically.

This model maps naturally to network engineering workflows:

- **Maintenance window as write transaction.** An engineer modifying OSPF costs across a set of interfaces opens a single write transaction. All cost changes are staged privately and become visible only upon commit, ensuring that no concurrent reader observes a *partially re-costed* topology.
- **Functor evaluation against a consistent snapshot.** When the functor evaluation engine (Section 25.4) re-evaluates overlays after a commit, it acquires a read snapshot that reflects the *entire* committed change set. Functors never observe a state where some attributes have been updated but others have not.
- **Concurrent read access.** Multiple engineers or automation consumers can query the plan topology simultaneously, each against their own snapshot, without interference or lock contention.

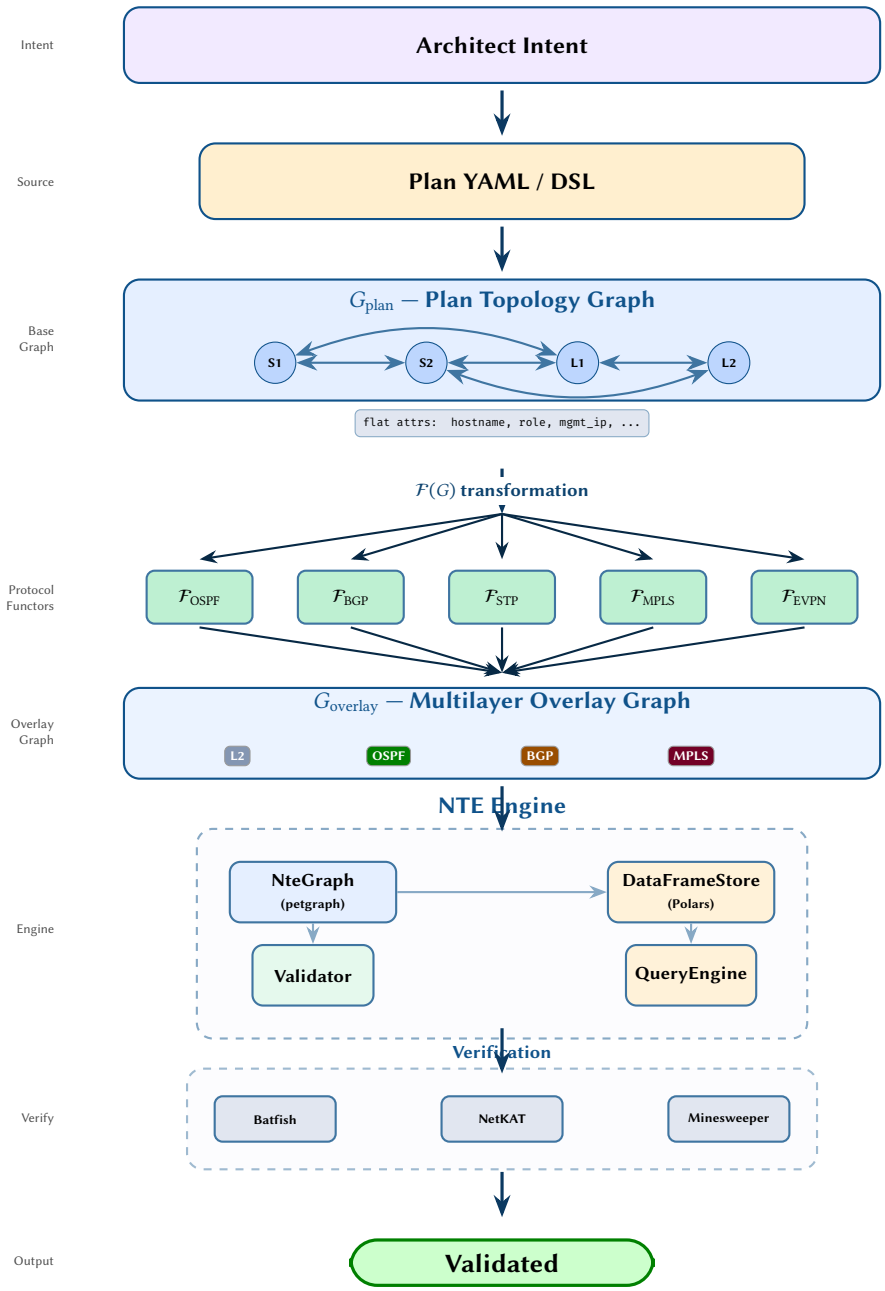


Figure 74. Complete system architecture showing the end-to-end pipeline from architect intent through plan specification, base topology graph construction, protocol functor overlay generation, NTE engine processing, and formal verification. The pipeline transforms high-level design intent into a validated, multilayer network model suitable for deployment.

Consistency guarantee. Every functor evaluation sees a *point-in-time consistent view* of all attributes in the `DataFrameStore` and all edges in the graph store. Formally, if a write transaction T_w modifies attributes $\{a_1, \ldots, a_k\}$ and commits at logical time t, then any read snapshot acquired at $t' \geq t$ observes *all* of $\{a_1, \ldots, a_k\}$ at their post-commit values, and any snapshot acquired at $t' < t$ observes *none* of them.

Design Decision 116: MVCC Properties

The NTE transaction model provides the following guarantees over the plan topology:

- **Snapshot isolation.** Read transactions see a frozen point-in-time view; concurrent writes are invisible until committed.
- **Atomic commits.** A write transaction either commits all mutations (graph edges, DataFrame rows, overlay markers) or none. There is no partial-commit state.
- **Rollback.** If validation fails (Section 25.2) or the caller explicitly aborts, the entire write set is discarded with zero side effects on the stored topology.

- **Non-blocking reads.** Readers never block writers and writers never block readers; contention is resolved via versioned storage, not locks.

25.2 Pre-Operation Validation

Insight: *MVCC transforms the plan topology into a safe shared data structure for collaborative network engineering. Without snapshot isolation, a functor evaluating the OSPF overlay could read a mix of old and new costs, producing an overlay graph that corresponds to no real configuration state.*

Before any attribute mutation is committed to the plan topology, a three-phase validation pipeline executes within the write transaction. This pipeline enforces the structural and semantic invariants established in Section 16.5, catching errors *before* they propagate to functor evaluation or downstream consumers.

Phase 1: Schema validation. Every attribute write is checked against the column type declared in the corresponding DataFrame schema. Type mismatches—such as writing a negative integer to a `u16` column, or a malformed IP address string to an `ipv4_address` column—are rejected immediately. This phase is analogous to compile-time type checking: it eliminates an entire class of errors at the point of entry.

Phase 2: Constraint validation. Inter-attribute consistency rules are evaluated next. These rules encode domain-specific invariants that span multiple columns or multiple rows within a single DataFrame. Examples include:

- **OSPF timer matching:** `ospf_dead_interval` must equal $4 \times$ `ospf_hello_interval` (or the explicitly configured value) on both endpoints of a connectivity edge.
- **BGP peer symmetry:** if endpoint e_1 lists `bgp_peer_address` $= a_2$, then the corresponding endpoint e_2 must list `bgp_peer_address` $= a_1$.
- **VLAN consistency:** a trunk endpoint's `switchport_allowed_vlans` must be a superset of the access VLANs on the connected access ports within the same broadcast domain.

Phase 3: Graph invariant validation. Structural integrity checks run against the graph topology itself:

- **No orphaned endpoints:** every endpoint must have exactly one ownership edge to a parent node.
- **No self-loops:** connectivity edges must connect distinct endpoints ($e_i \neq e_j$).
- **Endpoint uniqueness:** no two endpoints on the same node may share an identical `interface_name`.
- **Layer assignment coverage:** every endpoint participating in a layer overlay must have the required attributes populated in the corresponding DataFrame.

Fail-fast principle. Validation errors abort the transaction *before* any state changes are applied to the persistent store. The caller receives a structured error value identifying the failing phase, the offending attribute(s), and the violated constraint. Because the write set has not been committed, the plan topology remains in its pre-transaction state and no rollback logic is required—the MVCC layer simply discards the uncommitted version.

Design Decision 117: Validation Pipeline Phases

Phase	Scope	Checks	Example rejection
1	Single column	Type, range, format	`ospf_cost = -1` (not `u16`)
2	Cross-column / cross-row	Domain constraints	Hello/dead interval mismatch
3	Graph structure	Topological invariants	Endpoint with no parent node

Schema Validation Failure

An engineer sets `ospf_cost` to -1 on interface `GigabitEthernet0/1` of router `R1`. The attribute write enters Phase 1 (schema validation), which checks the target column type in the `ospf_config` DataFrame: The validation rejects the value as out of range for the `u16` column type (see Appendix C.12 for the detailed error output).
The transaction is aborted; no state change reaches the `DataFrameStore`. The engineer receives the error: `ValidationError::Schema { column: "ospf_cost", expected: "u16", got: "-1 (i64)" }`.

Insight: *Pre-operation validation converts the plan topology from a permissive key–value store into a* typed, constraint-checked *configuration database. Every attribute mutation that reaches the functor evaluation engine is guaranteed to be*

25.3 Query Patterns

The flat-attribute design of the plan topology enables a small set of composable query patterns that cover the vast majority of network engineering use cases. Because all attributes are stored as DataFrame columns keyed by node or endpoint ID, every query reduces to standard columnar operations: hash lookups, prefix scans, filters, joins, and subgraph projections.

The following catalog enumerates the six core query patterns, with pseudocode illustrating the Rust API surface: (1) attribute lookup ($O(1)$ hash lookup by endpoint ID), (2) prefix scan (all attributes for a given protocol prefix), (3) cross-device query (topology-wide column filtering), (4) neighbour walk (adjacency traversal), (5) subgraph extraction (role- or site-scoped views), and (6) join query (DataFrame join for cross-protocol analysis). Full Rust pseudocode appears in Appendix C.11.

Pattern analysis. Pattern 1 (attribute lookup) is the most frequent operation during functor evaluation: each functor reads specific attributes from specific endpoints. The `DataFrameStore` indexes by `id`, so this is a constant-time hash-map lookup.

Pattern 2 (prefix scan) exploits the naming convention established in Section 1: all attributes for a given protocol share a common prefix (`ospf_`, `bgp_`, `stp_`). A prefix scan retrieves the complete protocol configuration for one endpoint in a single operation.

Patterns 3 and 6 (cross-device query and join query) demonstrate the power of the columnar representation. Because attributes are stored as Polars DataFrames, filtering across all endpoints is a SIMD-accelerated column scan—not a per-node graph traversal. Join queries combine multiple DataFrames (e.g. `ospf_config` with `ip_addressing`) to answer questions that span protocol boundaries, such as "which interfaces have OSPF enabled but no IPv4 address assigned?"

Pattern 4 (neighbour walk) is a standard graph adjacency query, executed against the plan topology's connectivity edges.

Pattern 5 (subgraph extraction) produces a filtered view of the plan topology containing only nodes and endpoints that match a predicate. This is useful for role-based analysis (e.g. viewing only spine switches) or site-scoped evaluation (e.g. extracting a single campus for local functor evaluation).

Design Decision 118: Query Pattern Catalog

#	Pattern	Complexity	Primary use
1	Attribute lookup	$O(1)$	Functor evaluation
2	Prefix scan	$O(p)$	Protocol config retrieval
3	Cross-device query	$O(E)$	Topology-wide filtering
4	Neighbour walk	$O(\deg v)$	Adjacency discovery
5	Subgraph extraction	$O(N+E)$	Role/site scoping
6	Join query	$O(E)$	Cross-protocol analysis

p = number of attributes with the given prefix; E = total endpoints; N = total nodes; $\deg v$ = degree of the queried node.

25.4 Functor Evaluation Engine

Insight: *Flat attributes convert graph queries into DataFrame operations. This is the key implementation dividend of the flattening design: instead of writing graph traversals with complex visitor patterns, engineers compose queries from familiar tabular primitives— filter, join, group-by—that the Polars engine executes with vectorised, cache-friendly column scans.*

The functor evaluation engine is responsible for executing the layer functors $\mathcal{F}_\ell$ in an order that respects their inter-dependencies, while maximising parallelism where the dependency DAG (Section 17.5.1) permits.

Step 1: Topological sort. The engine performs a topological sort of the functor dependency DAG, producing a sequence of *evaluation levels* $L_0, L_1, \ldots, L_d$, where d is the maximum depth of the DAG. Functors within the same level have no mutual dependencies and may execute concurrently.

Step 2: Parallel evaluation. At each level L_i, the engine dispatches all functors in L_i to a thread pool. Each functor acquires a read snapshot of the plan topology (Section 25.1), evaluates its overlay construction logic, and writes the resulting overlay edges to a per-functor output buffer. Because functors at the same level read from the same snapshot and write to disjoint overlay namespaces (`Semantic("ospf")`, `Semantic("bgp")`, etc.), no synchronisation is required within a level.

Step 3: Result caching and invalidation. Each functor's output is cached, keyed by the set of input attribute versions it consumed. On subsequent evaluations (triggered by attribute changes), the engine computes the

change set—the set of attributes modified since the last evaluation—and invalidates only those cached results whose input attributes intersect the change set. This is the mechanism underlying the incremental re-evaluation described in Section 25.5.

Complexity analysis. Let L denote the total number of functors and d the maximum depth of the dependency DAG. Since each level executes in parallel, the total wall-clock evaluation time is $O(d \cdot C_{\max})$, where $C_{\max}$ is the cost of the most expensive functor at any level. For our ten-functor model (nine core functors plus $\mathcal{F}_{\text{Security}}$), the DAG has depth $d = 4$:

1. **Level 0:** $\mathcal{F}_{\text{L2}}$ (no dependencies)
2. **Level 1:** $\mathcal{F}_{\text{STP}}$, $\mathcal{F}_{\text{OSPF}}$, $\mathcal{F}_{\text{ISIS}}$ (depend on L2)
3. **Level 2:** $\mathcal{F}_{\text{BGP}}$, $\mathcal{F}_{\text{MPLS}}$ (depend on OSPF/ISIS)
4. **Level 3:** $\mathcal{F}_{\text{EVPN}}$ (depends on BGP)

The local-only functors $\mathcal{F}_{\text{QoS}}$, $\mathcal{F}_{\text{ACL}}$, and $\mathcal{F}_{\text{Security}}$ have no dependencies and no dependents; they execute at Level 0 in parallel with $\mathcal{F}_{\text{L2}}$. Thus, a full evaluation completes in four parallel rounds regardless of the total functor count.

Design Decision 119: Evaluation Engine Summary

- **Scheduling:** topological sort of the functor dependency DAG into d evaluation levels.
- **Parallelism:** all functors within a level execute concurrently on a shared thread pool.
- **Isolation:** each functor reads from an MVCC snapshot; writes target disjoint overlay namespaces.
- **Caching:** functor outputs are cached and invalidated based on input attribute change sets.
- **Depth:** $d = 4$ for the ten-functor model; full evaluation requires four parallel rounds.

25.5 Incremental Re-Evaluation

When a single attribute changes—for example, the `ospf_cost` on one interface—full re-evaluation of all layer functors is wasteful. The dependency DAG introduced in Section 17.5.1 enables *selective re-evaluation*: only functors whose input attributes are affected, or that transitively depend on an affected functor, need to be re-executed.

Impact set. Let p denote the attribute prefix that changed (e.g. `ospf_cost` has prefix `ospf`). Define the *impact set* recursively:

$$\text{impact}(p) \;=\; \Big\{\, \ell \mid p \text{ is consumed by } \mathcal{F}_\ell \Big\} \;\cup\; \Big\{\, \ell' \mid \exists\, \ell \in \text{impact}(p) \text{ s.t. } \ell \to \ell' \in \text{DAG} \Big\}.$$

Only functors $\mathcal{F}_\ell$ where $\ell \in \text{impact}(p)$ require re-evaluation; all other overlay subgraphs remain valid and can be reused from cache.

Worked examples. The following scenarios illustrate the impact set in practice:

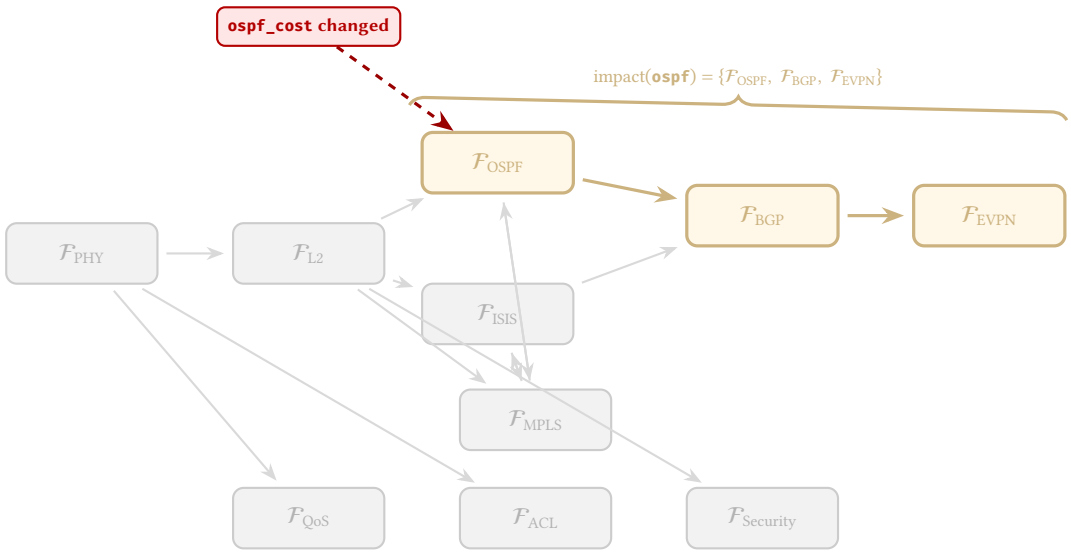
1. **Changing `ospf_cost` on one interface.** $\text{impact}(\texttt{ospf}) = \{\mathcal{F}_{\text{OSPF}},\ \mathcal{F}_{\text{BGP}},\ \mathcal{F}_{\text{EVPN}}\}$ — three functors re-evaluated, not all nine or more.
2. **Changing `qos_policy_in` on one interface.** $\text{impact}(\texttt{qos}) = \{\mathcal{F}_{\text{QoS}}\}$ — one functor, fully local; no downstream dependencies.
3. **Changing `switchport_access_vlan`.** $\text{impact}(\texttt{l2}) = \{\mathcal{F}_{\text{L2}},\ \mathcal{F}_{\text{STP}},\ \mathcal{F}_{\text{MPLS}},\ \mathcal{F}_{\text{OSPF}}/\mathcal{F}_{\text{ISIS}},\ \mathcal{F}_{\text{BGP}},\ \mathcal{F}_{\text{EVPN}}\}$ — six or more functors, reflecting the broad downstream impact of L2 topology changes.

Design Decision 120: Complexity: Full vs. Incremental Re-Evaluation

Let L denote the total number of layer functors, N the number of nodes, and E the number of endpoints in the plan topology.

- **Full re-evaluation:** $O\big(L \cdot (N+E) \log N\big)$ — every functor processes the entire graph.
- **Incremental re-evaluation:** $O\big(k \cdot (N+E) \log N\big)$ where $k = |\text{impact}(p)| \ll L$.
- **Local-only functors** ($\mathcal{F}_{\text{QoS}}$, $\mathcal{F}_{\text{ACL}}$): $O(1)$ per changed interface, since they read only the endpoint's own attributes and produce no downstream overlay edges.

For a typical campus or data-centre network with $L = 9$ functors, an `ospf_cost` change yields $k = 3$, reducing work by $\approx 67\%$. A `qos_policy_in` change yields $k = 1$, reducing work by $\approx 89\%$.



Coloured nodes and edges show the re-evaluation path.
Grey nodes are unaffected and reuse cached overlays.

Figure 75. Incremental re-evaluation for an `ospf_cost` change: only $\mathcal{F}_{OSPF}$, $\mathcal{F}_{BGP}$, and $\mathcal{F}_{EVPN}$ (coloured) are re-evaluated. All other functors (grey) reuse their cached overlay subgraphs.

25.6 Limitations and Scope

While the plan topology and functor framework capture a broad spectrum of network configuration and operational state, several aspects fall outside the current model.

Not Modelled

The following phenomena are explicitly excluded from the scope of this work:

- **Timing-sensitive convergence behaviour.** OSPF dead-interval expiry, BGP hold-timer negotiation, and other timer-based reconvergence dynamics are not represented; the model captures steady-state configuration, not transient protocol behaviour.
- **Vendor-specific CLI knobs.** Only standards-track parameters (RFC, IEEE 802.1) and widely implemented de facto attributes are included. Proprietary features such as vendor-specific route-map clauses or hardware-dependent buffering profiles are omitted.
- **Real-time telemetry sampling.** Streaming telemetry intervals, gNMI subscription modes, and SNMP polling rates are not modelled; the plan topology stores the *values* of operational counters, not their collection mechanism.
- **Control-plane resource constraints.** CPU utilisation, TCAM occupancy, RIB/FIB memory consumption, and process-level scheduling are outside scope.
- **Data-plane packet dynamics.** Queuing behaviour, buffer allocation, ECN marking thresholds, and micro-burst detection are not modelled at the graph level.

Scalability Bounds

- The plan topology has been designed and tested for networks of up to approximately 10,000 nodes with ~500,000 endpoints.
- Functor evaluation time grows with network size per the complexity bounds derived in Section 17.6. For the nine core functors, total evaluation on a 10K-node graph completes within the order of seconds on commodity hardware (single-threaded, no GPU acceleration).
- For very large networks (> 10K nodes), partition-based evaluation—splitting the plan topology along area or site boundaries and evaluating functors per partition—may be necessary to maintain interactive response times.

Design Decision 121: Scalability Estimates

Network size	Nodes	Endpoints	Full eval. (approx.)
Small campus	50	2 000	< 50 ms
Enterprise	500	25 000	< 200 ms
Large DC fabric	5 000	250 000	~ 1 s
Service provider	10 000	500 000	~ 3 s

Configuration Drift

- The plan topology represents a *point-in-time snapshot* of intended and operational state. It does not track temporal evolution or detect drift between intended configuration and actual device state.
- Drift detection is the responsibility of the automation system that consumes the plan topology (e.g. a continuous compliance engine comparing successive snapshots).
- The model could be extended with versioned attributes or a temporal graph model (e.g. bitemporal node properties), but this is outside the current scope.

Partial Protocol Support

- Some protocols have niche features not fully covered by the attribute schema: OSPF demand circuits (RFC 1793), BGP confederations with complex sub-AS topologies (RFC 5065), RSVP-TE Fast Reroute path computation (RFC 4090), and PIM Bidirectional mode (RFC 5015).
- The attribute schema is *extensible by design*: additional flat attributes can be introduced without modifying the functor framework or the plan topology structure.
- The flattening conventions established in Section 1 handle arbitrarily complex nested structures, so even deeply hierarchical vendor models can be projected onto the flat attribute space.

26 Synthesis and Analysis: Bidirectional Use of the Model

The preceding sections have presented the plan topology and its overlay functors as a *synthesis* tool: given a set of configuration attributes, the functors compute the intended overlay state. However, the same model can operate in the reverse direction as an *analysis* tool, and this duality is one of its most powerful properties.

26.1 Synthesis: Design-Time Configuration Generation

In synthesis mode, the workflow proceeds top-down:

1. An engineer or automation system writes intended state into G_{plan} as flat attributes.
2. The functor pipeline $\mathcal{F}(G_{\text{plan}})$ computes overlay graphs and derived state.
3. Validation predicates (Section 16.5) verify correctness *before deployment*.
4. Device-specific configurations are generated from the validated plan.

This is the mode described throughout this report: the plan topology captures *intended* state, and functors derive what the network *should* look like.

26.2 Analysis: Runtime State Verification

In analysis mode, the workflow is inverted. Operational state is collected from the live network—via streaming telemetry (gNMI), SNMP polling, or CLI show-command parsing—and projected onto the same plan topology schema:

1. Telemetry collectors populate a *runtime plan topology* G_{runtime} using the same flat attribute schema as the design-time plan. For example, a gNMI subscription to `/interfaces/interface/state/oper-status` maps to the `phy_oper_status` attribute on the corresponding endpoint vertex.
2. The same overlay functors $\mathcal{F}(G_{\text{runtime}})$ compute the *actual* overlay state from observed attributes.
3. Predicates that were applied at design time for validation are now applied at runtime for *compliance checking*: does the running network match the intended design?

26.3 Predicate Reuse Across Modes

A predicate is a boolean-valued function over the plan topology: $\phi : G \to \{\text{true}, \text{false}\}$. Examples include:

Insight:
The same predicates serve double duty: at design time they validate that a proposed configuration is correct; at runtime they verify that the deployed network conforms to the design. The model does not distinguish between these modes—the predicate logic is identical; only the source of the attribute values differs.

Design Decision 122: Reusable Predicates: Synthesis and Analysis

Predicate	Synthesis (Design-Time)	Analysis (Runtime)
OSPF hello/dead match	Validates that connected endpoints have matching timer values	Detects timer mismatches causing adjacency failures
BGP peer reachability	Confirms peer addresses are routable within the VRF	Identifies unreachable peers from route table state
VLAN trunk overlap	Ensures connected trunks share at least one common VLAN	Detects silent VLAN pruning on operational trunks
MTU consistency	Validates matching MTU on connected interfaces	Identifies MTU mismatches causing packet drops
STP root placement	Confirms intended root bridge has lowest priority	Detects unexpected root bridge elections

The predicate $\phi_{\text{OSPF-timers}}$, for instance, is defined as:

$$\phi_{\text{OSPF-timers}}(G) = \forall\,(e_i, e_j) \in E_{\text{conn}} \;:\; \pi(e_i, \texttt{ospf\_hello}) = \pi(e_j, \texttt{ospf\_hello}) \\ \wedge\; \pi(e_i, \texttt{ospf\_dead}) = \pi(e_j, \texttt{ospf\_dead})$$

This predicate is *identical* whether $G = G_{\text{plan}}$ (intended) or $G = G_{\text{runtime}}$ (observed).

26.4 Drift Detection via Graph Differencing

Given both the intended plan G_{plan} and the observed runtime state G_{runtime}, drift detection reduces to a graph differencing operation:

$$\Delta = G_{\text{plan}} \ominus G_{\text{runtime}}$$

where $\ominus$ denotes the symmetric attribute difference: for each vertex v present in both graphs, compute $\{k \mid \pi_{\text{plan}}(v, k) \neq \pi_{\text{runtime}}(v, k)\}$. The result Δ is a set of *drift records*, each identifying a specific attribute on a specific vertex where intended and operational state diverge.

Drift Detection Example

Suppose the plan specifies `ospf_cost = 10` on endpoint `R2:eth0`, but telemetry reports `ospf_cost = 100` (an engineer manually changed the cost on the device). The drift set Δ includes the record:

```
(R2:eth0, ospf_cost, intended=10, observed=100)
```

This is immediately actionable: the automation system can flag the drift, generate a remediation config, or trigger an alert.

26.5 Enabling Advanced Analysis Techniques

The annotated topology graph produced by the model—whether from intended or observed state—provides a structured substrate for advanced analytical techniques that go beyond predicate checking:

- **Graph Neural Networks (GNNs).** The plan topology, with its node/endpoint attributes and connectivity structure, is directly consumable by GNN architectures. Node features correspond to the flat attribute vectors; edge features correspond to connectivity and ownership semantics. GNNs trained on labelled topologies can predict failure modes, classify network health states, or detect anomalous configurations that simple predicates would miss.
- **Monte Carlo simulation.** By sampling attribute perturbations—e.g., randomly failing links, varying traffic demands, or introducing configuration errors—and re-evaluating the functor pipeline for each sample, Monte Carlo methods can estimate network resilience, identify single points of failure, and quantify the probability distribution of reachability under stochastic failures. The incremental re-evaluation mechanism (Section 25.5) makes this computationally tractable: each perturbation triggers only the affected functors, not a full re-evaluation.
- **Formal verification.** Tools like NetKAT [27], Header Space Analysis [25], and Minesweeper [28] operate on forwarding tables and reachability predicates. The overlay graphs produced by $\mathcal{F}(G)$ can serve as input to these tools, bridging the gap between configuration-level analysis and forwarding-plane verification.

- **Temporal analysis.** By storing successive snapshots of G_{runtime} (versioned via the NTE's MVCC transactions), temporal queries become possible: "When did this OSPF cost change?", "How has the BGP peering topology evolved over the last month?", "Which configuration changes preceded this outage?"

26.6 Future Directions

Insight: *The plan topology is not merely a configuration store; it is a* computational substrate *for network analysis. By standardising the representation of network state as a flat-attributed graph, the model enables any graph-based analytical technique—from classical predicate checking to machine-learning-driven anomaly detection—to operate on a consistent, vendor-neutral data structure.*

The synthesis–analysis duality opens several research directions that extend naturally from the model presented in this report:

1. **Closed-loop automation.** Combining synthesis (generating intended configs from the plan) with analysis (verifying runtime state against the plan) creates a closed-loop system: deploy → observe → compare → remediate. The plan topology is the shared data structure that makes this loop coherent.
2. **Predictive maintenance.** GNN or time-series models trained on historical G_{runtime} snapshots can predict impending failures (e.g., optical power degradation trends in DOM telemetry, BGP session flapping patterns) before they cause outages.
3. **What-if analysis.** Before deploying a configuration change, an engineer can modify G_{plan} in a transaction, re-evaluate the affected functors, and run the predicate suite to assess impact—all without touching any live device. This is the "digital twin" use case enabled by the model.
4. **Cross-domain correlation.** Because the model captures state across all OSI layers in a single graph, cross-layer failure analysis becomes tractable: a physical-layer optical degradation (DOM `rx_power` below threshold) can be correlated with L3 routing instability (OSPF adjacency flaps) through shared node/endpoint vertices, without requiring external correlation engines.

Each of these directions warrants its own detailed treatment and is the subject of ongoing work.

27 Conclusion

This report has presented five contributions:

1. An **exhaustive survey** of configurable and operational protocol state across all OSI layers, from physical transceiver DOM telemetry through BGP path attributes, grounded in specific RFCs and IEEE standards. The survey is organised in three tiers—core routing and switching, overlay and transport, and physical/security/management—and extends to policy and access control (ACLs, firewall chains, CoPP, route-maps, prefix-lists, PBR). A reference appendix (Appendix D) covers group-based segmentation (SGT, SXP, SGACL) for deployments requiring identity-based micro-segmentation.
2. A **comparative analysis of graph representations** (labeled property graphs, RDF/OWL, relational schemas, category-theoretic frameworks) with a systematic justification for the property hypergraph formalism.
3. A **plan topology** formalism G_{plan} that captures all protocol state as flat attributes on nodes and endpoints, eliminating nested hierarchies and edge-carried state. This design aligns with RFC 8345's structural model while providing SIMD-friendly columnar storage.
4. A **functorial overlay mapping** $\mathcal{F} : G_{\text{plan}} \to G_{\text{overlay}}$ that transforms the plan into protocol-specific multilayer graphs using algebraic graph rewriting (DPO rules), grounded in the category-theoretic framework of Baez–Fong and the multilayer network tensor formalism of De Domenico et al. Detailed specifications are provided for nine layer functors ($\mathcal{F}_{\text{BGP}}$, $\mathcal{F}_{\text{ISIS}}$, $\mathcal{F}_{\text{STP}}$, $\mathcal{F}_{\text{MPLS}}$, $\mathcal{F}_{\text{EVPN}}$, $\mathcal{F}_{\text{L2}}$, $\mathcal{F}_{\text{QoS}}$, $\mathcal{F}_{\text{ACL}}$, $\mathcal{F}_{\text{Security}}$), each with input attributes, pseudocode, DPO rules, and cross-layer dependency analysis.
5. A **synthesis–analysis duality** showing that the same plan topology and predicate framework can operate bidirectionally: generating intended configuration at design time (synthesis) and verifying observed state at runtime (analysis), with identical predicates applied in both modes.

The model is designed as a client application built on top of the Network Topology Engine (NTE), leveraging its dual-write graph/columnar architecture, pre-operation validation, and MVCC transaction support. The synthesis–analysis duality (Section 26) positions the plan topology not merely as a configuration store but as a computational substrate for network analysis, enabling graph neural network inference, Monte Carlo resilience estimation, and closed-loop drift remediation on the same vendor-neutral data structure.

Future work includes implementing the $\mathcal{F}_\ell$ layer functors in Rust, integrating with OpenConfig telemetry for operational state population, applying formal verification tools (NetKAT, Batfish) to the computed overlay state, extending the SGT model to cover cloud-native micro-segmentation (Kubernetes NetworkPolicy, AWS Security Groups), and developing the GNN and Monte Carlo analytical frameworks outlined in Section 26.5.

28 Advanced Implementation Mechanics

To elevate the mathematical framework into a functional orchestration engine, the NTE must bridge the gap between abstract user intent and the complex, stateful behaviors of real-world networks. This requires modeling directional asymmetry and translating structural baseline primitives into rigorous algebraic constraints.

28.1 Handling Stateful Middleboxes and Address Translation

Graph theory traditionally assumes edges are bidirectional or simply directed. Carrier-grade networks, however, are fiercely asymmetric. A packet flow from Host A to Server B might follow one path, while the return traffic follows another. This asymmetry fundamentally breaks stateful middleboxes, such as Firewalls and Network Address Translation (NAT) gateways, which require symmetric pathing to maintain session state.

In the multi-layer graph model, a stateful firewall is not merely a localized node; it represents a structural discontinuity in the graph. The supra-adjacency is strictly dependent on the temporal initiation of the transport layer (TCP/UDP) flow.

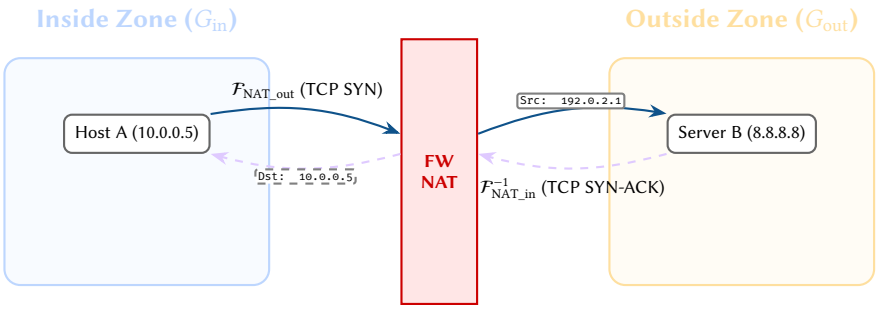


Figure 76. Stateful Firewall and NAT as a Functorial Mapping: The firewall boundary is not merely a node, but an algebraic discontinuity. It forces a mapping between two isolated IP graphs (Inside and Outside). The reverse mapping is strictly contingent upon the temporal initiation of the outbound flow, modeling stateful TCP tracking natively in the graph theory.

As depicted in Figure 76, the NTE models a NAT boundary as a functorial mapping between two mathematically isolated IP graphs (G_{in} and G_{out}). The forward mapping $\mathcal{F}_{NAT\_out}$ is instantiated by the initial packet. Crucially, the reverse mapping $\mathcal{F}^{-1}_{NAT\_in}$ only exists conditionally based on the state established by the forward flow. This proves the model natively accommodates overlapping IP spaces (e.g., overlapping VRFs or RFC1918 networks) because they are isolated across distinct parallel subgraphs stitched together by translation functors.

28.2 The Plan Compiler: From structural intent to Functorial Constraints

The mathematical depth of the supra-adjacency tensor and functor composition provides robustness, but it is entirely abstracted away from the network operator. Operators interface with the system via a Structural Composition strategy. They manipulate high-level primitives—such as drawing an "L3VPN" line between two sites.

The Plan Compiler acts as the translation layer. When a user defines an intent, the compiler expands this single primitive into a series of required flat attributes within the Plan Topology (G_{plan}). For example, an intent for "Secure L2 Stretch" translates into constraints requiring the allocation of a VLAN ID, MACsec keys, and EVPN Route Targets.

The compiler then executes the functorial mappings $\mathcal{F}(G_{plan})$. If the required attributes conflict, or if functor composition creates a cycle (as seen in route loops), the compilation fails. The operator receives an actionable error indicating the topological constraint violation without needing to understand the underlying graph algebra. This "narrow waist" architecture guarantees that diverse UI inputs—whether graphical whiteboards, YAML models, or API calls—are all rigorously validated against the universal algebraic graph.

28.3 Worked Example: Compiling structural intent to G_{plan}

To illustrate the developer experience (DX) and user experience (UX) of the Plan Compiler, we trace how a simple user intent is translated into mathematically rigorous graph state.

Step 1: User Intent (The Structural Baseline) The network operator uses a structural baseline UI to draw a line connecting two sites: `Site-A (Leaf-01)` and `Site-B (Leaf-02)`. They tag the line with the primitive `Secure L2 Stretch`.

Step 2: Primitive Expansion The NTE Plan Compiler intercepts this abstract primitive and consults the system's Design Rules. The rule for `Secure L2 Stretch` asserts that:

1. Both endpoints must possess an identical `vxlan_vni`.
2. Both endpoints must possess a matching EVPN Route Target (`evpn_rt_import` / `evpn_rt_export`).

3. The physical interfaces facing the WAN must have `macsec_enabled: true`.

Step 3: Attribute Allocation and Flattening The compiler interrogates the IP Address Management (IPAM) and ID resource pools, allocating VNI 50100 and Route Target 65000:100. It flattens these into the G_{plan} nodes.

Design Decision 123: Generated Plan Topology Fragment

```
# Node: Leaf-01 (Site A)
evpn_evi: 100
evpn_rt_import: "65000:100"
evpn_rt_export: "65000:100"
vxlan_vni: 50100
macsec_enabled: true
macsec_cipher_suite: "GCM-AES-XPN-256"

# Node: Leaf-02 (Site B)
evpn_evi: 100
evpn_rt_import: "65000:100"
evpn_rt_export: "65000:100"
vxlan_vni: 50100
macsec_enabled: true
macsec_cipher_suite: "GCM-AES-XPN-256"
```

Step 4: Functor Validation Before generating device configurations, the NTE feeds this fragment into the Functors $\mathcal{F}_{\text{EVPN}}$ and $\mathcal{F}_{\text{Security}}$. If `Leaf-02` is a legacy hardware switch that does not support the `GCM-AES-XPN-256` cipher, the structural type-checker within $\mathcal{F}_{\text{Security}}$ will throw a compile-time error. The operator receives an immediate alert: *"Cannot compile Secure L2 Stretch: Leaf-02 lacks MACsec XPN-256 capability."*

By decoupling The Structural Baseline UI from the Functor mathematics, the operator remains unaware of the complex algebraic validations occurring beneath the surface, enjoying a seamless "Intent-to-Attribute" workflow.

28.4 Worked Example: EVPN IRB Functorial Expansion

Integrated Routing and Bridging (IRB) in EVPN represents one of the most complex state-mapping problems in modern data centres. The mathematical model handles this complexity by treating the IRB mode (Asymmetric vs. Symmetric) as a parameter that drastically alters the output of the EVPN Functor ($\mathcal{F}_{\text{EVPN}}$).

The Scenario: Host A (VLAN 10) on Leaf-1 needs to communicate with Host B (VLAN 20) on Leaf-2.

Asymmetric IRB Mode: If the plan topology defines `evpn_irb_mode: asymmetric`, the Functor restricts the routing capability. Leaf-1 is only permitted to perform L2 bridging.

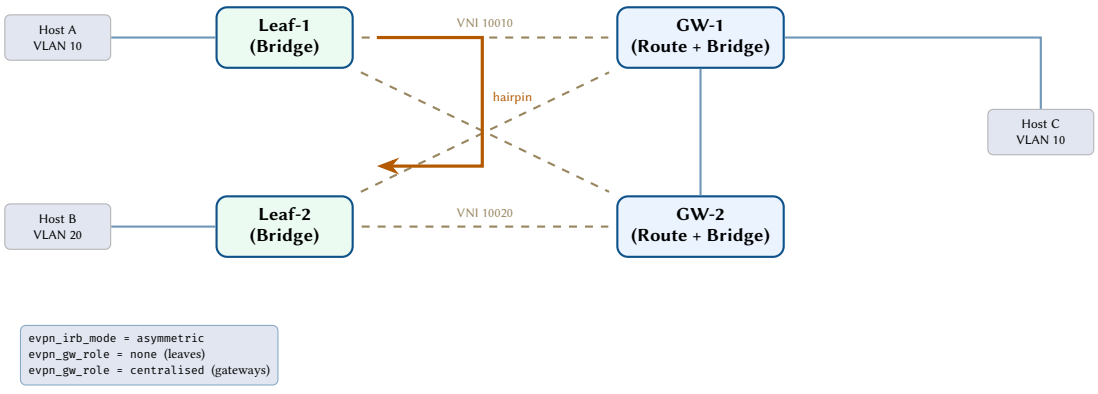


Figure 77. EVPN asymmetric IRB with centralised gateways. Leaf VTEPs perform bridging only; inter-subnet traffic hairpins through a gateway VTEP that performs routing. VXLAN tunnels (dashed) connect all VTEPs.

In this mode, the Functor generates an overlay graph where the L3 boundary is mathematically displaced to a centralised Gateway VTEP. The validation rules of the NTE assert that:

1. Leaf-1 must *not* contain an L3 SVI (Switched Virtual Interface) for VLAN 20.
2. The routing path must *hairpin* through the Gateway VTEP (GW-1 or GW-2).
3. The generated hardware TCAM rules must encapsulate the packet at Leaf-1 using the MAC address of the Gateway, not the destination host.

Symmetric IRB Mode (The Alternative): If the operator changes the attribute to `evpn_irb_mode: symmetric`, the Functor recompiles the graph. The new output generates SVIs for all VRFs on all leaf nodes. A Type-5 EVPN Route is injected into the BGP graph, establishing a direct, one-hop L3 VXLAN tunnel (L3VNI) between Leaf-1 and Leaf-2.

The value of the multi-layer graph model is that this massive structural shift—from a 2-hop L2 hairpin to a 1-hop L3 direct tunnel—is handled entirely by the algebraic rewriting rules of $\mathcal{F}_{\text{EVPN}}$. The underlying physical ($\mathcal{F}_{\text{PHY}}$) and underlay routing ($\mathcal{F}_{\text{IGP}}$) graphs remain mathematically untouched and do not require re-validation, proving the absolute isolation of the tensor layers.

28.5 Worked Example: Synthesizing the SGT Policy Matrix

Security Group Tags (SGT) present a unique challenge: they decouple security policy from IP topology. A traditional firewall ACL is a linear list of rules bound to a specific interface. An SGT Security Group Access Control List (SGACL), however, is a two-dimensional matrix (Source Tag × Destination Tag) that must be enforced globally across the fabric.

The Scenario: An operator defines an Intent where devices tagged as `IoT_Sensors (SGT: 40)` are explicitly denied access to `Financial_Servers (SGT: 90)`.

The Graph Representation: In the Plan Topology (G_{plan}), SGTs are flat identity attributes on endpoints. They possess no topological weight.

```
# Endpoint: eth1/1 (IoT Device)
sgt_value: 40
sgt_enforcement: true

# Global Policy Matrix
sgacl_policy:
  - src: 40
    dst: 90
    action: deny
```

The Functor Execution ($\mathcal{F}_{\text{Security}}$): When compiling the network state, the Security Functor does not simply copy the entire global matrix to every switch. Doing so would exhaust the hardware TCAM resources of the edge leaves. Instead, the Functor performs a highly targeted intersection query against the L2 and L3 adjacency graphs.

1. **Endpoint Discovery:** The Functor queries G_{plan} to find all endpoints where `sgt_value: 90` (the destination). It discovers these servers reside only on Leaf-3 and Leaf-4.
2. **Egress Enforcement Mapping:** In the Cisco TrustSec model, enforcement occurs at the egress node. The Functor algebraically projects the SGACL matrix onto the specific nodes holding the destination SGTs.
3. **TCAM Compilation:** The Functor generates the specific hardware rules for Leaf-3 and Leaf-4: *Deny traffic matching CMD header SGT 40, bound for local interfaces holding SGT 90.*

If a server with SGT 90 is migrated to Leaf-1 via The Structural Baseline (changing the endpoint's attached node in the graph), the $\mathcal{F}_{\text{Security}}$ Functor recalculates. It instantly deletes the TCAM rule from Leaf-4 and synthesizes the required rule on Leaf-1. The operator never configures an ACL; they simply define the algebraic constraint between two integer values, and the multi-layer tensor maps it perfectly to the topological hardware reality.

29 Day-2 Operations: Brownfield and Telemetry

While the mathematical framework strongly supports *Greenfield* deployments (where the NTE generates pristine configurations from scratch), real-world engineering requires adopting existing, messy networks (*Brownfield*) and managing state drift over time. The multi-layer graph model resolves these Day-2 operational challenges elegantly.

29.1 Brownfield Ingestion: The Anchor of Reality

A primary barrier to adopting modern intent-based orchestration is the complexity of translating legacy, unstructured CLI configurations into rigid, deeply nested data models (like YANG or complex YAML trees). Attempting to algorithmically populate a deep hierarchy from regex-parsed 'show' commands often requires brittle, highly custom middle-ware.

Because our Plan Topology (G_{plan}) fundamentally rejects deep hierarchies in favour of *flat, columnar attributes*, it acts as the ultimate ingestion schema. By syncing these attributes with a Network Source of Truth (NSoT), the graph establishes an absolute mathematical baseline—an **Anchor of Reality**.

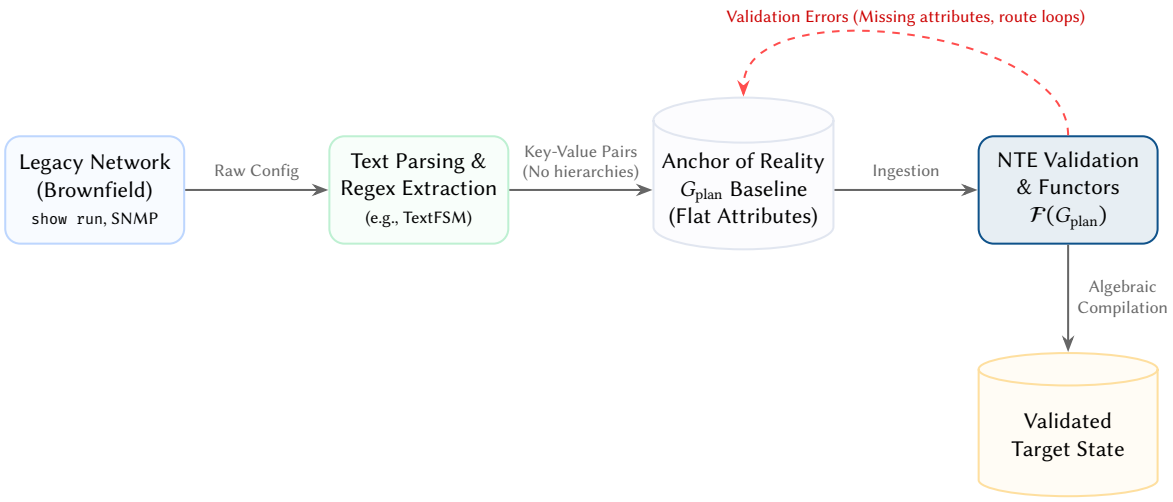


Figure 78. Brownfield Ingestion Workflow: Because the Plan Topology (G_{plan}) relies on flat, columnar attributes rather than deep YAML hierarchies, ingesting legacy configurations requires only basic regex key-value extraction. The mathematical burden of validating the state and building the relational links is offloaded entirely to the NTE Functors.

As shown in Figure 78, a migration workflow consists simply of scraping legacy text configurations and extracting flat key-value pairs (e.g., matching `router ospf 1` to `ospf_process_id: 1`). The ingestion script does not need to understand topology, interface hierarchies, or protocol dependencies; it simply dumps raw attributes onto unlinked node and endpoint records in the DataFrame.

Once the raw attributes are ingested, the NTE Functors ($\mathcal{F}$) take over. They attempt to algebraically compile the state. If the legacy configuration contains a routing loop, a missing symmetric key, or a disconnected interface, the Functor will fail to close the mathematical graph, throwing an explicit validation error. The operator fixes the flat attribute in the database, and the engine recompiles. This offloads the entire burden of structural validation from the ingestion scripts to the core mathematical engine.

29.2 State Drift and Operational Telemetry

Once a network is deployed, the "Intended State" (what we configured on The Structural Baseline) and the "Operational State" (what the network is actually doing) inevitably diverge. Links fail, BGP sessions get stuck in Active, or an engineer manually modifies an interface cost at 3:00 AM. We define this divergence as **Intent Drift**.

The tensor framework natively accommodates this duality (aligning conceptually with RFC 8342 NMDA). Telemetry data (SNMP, gNMI, streaming telemetry) is ingested and mapped through the same Functors, generating an **Operational Tensor ($\mathcal{A}_{op}$)** that lives in parallel with the **Intended Tensor ($\mathcal{A}_{int}$)**.

Because both states share an identical mathematical structure, calculating network drift is reduced to tensor subtraction:

$$\Delta = \mathcal{A}_{int} - \mathcal{A}_{op}$$

As illustrated in Figure 79, if the Intended State specifies an OSPF adjacency, but the Operational State telemetry shows the session is down, the resulting Δ matrix will be non-zero at that specific matrix coordinate.

This allows operators to query the drift tensor using standard graph algorithms to ask complex questions, such as: *"Show me all logical security boundaries (L5) that are currently compromised by operational link failures (L1) due to intent drift."* By treating telemetry as a mathematical structure rather than just a logging stream, the NTE closes the loop on Intent-Based Networking, enabling self-healing architectures.

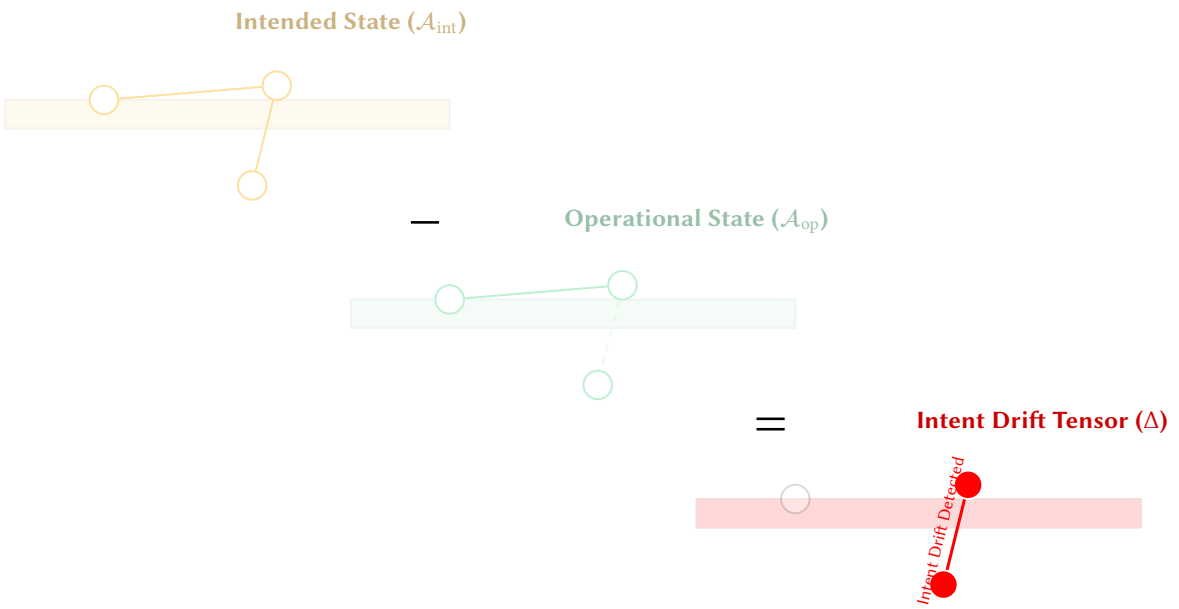


Figure 79. Day 2 State Drift as Tensor Subtraction. By mapping continuous telemetry into an Operational Tensor ($\mathcal{A}_{\text{op}}$), the NTE can perform an algebraic subtraction against the Intended Tensor ($\mathcal{A}_{\text{int}}$). The resulting Drift Matrix (Δ) isolates exactly which topological edges or attributes have deviated from the verified plan.

References

[1] M. Bjorklund, *The YANG 1.1 data modeling language*, RFC 7950, 2016.

[2] OpenConfig Working Group. "OpenConfig: Vendor-neutral network configuration models. "[Online]. Available: https://openconfig.net/

[3] NetBox Labs. "NetBox: The leading solution for network source of truth. "[Online]. Available: https://github.com/netbox-community/netbox

[4] A. Fogel et al., "A general approach to network configuration analysis," in *Proc. USENIX NSDI*, 2015.

[5] A. Clemm, J. Medved, R. Varga, N. Bahadur, H. Ananthakrishnan, and X. Liu, *A YANG data model for network topologies*, RFC 8345, 2018.

[6] A. Clemm, J. Medved, R. Varga, N. Bahadur, H. Ananthakrishnan, and X. Liu, *A YANG data model for layer 3 topologies*, RFC 8346, 2018.

[7] J. Dong, X. Wei, Q. Wu, M. Boucadair, and A. Liu, *A YANG data model for layer 2 network topologies*, RFC 8944, 2020.

[8] X. Liu, I. Bryskin, V. Beeram, T. Saad, H. Shah, and O. G. de Dios, *YANG data model for traffic engineering (TE) topologies*, RFC 8795, 2020.

[9] M. Bjorklund, *A YANG data model for interface management*, RFC 8343, 2018.

[10] M. Bjorklund, *A YANG data model for IP management*, RFC 8344, 2018.

[11] Q. Wu, S. Litkowski, L. Tomotaki, and K. Ogaki, *YANG data model for L3VPN service delivery*, RFC 8299, 2018.

[12] B. Wen, G. Fioccola, C. Xie, and L. Jalil, *A YANG data model for L2VPN service delivery*, RFC 8466, 2018.

[13] M. De Domenico et al., "Mathematical formulation of multilayer networks," *Physical Review X*, vol. 3, no. 4, p. 041 022, 2013.

[14] M. Kivelä, A. Arenas, M. Barthelemy, J. P. Gleeson, Y. Moreno, and M. A. Porter, "Multilayer networks," *Journal of Complex Networks*, vol. 2, no. 3, pp. 203–271, 2014.

[15] S. Boccaletti et al., "The structure and dynamics of multilayer networks," *Physics Reports*, vol. 544, no. 1, pp. 1–122, 2014.

[16] M. A. Porter, *What is a multilayer network?* Notices of the American Mathematical Society, 2018.

[17] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, and S. Havlin, "Catastrophic cascade of failures in interdependent networks," *Nature*, vol. 464, pp. 1025–1028, 2010.

[18] A. A. Shchurov, "A multilayer model of computer networks," *arXiv preprint arXiv:1509.00721*, 2015.

[19] J. C. Baez and B. Fong, "A compositional framework for passive linear networks," *Theory and Applications of Categories*, vol. 33, pp. 1158–1222, 2018.

[20] B. Fong, "The algebra of open and interconnected systems," Ph.D. dissertation, University of Oxford, 2016.

[21] D. I. Spivak, "The operad of wiring diagrams: Formalizing a graphical language for databases, recursion, and plug-and-play circuits," *arXiv preprint arXiv:1305.0297*, 2013.

[22] J. C. Baez and K. Courser, "Structured cospans," *Theory and Applications of Categories*, vol. 35, no. 48, 2020.
[23] M. Robinson, "Understanding networks and their behaviors using sheaf theory," *arXiv preprint arXiv:1308.4621*, 2013.
[24] J. Hansen, *A gentle introduction to sheaves on graphs*, 2020. [Online]. Available: https://www.jakobhansen.org/publications/gentleintroduction.pdf
[25] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *Proc. USENIX NSDI*, 2012.
[26] A. Khurshid, X. Zou, W. Zhou, M. Caesar, and P. B. Godfrey, "VeriFlow: Verifying network-wide invariants in real time," in *Proc. USENIX NSDI*, 2013.
[27] C. J. Anderson et al., "NetKAT: Semantic foundations for networks," in *Proc. POPL*, 2014.
[28] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proc. ACM SIGCOMM*, 2017.
[29] A. Brown, A. Fogel, et al., "Lessons from the evolution of the batfish configuration analysis tool," in *Proc. ACM SIGCOMM*, 2023.
[30] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Don't mind the gap: Bridging network-wide objectives and device-level configurations," in *Proc. ACM SIGCOMM*, 2016.
[31] R. Soulé et al., "Merlin: A language for provisioning network resources," in *Proc. CoNEXT*, 2014.
[32] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, "Hey, lumi! using natural language for intent-based network management," in *Proc. USENIX ATC*, 2021.
[33] Network to Code. "Nautobot: Network source of truth and automation platform. "[Online]. Available: https://github.com/nautobot/nautobot
[34] D. Barroso, E. Jasinska, et al. "NAPALM: Network automation and programmability abstraction layer with multivendor support. "[Online]. Available: https://napalm.readthedocs.io/
[35] ISO/IEC, *Information technology – open systems interconnection – basic reference model: The basic model*, Also published as ITU-T Recommendation X.200, 1994.
[36] J. Iyengar and M. Thomson, *QUIC: A UDP-based multiplexed and secure transport*, RFC 9000, 2021.
[37] J. Postel, *Internet protocol*, RFC 791, 1981.
[38] S. Deering and R. Hinden, *Internet protocol, version 6 (IPv6) specification*, RFC 8200, 2017.
[39] T. Narten, E. Nordmark, W. Simpson, and H. Soliman, *Neighbor discovery for IP version 6 (IPv6)*, RFC 4861, 2007.
[40] D. Plummer, *An ethernet address resolution protocol*, RFC 826, 1982.
[41] E. Rosen and Y. Rekhter, *BGP/MPLS IP virtual private networks (VPNs)*, RFC 4364, 2006.
[42] S. Nadas, *Virtual router redundancy protocol (VRRP) version 3*, RFC 5798, 2010.
[43] D. Katz and D. Ward, *Bidirectional forwarding detection (BFD)*, RFC 5880, 2010.
[44] P. Srisuresh and K. Egevang, *Traditional IP network address translator (traditional NAT)*, RFC 3022, 2001.
[45] B. Fenner et al., *Protocol independent multicast – sparse mode (PIM-SM)*, RFC 7761, 2016.
[46] B. Cain, S. Deering, I. Kouvelas, B. Fenner, and A. Thyagarajan, *Internet group management protocol, version 3*, RFC 3376, 2002.
[47] R. Vida and L. Costa, *Multicast listener discovery version 2 (MLDv2) for IPv6*, RFC 3810, 2004.
[48] B. Fenner and D. Meyer, *Multicast source discovery protocol (MSDP)*, RFC 3618, 2003.
[49] J. Moy, *OSPF version 2*, RFC 2328, 1998.
[50] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, *OSPF for IPv6*, RFC 5340, 2008.
[51] R. Callon, *Use of OSI IS-IS for routing in TCP/IP and dual environments*, RFC 1195, 1990.
[52] T. Li and H. Smit, *IS-IS extensions for traffic engineering*, RFC 5305, 2008.
[53] S. Previdi, L. Ginsberg, C. Filsfils, A. Bashandy, H. Gredler, and B. Decraene, *IS-IS extensions for segment routing*, RFC 8667, 2019.
[54] Y. Rekhter, T. Li, and S. Hares, *A border gateway protocol 4 (BGP-4)*, RFC 4271, 2006.
[55] T. Bates, E. Davies, and R. Chandra, *BGP route reflection: An alternative to full mesh internal BGP (IBGP)*, RFC 4456, 2006.
[56] P. Traina, D. McPherson, and J. Scudder, *Autonomous system confederations for BGP*, RFC 5065, 2007.
[57] Y. Rekhter, S. Sangli, and J. Scudder, *Graceful restart mechanism for BGP*, RFC 4724, 2007.
[58] T. Bates, R. Chandra, D. Katz, and Y. Rekhter, *Multiprotocol extensions for BGP-4*, RFC 4760, 2007.
[59] C. Loibl, S. Hares, R. Raszuk, D. McPherson, and M. Bacher, *Dissemination of flow specification rules*, RFC 8955, BGP Flowspec, 2020.
[60] IEEE, *IEEE standard for local and metropolitan area networks – bridges and bridged networks*, Incorporates STP (802.1D), RSTP (802.1w), MSTP (802.1s), 2022.
[61] S. Homchaudhuri and M. Foschiano, *Cisco systems' private VLANs: Scalable security in a multi-client environment*, RFC 5517, 2010.
[62] IEEE, *IEEE standard for shortest path bridging*, 2012.
[63] IEEE, *IEEE standard for local and metropolitan area networks – link aggregation*, Formerly 802.3ad, 2020.

[64] IEEE, *IEEE standard for local and metropolitan area network – station and media access control connectivity discovery*, Link Layer Discovery Protocol (LLDP), 2016.
[65] IEEE, *IEEE standard for port-based network access control*, 2020.
[66] IEEE, *IEEE standard for media access control security (MACsec)*, 2018.
[67] IEEE, *IEEE standard for connectivity fault management*, 2007.
[68] E. Rosen, A. Viswanathan, and R. Callon, *Multiprotocol label switching architecture*, RFC 3031, 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3031
[69] E. Rosen et al., *MPLS label stack encoding*, RFC 3032, 2001.
[70] K. Kompella, G. Swallow, C. Pignataro, N. Akiya, M. Chen, and S. Boutros, *Detecting multiprotocol label switched (MPLS) data-plane failures*, RFC 8029, LSP Ping, 2017.
[71] L. Andersson, I. Minei, and B. Thomas, *LDP specification*, RFC 5036, 2007.
[72] D. Awduche, L. Berger, D.-H. Gan, T. Li, V. Srinivasan, and G. Swallow, *RSVP-TE: Extensions to RSVP for LSP tunnels*, RFC 3209, 2001.
[73] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, *Segment routing architecture*, RFC 8402, 2018.
[74] S. Sivabalan, C. Filsfils, J. Tantsura, W. Henderickx, and J. Hardwick, *Segment routing policy architecture*, RFC 9256, 2022.
[75] A. Bashandy, C. Filsfils, S. Previdi, B. Decraene, S. Litkowski, and R. Shakir, *Segment routing with the MPLS data plane*, RFC 8660, 2019.
[76] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, and Z. Li, *SRv6 network programming*, RFC 8986, 2021.
[77] A. Sajassi et al., *BGP MPLS-Based ethernet VPN*, RFC 7432, 2015.
[78] A. Sajassi, J. Drake, N. Bitar, R. Shekhar, J. Uttaro, and W. Henderickx, *A network virtualization overlay solution using EVPN*, RFC 8365, 2018.
[79] K. Kompella and Y. Rekhter, *Virtual private LAN service (VPLS) using BGP for auto-discovery and signaling*, RFC 4761, 2007.
[80] M. Lasserre and V. Kompella, *Virtual private LAN service (VPLS) using label distribution protocol (LDP) signaling*, RFC 4762, 2007.
[81] S. Bryant, G. Swallow, L. Martini, and D. McPherson, *Pseudowire emulation edge-to-edge (PWE3) control word for use over an MPLS PSN*, RFC 4385, 2006.
[82] M. Jethanandani, S. Agarwal, L. Huang, and D. Blair, *YANG data model for network access control lists (ACLs)*, RFC 8519, 2019.
[83] K. Nichols, S. Blake, F. Baker, and D. Black, *Definition of the differentiated services field in the IPv4 and IPv6 headers*, RFC 2474, 1998.
[84] M. Mahalingam et al., *Virtual extensible local area network (VXLAN)*, RFC 7348, 2014.
[85] D. Farinacci, T. Li, S. Hanks, D. Meyer, and P. Traina, *Generic routing encapsulation (GRE)*, RFC 2784, 2000.
[86] S. Kent and K. Seo, *Security architecture for the internet protocol*, RFC 4301, 2005.
[87] J. Gross, I. Ganga, and T. Sridhar, *Geneve: Generic network virtualization encapsulation*, RFC 8926, 2020.
[88] W. Eddy, *Transmission control protocol (TCP)*, RFC 9293, 2022.
[89] J. Postel, *User datagram protocol*, RFC 768, 1980.
[90] OpenConfig, *gNMI – gRPC network management interface*, OpenConfig specification, https://github.com/openconfig/reference/blob/master/rpc/gnmi/gnmi-specification.md, 2024.
[91] R. Enns, M. Bjorklund, A. Bierman, and J. Schoenwaelder, *Network configuration protocol (NETCONF)*, RFC 6241, 2011.
[92] J. Vasseur and J. L. Roux, *Path computation element (PCE) communication protocol (PCEP)*, RFC 5440, 2009.
[93] M. Mathis, J. Mahdavi, S. Floyd, and A. Romanow, *TCP selective acknowledgment options*, RFC 2018, 1996.
[94] K. Ramakrishnan, S. Floyd, and D. Black, *The addition of explicit congestion notification (ECN) to IP*, RFC 3168, 2001.
[95] S. Bensley, D. Thaler, P. Balasubramanian, L. Eggert, and G. Judd, *Data center TCP (DCTCP): TCP congestion control for data centers*, RFC 8257, 2017.
[96] M. Thomson and S. Turner, *Using TLS to secure QUIC*, RFC 9001, 2021.
[97] A. Heffernan, *Protection of BGP sessions via the TCP MD5 signature option*, RFC 2385, 1998.
[98] J. Touch, A. Mankin, and R. Bonica, *The TCP authentication option*, RFC 5925, 2010.
[99] IEEE, *IEEE standard for ethernet*, 2022.
[100] ITU-T, *Timing and synchronization aspects in packet networks*, SyncE, 2019.
[101] IEEE, *IEEE standard for a precision clock synchronization protocol for networked measurement and control systems*, 2019.

[102] ITU-T, *Architecture and requirements for packet-based time and phase distribution*, PTP profiles for telecom, 2020.
[103] E. Rescorla, *The transport layer security (TLS) protocol version 1.3*, RFC 8446, 2018.
[104] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, *X.509 internet public key infrastructure online certificate status protocol – OCSP*, RFC 6960, 2013.
[105] E. Rescorla, H. Tschofenig, and N. Modadugu, *The datagram transport layer security (DTLS) protocol version 1.3*, RFC 9147, 2022.
[106] J. Chiba, G. Dommety, M. Eklund, D. Mitton, and B. Aboba, *Dynamic authorization extensions to remote authentication dial in user service (RADIUS)*, RFC 5176, 2008.
[107] D. Mills, J. Martin, J. Burbank, and W. Kasch, *Network time protocol version 4: Protocol and algorithms specification*, RFC 5905, 2010.
[108] P. Mockapetris, *Domain names – implementation and specification*, RFC 1035, 1987.
[109] D. Harrington, R. Presuhn, and B. Wijnen, *An architecture for describing simple network management protocol (SNMP) management frameworks*, RFC 3411, 2002.
[110] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd. O'Reilly Media, 2015.
[111] W3C, *RDF 1.1 concepts and abstract syntax*, W3C Recommendation, 2014. [Online]. Available: https://www.w3.org/TR/rdf11-concepts/
[112] B. Fong and D. I. Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press, 2019.
[113] S. Knight, "Plan topology schema reference: Attribute catalogue, design patterns, and validation rules," Tech. Rep. ANK-TR-2026-04, Mar. 2026, Companion document to ANK-TR-2026-03.
[114] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation* (Monographs in Theoretical Computer Science (EATCS Series)). Springer, 2006.
[115] M. Bjorklund, J. Schoenwaelder, P. Shafer, K. Watsen, and R. Wilton, *Network management datastore architecture (NMDA)*, RFC 8342, 2018.
[116] S. Barguil, O. G. de Dios, M. Boucadair, L. Munoz, and A. Aguado, *A YANG network data model for layer 3 VPNs*, RFC 9182, 2022.
[117] A. Farrel et al., *A framework for IETF network slices*, RFC 9543, 2024.
[118] P4.org Applications WG, *In-band network telemetry (INT) dataplane specification*, P4.org specification v2.1, 2020.
[119] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, *Internet key exchange protocol version 2 (IKEv2)*, RFC 7296, 2014.
[120] G. Huang, S. Beaulieu, and D. Bhatt, *A traffic-based method of detecting dead Internet Key Exchange (IKE) peers*, RFC 3706, 2004.
[121] P. Pan, G. Swallow, and A. Atlas, *Fast reroute extensions to RSVP-TE for LSP tunnels*, RFC 4090, 2005.

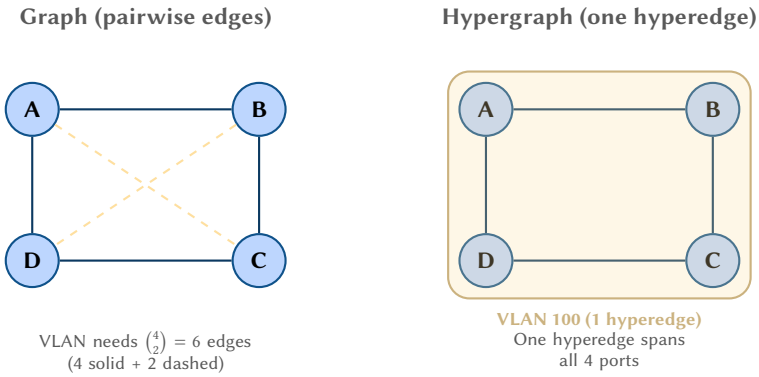
A Mathematical Concepts: Detailed Explanations

This appendix expands on the brief glossary in Section 1.2, providing detailed explanations of each mathematical concept used in this report. Every definition is accompanied by a concrete networking example and a visual illustration. The goal is to make the formal machinery accessible to network engineers who may not have a background in pure mathematics.

A.1 Graphs and Hypergraphs

A *graph* is the simplest formal model of a network: a set of *vertices* (nodes) connected by *edges* (links). Each edge connects exactly two vertices. In networking terms, the vertices are devices (routers, switches, hosts) and the edges are point-to-point links between them. Figure 80 shows a four-router network modelled as a graph with four vertices and four edges.

A *hypergraph* generalises this by allowing a single edge—called a *hyperedge*—to connect *any number* of vertices simultaneously. This matters because many networking constructs are inherently multi-endpoint: a VLAN spanning four switch ports is a single broadcast domain, not six separate pairwise connections. Modelling it as a hyperedge preserves this semantics directly. Other examples include multicast groups (one source, many receivers as a single distribution tree), VPLS instances, and bridge domains.



Insight: *Hyperedges capture "one-to-many" and "many-to-many" relationships natively. Without them, a VLAN with n ports requires $\binom{n}{2}$ pairwise edges, losing the information that all ports share a single broadcast domain.*

Figure 80. Ordinary graph vs. hypergraph representation of a VLAN. In the graph (left), the VLAN requires six pairwise edges. In the hypergraph (right), a single hyperedge captures the same broadcast domain.

A.2 Property Graphs

A *property graph* augments a graph by attaching key-value pairs ("properties") to both vertices and edges. This is distinct from a relational database in an important way: in a property graph, *relationships are first-class objects* that can carry data of their own, rather than being implicit joins between tables.

Consider a router `R1` in an OSPF network. In a relational model, the device data lives in a "devices" table, the interface data in an "interfaces" table, and the OSPF adjacency is an implicit join. In a property graph, the router is a vertex with properties (`hostname=R1`, `vendor=cisco`, `ospf_router_id=1.1.1.1`), its interface is another vertex with properties (`name=Gi0/0`, `ospf_cost=10`), and the OSPF adjacency is an edge with its own identity and type label. Traversing from router to adjacency to neighbour is a single graph walk, not a multi-table join.

Adding hyperedges to a property graph yields the *property hypergraph* used throughout this report.

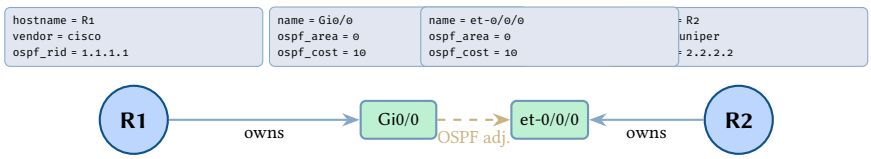


Figure 81. Property graph fragment showing two routers, their interfaces, and an OSPF adjacency edge. Key-value properties attach to every vertex; edges carry type labels.

A.3 Tensors and the Supra-Adjacency Matrix

At its simplest, an *adjacency matrix* records which nodes are connected in a graph: entry $A_{ij} = 1$ if node i connects to node j, and 0 otherwise. For a network with n nodes, this is an $n \times n$ matrix.

A *tensor* generalises a matrix to more dimensions. Just as a matrix is a 2-dimensional array of numbers, a tensor can have 3, 4, or more dimensions. In multilayer network theory, we need to record not only *which* nodes are connected but also *on which layer* the connection exists. The *supra-adjacency tensor* $M^{i\alpha}_{j\beta}$ is a rank-4 tensor that captures this: the indices i and j identify nodes, while α and β identify layers. An entry $M^{i\alpha}_{j\beta} = 1$ means "node i on layer α is connected to node j on layer β."

- When $\alpha = \beta$, this is an *intra-layer* edge (e.g., an OSPF adjacency between two routers on the routing layer).
- When $\alpha \neq \beta$, this is an *inter-layer* edge (e.g., a coupling between the physical port and the IP interface that runs on it).

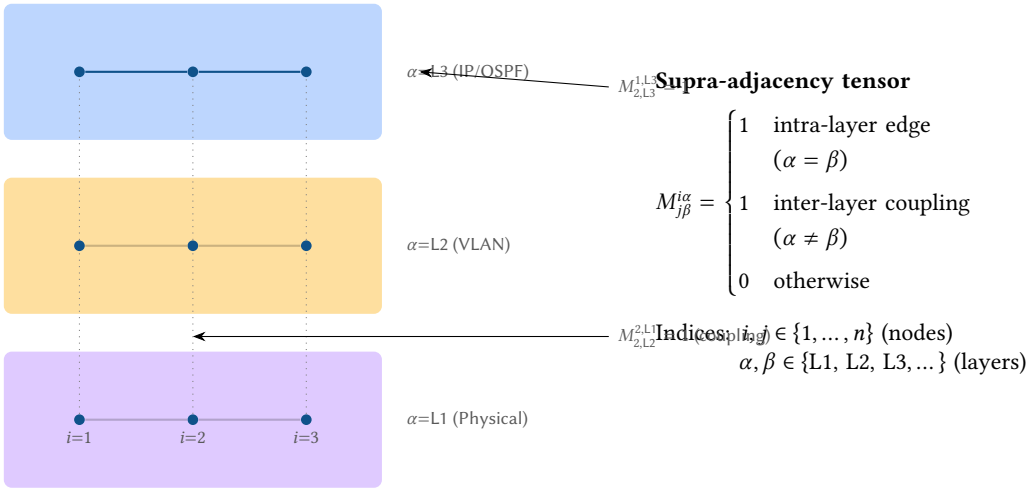


Figure 82. Stacked network layers and the supra-adjacency tensor. Solid lines are intra-layer edges ($\alpha = \beta$); dotted lines are inter-layer couplings ($\alpha \neq \beta$). The tensor $M^{i\alpha}_{j\beta}$ records both in a single data structure.

A.4 Categories and Functors

Insight: *The supra-adjacency tensor is conceptually just a "spreadsheet with four columns": source node, source layer, destination node, destination layer. Each non-zero entry records one relationship.*

A *category* is a mathematical structure consisting of two things: a collection of *objects* and a collection of *morphisms* (arrows) between those objects. Morphisms must compose associatively (if $f : A \to B$ and $g : B \to C$, then $g \circ f : A \to C$ exists), and every object has an identity morphism. Think of a category as a "type system for transformations": it tells you what kinds of structures exist and what kinds of transformations between them are valid.

In this report, two categories are central:

- **Plan**: the objects are plan topologies (property hypergraphs representing the full network state), and the morphisms are structure-preserving graph transformations (e.g., adding a new link, changing a configuration attribute).
- **Overlay**: the objects are overlay graphs (protocol-specific views like the OSPF adjacency graph or the BGP peering graph), and the morphisms are valid transformations of those overlays.

A *functor* $\mathcal{F}$: **Plan** $\to$ **Overlay** is a mapping that takes each plan topology to its corresponding overlay graph *and* takes each plan transformation to the corresponding overlay transformation. The critical property is the *composition law*: if you compose two plan changes f then g and compute the overlay, you get the same result as computing the overlay after f, then applying the overlay change induced by g. In symbols: $\mathcal{F}(g \circ f) = \mathcal{F}(g) \circ \mathcal{F}(f)$.

Why the Composition Law Matters

Suppose an engineer makes two configuration changes: (1) enable OSPF on interface Gi0/0, and (2) change the OSPF cost on Gi0/1. The composition law guarantees that computing the OSPF overlay *after both changes* gives the same result as computing the overlay after each change individually and combining the results. This is what makes incremental re-evaluation correct: you can process changes one at a time without worrying that the order of evaluation will produce a different final overlay.

A.5 Double Pushout (DPO) Rewriting

Double Pushout (DPO) rewriting is "find-and-replace for graphs." Just as a text editor finds a pattern and replaces it with new text, a DPO rule finds a subgraph pattern in a host graph and replaces it with a new subgraph. The rule is specified by three graphs and two morphisms:

- L (*left-hand side*): the pattern to find.
- K (*glue* or *interface*): the part of L that is preserved—the "context" that anchors the replacement.
- R (*right-hand side*): the replacement to produce.
- $l : K \hookrightarrow L$: identifies which parts of L are kept.

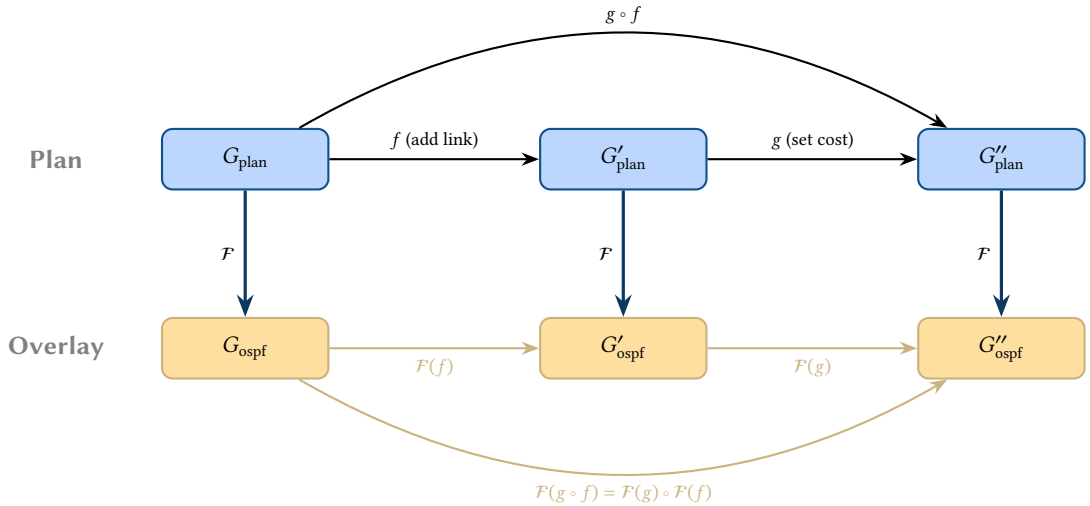


Figure 83. Commutative diagram for the overlay functor $\mathcal{F}$. Applying the functor after composing plan changes ($g \circ f$) yields the same overlay as applying the functor to each change separately and composing the results.

- $r : K \hookrightarrow R$: identifies where the kept parts appear in R.

The rule is applied by (1) finding a match $m : L \to G$ of the pattern in the host graph G, (2) removing $L \setminus K$ from G to produce a context graph D, and (3) gluing R onto D via K to produce the result graph H. The two "pushout" squares in the diagram guarantee that the rewrite is well-defined and does not create dangling edges.

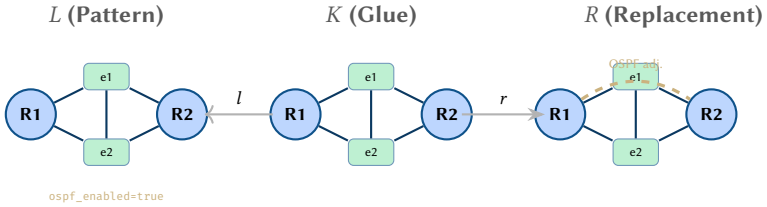


Figure 84. DPO rule for OSPF adjacency creation. The pattern L matches two routers with OSPF-enabled interfaces connected by a link. The glue K preserves all structure. The replacement R adds a new OSPF adjacency edge (dashed).

A.6 Cospans and Compositional Verification

A *cospan* is a diagram $A \to S \leftarrow B$ that models an *open system*: a network domain S with two sets of boundary interfaces, A (ingress) and B (egress). The arrows $\iota_A : A \hookrightarrow S$ and $\iota_B : B \hookrightarrow S$ indicate how the boundary interfaces embed into the internal structure.

The power of cospans lies in *composition*. Given two domains modelled as cospans, $A \to S_1 \leftarrow B$ and $B \to S_2 \leftarrow C$, they share the boundary B. Composing them means gluing S_1 and S_2 along B (formally, taking a pushout), producing a larger cospan $A \to S_1 \cup_B S_2 \leftarrow C$. The practical consequence is that you can *verify each domain independently* and then compose the verification results, rather than having to reason about the entire network at once.

Insight: *DPO rewriting makes graph transformations* precise *and* verifiable. *Because every rule is a formal triple $(L \leftarrow K \rightarrow R)$, you can mechanically check that a rule will not create dangling edges, will not violate invariants, and will produce exactly the intended result. This is the mathematical backbone of the overlay functor $\mathcal{F}$.*

Compositional Verification in Practice

Consider two OSPF areas sharing an Area Border Router (ABR). The ABR is the boundary B. You can verify reachability within Area 0 independently of Area 1 by treating each as a cospan. When you compose the two cospans, the pushout construction ensures that the ABR's route summaries are consistently propagated. If both areas pass their local verification, the composed result is guaranteed to satisfy the global reachability property—no need to re-verify the entire network from scratch.

A.7 Attribute Namespace Grammar

Every flat attribute key follows a structured naming grammar that ensures unambiguous protocol ownership and supports efficient prefix-based filtering. The following EBNF grammar defines the complete syntax.

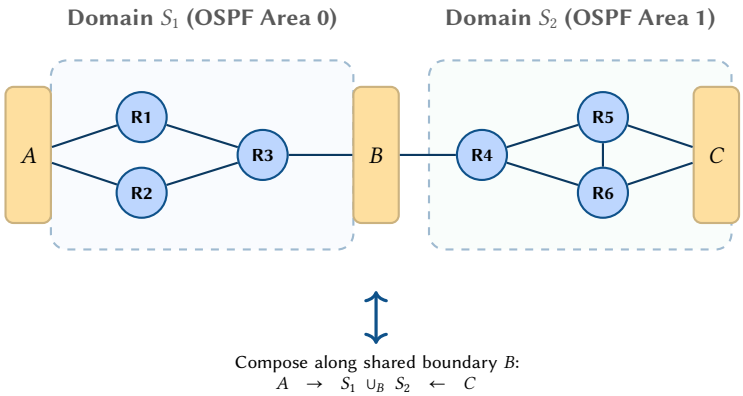


Figure 85. Two network domains modelled as cospans, sharing boundary B. Domain S_1 (OSPF Area 0) has boundary interfaces A and B; domain S_2 (OSPF Area 1) has boundaries B and C. Composing along B yields a single cospan $A \rightarrow S_1 \cup_B S_2 \leftarrow C$.

Design Decision 124: Attribute Key Grammar (EBNF)

```
attr_key        = protocol_prefix "_" field_name
protocol_prefix = "ospf" | "bgp" | "stp" | "mpls" | "isis" | "evpn"
                | "vxlan" | "gre" | "ipsec" | "ike" | "geneve"
                | "tcp" | "udp" | "quic" | "tls" | "macsec" | "dot1x"
                | "qos" | "acl" | "sgt" | "sgacl" | "sxp"
                | "phy" | "dom" | "synce" | "ptp" | "poe"
                | "switchport" | "lag" | "lacp" | "lldp" | "cfm"
                | "ipv4" | "ipv6" | "vrf" | "vrrp" | "bfd"
                | "dhcp" | "nat" | "pim" | "igmp" | "mld"
                | "ldp" | "rsvp" | "sr" | "srv6"
                | identifier
field_name      = identifier { "_" identifier }
identifier      = letter { letter | digit }
indexed_key     = attr_key "_" index
index           = digit { digit }
```

B Protocol Operation Reference

This appendix provides supplementary detail on protocol operation and runtime behaviour for readers unfamiliar with the protocols surveyed in Part II. The main text focuses on *configuration parameters*; this appendix explains *how the protocols use those parameters* at runtime.

B.1 VXLAN Control Plane Modes

The `vxlan_learning` attribute selects between two fundamentally different control-plane architectures. In *flood-and-learn* mode the VXLAN fabric behaves like a distributed bridge: unknown unicast, broadcast, and multicast (**BUM!** (**BUM!**)) frames are flooded to every remote VXLAN Tunnel Endpoint (VTEP) listed in `vxlan_flood_vteps`, and MAC addresses are learned from the data-plane source addresses of incoming encapsulated frames. This approach is straightforward to deploy but scales poorly because every VTEP must maintain a full flood list and data-plane learning introduces a race window during convergence.

When `vxlan_learning` is set to `evpn`, the fabric delegates host reachability to BGP EVPN (RFC 7432 [77]). Each VTEP advertises locally learned MAC and IP bindings as EVPN Type-2 routes, which BGP distributes to all peers. This control-plane approach eliminates data-plane flooding for known destinations and provides deterministic convergence because the Routing Information Base (RIB) is updated before the first packet arrives. The attribute therefore acts as a mode switch: changing it from `flood-and-learn` to `evpn` replaces the entire `vxlan_flood_vteps` list with BGP-learned remote VTEP state.

BUM Traffic Handling. Even under EVPN control, BUM traffic must still reach all VTEPs in a given VNI. Two replication strategies exist. *Multicast underlay* maps each VNI to an IP multicast group and relies on PIM to build distribution trees in the underlay, offloading replication to the network infrastructure. *Ingress replication* (also called head-end replication) copies the BUM frame at the originating VTEP and unicasts one copy to every remote VTEP. Ingress replication avoids the operational complexity of multicast routing but places higher load on the ingress node and its uplinks. The choice is encoded implicitly: when `vxlan_flood_vteps` is non-empty under EVPN mode, ingress replication is used; when empty, multicast group mappings from EVPN Type-3 (Inclusive Multicast Ethernet Tag) routes govern replication.

VTEP Discovery. In flood-and-learn deployments, VTEP peers must be statically enumerated in `vxlan_flood_vteps`. Under EVPN, VTEP discovery is automatic: each VTEP originates a Type-3 Inclusive Multicast route whose originator IP equals its loopback address (the `vxlan_source_intf` address). Receiving VTEPs add the originator IP to their per-VNI flood list. This dynamic discovery reduces configuration burden and enables elastic addition of leaf switches without touching existing nodes.

ARP Suppression. In dense VXLAN fabrics, ARP broadcasts constitute a significant portion of BUM traffic. EVPN ARP suppression allows each VTEP to intercept ARP requests, consult its local EVPN Type-2 cache (which includes MAC–IP bindings), and respond on behalf of the target host. The attribute `evpn_arp_suppress` (bool) enables this behaviour. When active, ARP traffic is converted from broadcast to unicast at the ingress VTEP, substantially reducing BUM load across the fabric.

MAC Mobility. When a host moves between VTEPs (e.g. VM live migration), both the old and the new VTEP may advertise the same MAC address. EVPN resolves this conflict using a *MAC mobility extended community* carrying a monotonically increasing sequence number. The new VTEP advertises the MAC with a sequence number one greater than the value in the existing route, causing all peers to prefer the new advertisement. If the sequence number reaches a configurable threshold, the MAC is flagged as a *duplicate* and further moves are dampened, preventing loops caused by misconfigured or flapping hosts. The model captures this behaviour through the existing EVPN route-type attributes; no additional per-VTEP knob is required because sequence numbering is an intrinsic property of the EVPN control plane.

B.2 Geneve Extensibility and INT

The Geneve header carries a variable-length set of Type-Length-Value (TLV) options after the fixed 8-byte base header. Each option is identified by a 16-bit *option class* (assigned by IANA or used privately) and an 8-bit *option type* whose high bit is the *critical bit*. When the critical bit is set, a transit device that does not recognise the option must drop the packet; when clear, unrecognised options may be safely ignored. This mechanism allows incremental deployment of new metadata without breaking existing forwarding paths.

A prominent use case is **INT!** (**INT!**) [118], where each hop appends telemetry metadata (queue depth, egress timestamps, link utilisation) as Geneve TLV options. The collector at the tunnel egress reconstructs per-hop metrics from the accumulated option stack. The model captures this with two control attributes: `geneve_int_enabled` activates INT insertion, and `geneve_option_class` together with `geneve_option_type` identify the TLV namespace.

B.3 IPsec SA Lifecycle and IKEv2

IKEv2 [119] structures SA establishment into three exchange types. `IKE_SA_INIT` performs a Diffie–Hellman key exchange and negotiates cryptographic algorithms in the clear; no authentication occurs at this stage. The `IKE_AUTH` exchange authenticates both peers (via the method in `ike_auth_method`) and establishes the first Child SA in a single round trip, reducing latency compared to IKEv1's six-message Phase 1. Subsequent Child SAs—for additional traffic selectors or rekeying—are created via `CREATE_CHILD_SA` exchanges that run inside the encrypted IKE SA.

Traffic Selectors. Each Child SA protects a specific traffic flow defined by *traffic selectors*: IP prefix and protocol/port tuples negotiated during `IKE_AUTH` or `CREATE_CHILD_SA`. The attributes `ike_traffic_selector_local` and `ike_traffic_selector_remote` express the initiator's proposed selectors; the responder may narrow them. In a route-based VPN the selectors are typically 0.0.0.0/0 ↔ 0.0.0.0/0, deferring packet selection to the routing table and a virtual tunnel interface.

Rekeying and Lifetime. The `ipsec_sa_lifetime` attribute sets the hard lifetime after which the SA expires. To avoid traffic interruption, rekeying begins before expiry: `ipsec_rekey_margin` specifies the number of seconds before expiry at which a `CREATE_CHILD_SA` exchange is initiated, and `ipsec_rekey_fuzz` adds a random jitter percentage to prevent synchronised rekeying storms across many tunnels.

Dead Peer Detection. DPD [120] probes liveness by sending IKEv2 Informational exchanges at the interval `ipsec_dpd_interval`. If `ipsec_dpd_retries` consecutive probes go unanswered, the IKE SA and all associated Child SAs are torn down. The model treats DPD attributes as timer parameters on the IKE SA rather than on individual Child SAs, reflecting the protocol's per-peer (not per-flow) liveness semantics.

Authentication. IKEv2 supports pre-shared keys (`psk`), RSA signatures (`rsa-sig`), and ECDSA (`ecdsa`). Certificate-based methods require the peer's distinguished name, captured in `ike_cert_dn`, to be matched during `IKE_AUTH`. Certificate authentication enables larger-scale deployments by avoiding per-peer shared secrets, at the cost of PKI infrastructure.

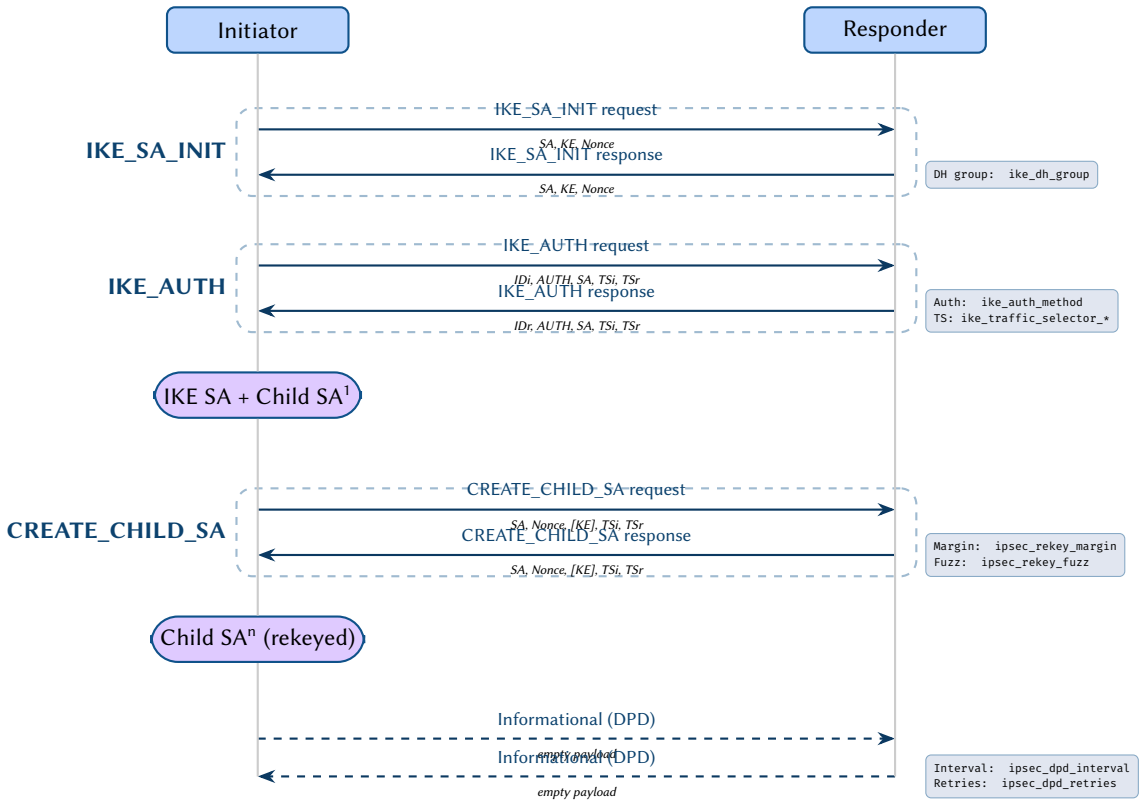


Figure 86. IKEv2 SA lifecycle. `IKE_SA_INIT` performs an unauthenticated DH exchange. `IKE_AUTH` authenticates peers and creates the first Child SA. Additional Child SAs or rekeying use `CREATE_CHILD_SA`. Dead Peer Detection (DPD) probes liveness via empty Informational exchanges. Attribute annotations (right) show the model parameters governing each phase.

B.4 LDP Operation

LDP establishes TCP sessions between neighbouring label-switching routers, exchanges label bindings, and maintains a label information base (LIB) that maps each FEC (forwarding equivalence class) to a local and remote label. In networks that use LDP, every interface participating in label distribution carries the LDP endpoint attributes defined in the main text.

B.5 RSVP-TE Signaling

Unlike LDP's hop-by-hop label distribution, RSVP-TE computes constraint-based paths using CSPF (Constrained Shortest Path First) and signals them end-to-end via PATH/RESV messages. Fast reroute [121] provides sub-50 ms protection switching via pre-computed backup LSPs.

B.6 OSPF and BGP Finite State Machines

The following diagrams illustrate the OSPF neighbour FSM and the BGP peer FSM with best-path selection, referenced from Section 4.

B.7 STP Operation

The following diagrams illustrate the STP finite state machine and bridge election process, referenced from Section 5.

B.8 MACsec Key Hierarchy

The following diagram illustrates the MACsec key derivation hierarchy (CAK, SAK, KEK), referenced from Section 11.

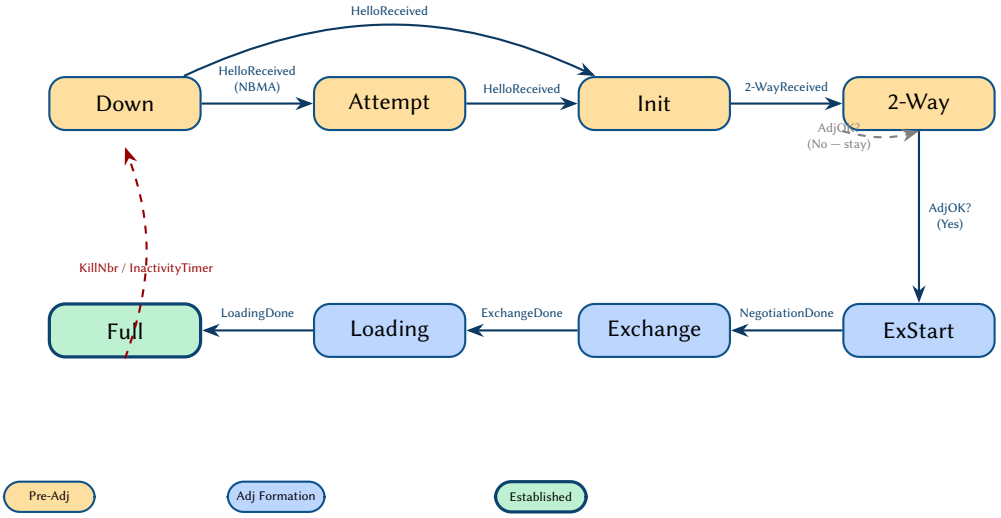


Figure 87. OSPF neighbor state machine showing the eight states from Down through Full adjacency. Pre-adjacency states (warm) handle initial hello exchange, adjacency formation states (blue) perform database synchronisation, and the Full state (green) indicates complete LSA synchronisation.

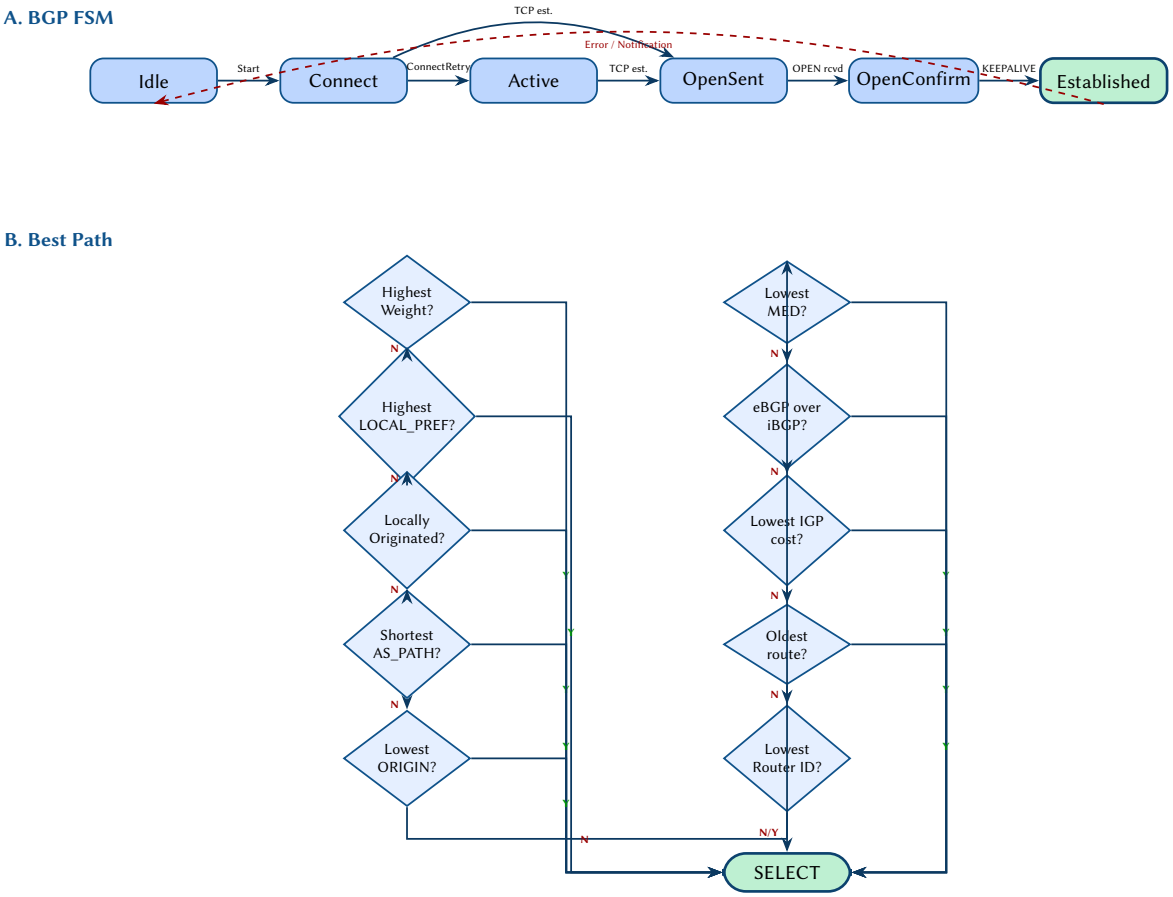


Figure 88. (A) BGP finite state machine with six states from Idle through Established. (B) BGP best path selection algorithm as a sequential decision chain, evaluating ten criteria in order of preference.

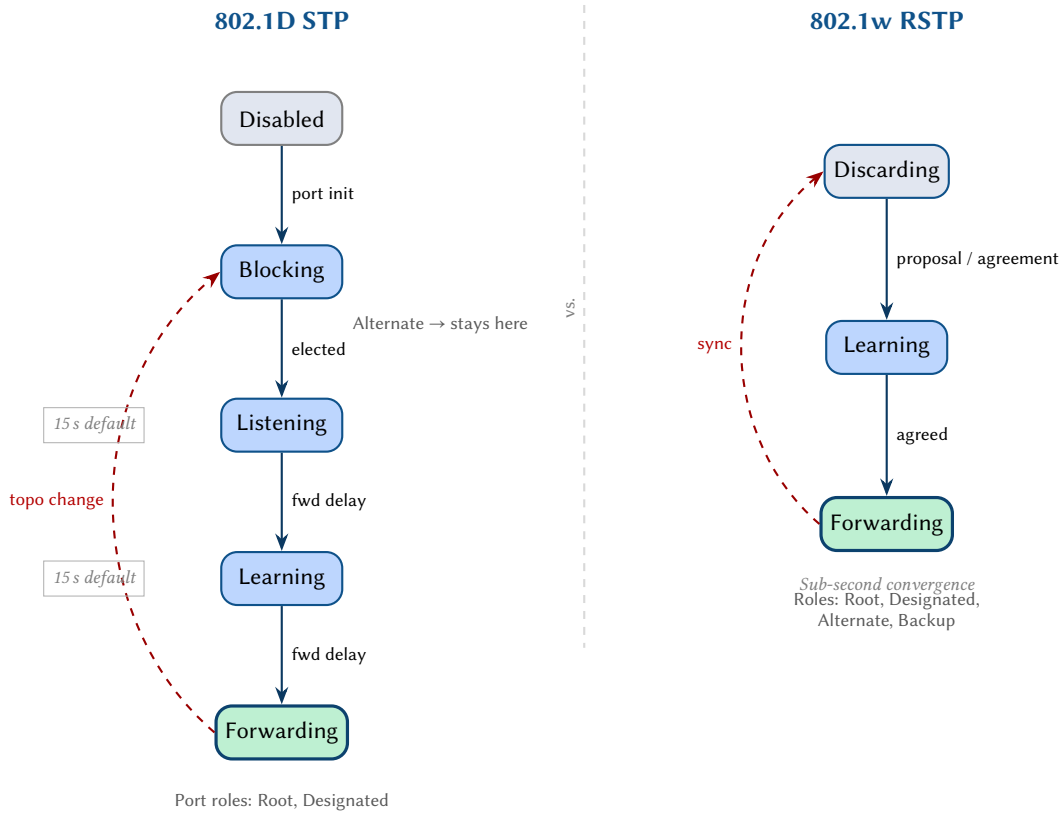


Figure 89. Spanning-tree port state machines compared. Classic 802.1D STP (left) uses five states with two timer-gated transitions of 15 s each. RSTP 802.1w (right) collapses Disabled, Blocking, and Listening into a single Discarding state and replaces timer-based convergence with a proposal/agreement handshake.

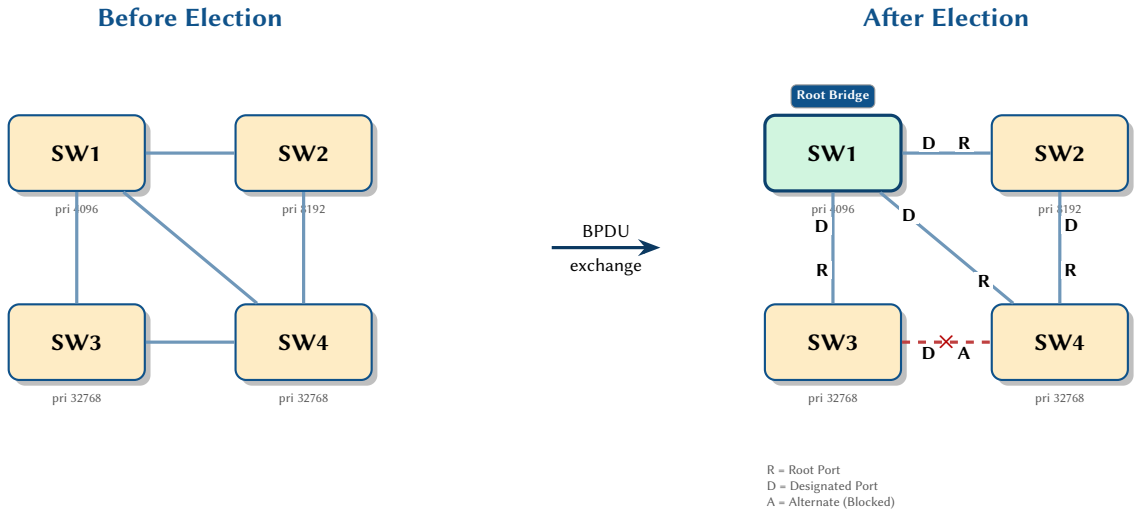


Figure 90. STP root bridge election. SW1 wins with the lowest bridge priority (4096). After convergence, each non-root switch selects one Root port toward SW1; remaining ports become Designated or Alternate. The SW3–SW4 link is blocked to break the loop.

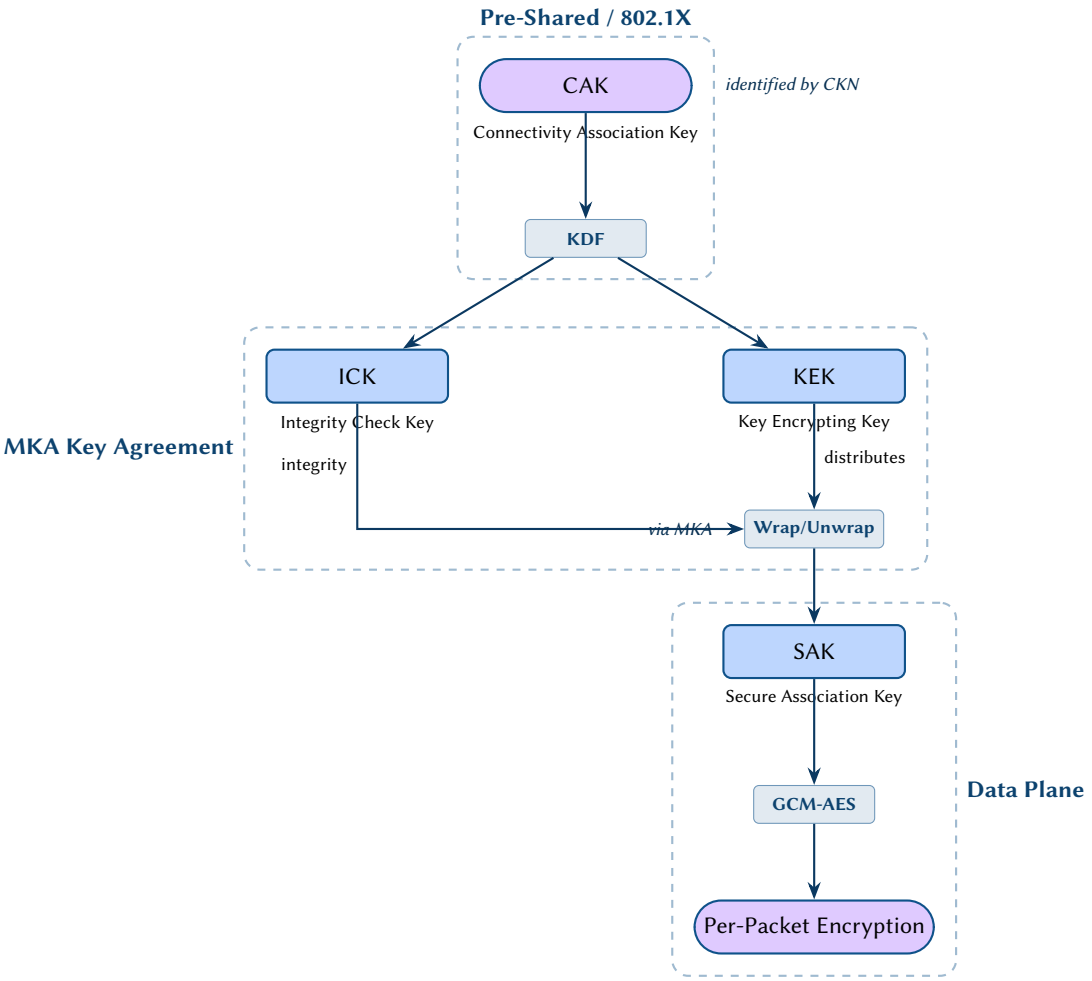


Figure 91. MACsec key hierarchy. A Connectivity Association Key (CAK), provisioned via pre-shared key or 802.1X, is processed through a KDF to produce the ICK (integrity) and KEK (key encryption). The MKA protocol uses the KEK to distribute the SAK, which drives per-packet GCM-AES encryption on the data plane.

C Functor Pseudocode Reference

This appendix contains the detailed pseudocode for each overlay functor described in Part III. The algorithms use Python-style notation with typed annotations for clarity.

C.1 $\mathcal{F}_{\text{OSPF}}$: OSPF Overlay Construction

```
F_OSPF(G_plan):
  endpoints ← filter(G_plan.endpoints, has_prefix("ospf_"))
  areas     ← group_by(endpoints, "ospf_area")
  for each area A in areas:
    adj_graph ← build_adjacency(A, G_plan.connectivity_edges)
    for each broadcast segment S in adj_graph:
      elect_dr_bdr(S)          # by ospf_priority, then router_id
    spf_tree  ← dijkstra(adj_graph, costs="ospf_cost")
    rib_entries ← extract_routes(spf_tree)
  return G_OSPF(vertices, adjacency_edges, rib_entries)
```

C.2 $\mathcal{F}_{\text{BGP}}$: BGP Overlay Construction

```
F_BGP(G_plan):
  peers <- filter(G_plan.endpoints, has_prefix("bgp_peer_"))
  for each peer pair (e_i, e_j) where bgp_peer_address matches:
    validate ASN, multihop, authentication
    afi_safi <- intersect(e_i.bgp_peer_afi_safi, e_j.bgp_peer_afi_safi)
    session <- create_session(e_i, e_j, afi_safi)
  for each session S:
    adj_rib_in <- apply_route_map(S.peer.route_map_in, received_routes)
    loc_rib    <- bestpath(adj_rib_in)
      # Decision order:
      #   weight > local_pref > locally_originated
      #   > AS_path_length > origin (IGP < EGP < incomplete)
      #   > MED > eBGP_over_iBGP > IGP_cost_to_next_hop
      #   > oldest_route > lowest_router_id > lowest_peer_address
    adj_rib_out <- apply_route_map(S.peer.route_map_out, loc_rib)
    if S.peer.next_hop_self:
      rewrite next-hop to local address
    if S.peer.rr_client and S.type == iBGP:
      reflect routes per RFC 4456
  return G_BGP(sessions, adj_rib_in, loc_rib, adj_rib_out)
```

C.3 $\mathcal{F}_{\text{STP}}$: STP Tree Computation

```
F_STP(G_plan):
  bridges <- filter(G_plan.nodes, stp_enabled=true)
  for each VLAN V (or MSTI instance):
    root <- min(bridges, key=(stp_bridge_priority, bridge_id))
    for each bridge B != root:
      for each port P in B:
        cost_to_root <- shortest_path_cost(P, root, weight=stp_port_cost)
      B.root_port <- argmin(B.ports, key=cost_to_root)
    for each segment S:
      designated <- min(S.ports, key=(cost_to_root, bridge_id, port_id))
      for each port P in S.ports:
        if P == designated:
          P.role <- DESIGNATED
        elif P == P.bridge.root_port:
          P.role <- ROOT
        else:
          P.role <- ALTERNATE  # or BLOCKED in classic STP
          block(P)
  return G_STP(bridges, port_roles, blocked_ports)
```

C.4 $\mathcal{F}_{\text{ISIS}}$: IS-IS Overlay Construction

```
F_ISIS(G_plan):
  endpoints <- filter(G_plan.endpoints, has_prefix("isis_"))
  partition by level:
    L1_endpoints <- filter(endpoints, isis_level in {level-1, level-1-2})
    L2_endpoints <- filter(endpoints, isis_level in {level-2, level-1-2})
  for each level L in {L1, L2}:
    adj_graph <- build_adjacency(L_endpoints, G_plan.connectivity_edges)
    for each broadcast segment S in adj_graph:
      elect_dis(S)  # by isis_priority, then SNPA
    spf_tree <- dijkstra(adj_graph, costs="isis_metric")
    rib_entries <- extract_routes(spf_tree)
  # L1->L2 route leaking for L1L2 routers
  for each L1L2 router R:
    leak L1 routes into L2 LSDB (filtered by policy)
    optionally leak L2 default into L1
  return G_ISIS(vertices, adjacency_edges, rib_entries, dis_elections)
```

C.5 $\mathcal{F}_{\text{MPLS}}$: MPLS Label Binding

```
F_MPLS(G_plan):
  lsrs <- filter(G_plan.nodes, mpls_enabled=true)
  ldp_bindings  <- {}
  sr_bindings   <- {}
  rsvp_bindings <- {}
  # --- LDP ---
  for each LDP-enabled pair (n_i, n_j) with IGP adjacency:
    for each FEC F reachable via n_j:
      label <- allocate(n_i.mpls_label_range)
      ldp_bindings[n_i, F] <- (label, n_j.label_for(F), next_hop=n_j)
  # --- Segment Routing ---
  for each SR-enabled node n:
    prefix_sid <- n.sr_global_block_min + n.sr_node_sid
    for each other SR node m:
      sr_bindings[m, n.prefix] <- (prefix_sid, swap/pop, IGP_next_hop)
    if n.sr_adj_sid_enabled:
      for each adjacency A of n:
        sr_bindings[n, A] <- (adj_sid, pop, A.interface)
  # --- RSVP-TE ---
  for each tunnel T in rsvp_tunnels:
    if T.rsvp_explicit_path:
      path <- T.rsvp_explicit_path
    else:
      path <- CSPF(T.source, T.rsvp_tunnel_dest,
                   bandwidth=T.rsvp_bandwidth,
                   priorities=(T.rsvp_setup_priority, T.rsvp_hold_priority))
    for each hop H in path:
      label <- allocate(H.mpls_label_range)
      rsvp_bindings[H, T] <- (label, next_label, next_hop)
  # --- Merge ---
  lfib <- merge(ldp_bindings, sr_bindings, rsvp_bindings)
    # Priority: SR > RSVP-TE > LDP (configurable)
  return G_MPLS(lsrs, lsp_edges, lfib)
```

C.6 $\mathcal{F}_{\text{EVPN}}$: EVPN Route Processing

```
F_EVPN(G_plan):
  vteps <- filter(G_plan.nodes, evpn_enabled=true)
  mac_ip_table <- {}
  vni_map      <- {}
  for each EVI E:
    members <- filter(vteps, evpn_evi=E.id)
    for each VTEP V in members:
      export Type-2 routes for locally learned MAC/IPs
```

```
      with RD=V.evpn_rd, RT=V.evpn_rt_export
    export Type-3 route for BUM membership
  for each VTEP V in members:
    import routes where RT intersects V.evpn_rt_import
    merge into mac_ip_table[E]
  vni_map[E] <- E.vxlan_vni
# --- Multi-homing ---
esi_groups <- group_by(vteps, evpn_esi)
for each ESI group G where |G| > 1:
  for each VLAN V in G:
    if df_election == modulo:
      df <- G.vteps[V.id mod |G.vteps|]
    else:
      df <- max(G.vteps, key=preference)
    mark non-DF VTEPs as backup for (ESI, V)
return G_EVPN(vteps, bgp_sessions, mac_ip_table, vni_map)
```

C.7 $\mathcal{F}_{\text{L2}}$: L2 VLAN Overlay Construction

```
F_L2(G_plan):
  ports <- filter(G_plan.endpoints, has_prefix("switchport_"))
  vlan_members <- {}
  for each port P in ports:
    if P.switchport_mode == "access":
      vlan_members[P.switchport_access_vlan].add(P)
    elif P.switchport_mode == "trunk":
      for each V in P.switchport_trunk_allowed_vlans:
        vlan_members[V].add(P)
  # Integrate STP port states
  stp_state <- F_STP(G_plan)
  for each VLAN V in vlan_members:
    for each port P in vlan_members[V]:
      if stp_state.is_blocked(P, V):
        vlan_members[V].remove(P)
  return G_L2(vlan_hyperedges=vlan_members, mac_table)
```

C.8 $\mathcal{F}_{\text{QoS}}$: QoS Policy Compilation

```
F_QoS(G_plan):
  interfaces <- filter(G_plan.endpoints, has_prefix("qos_"))
  for each interface I:
    policy_in  <- resolve(I.qos_policy_in)
    policy_out <- resolve(I.qos_policy_out)
    # Stage 1: Classify
    classify: map DSCP/CoS to internal queue
      based on qos_trust boundary
    # Stage 2: Police
    police: apply ingress rate limits
      from qos_policing_rate, qos_policing_burst
    # Stage 3: Mark
    mark: rewrite DSCP if qos_default_dscp set
      or if policy specifies remark action
    # Stage 4: Schedule
    schedule: compute queue weights from qos_scheduler type
      strict-priority queues get absolute priority
      WRR/WFQ queues share remaining bandwidth by weight
    # Stage 5: Shape
    shape: apply egress shaping from qos_shaping_rate
      token-bucket algorithm with configured burst
  return G_QoS(queue_assignments, dscp_tables, scheduler_configs)
```

C.9 $\mathcal{F}_{\text{ACL}}$: ACL to TCAM Compilation

```
F_ACL(G_plan):
```

```
  interfaces <- filter(G_plan.endpoints,
                       has("acl_in") or has("acl_out"))
  for each interface I:
    for direction in [ingress, egress]:
      acl_name <- I.acl_in or I.acl_out
      if acl_name is None: continue
      entries <- resolve_acl_entries(G_plan, acl_name)
        # expand indexed flat attributes: acl_<name>_seq_<n>_*
      for each entry E in sequence order:
        tcam_entry <- compile_to_tcam(E)
          # convert prefix + wildcard to ternary match
          # compute mask bits for protocol, ports, DSCP
          # set action bits (permit=1, deny=0)
        if E.acl_log:
          tcam_entry.flags |= LOG
        if E.acl_counter_packets is not None:
          tcam_entry.flags |= COUNTER
        append tcam_entry to TCAM[I, direction]
      # Append implicit deny-all as final TCAM entry
      append deny_all_entry to TCAM[I, direction]
  return G_ACL(tcam_tables, interface_bindings)
```

C.10 $\mathcal{F}_{\text{Security}}$: Group Policy Compilation

```
F_Security(G_plan):
  endpoints <- filter(G_plan.endpoints, has_prefix("sgt_"))

  # Phase 1: Resolve SGT assignments
  ip_sgt_table <- {}
  for each endpoint E with sgt_value:
    ip_sgt_table[E.ip_address] <- E.sgt_value

  # Phase 2: Propagate via SXP
  sxp_peers <- filter(G_plan.nodes, has_prefix("sxp_"))
  for each SXP speaker S:
    for each SXP listener L connected to S:
      propagate ip_sgt_table entries from S to L
      # L merges received bindings into its local table
      # conflict resolution: prefer static over dynamic

  # Phase 3: Build SGACL policy matrix
  sgacl_rules <- filter(G_plan.nodes, has_prefix("sgacl_"))
  policy_matrix <- {}
  for each rule R in sgacl_rules:
    key <- (R.sgacl_src_sgt, R.sgacl_dst_sgt)
    policy_matrix[key] <- (R.sgacl_action,
                           R.sgacl_protocol,
                           R.sgacl_port)

  # Phase 4: Compile per-enforcement-point policies
  for each node N with sgacl enforcement enabled:
    local_policy <- {}
    for each (src_sgt, dst_sgt) -> action in policy_matrix:
      if N serves traffic for dst_sgt:
        download (src_sgt, dst_sgt) -> action to N
        local_policy[(src_sgt, dst_sgt)] <- action

  return G_Security(sgt_assignments, sxp_bindings, policy_matrix)
```

C.11 NTE Query Patterns (Rust)

```rust
// 1. Attribute lookup — O(1) hash lookup by endpoint ID
let cost: u16 = plan.attr(endpoint_id, "ospf_cost")?;

// 2. Prefix scan — all OSPF attributes for one endpoint
```

```rust
let ospf_attrs: DataFrame = plan.attrs_with_prefix(endpoint_id, "ospf_")?;

// 3. Cross-device query — all area-0 OSPF interfaces in the topology
let area0: Vec<EndpointId> = plan.endpoints()
    .filter(|e| e.attr("ospf_area") == Some("0.0.0.0"))
    .collect();

// 4. Neighbour walk — all devices directly connected to a given device
let neighbours: Vec<NodeId> = plan.neighbors(device_id)?;

// 5. Subgraph extraction — spine-only view of the topology
let spine_graph: SubGraph = plan.subgraph(
    |v| v.attr("device_role") == Some("spine")
)?;

// 6. Join query — DataFrame join for composite analysis
let result: DataFrame = plan.endpoints_df()
    .join(&ospf_config_df, on: "id", how: JoinType::Inner)?
    .filter(col("ospf_passive").eq(lit(true)))?;
```

C.12 Validation Error Example

```
Attribute:  ospf_cost
Value:      -1 (i64)
Expected:   u16 (unsigned 16-bit integer, range 0..65535)
Result:     REJECTED — value out of range for column type
```

D Group-Based Security (SGT/SGACL) Reference

Traditional network segmentation relies on topology-bound constructs—VLANs partition Layer 2 broadcast domains, VRFs isolate Layer 3 routing tables, and firewall zones govern inter-zone traffic. While effective for north–south (client-to-server) traffic control, these mechanisms struggle with east–west (server-to-server, workload-to-workload) segmentation within the same broadcast domain. As data centre and campus architectures flatten and workloads migrate dynamically, access policy must decouple from network addressing entirely.

Intent-based segmentation addresses this gap by assigning traffic to *security groups* based on identity—user role, device type, application classification—rather than IP address or VLAN membership. The Scalable Group Tag (SGT) framework, originating from Cisco TrustSec and documented in IETF draft specifications, provides a concrete realisation of this model. In the zero-trust paradigm, SGTs enable micro-segmentation: every flow between security groups is subject to explicit policy, regardless of whether source and destination share a subnet.

This section models the full SGT lifecycle—assignment, propagation, and enforcement—as flat key-value attributes on nodes and endpoints within the property hypergraph.

D.1 Scalable Group Tags (SGT)

An SGT is a 16-bit tag (values 0–65 535) embedded in the data plane to classify traffic by security group membership. The tag travels with the packet, enabling enforcement at every hop without requiring per-flow ACL state keyed on IP addresses. Unlike VLAN-based segmentation, SGT assignment is independent of network topology: two endpoints on the same VLAN may carry different SGTs, and two endpoints on different VLANs may share the same SGT.

Design Decision 125: SGT Attributes

Attribute	Type	Description
`sgt_value`	u16	Assigned SGT value (0–65535)
`sgt_assignment_method`	enum	static, dynamic, vlan
`sgt_propagation`	enum	inline, sxp
`sgt_name`	str	Human-readable group name
`sgt_description`	str	Group description

Insight:
SGT values are pure identity labels—they carry no topological information. This makes them orthogonal to every other endpoint attribute in the plan topology: an endpoint's

D.2 SGT Assignment Methods

SGT assignment determines how an endpoint acquires its group tag. Three primary methods exist, forming a precedence hierarchy that balances automation against operational control.

Static assignment. The simplest method binds an SGT to a specific port, VLAN, or IP subnet on the enforcement device. Per-port assignment suits fixed infrastructure (printers, IP phones, IoT sensors) whose physical location does not change. Per-VLAN assignment maps all traffic in a VLAN to a single SGT—useful as a transitional strategy when migrating from VLAN-based to identity-based segmentation. Per-subnet assignment (IP-to-SGT binding) covers scenarios where devices obtain addresses from known pools but do not participate in 802.1X authentication.

Dynamic assignment. When a device authenticates via IEEE 802.1X, the RADIUS server returns an SGT value as a vendor-specific attribute (VSA) alongside the standard authorisation result. This is the preferred method in identity-aware deployments because the SGT reflects the authenticated identity rather than a topological assumption. If 802.1X is unavailable—for devices lacking a supplicant—MAC Authentication Bypass (MAB) provides a fallback: the switch sends the device's MAC address to RADIUS, which performs a lookup and returns an SGT based on MAC-to-group policy. Web authentication offers a third dynamic path for guest and BYOD devices.

IP-SGT binding database. Regardless of assignment method, the enforcement device maintains a local IP-to-SGT binding table that maps each active IP address to its assigned SGT. This table is the authoritative source for inline tagging decisions and is also the data structure propagated by SXP (Section D.3).

Design Decision 126: SGT Assignment Attributes

Attribute	Type	Description
`sgt_static_port`	u16	Static SGT for this port
`sgt_dynamic_source`	enum	dot1x, mab, webauth
`sgt_ip_binding_prefix`	prefix	IP-to-SGT static binding
`sgt_vlan_mapping`	map	VLAN-to-SGT mapping table

D.3 SGT Propagation

Once an SGT is assigned, it must be communicated to every enforcement point along the packet's path. Two propagation mechanisms exist, addressing different hardware capabilities.

Inline tagging (CMD). On hardware that supports TrustSec, the switch inserts a Cisco Meta Data (CMD) field into the Ethernet frame header between the 802.1Q tag and the EtherType/Length field. The CMD field carries the 16-bit SGT, enabling every intermediate device to classify and enforce policy in the data plane without consulting a central controller. Inline tagging imposes minimal overhead (16 bytes) and requires no control-plane protocol, but demands hardware support at every hop.

SXP (SGT Exchange Protocol). Where inline tagging is unavailable—legacy switches, third-party hardware, or WAN links—SXP provides a TCP-based control-plane protocol that propagates IP-SGT bindings between peers. Each SXP session has a *speaker* (the device that knows the bindings) and a *listener* (the device that needs them). A device may operate as both speaker and listener simultaneously. SXP sessions use TCP port 64999 with optional MD5 authentication, and bindings are refreshed via configurable hold timers.

In mixed environments, a common design pattern connects access-layer switches (which perform 802.1X and assign SGTs) as SXP speakers to distribution or data centre switches (which lack CMD hardware) as SXP listeners. The listeners install the received IP-SGT bindings into their local binding tables and enforce SGACLs in software.

Design Decision 127: SXP Attributes

Attribute	Type	Description
`sxp_peer_address`	ip	SXP peer IP address
`sxp_peer_mode`	enum	speaker, listener, both
`sxp_source_ip`	ip	Source IP for SXP connections
`sxp_vrf`	str	VRF for SXP peering
`sxp_password`	str	SXP authentication password
`sxp_holdtime_min`	u16	Minimum hold time (seconds)
`sxp_holdtime_max`	u16	Maximum hold time (seconds)

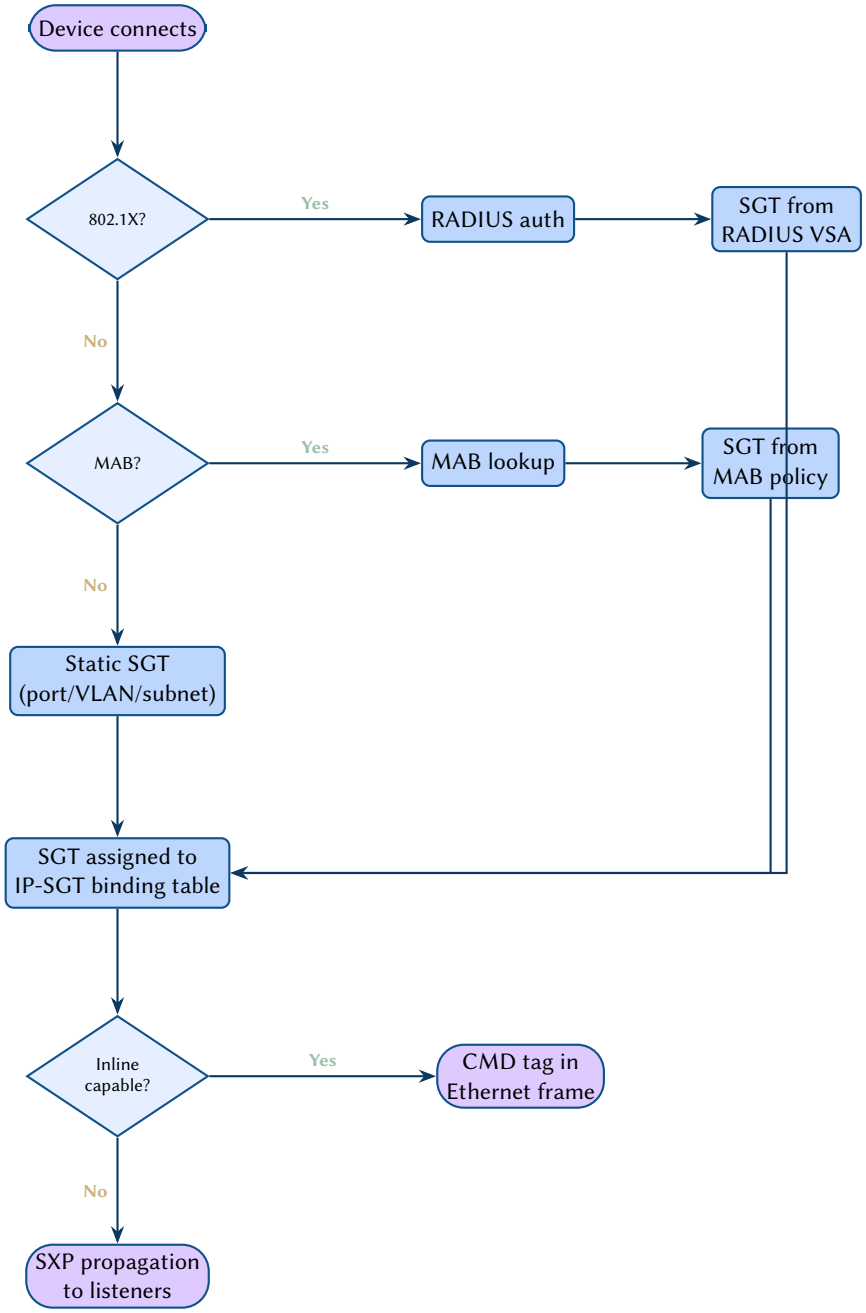


Figure 92. SGT assignment flow: from device authentication through RADIUS/MAB to tag assignment and inline or SXP propagation.

D.4 Security Group ACLs (SGACL)

With SGTs assigned and propagated, the enforcement mechanism is the Security Group ACL (SGACL). Unlike traditional ACLs, which match on source and destination IP addresses, SGACLs match on source SGT and destination SGT. This yields an $N \times N$ policy matrix where N is the number of defined security groups.

The scalability advantage is substantial. Consider a network with 1 000 servers across 10 roles. A traditional IP-based policy would require $O(1000^2)$ ACL entries, whereas an SGT-based policy requires only $O(10^2) = 100$ SGACL cells. Each cell specifies a compact rule set (permit, deny, or permit with logging/restrictions) that applies uniformly to all traffic between the two groups, regardless of how many individual hosts populate each group.

SGACLs are typically provisioned centrally (e.g., via Cisco ISE) and downloaded to enforcement devices over RADIUS or the Policy Administration Point (PAP) protocol. The device caches the SGACL policy and applies it in hardware when both source and destination SGTs are known.

Design Decision 128: SGACL Attributes

Attribute	Type	Description
`sgacl_name`	str	SGACL policy name
`sgacl_src_sgt`	u16	Source SGT value
`sgacl_dst_sgt`	u16	Destination SGT value
`sgacl_seq`	u32	Rule sequence number
`sgacl_action`	enum	permit, deny, log
`sgacl_protocol`	enum	ip, tcp, udp, icmp
`sgacl_dst_port`	u16	Destination port match

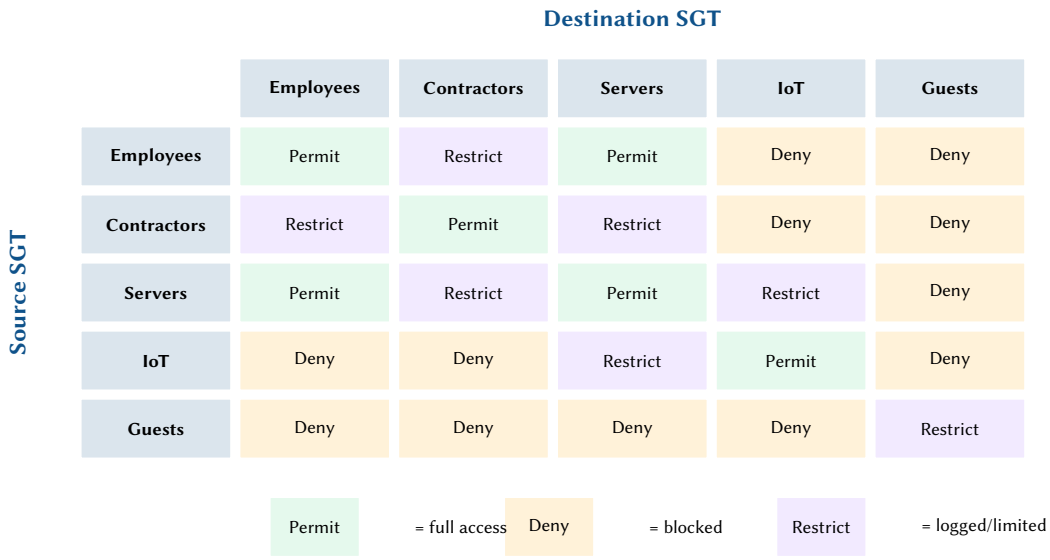


Figure 93. SGT source–destination policy matrix. Green cells indicate permit, red cells deny, and yellow cells permit with restrictions (logging, rate-limiting, or port-specific filtering).

D.5 Micro-Segmentation

Micro-segmentation extends the zero-trust principle to east–west traffic within a single broadcast domain. In a traditional segmented network, devices in the same VLAN communicate freely; inter-VLAN traffic is controlled at the Layer 3 boundary. With SGT-based micro-segmentation, policy enforcement occurs at the access layer, and two hosts on VLAN 100 carrying different SGTs are subject to SGACL policy just as if they resided in different VLANs.

Relationship to VXLAN/EVPN. In overlay fabrics, the VXLAN Group-Based Policy (GBP) extension carries the SGT value in a reserved field of the VXLAN header. This allows SGT semantics to traverse the overlay without requiring SXP or inline CMD in the underlay. The EVPN control plane distributes MAC/IP-to-SGT bindings as extended community attributes, enabling distributed enforcement at every VTEP.

VRF + SGT combined segmentation. A common enterprise design pattern uses VRFs for *macro-segmentation* (isolating tenants, business units, or compliance domains) and SGTs for *micro-segmentation* (controlling roles within each macro-segment). VRFs provide hard routing isolation; SGTs provide policy-driven access control within each VRF. The two mechanisms are orthogonal and composable: an endpoint carries both a `vrf_membership` attribute and an `sgt_value` attribute, and enforcement devices apply VRF routing decisions first, then SGACL policy within the resolved VRF context.

D.6 Hybrid-Cloud Policy Normalization

The group-based model is the primary mechanism for unifying security policy across hybrid-cloud environments. While the 16-bit SGT is the de facto standard for on-premise hardware, cloud providers use different classification primitives: AWS and Azure utilise *Security Groups* (SGs) keyed by resource IDs, while Kubernetes uses *Labels* and *Selectors*.

In the plan topology, the `sgt_value` (or a generalized `security_group_id`) serves as the **universal security identifier**. The $\mathcal{F}_{\text{Security}}$ functor maps this identifier to the platform-specific realization:

- **Campus/DC:** Rendered as SGTs in the CMD header or VXLAN GBP.
- **Public Cloud:** Rendered as AWS/Azure Security Group rules.
- **Container Orchestration:** Rendered as Kubernetes `NetworkPolicy` selectors (e.g., `matchLabels: { "sec-group": "100" }`).

By normalizing these disparate cloud-native and on-premise primitives into a singular group-based matrix in G_{plan}, the NTE provides a mathematically consistent security boundary that survives the transition from the physical wire to the virtualized cloud.

D.7 Groups in the Plan Topology

In the plan topology G_{plan}, group membership is modelled as a flat attribute on each endpoint, consistent with the single-graph principle (Section 16). The `sgt_value` attribute on an endpoint is no different in kind from `ipv4_address` or `ospf_area`: it is a scalar property that a functor may read and interpret.

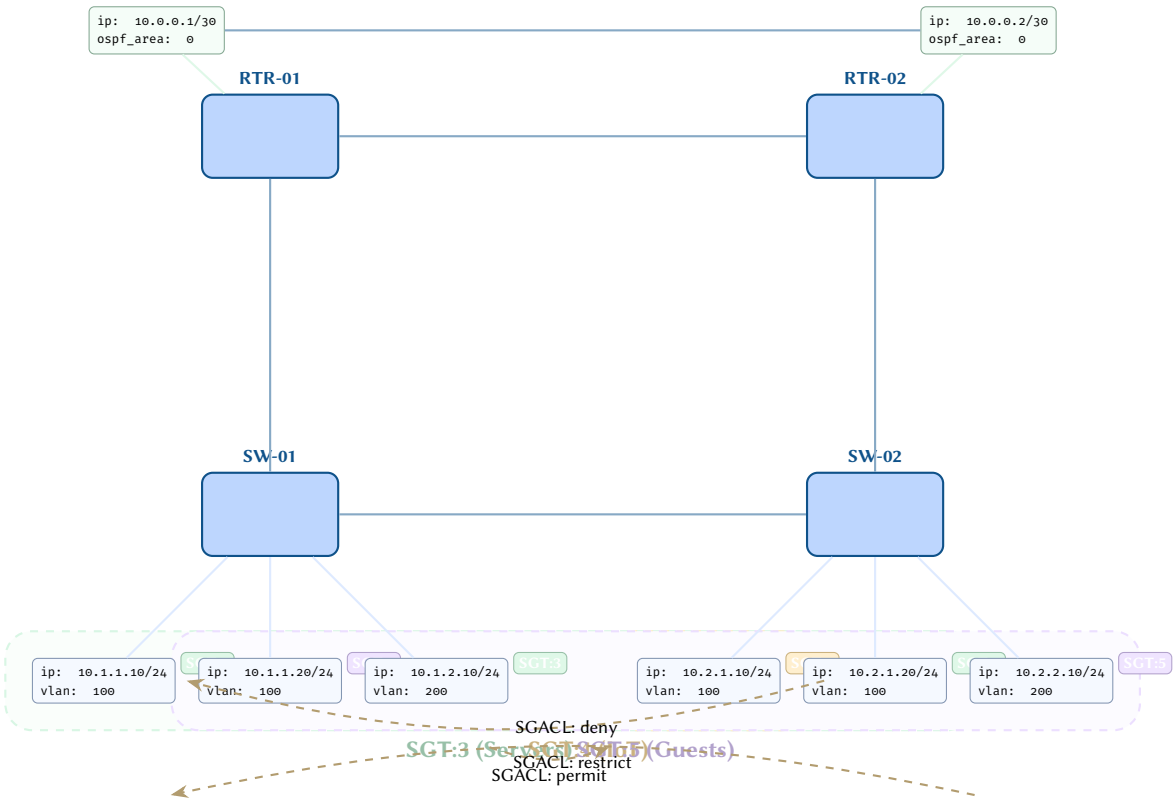
Group-to-group security policy, however, introduces a *second-order* structure: the SGACL matrix defines relationships not between individual endpoints but between sets of endpoints. We model this as a *security overlay graph* G_{sec} whose nodes are security groups (identified by SGT value) and whose directed edges are SGACL policies. The edge from group s to group d carries the policy attributes (`sgacl_action`, `sgacl_protocol`, `sgacl_dst_port`) that govern traffic from any endpoint with `sgt_value` $= s$ to any endpoint with `sgt_value` $= d$.

Design Rule 18: SGT as Cross-Cutting Attribute

SGT is *not* a new layer in the multilayer model. It is a cross-cutting attribute that feeds the $\mathcal{F}_{\text{Security}}$ functor (defined in Section 19.3). The security overlay graph G_{sec} has SGT groups as nodes and SGACL policies as directed edges. This graph is *derived from*, not stored in, the plan topology—the functor reads `sgt_value` attributes from endpoints and `sgacl_*` attributes from nodes to construct G_{sec}.

The security overlay graph is a compact representation: for k security groups, G_{sec} has at most k nodes and k^2 edges, regardless of the number of physical endpoints. This aligns with the functor decomposition principle—the $\mathcal{F}_{\text{Security}}$ functor extracts group structure from the plan topology and produces a small, semantically rich overlay that can be verified, diffed, and composed independently of the physical and protocol layers.

Insight: *Group membership naturally maps to hyperedge incidence—all endpoints with the same `sgt_value` belong to the same security group hyperedge. The SGACL matrix then defines directed relationships between these hyperedges, forming a quotient graph over the plan topology.*



Dashed edges = security overlay G_{sec}

Figure 94. Plan topology with SGT group membership shown as shaded regions (hyperedges) and group-to-group SGACL policies as dashed overlay edges forming G_{sec}. Endpoints on different switches and VLANs share the same SGT, demonstrating topology-independent grouping.

E Complete Attribute Reference

The following table provides a consolidated index of all attribute prefixes and their layer assignments:

Prefix	Layer	Target
`phy_`	L1 Physical	Endpoint
`dom_`	L1 Optics	Endpoint
`synce_`	L1 Timing	Endpoint
`ptp_`	L1 Timing	Node / Endpoint
`poe_`	L1 Power	Endpoint
`switchport_`	L2 VLAN	Endpoint
`stp_`	L2 STP	Node / Endpoint
`lag_`	L2 LAG	Endpoint
`lacp_`	L2 LACP	Endpoint
`lldp_`	L2 Discovery	Endpoint
`cfm_`	L2 OAM	Endpoint
`macsec_`	L2 Security	Endpoint
`dot1x_`	L2 Security	Endpoint
`mpls_`	L2.5 MPLS	Node / Endpoint
`ldp_`	L2.5 LDP	Node / Endpoint
`rsvp_`	L2.5 RSVP-TE	Node / Endpoint
`sr_`	L2.5 SR	Node / Endpoint
`srv6_`	L2.5 SRv6	Node

Prefix	Layer	Target
`evpn_`	L2.5 EVPN	Node / Endpoint
`ipv4_`	L3 IP	Endpoint
`ipv6_`	L3 IP	Endpoint
`vrf_`	L3 VRF	Node / Endpoint
`vrrp_`	L3 FHRP	Endpoint
`bfd_`	L3 BFD	Endpoint
`dhcp_`	L3 DHCP	Endpoint
`neighbor_`	L3 ARP/NDP	Node
`nat_`	L3 NAT	Endpoint
`ecmp_`	L3 Forwarding	Node
`pim_`	L3 Multicast	Node / Endpoint
`igmp_`	L3 Multicast	Endpoint
`mld_`	L3 Multicast	Endpoint
`msdp_`	L3 Multicast	Node
`ospf_`	L3.5 OSPF	Node / Endpoint
`isis_`	L3.5 IS-IS	Node / Endpoint
`bgp_`	L3.5 BGP	Node / Endpoint
`vxlan_`	L5 Overlay	Node
`gre_`	L5 Overlay	Endpoint
`ipsec_`	L5 Overlay	Endpoint
`ike_`	L5 Overlay	Endpoint
`geneve_`	L5 Overlay	Endpoint
`tls_`	L6 Security	Node
`qos_`	Cross-layer	Endpoint
`acl_`	Cross-layer	Endpoint
`fib_`	Cross-layer	Node
`tcam_`	Cross-layer	Node
`lfib_`	Cross-layer	Node
`mac_table_`	L2 Forwarding	Node
`mac_`	L2 Forwarding	Node
`tcp_`	L4 Transport	Endpoint
`udp_`	L4 Transport	Endpoint
`quic_`	L4 Transport	Endpoint
`nat64_`	L3 NAT	Endpoint
`mka_`	L2 MACsec	Endpoint
`dtls_`	L6 Security	Node
`port_security_`	L2 Security	Endpoint
`storm_ctrl_`	L2 Security	Endpoint
`dai_`	L2 Security	Endpoint
`ipsg_`	L2 Security	Endpoint
`dhcp_snooping_`	L2 Security	Endpoint
`y1731_`	L2 OAM	Endpoint
`ntp_`	L7 Management	Node
`dns_`	L7 Management	Node
`syslog_`	L7 Management	Node
`snmp_`	L7 Management	Node
`aaa_`	L7 Management	Node
`radius_`	L7 Management	Node
`tacacs_`	L7 Management	Node
`ssh_`	L7 Management	Node
`netconf_`	L7 Management	Node

Prefix	Layer	Target
`gnmi_`	L7 Management	Node
`restconf_`	L7 Management	Node
`vpls_`	L2.5 VPLS	Node
`pw_`	L2.5 Pseudowire	Endpoint
`te_`	L2.5 MPLS-TE	Node
`subif_`	L3 Sub-interface	Endpoint
`irb_`	L3 IRB/SVI	Endpoint
`sgt_`	Cross-layer	Endpoint
`sxp_`	Cross-layer	Node
`sgacl_`	Cross-layer	Node
`copp_`	Cross-layer	Node
`fw_`	Cross-layer	Node
`pbr_`	Cross-layer	Endpoint
`pl_`	Cross-layer	Node
`cl_`	Cross-layer	Node
`rmap_`	Cross-layer	Node